

Embedding Complex Languages in the Functional Programming Vernacular

Samantha Frohlich



A dissertation submitted to the University of Bristol in accordance with the requirements for award of the degree of PhD in Computer Science in the Faculty of Engineering.

School of Computer Science
April 2026

Word count: 62813

Abstract

Programming languages have come a long way since punching holes in cards. Increasingly, they feature powerful but complex language features to meet programming demands of today. For example, bidirectional programming techniques have developed to allow for two related operations to be programmed with one piece of code; and type systems have advanced to ensure the correct programming of quantum computers.

As we develop these powerful tools, it is important to ensure that their complexity is not felt by the programmer. The paradigm of functional programming, with its emphasis on abstraction, provides the perfect vernacular for programming these complexities unburdened of finer details. In particular, the functional technique of embedding, where a new language is created within another, has a lot of potential. It facilitates the quick prototyping of domain specific languages, which are well suited for supporting complexities because they are designed specifically for each domain.

The aim of this thesis is to bring benefits of the functional programming vernacular to complex languages via embedding. In particular, it focusses in on two functional idioms:

- **Monadic Composition:** this is introduced as a new paradigm for bidirectional programming (BX). This thesis uses this paradigm to contribute an exploration of logic programming as a new application domain for BX, and to enhance the power of bigenerators in the domain of property-based testing.
- **Binders:** this thesis contributes a type-safe technique for the embedding of ergonomic binders into languages with complex semantics and type systems.

Acknowledgements

First and foremost, I'd like to thank my supervisor Meng Wang. I am immeasurably grateful for your unwavering support and wise advice that has secured me the best opportunities. From unforgettable trips to Japan to never failing to magic me up a job, you have really helped me thrive.

More broadly, I'd like to thank all of [PLRG: Bristol](#) and my wider academic network. You've been friends, collaborators, colleagues, conference buddies, office mates, room mates, mentors and more. It is safe to say that my PhD would have been a lot less fun without you all. Thank you!

It is equally important to acknowledge the non-academic support that I have received over the years. Thank you to my friends and family who have supported me all the way through my seemingly endless PhD. Thank you also to my lockdown flatmates who kept me sane in those early and turbulent years of my PhD.

I'd also like to shout-out to those that shaped my academic path. To my school computing teacher, Mr Mulholland, who named my Advanced Higher project "as good as a PhD thesis", planting the seed. Alas, I could not sneak in an example featuring the Seven Dwarfs. To the Great and Powerful Zenzike, for showing me the ways of the force. Do, or do not, there is no try.

Finally, I'd like to thank the Engineering and Physical Sciences Research Council for providing the funding that made this all possible.

Statement on the inclusion of published and collaborative work

My dissertation does include work that has been published, submitted or prepared for publication with me as an author. My dissertation does include non-published collaborative work that contains data or contributions from others. Details of publications with me as an author and collaborative work is contained in the following table.

Chapter	Details	Author Contributions
2	<i>The Monadic Profunctor Paradigm of Bidirectional Programming</i> . Li-yao Xia, Samantha Frohlich, Dominic Orchard and Meng Wang. Under submission at the ACM Transactions on Programming Languages and Systems (TOPLAS) journal.	This Chapter is a journal extension of an existing paper [L.-y. Xia et al. 2019b], which I am not an author on. For completeness and readability, the journal extension has been included in its entirety. It is important to note that this includes much prior work as seen published in the original paper. It is the extensions, as listed in the change log (see 2.1) in the introduction of this chapter, that were contributed as part of this thesis. The main contributions are Sections 2.6, 2.9 and 2.10, with the latter two being substantial additions. I led the work and writing for these sections, supervised and helped by Dominic Orchard and Meng Wang, with the Roqc proofs done by Li-yao Xia.
3	<i>Reflecting on Random Generation</i> . Harrison Goldstein, Samantha Frohlich, Meng Wang, and Benjamin C. Pierce. Published as a Distinguished paper at the 2023 International Conference on Functional Programming (ICFP).	Harrison Goldstein of the University of Pennsylvania and I wrote an original version of this paper in equal collaboration, then Harry led the writing for the revised versions. Our supervisors, Meng Wang and Benjamin C. Pierce advised the project and helped with the editing process.
4	<i>Embedding by Unembedding</i> . Kazutaka Matsuda, Samantha Frohlich, Meng Wang, and Nicolas Wu. Published at the 2023 International Conference on Functional Programming (ICFP).	This work was done in collaboration with Kazutaka Matsuda, my supervisor Meng Wang, and Nicolas Wu. Kazutaka led the implementation, and I led the presentation of this work.
5	<i>Contextual Embeddings</i> . Samantha Frohlich, Alex Kavvos, Jessica Foster, and Meng Wang. Under submission at the 2026 Conference on Programming Language Design and Implementation (PLDI).	This work is a collaboration with Alex Kavvos and Jessica Foster led by me. As supervisor of Jess and I, Meng Wang took an advisory role in the project.

The information in this table counts as the references to my own publications if they are used as whole chapters or as whole sections of chapters. If work from my own publications have been incorporated in

smaller segments, I have included individual references to them in the relevant part of the dissertation in addition to completing the table.

Author's Declaration

I declare that the work in this dissertation was carried out in accordance with the requirements of the University's Regulations and Code of Practice for Research Degree Programmes and that it has not been submitted for any other academic award. Except where indicated by specific reference in the text, the work is the candidate's own work. Work done in collaboration with, or with the assistance of, others, is indicated as such. Any views expressed in the dissertation are those of the author.

A handwritten signature in black ink, appearing to read 'Samantha Frohlich', with a stylized, flowing script.

Samantha Frohlich
2 April 2026

Contents

1	Introduction	1
1.1	Part I: Monadic Composition	2
1.1.1	Chapter 2: The Monadic Profunctor Paradigm of Bidirectional Programming	2
1.1.2	Chapter 3: Reflecting on Random Generation	2
1.2	Part II: Binders	3
1.2.1	Chapter 4: Embedding by Unembedding	3
1.2.2	Chapter 5: Contextual Embeddings	4
I	Monadic Composition	5
2	The Monadic Profunctor Paradigm of Bidirectional Programming	7
2.1	Introduction	7
2.2	Background	9
2.3	Monadic bidirectional programming	10
2.3.1	Monadic biparsers	13
2.4	A unifying structure: partial monadic profunctors	15
2.4.1	Constructing Partial Monadic Profunctors (PMPs)	17
2.4.2	Deriving biparsers as PMP pairs	19
2.5	Reasoning about bidirectionality	20
2.5.1	Compositional properties of monadic bidirectional programming	21
2.5.2	Quasicompositionality for PMPs	26
2.5.3	Reasoning Recap	28
2.5.4	For comparison: separately defined parser and printer, and round tripping proof	28
2.6	Monadic bidirectional programming for picklers	29
2.6.1	Construction	29
2.6.2	Pickler Combinators	30
2.6.3	Round Tripping	30
2.7	Monadic bidirectional programming for lenses	31
2.7.1	Construction	32
2.7.2	A Small Example: Key-Value Maps	33
2.7.3	A Bigger Example: Trees	34
2.7.4	Round Tripping	36
2.8	Monadic bidirectional programming for generators	37
2.8.1	Construction	38
2.8.2	Examples	38
2.8.3	Round tripping	40
2.9	Monadic Bidirectional Logic Programming	42

2.9.1	Construction	42
2.9.2	Choice and Logic Programming	43
2.10	Generalising partial profunctors to right monadic actions	50
2.10.1	Summary	54
2.11	Discussion and Related Work	54
2.11.1	Applicative bidirectional programming	54
2.11.2	Lenses	55
2.11.3	Random generators and predicates	56
2.11.4	Future work	56
2.12	Conclusions	57
3	Reflecting on Random Generation	59
3.1	Introduction	59
3.2	Background	61
3.2.1	Monadic Random Generators	61
3.2.2	Free Generators	62
3.2.3	Partial Monadic Profunctors	62
3.3	The Reflective Generator Language	63
3.3.1	Intuition	64
3.3.2	Implementation	64
3.3.3	Interpretation	67
3.4	Theory of Reflective Generators	69
3.4.1	Correctness	69
3.4.2	Overlap	72
3.4.3	Expressiveness	73
3.5	Example-Based Generation	75
3.5.1	<i>Inputs from Hell</i>	75
3.5.2	Reflecting on Examples	76
3.6	Validity-Preserving Shrinking and Mutation	78
3.6.1	Test-Case Reduction in Hypothesis	78
3.6.2	Reflective Shrinking	79
3.6.3	Reflective Mutation	81
3.7	Improving the Testing Workflow	81
3.7.1	Reflective Checkers	81
3.7.2	Reflective Completers	82
3.7.3	Reflective Producers	83
3.8	Related Work	83
3.9	Conclusion and Future Directions	84
II	Binders	85
4	Embedding by Unembedding	87
4.1	Introduction	87
4.2	The Problem with Higher-Order Abstract Syntax (HOAS)	89
4.2.1	Background: HOAS	89
4.2.2	The Problem	89
4.3	Background: The Technique of Unembedding	90

4.3.1	What and Why	90
4.3.2	The de Bruijn Representation	91
4.3.3	The HOAS Representation	92
4.3.4	Converting between HOAS and de Bruijn	93
4.4	Embedding by Unembedding	95
4.4.1	Applicability	95
4.4.2	Embedding	95
4.4.3	Unembedding	97
4.4.4	Embedding by Unembedding	99
4.5	Case Study: Incremental λ -Calculus	101
4.5.1	Cache-Transfer-Style Incremental Functions	101
4.5.2	Why Unembedding is Desired and Applicable	101
4.5.3	Unembedding Details of Interest	102
4.6	Case Study: Bidirectional Programming Language	105
4.6.1	Lenses	106
4.6.2	Why Unembedding is Desired and Applicable	106
4.6.3	Unembedding Details of Interest	107
4.7	Related Work	109
4.8	Future Work	111
4.9	Conclusion	112
5	Contextual Embeddings	115
5.1	Introduction	115
5.2	Background: First-Order Embeddings	117
5.2.1	First-Order Embedding of the STLC	118
5.2.2	Dealbreaking Trade-off	119
5.2.3	Rise of the Unembed	119
5.3	Contextual Embeddings	120
5.3.1	Towards an Ergonomic First-Order Embedding	120
5.3.2	Contextual Embedding of the STLC	122
5.3.3	Isomorphic to First-Order Embedding	123
5.3.4	Assigning Semantics	125
5.3.5	Scope of Contextual Embeddings	126
5.3.6	Performance of Contextual Embeddings	127
5.4	Modal Example: Fitch-Style Modal Lambda Calculus	128
5.5	Substructural Example: Linear λ -Calculus (LLC)	130
5.6	Contextual Embeddings Levelled Up	132
5.6.1	Contextual Embedding with Levels	133
5.6.2	Contextual Embedding of STLC with Levels	133
5.6.3	Justification of Levels	134
5.7	Substructural Example: LLC Levelled Up	135
5.8	Further Examples	135
5.9	Related Work	136
5.9.1	Higher-Order Abstract Syntax (HOAS) and Friends	136
5.9.2	Embedding by Unembedding	137
5.9.3	Names for Free (NFF)	137
5.9.4	Embedding a full Linear λ -Calculus (LLC) in Haskell	137
5.9.5	Locally Nameless	138

5.9.6	Nominal Approaches	138
5.9.7	First-Order Formalisation	138
5.9.8	Unpublished Related Work	139
5.10	Conclusion and Future Work	139
6	Conclusion	141
	Bibliography	143
	Appendix	157
1	Partial Arrows are PMPs	157
2	Generator Upgrade Progression	157
3	Proofs of Lemma 3.4.1 (Laws)	158
4	Polymorphic Interpretation Function	159
5	Proof of Lemma 3.4.4 (Weak Completeness)	160
6	Hypothesis Generator Example	161
7	JSON with Hash Code Generator	161
8	Example of Reduced package.json	162
9	LeanCheck BST Enumerator	162
10	Implementation of liftS0	163
11	Unembedding the Incremental λ -Calculus	164
11.1	Unembedding Incremental λ -Calculus (ILC)	165
11.2	Extending ILC with Sequences	167
12	Syntax and Typing Rules of Languages to Embed	169
12.1	Syntax and Typing Rules of ILC/CTS DSL	169
12.2	Syntax and Typing Rules of HOBiT to Embed	169
13	More Details of Embedded Incremental Language	170
13.1	Concrete Definition of mapSem'	170
13.2	Pseudo Functions	170
13.3	Programming Example: Incremental HaXML Filters	172
13.4	More Efficient Implementation	175
14	Related Work of Embedded Incremental Language	176
15	More Details of Embedded Bidirectional Language	176
15.1	Round-tripping Laws	176
15.2	Pattern Combinators	177

List of Tables

1	Introduction	1
2	The Monadic Profunctor Paradigm of Bidirectional Programming	7
3	Reflecting on Random Generation	59
3.1	Average size of shrunk outputs after reflective shrinking, compared with Hypothesis shrinking, QuickCheck’s genericShrink, and un-shrunk inputs. (Mean and two standard-deviation range.) *Hypothesis experiments not re-run, data taken from [David R. MacIver and Donaldson 2020].	80
4	Embedding by Unembedding	87
5	Contextual Embeddings	115
6	Conclusion	141
	Bibliography	143
	Appendix	157

List of Figures

1	Introduction	1
2	The Monadic Profunctor Paradigm of Bidirectional Programming	7
2.1	Addams Family Tree	52
3	Reflecting on Random Generation	59
3.1	Generators for binary search trees.	59
3.2	Background definitions.	61
3.3	An intermediate generator with <code>Void</code> as its contravariant type, using <code>voidAnn</code> .	67
3.4	The “generate” interpretation.	67
3.5	The “reflect” interpretation.	68
3.6	Reflective generators with unit, exponential, and infinite overlap.	72
3.7	Analysis of <code>withChecksum</code> tuned by example in the style of <i>Inputs from Hell</i> .	77
3.8	A Hypothesis-inspired reflective generator and a tree that the generator might produce.	78
3.9	Shrinking strategies.	80
4	Embedding by Unembedding	87
4.1	Example of Advanced Semantics: Reversible Computation	87
4.2	STLC HOAS Representation	92
4.3	Experimental Results: incremental computation of <i>cartesian xs ys</i> when <i>length xs = length ys = 200</i> . Key: scratch = recomputation, inc = incremental computation, inc-opt = optimized incremental computation (optimizes away change propagation overhead).	104
4.4	Semantic Function for Unpairing	107
4.5	Comparison of <i>linesB</i> in <code>HOBiT</code> and <code>eHOBiT</code> to showcase the faithfulness of the embedding.	108
5	Contextual Embeddings	115
5.1	The Simply-typed λ -Calculus (STLC), as a Generalised Algebraic Data Type (GADT)	118
5.2	Compile-time: CE Vs. FO	127
5.3	Compile-time: CE Vs. HOAS	128
5.4	Runtime Cost	129
5.5	The First-Order Linear λ -Calculus (LLC), as a GADT	131
5.6	A First-Order Embedding of the STLC with Levels	133
5.7	A Contextual Embedding of the STLC with Levels	134
5.8	A Contextual Embedding of the LLC with Levels	136
6	Conclusion	141
	Bibliography	143

Appendix	.157
1 Implementation of liftS0	164
2 An implementation of mapSem', where mapEnv :: (forall x. f x -> g x) -> Env f as -> Env g as is the mapping function for Env.	171
3 Construction of PFun and an example of its use: we abused the Haskell syntax to use the bare exists.	172
4 Input and Output XMLs for Q1 (taken from https://www.w3.org/TR/xquery-use-cases/)	173
5 Extended Language for HaXML Filters	174
6 Derived Filters and Filter Combinators (Excerpt)	174
7 Query Q1	174

List of Abbreviations

AST Abstract Syntax Tree

BST Binary Search Tree

CTS Cache-transfer-style

GADT Generalised Algebraic Data Type

HOAS Higher-Order Abstract Syntax

ILC Incremental λ -Calculus

LIFO Last In, First Out

LLC Linear λ -Calculus

NFF Names for Free

PHOAS Parametric Higher-Order Abstract Syntax

PMP Partial Monadic Profunctor

STLC Simply-typed λ -Calculus

ULC Untyped λ -Calculus

1. Introduction

In today’s world, there is increasing demand for more **complex** language features to support and verify the programming world of tomorrow. It is essential to have powerful ways to check and enforce how a program should behave, such as that provided by strong and exacting type systems, and thorough testing. For example, *substructural type systems* [B. Pierce 2004] are one such complex feature that is gaining traction, with *linear type systems* [J. Bernardy et al. 2018] proving their value in the domain of quantum computing [Fu et al. 2020], and languages like Rust growing in popularity. Other complex language features, such as *bidirectional programming* [J. Nathan Foster et al. 2005; Matsuda, Hu, et al. 2007; Voigtländer 2009] are also coming into their own.

As language features get necessarily more complex, it is important to ensure that the difficulty of programming does not. The paradigm of functional programming, with its emphasis on abstraction: what a solution should be instead of how a solution is reached, is a natural fit. In fact, the functional paradigm has already shown itself to be influential in the development and propagation of new language features. Useful functional features such as higher-order and anonymous functions (lambda expressions), types, polymorphism, monadic composition, property-based testing, algebraic datatypes, and typeclasses have since been adopted by mainstream languages such as Python, Java, C#, C++, Scala and Rust [David R MacIver, Hatfield-Dodds, et al. 2019; Odersky and Wadler 1997]. The impact of the **functional programming vernacular** is well summarised by Hughes [1990] and Hu et al. [2015], who both praise its value in terms of compositionality. Hughes [1990] advocates that to facilitate modular, divide and conquer code, a language must provide good methods for combining together the solutions of sub-problems. Hu et al. [2015] then pick up the topic years later, evidencing in numerous ways that functional programming has truly mattered.

Embedding, a technique that is particularly popular in the functional paradigm, is at the frontier of the development of new language features. This is where a new *guest* language is created within another existing *host* language. This technique has numerous benefits [Hudak 1996]: it expedites the creation of the guest through reuse of host language infrastructure; embedded languages are highly interoperable with other languages, particularly those embedded in the same host; and it is easy for users of the host language to pick up the guest language due to existing familiarity with the host language’s environment and syntax. To enable embedded languages to stay at the forefront of feature development, and allow the creators of new language features to access these attractive benefits, it is essential that the state-of-the-art features of modern languages can be embedded. Moreover, it would be desirable for this to be done in the functional programming vernacular.

This thesis focuses on **bringing the benefits of the functional programming vernacular to the complex languages of tomorrow via embedding**.

The structure of this thesis is in two parts. Each part takes an influential idiom from the functional programming vernacular and explores its impact on the embedding of complex language features. In particular, the thesis focuses on the features that grant the compositionality advocated for by Hughes [1990]. In Part I, we will explore bringing the *monadic do notation* praised by Hu et al. [2015] to the domain of bidirectional programming; and in Part II, we will tackle the difficulties of embedding *binders* [B. E. Aydemir et al. 2005; Pfenning and Elliott 1988] into languages with complex semantics, which will unlock the benefits of higher-order functions highlighted by Hughes [1990].

1.1. Part I: Monadic Composition

Monads, hailed by [Wadler \[1992\]](#) as the “essence” of functional programming, provide a principled way of structuring code with side-effects in a pure functional language. They enjoy an expressive distinguished syntax: monadic `do` notation. It is this interface that this part of the thesis focuses on bringing to the domain of bidirectional programming, which hitherto had been dominated by less expressive combinator based embedded languages [[J. Nathan Foster et al. 2005](#)]. Chapter 2 introduces monadic composition as a powerful new paradigm for bidirectional programming. Then, Chapter 3 builds upon this in the domain of property-based testing, increasing the power of random data generators [[Claessen and Hughes 2000](#)].

1.1.1. Chapter 2: The Monadic Profunctor Paradigm of Bidirectional Programming

In this chapter, monadic composition is brought to bidirectional programming using *partial monadic profunctors*. Profunctors [[Edward Kmett 2011](#); [Pickering et al. 2017](#)] are a standard construction from category theory, generalizing ordinary functors (structure-preserving maps) to allow for both covariant and contravariant mapping operations. It is the duality of directions given to profunctors by their co- and contravariance that allows them to accommodate the related (and often near-inverse) operations of bidirectional programs.

This proves to be a successful and unifying structure, providing expressive monadic composition to numerous existing bidirectional domains: parsers/printers [[Matsuda and M. Wang 2018c](#)], picklers [[Kennedy 2004](#)], lenses [[J. Nathan Foster et al. 2005](#)], and generators/predicates for structured data [[L.-y. Xia et al. 2019a](#)]. In fact, this chapter also uses its monadic profunctor paradigm to explore logic programming [[Braßel et al. 2010](#); [Hanus et al. 1995](#)] as a new bidirectional domain.

The content of this chapter is taken from *The Monadic Profunctor Paradigm of Bidirectional Programming*, a journal extension of *Composing Bidirectional Programs Monadically* [[L.-y. Xia et al. 2019a](#)], which is currently under submission at the ACM Transactions on Programming Languages and Systems (TOPLAS) journal.

The impact of bringing this essential part of the functional programming vernacular, expressive monadic composition, to the complex domain of bidirectional programming is to allow these programs to be written in a more natural and expressive manner. Many of the applications of this chapter already have a monadic interface for at least one direction of the bidirectional transformation. This allows programmers to explore the connections between the unidirectional programming that they are familiar and bidirectional features. The idea is that this will lessen the complexity of programming with this language feature.

1.1.2. Chapter 3: Reflecting on Random Generation

Property-based testing, popularised by Haskell’s QuickCheck [[Claessen and Hughes 2000](#)] is a paradigm of testing that achieves more thorough testing than standard unit tests. As opposed to testing specific examples, *properties* that should hold of the code are tested against randomly generated inputs. As such, random data generators are at the core of property-based testing. Chapter 2 features *bigenators* as an example of a bidirectional transformation that extends generators to also be able to check whether they could have generated a value. This chapter takes this to the next level, combining them with *free generators* [[Goldstein and Pierce 2022](#)] to extend their applicability to grammar-based testing [[Soremekun et al. 2020](#)], and shrinking / mutation [[David R MacIver, Hatfield-Dodds, et al. 2019](#)].

The combination of bigenerators with free generators to create *reflective generators* is an impactful

match. The paper that this chapter is taken from, *Reflecting on Random Generation* [H. Goldstein et al. 2023], was published as a Distinguished paper at the 2023 International Conference on Functional Programming. Its key contributions include using bidirectionally to increase the applicability of shrinkers like that of the popular Hypothesis library [David R MacIver, Hatfield-Dodds, et al. 2019], which has 6,500 stars on GitHub and an estimated 500,000 users [Dodds 2022].

Reflective generators epitomise the focus of this thesis. They are an embedded language that supports the complex languages of the future by providing thorough testing that leverages bidirectional programming, and this is done in the functional programming vernacular through property-based testing that is composed monadically.

1.2. Part II: Binders

In the second act of this thesis, the focus will be on enabling another form of compositionality in the functional programming vernacular: higher-order functions. This is the powerful programming tool highlighted by Hughes [1990]. It allows for the breaking down of tasks into sub-tasks, achieving a separation of concerns.

Embedding a higher-order language depends upon embedding *binders*. Binders are language constructs that close over, or *bind* variables, such as `let` or `lambda`. Unfortunately, despite being a very well explored area [Atkey 2009; Carette et al. 2009; Chlipala 2008; Despeyroux, Felty, et al. 1995; Despeyroux and Hirschowitz 1994; Despeyroux, Pfenning, et al. 1997; G. P. Huet and Lang 1978; Matsuda, Frohlich, et al. 2023; Washburn and Weirich 2003, 2008], embedding languages with binders is not a solved problem. Both chapters in this part of the thesis address the challenge of embedding languages with binders.

Chapter 4 makes a fruitful start to this quest by noticing that the technique of *unembedding* [Atkey et al. 2009] can be used to connect two representations of syntax with binders to combine their benefits. This allows for an ergonomic *Higher-Order Abstract Syntax (HOAS)* [Church 1940; G. P. Huet and Lang 1978; Miller and Nadathur 1987; Pfenning and Elliott 1988] interface to be connected to an amenable *de Bruijn indices* [de Bruijn 1972] backend. This is applied to embedding modern languages with complex semantics and binders. Chapter 5 builds upon this by removing the previous work’s reliance on a difficult external proof [Atkey 2009] and expansion to support languages with modal and substructural type systems.

1.2.1. Chapter 4: Embedding by Unembedding

This chapter introduces Embedding by Unembedding, a technique for embedding languages with complex semantics and binders. Specifically, this paper addresses the problem that the desirable interface of **HOAS** does not directly support complex semantics defined on terms-in-contexts. This includes semantics in the domains of bidirectional programming and incremental computation [Cai et al. 2014; Giarrusso et al. 2019]. The solution employs unembedding, a bijection between **HOAS** and de Bruijn representations of syntax, to achieve embeddings of such languages. The technique is demonstrated on two case-studies: **HOBiT** [Matsuda and M. Wang 2018d], a higher-order lens language; and the Cache-transfer-style (**CTS**) [Giarrusso et al. 2019] variant of the Incremental λ -Calculus (**ILC**) [Cai et al. 2014].

The content of this chapter is taken from *Embedding by Unembedding* [Matsuda, Frohlich, et al. 2023], which was published at the 2023 International Conference on Functional Programming. It advances work on binder representations for embedded languages to support languages with complex semantics by seeing the potential of unembedding.

1. Introduction

Bringing ergonomic binders to embedded languages with complex semantics is an important step towards the goal of this thesis. Previously, the complex languages of tomorrow that included binders could not enjoy the benefits of embedding without sacrificing usability. Now, this core part of the functional programming vernacular can be embedded, even if the guest has complex semantics.

1.2.2. Chapter 5: Contextual Embeddings

Picking up where Embedding by Unembedding left off, Contextual Embeddings remove the need for external arguments [Atkey 2009] justifying unsafe operations when unembedding. They do this using a *type-safe context-directed isomorphism*. Moreover they introduce support for languages with complex type systems, making unembedding relevant for embedding even more modern languages.

Contextual Embeddings is currently under submission at the 2026 Conference on Programming Language Design and Implementation (PLDI). It showcases its advances by embedding modal λ -calculi [Clouston 2017; Davies and Pfenning 2001] and the Linear λ -Calculus (LLC) [Grenrus 2019; Hodas 1995; Polakow 2015] with an ergonomic interface in a type-safe manner.

This work rounds this thesis off nicely by combining together both features we value and in an embedded language: a compositional functional vernacular via binders, and support for complex modern languages such as those with substructural type systems.

Part I.

Monadic Composition

2. The Monadic Profunctor Paradigm of Bidirectional Programming

We begin our quest of bringing the functional programming vernacular to complex languages via embedding with bringing monadic composition to the complex domain of bidirectional programming. Monads are the linchpin of structuring effectful code in functional languages. It is their expressive notation that this chapter focusses on bringing to bidirectional programming, allowing these programs to be written in a more natural and functional vernacular. In fact, it introduces the monadic composition style as a new paradigm for bidirectional programming.

This chapter’s content is taken from *The Monadic Profunctor Paradigm of Bidirectional Programming*, a journal extension of *Composing Bidirectional Programs Monadically* [L.-y. Xia et al. 2019a]. This is currently under submission at the ACM Transactions on Programming Languages and Systems (TOPLAS) journal. The extensions listed in the change log (see 2.1) were led and mostly done by me, supervised and helped by Dominic Orchard and Meng Wang, with the Rocq proofs and the new notion of “idiomatic compositionality” done by Li-yao Xia.

2.1. Introduction

A *bidirectional transformation* (BX) is a pair of mutually related mappings between source and target data objects. A classic application of this idea is the *view-update problem* [Bancilhon and Spyrtos 1981] from relational database design, where a *view* is a derived database table, computed from concrete *source* tables by a query. The problem is to map an update of the view back to a corresponding update on the source tables. This can be neatly captured as a bidirectional transformation. The bidirectional pattern is found in a broad range of applications, including parsing [Matsuda and M. Wang 2018c; Rendel and Ostermann 2010], pickling [Kennedy 2004], refactoring [Schuster et al. 2016], code generation [Mayer et al. 2018; Pombrio and Krishnamurthi 2014], model transformation [Stevens 2008b], and XML transformation [Pacheco, Zan, et al. 2014].

When programming a bidirectional transformation, one can separately construct the “forward” (view) and “backward” (update) functions. However, this approach duplicates effort, is prone to error, and causes subsequent maintenance issues as both directions must be kept in sync. These problems can be avoided by using specialised programming languages that generate both directions from a single definition [J. Nathan Foster et al. 2005; Matsuda, Hu, et al. 2007; Voigtländer 2009], a discipline known as *bidirectional programming*.

The most well-known language family for BX programming is *lenses* [J. Nathan Foster et al. 2005]. A lens captures transformations between sources S and views V via a pair of functions $\text{get} : S \rightarrow V$ and $\text{put} : V \rightarrow S \rightarrow S$. The *get* function extracts a view from a source, and *put* takes an updated view and a source as inputs to produce an updated source. The asymmetrical nature of *get* and *put* makes it possible for *put* to recover some of the source data that is not present in the view. In other words, *get* does not have to be injective to have a corresponding *put*.

Typically, bidirectional transformations respect *round tripping* laws, capturing the extent to which the transformations preserve information between the two data representations. For example, *well-behaved*

2. The Monadic Profunctor Paradigm of Bidirectional Programming

lenses [Bohannon, B. C. Pierce, et al. 2006; J. Nathan Foster et al. 2005] should satisfy:

$$\text{put } (\text{get } s) \ s = s \qquad \text{get } (\text{put } v \ s) = v$$

Lens languages are typically designed to enforce these properties *by construction*. This focus on unconditional correctness inevitably leads to reduced practicality: lens combinators—the main way to program lenses—are often stylised and disconnected from established programming idioms.

In this chapter, we instead focus on expressing bidirectional programs directly in idiomatic functional code, using monads as an interface for sequential composition. Monads are a popular pattern [Wadler 1995] (especially in Haskell) that combinator libraries in other (non-bidirectional) domains routinely exploit. Introducing monadic composition to BX programming significantly expands the expressiveness of BX languages and opens up a route for programmers to explore the connection between BX programming and mainstream uni-directional programming. Moreover, it appears that many applications of bidirectional transformations (*e.g.*, parsers and printers [Matsuda and M. Wang 2018c]) do not share the lens *get/put* pattern, and as a result have not been sufficiently explored. However, monadic composition is known to be an effective way to construct at least one direction of such transformations (*e.g.*, parsers). We show, amongst many other examples, that bidirectional parsing-and-printing can be given a monadic treatment.

Contributions In this paper, we deliberately avoid the well-tried approach of specialised lens languages, instead exploring a novel point in the BX design space based instead on monadic programming, naturally reusing host language constructs. We prove the validity of our approach by revisiting lenses and other known bidirectional domains, and test its boundaries by using it to explore completely new domains. By being uncompromising about the monadic interface, we expose the essential ideas behind our framework whilst maximising its utility. The trade off with our approach is that we can no longer enforce correctness in the same way as conventional lens languages: our interface does not rule out all non-round-tripping BXs. We tackle this issue by proposing a new compositional reasoning framework that is flexible enough to characterise a variety of round tripping properties, and simplifies the necessary reasoning.

Specifically, we make the following contributions:

- We describe a method to enable *monadic composition* for bidirectional programs (Section 2.4). Our approach is based on a construction that specifies a *Partial Monadic Profunctor (PMP)*, parameterised by two application-specific monads, which are used to generate the *forward* and *backward* directions. We later generalise our approach to include a *right monadic coaction* over PMPs which captures a common related programming pattern, increasing the power of the backwards direction (Section 2.10).
- To demonstrate the flexibility of our approach, we apply the above method to five different problem domains: parsers/printers (Section 2.4 and 2.5), picklers (Section 2.6), lenses (Section 2.7), generators/predicates for structured data (Section 2.8), and logic programming (Sections 2.9 and 2.10). While parsers/printers and lenses are well-explored areas in the bidirectional programming literature, generators are a completely new application domain, which we fully explore through their generalisation that enables logic programming (Section 2.9).
- We present a scalable reasoning framework, capturing notions of *compositionality* for bidirectional properties (Section 2.5). We define classes of round tripping properties inherent to bidirectionality, which can be verified by following simple criteria. These principles are demonstrated with our examples. We include some proofs for illustration in the paper. The supplementary material [Li-yao Xia et al. 2025] contains machine-checked Rocq proofs for the main theorems.

- We have implemented these ideas as Haskell libraries [Liyao Xia 2018], with two wrappers around `attoparsec` for parsers and printers, and `QuickCheck` for generators and predicates, showing the viability of our approach for real programs.

We use Haskell for concrete examples, but the programming patterns can be easily expressed in many functional languages. All of the proofs for this paper, apart from more complex examples in Section 2.8 and Section 2.9, are mechanised in Rocq [Li-yao Xia et al. 2025].

Change log This work extends “Composing bidirectional programs monadically” [L.-y. Xia et al. 2019b] with the following updates and additions:

- Inlining of relevant appendix material, more examples, and expanded explanations.
- Unification with the profunctors package¹ enabling the results of this work to be more easily applied within the Haskell ecosystem.
- New application domains: picklers [Kennedy 2004] (Section 2.6) and logic programming (Sections 2.9 and 2.10), where the latter is a substantial addition, generalising monadic profunctor generators to solve classic logic puzzles such as *n*-queens or the river crossing puzzle.
- For bigenerators, our framework previously only enabled proofs of *completeness*. To handle *soundness*, we propose *idiomatic compositionality*, a new notion of compositionality more restrictive than quasicompositionality.
- Insight into, and generalisation of, partial profunctors as *right monad coactions*, enabling the above application to logic programming (Section 2.10).
- [Li-yao Xia et al. 2025] contains updated Haskell code and Rocq proofs.

2.2. Background

We introduced lenses briefly above. We now introduce *parsers/printers* and *generators/predicates*. As *picklers* build upon *parsers/printers* and our *logic programming* approach is achieved by generalising generators, these will be introduced later.

Parsing and printing Programming language tools (such as interpreters, compilers, and refactoring tools) typically require two intimately linked components: *parsers* and *printers*, respectively mapping from source code to ASTs and back. An (overly) simplistic implementation of these two functions can be given with types:

```
parser :: String → AST
printer :: AST → String
```

Parsers and printers are rarely actual inverses to each other, but instead typically exhibit a variant of round tripping such as satisfying properties about the interaction of partially-parsed inputs with the printer and parser, *e.g.* if `parser :: String → (AST, String)`:

$$(\text{let } (x, s') = \text{parser } s \text{ in } \text{printer } ((\text{printer } x) ++ s')) \equiv \text{parser } s$$

¹<https://hackage.haskell.org/package/profunctors>

2. The Monadic Profunctor Paradigm of Bidirectional Programming

i.e., after parsing s into an AST fragment x and remaining text s' , printing x back to a string, concatenating it with the remained s' , and re-parsing is just the same as the original parsing. This pattern resembles lenses between a source (source code) and view (ASTs), but with a notion of composition for the source, and partial “gets” which consume some of the source leaving a remainder. Thus, parsing and printing follows a pattern of inverse-like functions which does not fit the lens paradigm.

Writing parsers and printers by hand is often tedious due to the redundancy implied by their inverse-like relation. Thus, various approaches have been proposed for reducing the effort of writing parsers/printers by generating both from a common definition [Matsuda and M. Wang 2018b,c; Rendel and Ostermann 2010]. Bidirectional parsers/printers will be the running example in the next section.

Generating and checking Property-based testing (*e.g.* QuickCheck) [Claessen and Hughes 2000] expresses program properties as executable predicates. For instance, the following property checks that an insertion function `insert`, given a sorted list — as checked by the predicate `isSorted :: [Int] → Bool` — produces another sorted list. The combinator $\implies$ is implication for properties:

```
propInsert :: Int → [Int] → Property
propInsert val list = isSorted list  $\implies$  isSorted (insert val list)
```

To test this property, a testing framework generates random inputs for `val` and `list`. The implementation of $\implies$ applied here first checks whether `list` is sorted, and if it is, checks that `insert val list` is sorted as well. This process is repeated with further random inputs until either a counterexample is found or a predetermined number of test cases pass.

However, this naïve method is inefficient: many properties such as `propInsert` have preconditions which are satisfied by an extremely small fraction of inputs. In this case, the ratio of sorted lists among lists of length n is inversely proportional to $n!$, so most generated inputs will be discarded for not satisfying the `isSorted` precondition. Such tests give no information about the validity of the predicate being tested and thus are prohibitively inefficient.

When too many inputs are being discarded, the user must instead supply the framework with *custom generators* of values satisfying the precondition: `genSorted :: Gen [Int]`.

One can expect two complementary properties of such a generator. A generator is *sound* with respect to the predicate `isSorted` if it generates only values satisfying `isSorted`; soundness means that no tests are discarded, hence the tested property is better exercised. A generator is *complete* with respect to `isSorted` if it can generate all satisfying values; completeness ensures the correctness of testing a property with `isSorted` as a precondition, in the sense that if there is a counterexample, it will be eventually generated. In this setting of testing, completeness is arguably more important than soundness, as completeness affects the potential adequacy of testing whilst soundness affects only efficiency.

It is clear that generators and predicates are closely related, forming a pattern similar to that of bidirectional transformations. Given that good generators are usually difficult to construct, the ability to extract both from a common specification with bidirectional programming is a very attractive alternative. We return to this example in detail in Section 2.8

2.3. Monadic bidirectional programming

A bidirectional parser, or *biparser*, combines both a parsing direction and printing direction. Our first novelty here is to express biparsers monadically.

In code samples, we use the Haskell pun of naming variables after their types, *e.g.*, a variable of some polymorphic type v will also be called v . Similarly, for some type constructor m , a variable of type $m\ v$ will be called mv . A function $u \rightarrow m\ v$ (a Kleisli arrow for a monad m) will be called kv .

Monadic parsers The following data type provides the standard way to describe parsers of values of type v which may consume only part of the input string:

```
data Parser v = Parser {parse :: String → Maybe (v, String)}
```

Here a `Parser` consists of a function that takes a `String` (the input to be parsed), and from it *maybe* produces the resulting parsed value of type v paired with a `String`, which contains the remainder of the input that was not used in parsing. It is well-known that such parsers are monadic [Wadler 1995], *i.e.*, they have a notion of sequential composition embodied by the interface:

```
instance Monad Parser where
  (≫=) :: Parser v → (v → Parser u) → Parser u
  return :: v → Parser v
```

The sequential composition operator ($\gg=$), called *bind*, describes the scheme of constructing a parser by sequentially composing two sub-parsers where the second depends on the output of the first. Indeed, this is the implementation given to the monadic interface:

```
pv ≫= ku = Parser (λs → parse pv s ≫= λ(v, s') → parse (ku v) s')
return v = Parser (λs → Just (v, s))
```

Bind first runs the parser pv on an input string s , resulting in maybe a value v and that rest of the string s' . Then, v is passed on using the bind of `Maybe` to create the parser $ku\ v$, which is in turn run on the remaining input s' to produce parsed values of type u . The return operation creates a trivial parser for any value v which does not consume any input but simply produces v .

In practice, parsers composed with ($\gg=$) often have a relationship between the output types of the two operands: one “contains” the other in some sense. For example, we might parse an expression and compose this with a parser for statements, where statements contain expressions. This relationship will be useful later when we consider printers.

As a shorthand, we can discard the remaining unparsed string of a parser using projection, giving a helper function `parser :: Parser v → (String → Maybe v)`.

Monadic printers Our goal is to augment parsers with their inverse printer, such that we have a monadic type `Biparser` which provides two complementary (bi-directional) transformations:

```
parser  :: Biparser v → Maybe (String → v)
printer :: Biparser v →      (v → String)
```

However, this type of printer $v \rightarrow \text{String}$ (shown also in Section 2.2) cannot form a monad because it is *contravariant* in its type parameter v . Concretely, we cannot implement the bind ($\gg=$) operator for values with types of this form:

```
-- Failed attempt
bind :: (v → String) → (v → (u → String)) → (u → String)
bind pv ku = λu → let v = (??) in pv v ++ ku v u
```

2. The Monadic Profunctor Paradigm of Bidirectional Programming

We are stuck trying to fill the hole (??) as there is no way to get a value of type v to pass as an argument to pv (first printer) and ku (second printer which depends on a v). Subsequently, we cannot construct a monadic biparser by simply taking a product of the parser monad and $v \rightarrow \text{String}$ and leveraging the result that the product of two monads is a monad.

But what if the type variables of bind were related by containment, such that v is contained within u and thus we have a projection $u \rightarrow v$? We could use this projection to fill the hole in the failed attempt above, defining a bind -like operator:

```
bind' :: (u → v) → (v → String) → (v → (u → String)) → (u → String)
bind' from pv ku = λu → let v = from u in pv v ++ ku v u
```

This is closer to the monadic form, where $\text{from} :: u \rightarrow v$ resolves the difficulty of contravariance by “contextualizing” the printers. Thus, the first printer is no longer just “a printer of v ”, but “a printer of v extracted from u ”. In the context of constructing a bidirectional parser, having such a function to hand is not an unrealistic expectation: when we compose two parsers with $(\gg=)$, typically the values of the first parser for v will be contained within the values returned by the second parser for u , thus a notion of projection can be defined and used here to recover a v in order to build the corresponding printer compositionally.

Of course, this is still not a monad. However, it suggests a way to generate a monadic form by putting the printer and the contextualizing projection together, $(u \rightarrow v, v \rightarrow \text{String})$ and fusing them into $(u \rightarrow (v, \text{String}))$. This has the advantage of removing the contravariant occurrence of v , yielding a data type:

```
data Printer u v = Printer {print :: u → (v, String)}
```

If we fix the first parameter type u , then the type $\text{Printer } u$ of printers for u values is indeed a monad, combining a *reader monad* (for some global read-only parameter of type u) and a *writer monad* (for strings) [M. P. Jones 1995], with implementation:

```
instance Monad (Printer u) where
  return :: v → Printer u v
  return = λv → Printer (λ_ → (v, ""))

  (≫=) :: Printer u v → (v → Printer u t) → Printer u t
  pv ≻= kt = Printer (λu → let (v, s) = print pv u
                              (t, s') = print (kt v) u
                              in (t, s ++ s'))
```

The printer $\text{return } v$ ignores its input and prints nothing. For bind , an input u is shared by both printers and the resulting strings are concatenated.

We can adapt the contextualisation of a printer by the following operation which amounts to pre-composition, witnessing the fact that Printer is a contravariant functor in its first parameter:

```
comapPrinter :: (u → u') → Printer u' v → Printer u v
comapPrinter from (Printer p) = Printer (p ∘ from)
```

2.3.1. Monadic biparsers

So far so good: we now have a monadic notion of printers. However, our goal is to combine parsers and printers in a single type. Since we have two monads, their product is a monad, defining *biparsers*:

```
data Biparser u v = Biparser { biParse :: String → Maybe (v, String)
                              , biPrint :: u      →      (v, String) }
```

By pairing parsers and printers we have to unify their covariant parameters. When both the type parameters of `Biparser` are the same it is easy to interpret this type: a biparser `Biparser v v` is a parser from strings to `v` values and printer from `v` values to strings. We refer to biparsers of this type as *aligned* biparsers. What about when the type parameters differ? A biparser of type `Biparser u v` provides a parser from strings to `v` values and a printer from `u` values to strings, but where the printers can compute `v` values from `u` values, *i.e.*, `u` is some common broader representation which contains relevant `v`-typed subcomponents. A biparser `Biparser u v` can be thought of as printing a certain subtree `v` from the broader representation of a syntax tree `u`.

The corresponding monad for `Biparser` is the product of the previous two monad definitions for `Parser` and `Printer`, allowing both to be composed sequentially at the same time:

```
instance Monad (Biparser u) where
  return :: v → Biparser u v
  return v = Biparser (λs → Just (v, s)) (λ_ → (v, ""))

  (≫=) :: Biparser u v → (v → Biparser u w) → Biparser u w
  pu ≻= kw = Biparser parse' print' where
    parse' s = biParse pu s ≻= λ(v, s') → biParse (kw v) s'
    print' u = let (v, s) = biPrint pu u
                (w, s') = biPrint (kw v) u
                in (w, s ++ s')
```

In the above instance, the product construction is in evidence: in both `return` and `(≫=)`, the `Biparser` constructor is provided with first the `Parser` version of the function and then the `Printer` version.

We can also lift the previous notion of `comap` from printers to biparsers, which gives us a way to contextualise a printer:

```
comapBiparser :: (u → u') → Biparser u' v → Biparser u v
comapBiparser f (Biparser parse print) = Biparser parse (print ∘ f)

upon :: Biparser u' v → (u → u') → Biparser u v
upon = flip comapBiparser
```

In the rest of this section, we use the alias “upon” for `comap` with flipped parameters where we read `p` ‘upon’ subpart as applying the printer of `p :: Biparser u' v` on a subpart of an input of type `u` calculated by the function subpart `:: u → u'`, thus yielding a biparser of type `Biparser u v`.

Often, the desired subpart function, that “inverts” the forward computation, is partial. For example, when `u` is a sum type (and the subpart in question is only one side of the sum), or when `u` is a list that needs decomposing (e.g., by head). Thus the more appropriate type is subpart `:: u → Maybe u'`. We thus adjust the type of `comapBiparser` accordingly. To avoid handling `Maybe` within each `Biparser`, we also add the possibility of failure to the backward direction:

2. The Monadic Profunctor Paradigm of Bidirectional Programming

```

data BiparserM u v = BiparserM { biParseM :: String → Maybe (v, String)
                                   , biPrintM  :: u      → Maybe (v, String) }

comapBiparserM :: (u → Maybe u') → BiparserM u' v → BiparserM u v
comapBiparserM f (BiparserM parse print) = BiparserM parse (λu → f u >>= print)

uponM :: BiparserM u' v → (u → Maybe u') → BiparserM u v
uponM = flip comapBiparserM

```

An example biparser Let us write a biparser, `string :: BiparserM String String`, for strings that are prefixed by their length and a space. For example, the following unit tests should succeed:

```

test1 = biParseM string "6 lambda calculus" ≡ Just ("lambda", " calculus")
test2 = biPrintM string "SKI" ≡ Just ("SKI", "3 SKI")

```

In `test1`, the parser consumes the number 6 followed by a space. This tells it to gobble the next six characters following the space. Hence, the expected result is the six letters of "lambda" to be parsed, and the rest to be left. When it comes to the printer, `test2` presents the word "SKI". Since this is three letters long, the printer will print the number three and a space followed by the word. For brevity, we will not handle failing cases, assuming inputs always have the expected form.

We start by defining a primitive biparser of single characters as:

```

char :: BiparserM Char Char
char = BiparserM parseChar (λc → Just (c, [c]))
  where parseChar [] = Nothing
        parseChar (c:s) = Just (c, s)

```

A character is parsed by deconstructing the source string into its head and tail. A character `c` is printed as its single-letter string (a singleton list) paired with `c`.

In our example, we first need to parse the number. We will do this by parsing the digits of the number using `char` until there is a space, using the combinator `manyUntilBP`:

```

manyUntilBP :: BiparserM u a → (a → Bool) → BiparserM [u] [a]
manyUntilBP one p = do
  x ← one 'uponM' headM
  if p x
  then return []
  else do
    xs ← (manyUntilBP one p) 'uponM' tailM
    return (x:xs)

```

where `headM :: [a] → Maybe a` and `tailM :: [a] → Maybe [a]`.

This biparser takes as input a biparser to repeat, and a condition on what it is parsing to signal when to stop the repetition; it uses the given parser to parse one example, then if the condition is met, it stops, otherwise it continues. Note that in Haskell, the **do**-notation statement “`x ← one 'uponM' headM`” desugars to “`one 'uponM' headM >>= λc → ...`”, which uses (`>>=`) and a function binding `c` in the scope

of the rest of the desugared block. In the case of parsing digits to make up an integer, we want to keep parsing until we reach a space:

```
int :: BiparserM Int Int
int = do
  ds ← manyUntilBP char isSpace 'uponM' printedInt
  return (read ds)
  where
    printedInt n = Just $ show n ++ " "
```

After parsing an integer n , we can parse the string following it by iterating the biparser `char` n times. This is captured by the `replicateBP` combinator below. This combinator is akin to `replicateM` from Haskell's standard library. To iterate n times a biparser `pv`: if $n \equiv 0$, there is nothing to do and we return the empty list; otherwise for $n > 0$, we run `pv` once to get the head v , and recursively iterate $n - 1$ times to get the tail vs .

Note that although not reflected in its type, as a printer, `replicateBP n pv` expects a list, l , of length n : if $n \equiv 0$, there is nothing to print; if $n > 0$, 'upon' head extracts the head of l to print it with `pv`, and 'upon' tail extracts its tail, of length $n - 1$, to print it recursively.

```
replicateBP :: Int → BiparserM u v → BiparserM [u] [v]
replicateBP 0 pv = return []
replicateBP n pv = do
  v ← pv 'uponM' headM
  vs ← (replicateBP (n - 1) pv) 'uponM' tailM
  return (v:vs)
```

We can now fulfil our example task to parse and print length-prefixed strings:

```
string :: BiparserM String String
string = int 'uponM' (Just ∘ length) >>= λn → replicateBP n char
```

Interestingly, if we erase applications of `upon`, *i.e.*, we substitute every expression of the form `py 'upon' f` with `py` and ignore the second parameter of the types, we obtain what is essentially the definition of a parser in an idiomatic style for monadic parsing. This is because 'upon' f is the identity on the parser component of `Biparser`. Thus the biparser code closely resembles standard, idiomatic monadic parser code but annotated using `upon` to access the relevant subparts of the input when running the *backward* direction of printing.

Despite its simplicity, the syntax of length-prefixed strings is notably context-sensitive. Thus the example makes crucial use of the monadic interface for bidirectional programming: a value (the length) must first be extracted to dynamically delimit the string that is parsed next. Context-sensitivity is standard for parser combinators in contrast with parser generators, *e.g.*, Yacc, and applicative parsers, which are mostly restricted to context-free languages. By our monadic BX approach, we can now bring this power to bear on *bidirectional* parsing.

2.4. A unifying structure: partial monadic profunctors

The biparser examples of the previous section were enabled by both the monadic structure of `Biparser` and its covariant functor structure provided by the `comapBiparserM` operation (also called `uponM`, with flipped arguments). We call the combined structure a *Partial Monadic Profunctor (PMP)*, which we

2. The Monadic Profunctor Paradigm of Bidirectional Programming

view as the conjunction of two algebraic structures: monads, of course, and profunctors [Edward Kmett 2011; Pickering et al. 2017], which have partiality injected into the first type parameter. Here, we will show that **PMPs** characterise a key class of structures for bidirectional programs. Furthermore, we show a construction of **PMPs** from pairs of monads which elicits the necessary structure for monadic bidirectional programming in the style of the previous section.

Definition 2.4.1. A binary type constructor p is a *profunctor* if it is contravariant in its first parameter and covariant in its second. Its operation given by the following class:

```
class Profunctor p where
  dimap :: (a → b) → (c → d) → p b c → p a d
```

Profunctors are subject to the laws of a bifunctor (contravariant functoriality in the first argument and covariant functoriality in the second):

$$\text{dimap id id} = \text{id} \qquad \text{dimap } (g \circ f) (h \circ k) = \text{dimap } f h \circ \text{dimap } g k$$

In category theory terminology, a profunctor is more generally a functor $\mathcal{D}^{\text{op}} \times \mathcal{C} \rightarrow \mathbf{Set}$, from the product of two categories $\mathcal{D}^{\text{op}}$ and $\mathcal{C}$ to the category of sets. The naming in Haskell, coming from the existing Profunctor class [Edward Kmett 2011], derives from this terminology, but instead essentially describes profunctors $\mathbf{Set}^{\text{op}} \times \mathbf{Set} \rightarrow \mathbf{Set}$ if we identify Haskell types in an idealised way with sets. We adopt this programming-oriented terminology, and utilise Haskell’s profunctor package.

Every profunctor is a functor in its second parameter. The following function, `rmap`, can be used to implement `fmap` in the Functor class. Provided the functor laws are satisfied, `rmap = fmap` is guaranteed by parametricity:

```
rmap :: Profunctor p ⇒ (a → b) → p u a → p u b
rmap = dimap id
```

We introduce the following subclass of Profunctor to enable partiality.

Definition 2.4.2. A *partial profunctor* p is a profunctor equipped with a natural transformation `internaliseMaybe`:

```
class Profunctor p ⇒ PartialProfunctor p where
  internaliseMaybe :: p u v → p (Maybe u) v
  comap :: PartialProfunctor p ⇒ (u → Maybe u') → p u' v → p u v
  comap f = dimap f id ∘ internaliseMaybe
```

`internaliseMaybe` is subject to the following laws, where `fmap`, `return` and `join` are specialised to the **Maybe** monad:

$$\begin{aligned} \text{internaliseMaybe} \circ \text{dimap } f g &= \text{dimap } (\text{fmap } f) g \circ \text{internaliseMaybe} \\ \text{dimap return id} \circ \text{internaliseMaybe} &= \text{id} \\ \text{dimap join id} \circ \text{internaliseMaybe} &= \text{internaliseMaybe} \circ \text{internaliseMaybe} \end{aligned}$$

The first law is the naturality of `internaliseMaybe`. That law is satisfied “for free” by parametricity [Wadler 1989]. The last two laws regulate the interaction of the profunctor p with the **Maybe** monad. Using `internaliseMaybe` and `dimap`, we can define the `comap_M` operation of the previous section to map over the first type parameter of p with a Kleisli arrow in the **Maybe** monad. This is a default method of the class.

As a consequence of the partial profunctor laws, `comap` obeys the following functor laws:

```
comap return = id
comap (f <=< g) = comap f ∘ comap g
```

Definition 2.4.3. A *Partial Monadic Profunctor (PMP)* is a partial profunctor `p` (in the sense of Definition 2.4.2) such that `p u` is a monad for all `u`, and `dimap f id` is a monad morphism for all `f :: u' → u`:

```
dimap f id (u >>= k) = (dimap f id u) >>= ((dimap f id) ∘ k)
dimap f id (return x) = return x
```

Those equations follow from parametricity, so we may represent **PMPs** with the following empty class, or “class synonym”, which inherits all its methods from its superclasses, and which is instantiated whenever its superclasses are. The constraint $\forall u. \text{Monad } (p\ u)$ is enabled by the `QuantifiedConstraints` extension [Bottu et al. 2017].

```
class (PartialProfunctor p, ∀u. Monad (p u)) ⇒ Profmonad p
instance (PartialProfunctor p, ∀u. Monad (p u)) ⇒ Profmonad p
```

One might also expect `comap` to be a monad morphism. However, this is not the case for the **Printer PMP** of the previous section—nor for **Bwd** which will generalise **Printer**, in §2.4.1.

Note that `dimap` is uniquely determined by `comap` and `fmap`, and `internaliseMaybe` is uniquely determined by `comap`:

```
dimap f g = comap (Just ∘ f) ∘ fmap g
internaliseMaybe = comap (λ_ → Nothing)
```

Therefore, an implementation of a **PMP** can be derived from `return`, `(>>=)`, and `comap`.

Biparsers have lawful instances of `Monad` (**Biparser** `u`) as defined in Section 2.3.1 (p. 13) and an instance `Profunctor` **Biparser** thus defined:

```
instance Profunctor Biparser where
  dimap f g (Biparser parse print) = Biparser parse' print'
  where
    print' = λu → let (v, s) = print (f u) in (g v, s)
    parse' = λs → let (v, s) = parse s in (g v, s)
```

Lastly, we introduce a useful piece of terminology (mentioned in the previous section on biparsers) for describing values of a profunctor of a particular form.

Definition 2.4.4. A value `p :: P u v` of a profunctor `P` is called *aligned* if `u = v`.

2.4.1. Constructing PMPs

Our examples (parsers/printers, picklers, lenses, generators/predicates, and generalise generators) all share **PMPs** as an abstraction, making it possible to write different kinds of bidirectional transformations monadically. Underlying these definitions of **PMPs** is a common structure, which we explain here using biparsers, and which will be replayed in subsequent sections for each of our examples, starting with

2. The Monadic Profunctor Paradigm of Bidirectional Programming

picklers in Section 2.6, which is very similar to the parsers/printers construction, and ending with bigenerators and their generalisation in Sections 2.8 and 2.9.

There are two simple ways in which a functor, m (resp. a monad) gives rise to a profunctor (resp. a PMP). The first is by constructing a profunctor in which the contravariant parameter is discarded, i.e., $p \ u \ v = m \ v$; the second is as a function type from the contravariant parameter u to $m \ v$, i.e., $p \ u \ v = u \rightarrow m \ v$. These are standard mathematical constructions, and the latter appears in the Haskell profunctors package with the name *Star*. Our core construction is based on these two ways of creating a profunctor, which we call **Fwd** and **Bwd** respectively:

```
data Fwd  $m \ u \ v = \text{Fwd} \ \{\text{unFwd} :: m \ v\} \quad \text{-- ignore contrv. parameter}$ 
data Bwd  $m \ u \ v = \text{Bwd} \ \{\text{unBwd} :: u \rightarrow m \ v\} \quad \text{-- maps from contrv. parameter}$ 
```

The naming reflects the idea that these two constructions will together capture a bidirectional transformation and are related by domain-specific round tripping properties in our framework. Both **Fwd** and **Bwd** map any functor into a profunctor by the following type class instances:

```
instance Functor  $m \Rightarrow$  Profunctor (Fwd  $m$ ) where
  dimap  $- \ g \ (\text{Fwd} \ mx) = \text{Fwd} \ (\text{fmap} \ g \ mx)$ 
```

```
instance Functor  $m \Rightarrow$  Profunctor (Bwd  $m$ ) where
  dimap  $f \ g \ (\text{Bwd} \ kx) = \text{Bwd} \ ((\text{fmap} \ g) \circ kx \circ f)$ 
```

Since the **Fwd** construction ignores the contravariant type parameter, there is nothing to apply `dimap`'s first argument to, meaning that the **Fwd** type can be passed through unchanged.

Furthermore, **Fwd** and **Bwd** both map any monad into a PMP:

```
instance Monad  $m \Rightarrow$  Monad (Fwd  $m \ u$ ) where
  return  $x = \text{Fwd} \ (\text{return} \ x)$ 
  Fwd  $py \gg= kz =$ 
    Fwd  $(py \gg= \text{unFwd} \circ kz)$ 
```

```
instance Monad  $m \Rightarrow$  Monad (Bwd  $m \ u$ ) where
  return  $x = \text{Bwd} \ (\lambda_- \rightarrow \text{return} \ x)$ 
  Bwd  $my \gg= kz = \text{Bwd}$ 
     $(\lambda u \rightarrow my \ u \gg= (\lambda y \rightarrow \text{unBwd} \ (kz \ y) \ u))$ 
```

The **Fwd** is trivially a partial profunctor:

```
instance Functor  $m \Rightarrow$  PartialProfunctor (Fwd  $m$ ) where
  internaliseMaybe (Fwd  $x$ ) = Fwd  $x$ 
```

For **Bwd** to be a partial profunctor, we require that the monad m has a notion of failure, via the following MonadFail class (which is part of the Haskell standard library):

```
class Monad  $m \Rightarrow$  MonadFail  $m$  where fail :: String  $\rightarrow m \ a$ 
```

Instances of the type class are expected to obey the following axiom for all $s :: \text{String}$ (which represents an error message):

$$(\text{fail} \ s \gg= k) = \text{fail} \ s$$

If m is a monad with a fail, we can define `internaliseMaybe` by producing fail when the input is **Nothing**.

```

instance MonadFail m  $\Rightarrow$  PartialProfunctor (Bwd m) where
  internaliseMaybe (Bwd k) = Bwd k' where
    k' Nothing = fail "Error"
    k' (Just x) = k x

```

Standard instances The `Maybe` type constructor is the prototypical instance of `MonadFail`:

```

instance MonadFail Maybe where fail _ = Nothing

```

For every monad transformer `t`, we also get an instance `MonadFail m  $\Rightarrow$  MonadFail (t m)`. We define it as an individual instance for each transformer to avoid overlapping instances.

```

instance (Monoid w, MonadFail m)  $\Rightarrow$  MonadFail (WriterT w m) where
  fail e = WriterT (fail e)

```

```

instance MonadFail m  $\Rightarrow$  MonadFail (StateT s m) where
  fail e = StateT ( $\lambda$ _  $\rightarrow$  fail e)

```

The product of two **PMPs** is also a **PMP**. This follows from the fact that the product of two monads is a monad and the product of two functors is a functor:

```

data (.:) p q u v = (.:) {pfst :: p u v, psnd :: q u v}

```

```

instance (Monad (p u), Monad (q u))  $\Rightarrow$  Monad ((p.:q) u) where
  return y = return y.:return y
  py.:qy  $\gg$  kz = (py  $\gg$  pfst o kz).: (qy  $\gg$  psnd o kz)

```

```

instance (Profunctor p, Profunctor q)  $\Rightarrow$  Profunctor (p.:q) where
  dimap f g (py.:qy) = dimap f g py.:dimap f g qy

```

These instances are a simple dance around the `(.:)` constructor that sticks the two monads together, applying the corresponding underlying instance to the correct monad.

Finally, the product of two partial profunctors is a partial profunctor:

```

instance (PartialProfunctor p, PartialProfunctor q)
 $\Rightarrow$  PartialProfunctor (p.:q) where
  internaliseMaybe (py.:qy) = (internaliseMaybe py).: (internaliseMaybe qy)

```

Therefore, we can form **PMPs** by a pair of `Fwd` and `Bwd` constructions.

2.4.2. Deriving biparsers as **PMP** pairs

We can redefine biparsers in terms of the above data types, their instances, and two standard monads, the state and writer monads (we are using the monad transformer versions, so the results can be wrapped in `Maybe`):

```

newtype StateT s m a = StateT (s  $\rightarrow$  m (a, s))
newtype WriterT w m a = WriterT (m (a, w))
type Biparser = Fwd (StateT String Maybe).:Bwd (WriterT String Maybe)

```

2. The Monadic Profunctor Paradigm of Bidirectional Programming

The forward component corresponds to parsers, and the backward to printers. Both sides may fail (**Maybe**): the parser because parsing is a partial process; the printer because it is necessary to obtain a partial contravariant map `comap` via the `PartialProfunctor` class.

$$\begin{aligned} \text{Fwd } (\text{StateT String Maybe}) \text{ } u \text{ } v &\cong \text{String} \rightarrow \text{Maybe } (v, \text{String}) \\ \text{Bwd } (\text{WriterT String Maybe}) \text{ } u \text{ } v &\cong u \rightarrow \text{Maybe } (v, \text{String}) \end{aligned}$$

We define smart constructors and destructors for accessing the two components of biparsers:

```
parse :: Biparser u v → (String → Maybe (v, String))
print :: Biparser u v → (u → Maybe (v, String))
mkBP :: (String → Maybe (v, String)) → (u → Maybe (v, String)) → Biparser u v
```

The **PMP** definition for biparsers now comes for free from the constructions in Section 2.4.1. The example biparsers from Section 2.3.1 can be easily redefined using the structure here. For example, the primitive biparser `char` becomes:

```
char :: Biparser Char Char
char = mkBP (\(c:s) → Just (c, s)) (\c → Just (c, [c]))
```

In a similar manner, we will use this **PMP** construction to define monadic bidirectional transformations for our other examples.

Codec library The codec library [Chilton 2018] provides a general type for bidirectional programming isomorphic to our composite type `Fwd r : * : Bwd w`:

```
data Codec r w c a = Codec { codecIn :: r a, codecOut :: c → w a }
```

Though the original codec library was developed independently, its current form is a result of this work. In particular, the second component was generalised from `codecOut :: c → w ()` in order to support a `Monad` instance for monadic bidirectional programming with codecs.

2.5. Reasoning about bidirectionality

So far we have seen how the **PMP** structure provides a way to define biparsers using familiar operations and syntax: monads and **do**-notation. This structuring allows both the forward and backward components of a biparser to be defined simultaneously in a single compact definition.

This section studies the interaction of **PMPs** with the *round tripping laws* that relate the two components of a bidirectional program. For every bidirectional transformation we can define dual properties: *backward round tripping* (going backward-then-forward) and *forward round tripping* (going forward-then-backward). In each BX domain, such properties also capture additional domain-specific information flow inherent to the transformations. We use biparsers as the running example. We then apply the same principles to our other examples. For brevity, we use **Bp** as an alias for **Biparser**. We separate the string and value components of printers via helper functions:

```
printS :: Bp u v → u → Maybe String      printV :: Bp u v → u → Maybe v
printS p = fmap snd ∘ print p              printV p = fmap fst ∘ print p
```

To begin, we consider the simpler case of *aligned* biparsers (Definition 2.4.4), of type $\text{Bp } u \ u$, using the same type u for both the input of the printer and for the output of the parser. When the inputs and outputs of both direction are thus aligned, it is straightforward to compose them in one of two ways, and equate the output with the input.

Starting with a value x , print it (going “backward”) into a string s , and parse that string, the result should be equal to the initial value x . In the following definitions, underscores abbreviate existentially quantified values; e.g. $\text{parse } p \ s = (x, _)$ is shorthand for $\exists s', \text{parse } p \ s = (x, s')$.

Definition 2.5.1. A biparser $p :: \text{Bp } u \ u$ is *backward round tripping* if, for all $x :: u$ and $s :: \text{String}$, this implication holds:

$$\text{print } p \ x = \text{Just } (_, s) \implies \text{parse } p \ s = \text{Just } (x, _).$$

The dual property is forward round tripping: a source string s is parsed (going “forward”) into some value x which when printed produces the initial source s :

Definition 2.5.2. A biparser $p :: \text{Bp } u \ u$ is *forward round tripping* if for every $x :: u$ and $s :: \text{String}$, this implication holds:

$$\text{parse } p \ s = \text{Just } (x, _) \implies \text{print } p \ x = \text{Just } (_, s).$$

Note, in some applications, that definition of forward round tripping is too strong. Here it requires that every printed value corresponds to at most one source string. This is often not the case as ASTs typically discard formatting and comments so that pretty-printed code is lexically different to the original source. Coarser equivalence relations on strings could enable more flexible forward round tripping properties for such cases.

The biparser $\text{char} :: \text{Bp } \text{Char } \text{Char}$ is aligned, so we can check its round tripping properties.

Proposition 2.5.3. The biparser char (§2.4.2) is both backward and forward round tripping.

Proof. By expanding definitions and algebraic reasoning. For backward round tripping, if $\text{print } \text{char } c = \text{Just } (_, [c'])$, then $c = c'$ and we can show by the expanding of definitions that $\text{parse } \text{char } [c'] = \text{Just } (c, _)$:

$$\begin{aligned} & \text{parse } \text{char } [c'] \\ \equiv & (\lambda(c0:s0) \rightarrow \text{Just } (c0, s0)) [c'] && \{ \text{Def. parse char, } (:) \text{ case } \} \\ \equiv & \text{Just } (c', _) && \{ \beta\text{-reduction } \} \\ \equiv & \text{Just } (c, _) && \{ \text{Assumption } \} \end{aligned}$$

For forward round tripping, $\text{parse } p \ s = \text{Just } (c, _)$ means that s must be $[c]$:

$$\begin{aligned} & \text{print } \text{char } c \\ \equiv & (\lambda c0 \rightarrow \text{Just } (c0, [c0])) c && \{ \text{Def. print char } \} \\ \equiv & \text{Just } (c, [c]) && \{ \beta\text{-reduction } \} \end{aligned}$$

□

2.5.1. Compositional properties of monadic bidirectional programming

As we compose parsers from smaller pieces, one naturally expects to prove round tripping properties in a similarly modular manner. A property is *compositional* with respect to a **PMP** if it is preserved by its operations (return , $(\gg=)$, dimap , and internaliseMaybe , where comap is also sufficient to cover the last two). Compositional properties are guaranteed “by construction” for programs built from those components.

2. The Monadic Profunctor Paradigm of Bidirectional Programming

Definition 2.5.4. A *property* $\mathcal{R}$ over a **PMP** P is a family of subsets $\mathcal{R}_v^u \subseteq P \text{ } u \text{ } v$ indexed by types u and v .

Definition 2.5.5. A property $\mathcal{R}$ is *compositional* with respect to the operations of a (partial) **PMP** if the following conditions hold for all types u, v, w , for all $x :: v, p :: P \text{ } u \text{ } v, k :: v \rightarrow P \text{ } u \text{ } w, f :: u' \rightarrow \text{Maybe } u$,

$$\begin{aligned} & (\text{return } x) \in \mathcal{R}_v^u && \text{(comp-return)} \\ (p \in \mathcal{R}_v^u) \wedge (\forall x. (k \text{ } x) \in \mathcal{R}_w^u) &\implies (p \gg= k) \in \mathcal{R}_w^u && \text{(comp-bind)} \\ p \in \mathcal{R}_v^u &\implies (\text{comap } f \text{ } p) \in \mathcal{R}_v^{u'} && \text{(comp-comap)} \end{aligned}$$

Unfortunately, forward and backward round tripping as defined above for biparsers are *not* compositional. For one, compositionality considers biparsers which might not even be aligned. Furthermore, even when starting with round tripping biparsers, arbitrary compositions using the above operations are not backward round tripping. For instance, consider a biparser $\text{char} \gg= \lambda z \rightarrow \text{return } (\text{succ } z)$. As a parser, it can parse any character and return its successor, *e.g.*, "a" is parsed to 'b', but as a printer it just prints the given character, *e.g.*, 'b' is printed as "b", different from the original "a".

Counterexample 2.5.6. The return operation is not backward round tripping for **Biparser**.

Proof. For return to be backward round tripping the following must hold:

$$\text{print } (\text{return } y) \text{ } x = \text{Just } (_, s) \implies \text{parse } (\text{return } y) \text{ } s = \text{Just } (x, _).$$

Let $x, y :: u$ and $s, s' :: \text{String}$, then we proceed by the following reasoning:

$$\begin{aligned} & \text{print } (\text{return } y) \text{ } x \\ \equiv & (\lambda_ \rightarrow \text{Just } (y, "")) \text{ } x && \{ \text{Def. return} \} \\ \equiv & \text{Just } (y, "") && \{ \beta\text{-reduction} \} \end{aligned}$$

Thus $s \equiv ""$. However the consequent of backward round tripping does not hold:

$$\begin{aligned} & \text{parse } (\text{return } y) \text{ } "" \\ \equiv & (\lambda s \rightarrow \text{Just } (y, s)) \text{ } "" && \{ \text{Def. parse } (\text{return } y) \} \\ \equiv & \text{Just } (y, "") && \{ \beta\text{-reduction} \} \\ \neq & \text{Just } (x, "") \end{aligned}$$

Thus, return is not backward round tripping. □

This suggests a need for more general properties that capture the full gamut of possible biparsers.

We first focus on backward (printing-then-parsing) round tripping. To obtain a more compositional version, we generalise it by adding an arbitrary suffix before parsing. The parser should not touch or look at the suffix: the parser is *context agnostic*.

We also allow the two type parameters of **Bp** to vary independently. If we print (going backward) and parse the resulting output (going forward) then we should get back the initial value. However, in a general biparser $p :: \text{Bp } u \text{ } v$, the input type of the printer u differs from the output type of the parser v , so we cannot compare them. Our actual intent for printers is that what we print is a fragment of u , which is produced as the output of the printer along with the string. By thus comparing the outputs of both the parser and printer, we obtain the following variant of backward round tripping:

Definition 2.5.7. A biparser $p :: \text{Bp } u \text{ } v$ is *weak backward round tripping* if for all $x :: u, y :: v$, and $s, s' :: \text{String}$ then:

$$\text{print } p \text{ } x = \text{Just } (y, s) \implies \text{parse } p \text{ } (s ++ s') = \text{Just } (y, s')$$

This differs from backward round tripping in that we now name and compare against y instead of the original value x .

Weakening the backward round tripping property in that way allows return to satisfy it:

Lemma 2.5.8. The return operation for the **Biparser PMP** is weak backward round tripping.

Proof. For return to be *weak* backward round tripping the following must hold:

$$\text{print (return } y) \ x = \text{Just } (y, s) \implies \text{parse (return } y) \ (s \ ++ \ s') = (y, s')$$

Let $x, y :: u$ and $s, s' :: \text{String}$:

$$\begin{aligned} & \text{print (return } y) \ x \\ \equiv & (\lambda_ \rightarrow \text{Just } (y, "")) \ x && \{ \text{Def. print (return } y) \} \\ \equiv & \text{Just } (y, "") && \{ \beta\text{-reduction} \} \end{aligned}$$

Thus $s \equiv ""$. Now we must prove the consequent of weak backward round tripping:

$$\begin{aligned} & \text{parse (return } y) \ (" \ ++ \ s') \\ \equiv & \text{parse (return } y) \ s' && \{ \text{Def. (++)} \} \\ \equiv & (\lambda s \rightarrow \text{Just } (y, s)) \ s' && \{ \text{Def. parse (return } y) \} \\ \equiv & \text{Just } (y, s') && \{ \beta\text{-reduction} \} \end{aligned}$$

□

Removing backward round tripping's restriction to aligned biparsers and using the result $y :: v$ of the printer gives us a property that *is* compositional:

Proposition 2.5.9. Weak backward round tripping of biparsers is compositional.

Proof. We consider in turn the three cases that need to hold for compositionality:

CASE return: Shown above.

CASE ($\gg=$):

Assume the weak round tripping of p :

$$\begin{aligned} & \text{print } p \ u = \text{Just } (w, sp) \\ \implies & \text{parse } p \ (sp \ ++ \ s') = \text{Just } (w, s') \end{aligned}$$

Assume the weak round tripping of k for all v :

$$\begin{aligned} & \text{print } (k \ v) \ u = \text{Just } (w, sk) \\ \implies & \text{parse } (k \ v) \ (sk \ ++ \ s') = \text{Just } (w, s') \end{aligned}$$

Our goal is to prove:

$$\begin{aligned} & \text{print } (p \ \gg= \ k) \ x = \text{Just } (w, s) \\ \implies & \text{parse } (p \ \gg= \ k) \ (s \ ++ \ s') = \text{Just } (w, s') \end{aligned}$$

Assume premise: $\text{print } (p \ \gg= \ k) \ x = \text{Just } (w, s)$.

Before proving our goal, we work on the premise to learn about the shape of s and discharge the premises of our assumptions:

$$\begin{aligned} & \text{print } (p \ \gg= \ k) \ u \\ \equiv & (w, sp \ ++ \ sk) && \{ \text{Def. } (\gg=) \} \end{aligned}$$

2. The Monadic Profunctor Paradigm of Bidirectional Programming

(Part of s comes from p , and part from k , we name these sp and sk .) We can now achieve our goal:

$$\begin{aligned}
 & \text{parse } (p \gg k) (sp ++ sk ++ s') \\
 \equiv & \text{parse } (k \ v) (sk ++ s') & \{ \text{Weak round tripping of } p \} \\
 \equiv & \text{Just } (w, s') & \{ \text{Weak round tripping of } k \}
 \end{aligned}$$

CASE comap:

For $f :: u \rightarrow \text{Maybe } u'$ and $p :: P \ u' \ v$, our goal is to prove:

$$\begin{aligned}
 & \text{print } (\text{comap } f \ p) \ x = \text{Just } (y, s) \\
 \implies & \text{parse } (\text{comap } f \ p) (s ++ s') = \text{Just } (y, s')
 \end{aligned}$$

Assume that p is weak backward round tripping, that is:

$$\forall x', y, s, s'. \text{print } p \ x' = \text{Just } (y, s) \implies \text{parse } p (s ++ s') = \text{Just } (y, s'). \quad (*)$$

Assume the antecedent: $\text{print } (\text{comap } f \ p) \ x = \text{Just } (y, s)$.

Before proving our goal, we work on this antecedent to discharge the premises of our assumptions:

$$\begin{aligned}
 & \text{print } (\text{comap } f \ p) \ x = \text{Just } (y, s) \\
 \equiv & f \ x = \text{Just } y' & \{ \text{Partial function succeeded} \} \\
 \equiv & \text{print } p \ y' = \text{Just } (y, s) & \{ \text{Simplify} \}
 \end{aligned}$$

Thus $\text{parse } p (s ++ s') = \text{Just } (y, s')$ holds by $(*)$, and we conclude:

$$\begin{aligned}
 & \text{parse } (\text{comap } f \ p) (s ++ s') \\
 \equiv & \text{parse } p (s ++ s') & \{ \text{Def.} \} \\
 \equiv & \text{Just } (y, s') & \{ \text{Def.} + \beta \}
 \end{aligned}$$

(Recall that comap does nothing in the forward (in this case parsing) direction.) □

Proposition 2.5.10. The primitive biparser `char` is weak backward round tripping, and `replicateBP` preserves weak backward round tripping.

We can then compositionally show weak backward round tripping for a composite biparser:

Example 2.5.11. `string` is weak backward round tripping. (By Proposition 2.5.9 and Proposition 2.5.10.)

This property is “weak” as it does not constrain the relationship between the input u of the printer and its output v . In fact, there is no hope for a compositional property to do so: the **PMP** combinators do not enforce a relationship between u and v . However, we can regain the stronger backward round tripping property by combining the weak compositional property with an additional non-compositional property on the relationship between the printer’s input and output. This relationship is represented by the function that results from ignoring the printed string, which amounts to removing the main effect of the printer. Thus we call this operation a *purification*:

$$\begin{aligned}
 \text{purify} & :: \forall u \ v. \text{Bp } u \ v \rightarrow u \rightarrow \text{Maybe } v \\
 \text{purify } p \ u & = \text{fmap fst } (\text{print } p \ u)
 \end{aligned}$$

Ultimately, when a biparser is aligned ($p :: \text{Bp } u \ u$) we want an input to the printer to be returned in its output, *i.e.*, $\text{purify } p$ should equal $\lambda x \rightarrow \text{Just } x$. If this is the case, we recover the original backward round tripping property:

Theorem 2.5.12. If $p :: P\ u\ u$ is weak backward round tripping, and for all $x :: u$, $\text{purify } p\ x = \text{Just } x$, then p is backward round tripping.

Proof. The definition of $\text{purify } p\ x = \text{fmap fst } (\text{print } p\ x)$ combined with the property $\text{purify } p\ x = \text{Just } x$, and the antecedent of backward round tripping ($\text{fmap snd } (\text{print } p\ x) = \text{Just } s$), imply that $\text{print } p\ x = \text{Just } (x, s)$. This satisfies the antecedent of weak backward round tripping, thus we can conclude $\text{parse } p\ (s ++ s') = \text{Just } (x, s')$, so backward round tripping holds for p . $\square$

Example 2.5.13. The biparser `string` is backward round tripping. (By Theorem 2.5.12 with Example 2.5.11 and Proposition 2.5.15.)

Proof. First we need to prove the following properties of biparsers `char`, `int`, and `replicateBP` :: `Int` $\rightarrow$ `Bp` $u\ v \rightarrow$ `Bp` `[u]` `[v]` (writing `proj` for `purify`):

$$\text{proj char } n \equiv \text{Just } n \quad (2.1)$$

$$\text{proj int } n \equiv \text{Just } n \quad (2.2)$$

$$\text{proj } (\text{replicateBP } (\text{length } xs) \ p) \ xs \equiv \text{mapM } (\text{proj } p) \ xs \quad (2.3)$$

The first two are straightforward from their definitions, here is the first, the second is similar:

$$\begin{aligned} & \text{proj char } n \\ \equiv & \text{fmap fst } (\text{print char } n) && \{ \text{Def. proj} \} \\ \equiv & \text{fmap fst } (\text{Just } (n, [n])) && \{ \text{Def. print} \} \\ \equiv & \text{Just } n && \{ \text{Def. fmap} + \beta\text{-reductions} \} \end{aligned}$$

Let us take a closer look at the final equation, Equation 2.3. As a printer, `replicateBP (length xs)` applies the printer `p` to every element of the input list `xs`, and if we ignore the output string with `proj`, that yields `mapM (proj p) xs`. When `p` is aligned and has `proj p = Just`, then all applications in the list succeed and return a `Just` value, so `mapM (proj p) xs` as a whole succeeds and returns the whole list of results. Therefore, `replicateBP (length xs) p xs = Just xs`. From these and the homomorphism properties we can prove `proj string = Just`:

$$\begin{aligned} & \text{proj string } xs \\ \equiv & \text{proj } (\text{comap length int } \gg= \lambda n \rightarrow \text{replicateBp } n \text{ char}) \ xs && \{ \text{Defs.} \} \\ \equiv & (\text{comap length } (\text{proj int}) \gg= \lambda n \rightarrow \text{proj } (\text{replicateBp } n \text{ char})) \ xs && \{ \text{Prop. 2.5.15} \} \\ \equiv & (\text{comap length Just } \gg= \lambda n \rightarrow \text{proj } (\text{replicateBp } n \text{ char})) \ xs && \{ \text{Eq. (2.2)} \} \\ \equiv & \text{proj } (\text{replicateBp } (\text{length } xs) \text{ char}) \ xs && \{ \text{Def. 2.4.3} \} \\ \equiv & \text{mapM } (\text{proj char}) \ xs && \{ \text{Eq. (2.3)} \} \\ \equiv & \text{mapM Just } xs && \{ \text{Eq. (2.1)} \} \\ \equiv & \text{Just } xs && \{ \text{Def. monad} \} \end{aligned}$$

Combining `proj string = Just` with Example 2.5.11 (`string` is weak backward round tripping) enables Theorem 2.5.12, proving that `string` is backward round tripping. $\square$

Thus, for any biparser p , we can get backward round tripping by proving that its atomic subcomponents are weak backward round tripping, and proving that $\text{purify } p\ x = \text{Just } x$. The interesting aspect of the purification condition here is that it renders irrelevant the domain-specific effects of the biparser, *i.e.*, those related to manipulating source strings. This considerably simplifies any proof. Furthermore, `purify` is a *PMP homomorphism*: it commutes with the *PMP* operations.

2. The Monadic Profunctor Paradigm of Bidirectional Programming

Definition 2.5.14. A **PMP** homomorphism between **PMP**s P and Q is a polymorphic function $\text{proj} :: P\ u\ v \rightarrow Q\ u\ v$ such that:

$$\begin{aligned} \text{proj} (\text{comap}_P\ f\ p) &\equiv \text{comap}_Q\ f\ (\text{proj}\ p) \\ \text{proj} (p \gg_P k) &\equiv (\text{proj}\ p) \gg_Q (\lambda x \rightarrow \text{proj}\ (k\ x)) \\ \text{proj} (\text{return}_P\ x) &\equiv \text{return}_Q\ x \end{aligned}$$

Proposition 2.5.15. The $\text{purify} :: Bp\ u\ v \rightarrow u \rightarrow \text{Maybe}\ v$ operation for biparsers (above) is a **PMP** homomorphism between Bp and the **PMP** $\text{PartialFun}\ u\ v = u \rightarrow \text{Maybe}\ v$.

The other examples in this paper also permit a definition of purify . We capture the general pattern as follows:

Definition 2.5.16. A *purifiable PMP* is a **PMP** P equipped with a homomorphism proj from P to the **PMP** of partial functions $- \rightarrow \text{Maybe}\ -$. We say that $\text{proj}\ p$ is the *pure projection* of p .

Definition 2.5.17. A pure projection $\text{proj}\ p :: u \rightarrow \text{Maybe}\ u$ is called the *identity projection* when $\text{proj}\ p\ x = \text{Just}\ x$ for all $x :: u$. We say that p *projects to the identity*.

Here and for the other examples, identity projections enable compositional round tripping properties to be derived from more general non-compositional properties, as seen above for backward round tripping of biparsers.

Sometimes, a computation is only expected to work on a subset of the values of a type. For example, biparsers in Section 2.3 may only parse a subset of values within the type and in §2.8 a generator for binary trees bst may generate or accept only the subset of binary trees which are *binary search* trees. In those cases, the pure projection is only a restriction of the identity function.

Definition 2.5.18. Let $p :: P\ u\ u$ and let v be a subset of u . We say that p *projects to the identity on* v if, for all $x \in v$, $\text{proj}\ p\ x = \text{Just}\ x$.

A related issue may arise when proving that a computation $p \gg k$ satisfies a compositional property $\mathcal{R}$. While Definition 2.5.5 requires $p \in \mathcal{R}_v^u$ and $\forall x. (k\ x) \in \mathcal{R}_w^u$, it might be that p only produces values in a subset of v and k only behaves well on that same subset. A possible solution is to use more precise types, choosing v so that $p :: P\ u\ v$ may in fact produce all values in v . Another solution is to index the property $\mathcal{R}$ by arbitrary subsets to describe *a posteriori* the set of elements actually produced by a computation.

2.5.2. Quasicompositionality for **PMP**s

We have neglected forward round tripping, which is not compositional, not even in a weakened form. However, we can generalise compositionality to enable forward round tripping. We call this generalised metaproperty *quasicompositionality*.

This generalisation of compositionality that admits a proof of forward round tripping depends on *injectivity*. An injective function $f : A \rightarrow B$ is a function for which there exists a left inverse $f^{-1} : B \rightarrow A$, so that $f^{-1} \circ f = \text{id}$, i.e., every element in A has a unique image in B by f . We can see this pair of functions as a simple kind of bidirectional program, with a forward round tripping property (assuming f is the forward direction). We can lift the notion of injectivity to the **PMP** setting and capture forward round tripping properties that are preserved by the **PMP** operations, given an additional injectivity-like restriction. We first formalise the notion of an *injective arrow*. An injective arrow $k :: v \rightarrow m\ w$ produces an output from which the input can be recalculated:

Definition 2.5.19. Let m be a monad. A function $k :: v \rightarrow m\ w$ is an *injective arrow* if there exists $k' :: w \rightarrow v$ (the *left arrow inverse* of k) such that for all $x :: v$:

$$k\ x \gg= \lambda y \rightarrow \text{return}\ (x, y) \quad \equiv \quad k\ x \gg= \lambda y \rightarrow \text{return}\ (k'\ y, y)$$

Next, we define *quasicompositionality* which extends the compositionality meta-property with the requirement for $(\gg=)$ to be applied to injective arrows:

Definition 2.5.20. Let P be a **PMP**. A property $\mathcal{R}_v^u \subseteq P\ u\ v$ indexed by types u and v is *quasicompositional* if the following holds, for all $x :: v$, $p :: P\ u\ v$, $k :: v \rightarrow P\ u\ w$, $f :: u' \rightarrow \text{Maybe}\ u$:

$$(\text{return}\ x) \in \mathcal{R}_v^u \quad (\text{qcomp-return})$$

if k is an injective arrow,

$$(p \in \mathcal{R}_v^u) \wedge (\forall x. (k\ x) \in \mathcal{R}_w^u) \implies (p \gg= k) \in \mathcal{R}_w^u \quad (\text{qcomp-bind})$$

$$p \in \mathcal{R}_v^{u'} \implies (\text{comap}\ f\ p) \in \mathcal{R}_w^u \quad (\text{qcomp-comap})$$

We now formulate a weakening of forward round tripping. As with weak backward round tripping, we rely on the idea that the printer *outputs* both a string and the value that was printed, so that we need to compare the outputs of both the parser and the printer, as opposed to comparing the output of the parser with the input of the printer as in (strong) forward round tripping. If running the parser component of a biparser on a string $s01$ yields a value y and a remaining string $s1$, and the printer outputs that same value y along with a string $s0$, then $s0$ is the prefix of $s01$ that was consumed by the parser, *i.e.*, $s01 = s0 \mathbin{++} s1$ for some $s1$.

Definition 2.5.21. A biparser $p:\text{Bp}\ u\ v$ is *weak forward round tripping* if the following holds for all $x :: u$, $y :: v$, and $s0, s1, s01 :: \text{String}$:

$$\text{parse}\ p\ s01 = \text{Just}\ (y, s1) \quad \wedge \quad \text{print}\ p\ x = \text{Just}\ (y, s0) \quad \implies \quad s01 = s0 \mathbin{++} s1$$

Proposition 2.5.22. Weak forward round tripping is quasicompositional.

Proof. We sketch the **qcomp-bind** case, where $p = (m \gg= k)$ for some m and k that are weak forward round tripping.

From $\text{parse}\ (m \gg= k)\ s01 = \text{Just}\ (y, s1)$, it follows that there exists z, s such that $\text{parse}\ m\ s01 = \text{Just}\ (z, s)$ and $\text{parse}\ (k\ z)\ s = \text{Just}\ (y, s1)$. Similarly, the second conjunct $\text{print}\ (m \gg= k)\ x = \text{Just}\ (y, s0)$ implies there exists $\{z', s0'\}$ such that $\text{print}\ m\ x = \text{Just}\ (z', s0')$ and $\text{print}\ (k\ z')\ x = \text{Just}\ (y, s1')$ and $s0 = s0' \mathbin{++} s1'$.

Because k is an injective arrow, we have $z = z'$. This comes from the following: recall we have: $\text{parse}\ (k\ z)\ s = \text{Just}\ (y, s1)$ and $\text{print}\ (k\ z')\ x = \text{Just}\ (y, s1')$, let k' be the left arrow inverse of k , then (from Definition 2.5.19):

$$(k\ z \gg= \lambda y \rightarrow \text{return}\ (z, y)) = (k\ z \gg= \lambda y \rightarrow \text{return}\ (k'\ y, y))$$

(and similarly with z'). Plugging both sides of this equation into “ $\text{parse}\ _ s$ ” and “ $\text{print}\ _ x$ ” respectively, and using the fact that print and parse are monad morphisms, followed by the above two equations, yields the following equalities:

$$\begin{aligned} \text{parse}\ (\dots)\ s &= \text{Just}\ ((z, y), s1) = \text{Just}\ ((k'\ y, y), s1) \\ \text{print}\ (\dots)\ x &= \text{Just}\ ((z', y), s1') = \text{Just}\ ((k'\ y, y), s1') \end{aligned}$$

2. The Monadic Profunctor Paradigm of Bidirectional Programming

Thus we can conclude (by injectivity of data constructors) that $z = (k' \ y) = z'$.

We then use the assumption that m and k are weak forward round tripping on m and on k a , and deduce that $s01 = s0' \ ++ \ s$ and $s = s1' \ ++ \ s1$. Therefore $s01 = s0' \ ++ \ (s1' \ ++ \ s1)$ which reassociates to $s01 = (s0' \ ++ \ s1') \ ++ \ s1$ which equals $s01 = s0 \ ++ \ s1$ (since $s0 = s0' \ ++ \ s1'$, shown above). $\square$

Proposition 2.5.23. The `char` biparser is weak forward round tripping, and `manyMaybe` preserves weak forward round tripping.

Corollary 2.5.24. `string` is weak forward round tripping (by Propositions 2.5.22 and 2.5.23)

Thus, quasicompositionality gives us scalable reasoning for weak forward round tripping, which is by construction for biparsers: we just need to prove this property for individual atomic biparsers and check that arrows are injective. Similarly to backward round tripping, we can prove forward round tripping by combining weak forward round tripping with the identity projection property:

Theorem 2.5.25. If $p :: P \ u \ u$ is weak forward round tripping, and for all $x :: u$, `purify p x = Just x`, then p is forward round tripping.

Corollary 2.5.26. The biparser `string` is forward round tripping by the above theorem (with identity projection shown in the proof of Example 2.5.13) and Corollary 2.5.24.

2.5.3. Reasoning Recap

In summary, for any bidirectional transformation, we can consider two round tripping properties: forward-then-backward and backward-then-forward, called just *forward* and *backward* here respectively. Whilst combinator-based approaches usually guarantee round tripping by construction, we must compromise to get greater expressivity in the monadic approach. To reason about bidirectional transformations in a manageable, scalable way, it is desirable for round tripping properties to be compositional. Unfortunately, due to the **PMP** structuring, this tends not to be the case. Instead, weakened round tripping properties can be compositional or quasicompositional (adding injectivity). We then recover the stronger property by proving a simple property on aligned transformations: that the backward direction faithfully reproduces its input as its output (*identity projection*). The next subsection, compares this reasoning approach to a proof of backward round tripping for separately implemented parsers and printers (not using our combined monadic approach).

2.5.4. For comparison: separately defined parser and printer, and round tripping proof

The following provides a comparison between the **PMP** biparser of Section 2.3 and the alternative without our approach: having to write two separate definitions of a parser and a printer. With these examples, we also compare our reasoning approach of Section 2.5 with having to manually prove round tripping on the separated definitions. The main points of comparison are:

- This parser has the same structure as the biparser in Section 2.3 (without any upon annotations).
- The standalone printer for this example is extremely simple, in line with a common pattern that most of the benefit of bidirectional programming is felt in the reduction of proof and maintenance effort.
- The auxiliary lemmas for `parseInt`, `parseDigits`, and `replicateM` correspond to the round tripping properties of their bidirectional counterparts (`int` and `replicateBP`).

- These lemmas must all manipulate source strings explicitly, whereas our framework uses compositionality to handle those details, which reduces the proof burden by modularisation.

2.6. Monadic bidirectional programming for picklers

The BiParser **PMP** can be tweaked to unlock a similar bidirectional domain. Exchanging **String** for **Endo String** using Cayley’s theorem [Jacobson 2012] yields a monadic bidirectional representation of picklers:

```
Fwd (StateT String Maybe) :: Bwd (WriterT      String Maybe)
Fwd (StateT String Maybe) :: Bwd (WriterT (Endo String) Maybe)
```

Picklers transform data to be stored or transmitted. They perform both the preparation of the data to be stored/sent (“serialisation” or “marshalling”) and the reversing of the process at the other end. Similarly to parsers, this domain converts between structured and unstructured data (we’ll use a **String** as our example), where just like for biparsers, we are working in the simplified setting where both directions succeed.

```
pickle  : v → String      unpickle : String → Maybe v
```

2.6.1. Construction

```
type BiPi = Fwd (StateT String Maybe) :: Bwd (WriterT (Endo String) Maybe)
```

The augmented BiParser **PMP** packages up pickler forward and backward operations, which can be made easily accessible with smart destructors:

```
unpick :: BiPi u v → (String → Maybe (v, String))
pick   :: BiPi u v → (u → Maybe (v, Endo String))
```

The forward direction is identical to parsing: unstructured **String** data is maybe decoded into the value that was pickled paired with the remainder of the input that was not used. In the backward direction, a value is pickled and returned paired with a series of **String** changes. These can be thought of as the pickled value, as that is what applying them to the empty string yields. Note that partiality has been artificially injected into the pickling operator to allow for the partial and contravariant **comap** operation of **PMPs**. The original pickler interface, modulo this partiality, can be recovered by applying the series of changes to the empty string, and ignoring the second argument of **unpick**:

```
pickle :: BiPi u v → u → Maybe String
pickle p value
  = let
      f :: (v, Endo String) → String
      f (v, Endo g) = g []
      in fmap f (pick p value)

unpickle :: BiPi u v → String → Maybe v
unpickle p s = fmap fst $ unpick p s
```

2.6.2. Pickler Combinators

Existing combinator libraries for picklers, such as “Pickler Combinators” [Kennedy 2004], have already inadvertently almost used **PMPs**, with operations and combinators that correspond to our framework. Kennedy actually only considers the simpler case where unpickling always succeeds, but since our framework makes it easy to add partiality to the forwards direction, we have done so.

They present a pickler as two functions packaged up together using record syntax:

```
data PU a s = PU {appP :: (a, s) → s, appU :: s → (a, s)}
```

Looking at these operations, appU is already the state monad, meaning that it can be lifted into our framework with the Fwd constructor. appP requires the Bwd construction, which nicely demonstrates why pickling necessitates a **PMP** structure, not just monadic one. appP is similar to the writer monad over endomorphisms $(a, s \rightarrow s)$, except for the fact that the s it produces can depend upon the inputted a . Combining this writer monad with the Bwd **PMP** construction recovers this: the contravariant parameter that we map from contains the covariant parameter, which in this case is a as desired. Thus we can arrive at our pickler construction from theirs.

Their combinators also featured ones that corresponded to the operations of **PMPs**:

```
lift :: a → BiPi u a
lift = return

sequ :: (b → a) → BiPi a a → (a → BiPi b b) → BiPi b b
sequ f pa k = comap (Just o f) pa >>= k
```

lift corresponds directly to return, while sequ corresponds to a bind where the backward comap annotation is enforced. The only difference is that in their restriction the backward annotation will always succeed. Kennedy notes that this forms a monad, missing only our key insight that the addition of profunctors creates the appropriate abstraction for monadic bidirectional programming.

Building from these two core combinators, it is not challenging to reimplement the rest, enabling direct porting of their examples.

As a combinator library, they guarantee round tripping laws by construction; something that in our framework is proved compositionally.

2.6.3. Round Tripping

Pickler correctness can be encapsulated into two laws:

Definition 2.6.1. A pickler $p :: \text{BiPi } u \ v$ is *backward round tripping* when

$$\text{unpickle } p \ u = \text{Just } u' \implies \text{pickle } p \ u' = u$$

Definition 2.6.2. A pickler $p :: \text{BiPi } u \ v$ is *forward round tripping* when

$$\text{pickle } p \ s = \text{Just } u \implies \text{unpickle } p \ u = \text{Just } s$$

Unpacking, then repacking something should not change it; and if a serialisation was successful, unpacking it should obtain the original value.

As with biparsers, these laws cannot be proven compositionally as they are: the backward direction must be broken down into a weakened form with a pure projection; and the forward must also be weakened then instead be proven using quasicompositionality. We will look at each in turn.

The backward round tripping law has to be weakened to support the monadic bind operation, removing reliance on a connection between the two type parameters (u and v). A connection which is recovered later via purification.

Definition 2.6.3. A pickler $p :: \text{BiPi } u \ v$ is *weak backward round tripping* when for all $x :: u$ and $s :: \text{String}$ and $f :: \text{String} \rightarrow \text{String}$, we have:

$$\text{pick } p \ x = \text{Just } (v, f) \implies \text{unpick } p \ (\text{appEndo } f \ s) = \text{Just } (v, s)$$

The connection between u and v is broken by having pick output v , representing the value it has pickled (x), but of the correct type. v is then used to verify the behaviour of unpick. This weakened property is similar to that of biparsers, where s is the remaining string, except this time `appEndo` is used instead of `++`. `appEndo :: Endo a → a → a` unpacks the `Endo` type, which is a **newtype** wrapper around endomorphisms in Haskell allowing them to have a monoid instance.

Theorem 2.6.4. Weak backward round tripping is a compositional property.

To recover full backward round tripping, all that remains is the definition of `proj` to show that `BiPi` is a purifiable **PMP**. Since all that has changed between biparsers and picklers is the writer's monoid, the definition uses the same structure.

Proposition 2.6.5. `BiPi` is a *purifiable PMP* (Definition 2.5.16), with `proj` defined as:

$$\begin{aligned} \text{proj} &:: \text{BiPi } u \ v \rightarrow (u \rightarrow \text{Maybe } v) \\ \text{proj } b \ u &= \text{fmap } \text{fst} \ (\text{pick } b \ u) \end{aligned}$$

Thus backward round tripping for picklers is compositional.

When it comes to the forward direction, the round tripping cannot be proven compositionally; instead quasicompositionally is needed. To prove quasicompositionality the round tripping law again needs to be weakened:

Definition 2.6.6. A pickler $p :: \text{BiPi } u \ v$ is *weak forward round tripping* when for all $s1 :: \text{String}$ and $s2 :: \text{String}$, we have:

$$\text{unpick } p \ (s1 ++ s2) = \text{Just } (x, s2) \implies \text{pickle } p \ x = \text{Just } s1$$

If some value is unpacked, leaving some remainder ($s2$), repacking that value should yield the prefix of the original string ($s1$).

Theorem 2.6.7. Weak forward round tripping is a quasicompositional property.

Thus, using our reasoning framework, round tripping for picklers can be proven compositionally.

2.7. Monadic bidirectional programming for lenses

Lenses are a common object of study in bidirectional programming, comprising a pair of functions (`get : S → V`, `put : V → S → S`) satisfying *well-behaved lens* laws that we introduced in Section 2.1:

$$\text{put } (\text{get } s) \ s = s \qquad \text{get } (\text{put } v \ s) = v$$

2.7.1. Construction

Previously, when considering the monadic structure of parsers and printers, the starting point was that parsers already have a well-known monadic structure. The challenge came in finding a reasonable monadic characterisation for printers that was compatible with the parser monad. In the end, this construction was expressed by a product of two PMPs `Fwd m` and `Bwd n` for monads `m` and `n`. For lenses we are in the same position: the forward direction, `get`, is already a monad—the reader monad. The backward direction, `put`, is not a monad since it is contravariant in its parameter; the same situation as printers. The same approach of “monadisation” used for parsers and printers yields the following new data type for lenses:

```
data L s u v = L {
  get :: s → v,
  put :: u → s → (v, s)}
```

The result of `put` is paired with a covariant parameter `v` (the result type of `get`) in the same way as monadic printers. Instead of mapping a view and a source to a source, `put` now maps values of a different type `u`, which we call a *pre-view*, along with a source `s` into a pair of a view `v` and source `s`. Note that this is essentially the state monad. This definition can be structured as a PMP via a pair of `Fwd` and `Bwd` constructions:

```
type L s = Fwd (Reader s) :∗ Bwd (State s)
```

Thus by the results of Section 2.4, we now have a PMP characterisation of lenses that allows us to compose lenses via the monadic interface.

Ideally, `get` and `put` should be total, but this is impossible without a way to restrict the domains. In particular, there is the known problem of “duplication” [Mu et al. 2004], where source data may appear more than once in the view, and a necessary condition for `put` to be well-behaved is that the duplicates remain equal amid view updates. This problem is inherent to all bidirectional transformations, and bidirectional languages have to rule out inconsistent updates of duplicates either statically [J. Nathan Foster et al. 2005] or dynamically [Mu et al. 2004]. To remedy this, we capture both partiality of `get` with `Maybe`, and a predicate `(s → Bool)` on sources in `put` for additional dynamic checking. This is provided by the following `Fwd` and `Bwd` PMPs:

```
type ReaderT r m a = r → m a
type StateT s m a = s → m (a, s)
type WriterT w m a = m (a, w)

type L s = Fwd (ReaderT s Maybe)
          :∗ Bwd (StateT s (WriterT (s → Bool) Maybe))
```

```
-- Smart constructor and destructors:
mkLens :: (s → Maybe v) → (u → s → Maybe ((v, s), s → Bool)) → L s u v
get :: L s u v → (s → Maybe v)
put :: L s u v → (u → s → Maybe ((v, s), s → Bool))
```

Going forward, *getting* a view `v` from a source `s` may fail if there is no view for the current source. Going backward, *putting* a pre-view `u` updates some source `s` (via the state transformer `StateT s`), but with some further structure returned, provided by `WriterT (s → Bool) Maybe` (similar to the writer

transformer used for biparsers, § 2.4.2, p. 19). The `Maybe` here captures the possibility that `put` can fail. The `WriterT (s → Bool)` structure provides a predicate which detects the “duplication” issue mentioned earlier. Informally, the predicate can be used to check that previously modified locations in the source are not modified again. For example, if a lens has a source made up of a bit vector, and a `put` sets bit i to 1, then the returned predicate will return `True` for all bit vectors where bit i is 1, and `False` otherwise. This predicate can then be used to test whether further `put` operations on the source have modified bit i .

Similarly to biparsers, a pre-view, u , can be understood as *containing* the view, v , that is to be merged with the source, and which is returned with the updated source. Ultimately, we wish to form lenses of matching input and output types (*i.e.*, $L\ s\ v\ v$) satisfying the standard lens well-behavedness laws, modulo explicit management of partiality via `Maybe` and testing for conflicts via the predicate:

$$\begin{aligned} \text{put } l\ x\ s = \text{Just } ((-, s'), p) \ \wedge \ p\ s' &\implies \text{get } l\ s' = \text{Just } x && \text{(Acceptability)} \\ \text{get } l\ s = \text{Just } x &\implies \text{put } l\ x\ s = \text{Just } ((-, s), -) && \text{(Consistency)} \end{aligned}$$

Acceptability and **Consistency** [Matsuda and M. Wang 2015] are backward and forward round tripping respectively, and collectively, the two laws define *well-behavedness* [Bancilhon and Spyratos 1981; J. Nathan Foster et al. 2005; Hegner 1990]. Some lenses, such as the later example, are not defined for all views. In that case we may say that the lens is backward/forward round tripping in some subset $P \subseteq u$ when the above properties only hold on elements x of P .

For every source type s , the lens type $L\ s$ is automatically a **PMP** by its definition as the pairing of **Fwd** and **Bwd** (Section 2.4.1), and the following instance of **Monoid** to satisfy the requirements of the writer monad, which is the $(\rightarrow)$ monoid composed with the **And** monoid:

```
instance Monoid (s → Bool) where
  mempty = λ_ → True
  mappend h j = λs → h s ∧ j s
```

2.7.2. A Small Example: Key-Value Maps

A simple lens example that operates on key-value maps. For keys of type `Key` and values of type `Value`, we have the following source type and a simple lens:

```
type Src = Map Key Value
atKey :: Key → L Src Value Value -- Key-focused lens
atKey k = mkLens (lookup k)
  (λv → λmap → Just ((v, insert k v map), λm' → lookup k m' ≡ Just v))
```

The `get` component of the `atKey` lens looks up the key k in a map, producing an optional `Value` if found. The `put` component inserts a value for key k . When the key already exists, `put` overwrites its associated value.

Thanks to our approach, multiple calls to `atKey` can be composed monadically, giving a lens that gets/sets multiple key-value pairs at once. The list of keys and the list of values are passed separately, and are expected to be the same length.

```
atKeys :: [Key] → L Src [Value] [Value]
atKeys [] = return []
atKeys (k:ks) = do
  x ← comap headM (atKey k)
```

2. The Monadic Profunctor Paradigm of Bidirectional Programming

```
xs ← comap tailM (atKeys ks)
return (x:xs)
```

2.7.3. A Bigger Example: Trees

For another example of programming with monadic lenses, we consider lenses over the following data type of binary trees labeled by integers.

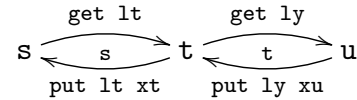
```
data Tree = Leaf | Node Tree Int Tree deriving Eq
```

In this example, our aim is to build a lens whose forward direction gets the right spine of the tree as a list of integers. The backward direction will then allow a tree to be updated with a new right spine (represented as a list of integers), which may produce a larger source tree.

We start by defining a classical lens combinator. Given a lens `lt` to view `s` as `t`, and a lens `ly` to view `t` as `u`, the combinator `(>>>)` creates a lens to view `s` as `u`. We illustrate and explain the composition on the right.

```
(>>>) :: L s t t → L t u u → L s u u
lt >>> ly = mkLens get' put' where
  -- get' :: s -> Maybe u
  get' s = get lt s >>= get ly
  -- put' :: u -> s ->
  -- Maybe ((u, s), s -> Bool)
  put' xu s = do
    t ← get lt s
    ((y, xt), q') ← put ly xu t
    ((-, s'), p') ← put lt xt s
    let check s' =
      p' s' ∧ any q' (get lt s')
    Just ((y, s'), check)
```

Illustration of the composition of lenses in `(>>>)`:



For individual lenses, the `put` action takes the source as its last parameter (shown above the lower arrows here). In the case of the composite lens, `put'` has a source of type `s`, thus we need to create an intermediate source of type `t` in order to use `put ly`. This intermediate source is provided by first using `get lt s`.

In the last three lines of `put'` in `(>>>)`, for the composite backward direction to succeed, the returned intermediate store `xt` must be consistent (free of conflict) as checked by `q'`.

We define two primitive lenses: `rootL` and `rightL` for the root and right child of a tree. The `rootL` lens accesses the label at the root if it is a `Node`, otherwise returning `Nothing`. Note that `Maybe` type used here is different to the use of `Maybe` inside the definition of `L`: internally `L` uses `Maybe` to represent failure, here at the top-level we are using it to explicitly indicate presence or absence of a label without failing.

```
rootL :: L Tree (Maybe Int) (Maybe Int)
rootL = mkLens getter putter
where
  getter (Node _ n _) = Just (Just n)
  getter Leaf = Just Nothing
  putter n' t = Just ((n', t'), p)
    where
      t' = case (t, n') of
```

```

    (←, Nothing) → Leaf
    (Leaf, Just n) → Node Leaf n Leaf
    (Node l _ r, Just n) → Node l n r
    p t'' = getter t' ≡ getter t''

```

The `rightL` lens accesses the right child of a tree. If the source tree is a `Leaf`, this lens fails rather than return a `Maybe` view. In both lenses `rootL` and `rightL`, the `put` operations return predicates that check that the view of a (possibly updated) source is equal to the view provided to `put`.

```

rightL :: L Tree Tree Tree
rightL = mkLens getter putter
  where
    getter (Node _ _ r) = Just r
    getter _ = Nothing
    putter r Leaf = Nothing
    putter r (Node l n _) = Just
      ((r, Node l n r),
       λt' → Just r ≡ getter t')

```

We compose these two primitive lenses monadically to define the `spineL` lens to view and update the right spine of a tree:

```

spineL :: L Tree [Int] [Int]
spineL = do
  hd ← comap (Just ∘ safeHead) rootL
  case hd of
    Nothing → return []
    Just n → do
      tl ← comap safeTail (rightL >>> spineL)
      return (n:tl)

```

Auxiliary functions `safeHead` and `safeTail` are defined:

```

safeHead :: [a] → Maybe a
safeHead (a:_) = Just a
safeHead [] = Nothing

safeTail :: [a] → Maybe [a]
safeTail (_:as) = Just as
safeTail [] = Nothing

```

The lens `spineL`, interpreted as a `get`, first views the root of the source tree through `rootL` as `hd`, and recurses if the root is a non-empty node (with label `n`), using `rightL` to shift the context. As a `put`, it updates the root using the head of the list, which is returned as the view `hd`, and continues with the same logic. To illustrate the action of this lens, consider a tree:

```

t0 = Node (Node Leaf 0 Leaf) 1 (Node Leaf 2 Leaf)

```

Getting the right spine (`get spineL t0`) yields the list `[1, 2]`. The tree spine can be updated to `[3, 4, 5]` yielding the following tree:

```

fmap fst (put spineL [3, 4, 5] t0)
= Just ([3, 4, 5], Node (Node Leaf 0 Leaf) 3
  (Node Leaf 4 (Node Leaf 5 Leaf)))

```

2.7.4. Round Tripping

To apply the reasoning framework of Section 2.5, to **Acceptability** and **Consistency** we must consider a couple of things to ensure compositionality. We first weaken backward round tripping (**Acceptability**) to be compositional. Here, our first consideration is that we can no longer rely on the lens being aligned. This means that we cannot compare the input to put with the output of get as they will have different types. Instead, we must make use of the output of put, which has the correct type. Next, we must consider the predicate that tests for conflicts under composition. By sequentially composing lenses such as in $l \gg k$, the output source s_1 of put l will be further modified by put $(k \ x)$, so this round tripping property must constrain all potential modifications of s_1 . In fact, the predicate p ensures exactly that the view get l has not changed and is still x . It is not even necessary to refer to the intermediate source s_1 , which is just one source for which we expect p to be **True**. These changes give us our weak backward round tripping:

Definition 2.7.1. A lens $l :: L \ s \ u \ v$ is *weak backward round tripping* if for all $x :: u$, $y :: v$, for all sources s, s' , and for all $p :: s \rightarrow \text{Bool}$, we have:

$$\text{put } l \ x \ s = \text{Just } ((y, s'), p) \wedge p \ s' \implies \text{get } l \ s' = \text{Just } y$$

Theorem 2.7.2. Weak backward round tripping is a compositional property.

Again, we complement this weakened version of round tripping with the notion of purification to recover the regular backward round tripping property.

Proposition 2.7.3. Our lens type L is a *purifiable PMP* (Definition 2.5.16), with a family of pure projections $\text{proj } s$ indexed by a source s , defined as:

$$\begin{aligned} \text{proj} &:: s \rightarrow L \ s \ u \ v \rightarrow (u \rightarrow \text{Maybe } v) \\ \text{proj } s &= \lambda l \ u \rightarrow \text{fmap } (\text{fst} \circ \text{fst}) (\text{put } l \ u \ s) \end{aligned}$$

Theorem 2.7.4. If a lens $l :: L \ s \ u \ u$ is weak backward round tripping and projects to the identity on some subset $P \subseteq u$ (i.e., $x \in P \implies \text{proj } s \ l \ x = \text{Just } x$, cf. Definition 2.5.18) then l is also backward round tripping on all $x \in P$.

Proof. Assume $\text{put } l \ x \ s = \text{Just } ((y, s'), p) \wedge p \ s' = \text{True}$. By the identity projection assumption, $x = y$. We can now apply weak backward round tripping to deduce $\text{get } l \ s' = \text{Just } x$. $\square$

To demonstrate, we apply this result to $\text{atKeys} :: [\text{Key}] \rightarrow L \ \text{Src} \ [\text{Value}] \ [\text{Value}]$.

Proposition 2.7.5. The lens $\text{atKey } k$ is weak backward round tripping.

Proof. Recall $\text{atKey} :: L \ \text{Src} \ \text{Value} \ \text{Value}$. Assuming the antecedent of backward round tripping, we get the following information:

$$\begin{aligned} \text{put } l \ x \ (\text{atKey } k) \ m &= \text{Just } ((x, \text{insert } k \ x \ m), \lambda m' \rightarrow \text{lookup } k \ m' \equiv \text{Just } x) \\ (\lambda m' \rightarrow \text{lookup } k \ m' \equiv \text{Just } x) \ s' &= \text{True} \end{aligned}$$

We then need to prove $\text{get } l \ s' = \text{Just } x$. By the definition of get:

$$\text{get } (\text{atKey } k) \ s' = \text{lookup } k \ s'$$

By the second conjunct of the antecedent we know $\text{lookup } k \ s' = \text{Just } x$, giving the required consequent. $\square$

Proposition 2.7.6. The lens `atKey k` projects to the identity: `proj z (atKey k) = Just`.

Proof. For all `s :: s`, following the definition we get:

$$\begin{aligned}
& \text{proj } s \text{ (atKey } k) \\
& \equiv \lambda u \rightarrow \text{fmap } (\text{fst} \circ \text{fst}) \text{ (put (atKey } k) \text{ } u \text{ } s) \\
& \equiv \lambda u \rightarrow \text{fmap } (\text{fst} \circ \text{fst}) \text{ (Just ((u, ...), ...))} \\
& \equiv \lambda u \rightarrow \text{Just } u
\end{aligned}$$

□

Our lens `atKeys ks` is therefore weak backward round tripping by construction. We now interpret/purify `atKeys ks` as a partial function, which is actually the identity function when restricted to lists of the same length as `ks`.

Proposition 2.7.7. For all `vs :: [Value]` such that `length vs = length ks`, and for all `s :: Src` then `proj s (atKeys ks) vs = Just vs`.

Corollary 2.7.8. By the above results, `atKeys ks :: L Src [Value] [Value]` for all `ks` is backward round tripping on lists of length `length ks`.

The other direction, forward round tripping (*Consistency*), follows a similar story. We first restate it as a quasicompositional property.

Definition 2.7.9. A lens `l :: L s u v` is *weak forward round tripping* if for all `x :: u`, `y :: v`, for all sources `s, s'`, and for all `p :: s → Bool`, we have:

$$\text{get } l \text{ } s = \text{Just } y \wedge \text{put } l \text{ } x \text{ } s = \text{Just } ((y, s'), _) \implies s = s'$$

Theorem 2.7.10. Weak forward round tripping is a quasicompositional property.

Along with identity projection, this gives the forward round tripping property.

Theorem 2.7.11. If a lens `l` is weak forward round tripping and projects to the identity on some subset `P` (i.e., `x ∈ P ⇒ proj s l x = Just x`) then `l` is also forward round tripping on `P`.

We can thus apply this result to our example (details omitted).

Proposition 2.7.12. For all `ks`, the lens `atKeys ks :: L Src [Value] [Value]` is forward round tripping on lists of length `length ks`.

2.8. Monadic bidirectional programming for generators

This section captures the novel notion of *bidirectional generators* (*bigenators*). These expand the applicability of bidirectional programming to random generators in property-based testing frameworks like *QuickCheck* [Claessen and Hughes 2000].

2.8.1. Construction

The forward direction generates values conforming to a specification; the backward direction checks whether values conform to a predicate. We capture the two together via our **PMP** pair as **G**:

```
type G = (Fwd Gen):*(Bwd Maybe)
-- ... with destructors and constructors
generate :: G u v → Gen v      -- forward direction
check :: G u v → u → Maybe v  -- backward direction
mkG      :: Gen v → (u → Maybe v) → G u v
```

The forward direction of the bigenerator is given by `generate` that returns a generator of type `Gen v`. The backward direction is given by `check` that returns a partial function of type `u → Maybe v`. The application `check g` defines a predicate on `u` that is true if and only if `check g u` returns `Just x` for some value `x`. This predicate can be extracted from the backward direction using the function `toPredicate`:

```
toPredicate :: G u v → u → Bool
toPredicate g x = isJust (check g x) where
  isJust (Just _) = True
  isJust Nothing = False
```

The bigenerator type **G** is automatically a **PMP** due to our construction (§2.4). Thus, monad and profunctor instances come for free, modulo (un)wrapping of constructors.

2.8.2. Examples

This section provides code examples for our notion of *bigenerators* (*bidirectional generators*) extending random generators in property-based testing frameworks like *QuickCheck* [Claessen and Hughes 2000] to a bidirectional setting.

We assume that we have been given a **Gen** monad of random generators (*e.g.*, as defined in the *QuickCheck* library for Haskell) and two primitive generators: `genBool :: Double → Gen Bool` generates a random boolean according to a Bernoulli distribution with a given parameter $p \in [0, 1]$; `choose :: (Int, Int) → Gen Int` generates a random integer uniformly in a given inclusive range $[\text{min}, \text{max}]$.

Generators for binary search trees We consider again the type of trees from the previous section. A *binary search tree* (BST) is a **Tree** whose nodes are in sorted order. Inductively, a BST is either a **Leaf**, or some **Node** `l n r` where `l` and `r` are both binary search trees, nodes in `l` have smaller values than `n`, and nodes in `r` have greater values than `n`.

As a working example, we are given some function `insert :: Tree → Int → Tree` which inserts an integer in a BST. We want to test the invariant that BSTs are mapped to BSTs, by *generating* a BST and an integer to apply the `insert` function, and *check* that the output is also a BST.

With the **Gen** monad, we can write a simple generator of BSTs recursively: given some bounds on the values of the nodes, if the bounds describe a nonempty interval, we flip a coin to decide whether to generate a leaf or a node, and if it is a node, we recursively generate binary search trees, following the inductive definition above. We can similarly write a *checker* for binary search trees as a predicate.

```

genBST :: Int → Int → Gen Tree
genBST lo hi | hi < lo = return Leaf
genBST lo hi = do
  isLeaf' ← genBool 0.5
  if isLeaf' then return Leaf
  else do n ← choose (lo, hi)
    l ← genBST lo (n - 1)
    r ← genBST (n + 1) hi
    return (Node l n r)

```

```

checkBST :: Int → Int → Tree → Bool
checkBST lo hi Leaf = True
checkBST lo hi (Node l n r) =
  lo ≤ n
  ∧ n ≤ hi
  ∧ checkBST lo n l
  ∧ checkBST n hi r

```

Bigenerator A generator of values v and a predicate on v (modelled by $v \rightarrow \text{Bool}$) together define a bidirectional generator with the same pre-view and view type, provided here by a smart constructor: `mkAlignedG`:

```

mkAlignedG :: Gen v → (v → Bool) → G v v
mkAlignedG gen check = mkG gen (λy → if check y then Just y else Nothing)

```

The generator `genBST` above depends on two primitives: `genBool` to decide whether to generate a leaf or a node, and `choose` to generate a value for a node. We wrap them as the following bidirectional generators, `bool` and `inRange`. As predicates, `bool` makes no assertion, `inRange` checks that the input integer is within the given range.

```

bool :: Double → G Bool Bool
bool p = mkAlignedG
  (genBool p)
  (λ_ → True)

inRange :: (Int, Int) → G Int Int
inRange (min, max) = mkAlignedG
  (choose (min, max))
  (λx → min ≤ x ∧ x ≤ max)

```

We consider again a type of labelled trees, with some field accessors. On the bottom right, `leaf` is a simple bigenerator for leaves.

```

data Tree = Leaf | Node Tree Int Tree

```

```

nodeValue :: Tree → Maybe Int
nodeValue (Node _ n _) = Just n
nodeValue _ = Nothing

```

```

nodeLeft :: Tree → Maybe Tree
nodeLeft (Node l _ _) = Just l
nodeLeft _ = Nothing

```

```

nodeRight :: Tree → Maybe Tree
nodeRight (Node _ _ r) = Just r
nodeRight _ = Nothing

```

```

isLeaf :: Tree → Bool
isLeaf Leaf = True
isLeaf (Node _ _ _) = False

```

```

leaf :: G Tree Tree
leaf = mkAlignedG (return Leaf) isLeaf

```

2. The Monadic Profunctor Paradigm of Bidirectional Programming

We then define a specification of binary search trees (bst below). A corresponding generator and predicate are extracted on the right from this bigenerator:

```

bst :: (Int, Int) → G Tree Tree
bst (min, max) | min > max = leaf
bst (min, max) = do
  isLeaf' ← comap (Just ∘ isLeaf) (bool 0.5)
  if isLeaf' then return Leaf
  else do
    n ← comap nodeValue (inRange (min, max))
    l ← comap nodeLeft (bst (min, n - 1))
    r ← comap nodeRight (bst (n + 1, max))
    return (Node l n r)

genBST :: Gen Tree
genBST =
  generate (bst (0, 20))

checkBST :: Tree → Bool
checkBST =
  toPredicate (bst (0, 20))

```

As a predicate, bst first checks whether the root is a leaf (isLeaf); returning a boolean allows us to reuse the same case expression as for the generator. If it is a node, we check that the value is within the given range and then recursively check the subtrees.

2.8.3. Round tripping

A random generator can be interpreted as the set of values it may generate, while a predicate represents the set of values satisfying it. For a bigenerator g , we write $x \in \text{generate } g$ when x is a possible output of the generator. The generator and the predicate of a bigenerator g must match. This requirement equates to round tripping properties: a bigenerator is *sound* if every value which it can generate satisfies the predicate (forward round tripping); a bigenerator is *complete* if every value which satisfies the predicate can be generated (backward round tripping). Completeness is often more important than soundness in testing because unsound tests can be filtered out by a predicate (e.g. a Haskell function `isBST :: Tree → Bool`), but completeness determines the potential adequacy of testing, a generator should be able to generate all values in the test space. In other words, overfitting the test space can be fixed by filtering, but underfitting is an issue.

Definition 2.8.1. A bigenerator $g :: G \ u \ u$ is *complete* (backward round tripping) when $\text{toPredicate } g \ x = \text{True}$ implies $x \in \text{generate } g$.

Definition 2.8.2. A bigenerator $g :: G \ u \ u$ is *sound* (forward round tripping) if for all $x :: u$, $x \in \text{generate } g$ implies that $\text{toPredicate } g \ x = \text{True}$.

Similarly to backward round tripping of biparsers and lenses, completeness can be split into a compositional weak completeness and a purification property.

As before, the compositional weakening of completeness relates the forward and backward components by their outputs, which have the same type.

Definition 2.8.3. A bigenerator $g :: G \ u \ v$ is *weak-complete* when

$$\text{check } g \ x = \text{Just } y \implies y \in \text{generate } g.$$

Theorem 2.8.4. Weak completeness is compositional.

In a separate step, we connect the input of the backward direction, *i.e.*, the checker, by reasoning directly about its pure projection (via a more general form of identity projection) which is defined to be the checker itself:

Theorem 2.8.5. A bigenerator $g :: G \ u \ u$ is complete if it is weak-complete and its checker satisfies a pure projection property: $\text{check } g \ x = \text{Just } x' \Rightarrow x = x'$ (that is, if a succesful bigenerator g “projects to the identity” in the terminology of Definition 2.5.17).

Thus to prove completeness of a bigenerator $g :: G \ u \ u$, we first have weak-completeness by construction, and we can then show that $\text{check } g$ is a restriction of the identity function, interpreting all bigenerators simply as partial functions.

Considering the other direction, soundness, there is no decomposition of soundness into a quasicompositional property and the pure projection property. The following (counter)example shows why:

Example 2.8.6. Let $\text{bool} :: G \ \text{Bool} \ \text{Bool}$ be a random uniform bigenerator of booleans, then consider two contextualisations of bool :

$$\text{comap } \text{isTrue } \text{bool} \tag{2.4}$$

$$\text{comap } \text{isTrue } (\text{return } \text{True}) \tag{2.5}$$

where $\text{isTrue } \text{True} = \text{Just } \text{True}$ and $\text{isTrue } \text{False} = \text{Nothing}$.

Both the above satisfy any quasicompositional property satisfied by bool , and both have the same pure projection isTrue , and yet (2.4) is unsound—in the forwards direction it can generate False , which is rejected by isTrue , returning Nothing —whilst (2.5) is sound.

To reason about soundness, we must look beyond quasicompositionality. All of the monadic bidirectional programs presented in this paper follow a regular structure: every $(\gg=)$ is associated with a comap , and the function argument of comap is the sagittal inverse of the function argument of $(\gg=)$. In other words, we use $(\gg=)$ to apply mappings in the “forward” direction, and comap to apply mappings “backwards”, and these mappings should be inverse of each other. In the previous sections, we used the notion of pure projection to express this inverse relation, and the counterexample above shows that that approach is not suitable to reason about completeness of bigenerators. Instead, we introduce the following notion of *idiomatic compositionality* which constrains $(\gg=)$ and comap to use an injective function and its inverse.

Definition 2.8.7. Let P be a **PMP**. A property $\mathcal{R}_u \subseteq P \ u \ u$ indexed by a type u is *idiomatically compositional* if the following holds, for all $x :: u$, $p :: P \ u \ u$, $k :: u \rightarrow P \ v \ v$, $f :: v \rightarrow \text{Maybe } u$:

$$\mathcal{R}_u(\text{return } x) \tag{icomp-return}$$

if k is an injective arrow with inverse f ,

$$(\mathcal{R}_u p) \wedge (\forall x. \mathcal{R}_v(k \ x)) \implies \mathcal{R}_v(\text{comap } f \ p \gg= k) \tag{icomp-bind}$$

Like quasicompositionality, it requires $(\gg=)$ to be applied to an injective arrow k , but idiomatic compositionality adds the constraint that the inverse of k appears as the argument of the corresponding comap .

Theorem 2.8.8. Quasicompositionality implies idiomatic compositionality.

One notable difference is that idiomatic compositionality only considers computations of type $P \ u \ u$, where the “input” type and the “output” type are the same u . That allows the property $\mathcal{R}$ to encode a notion of round trip between those types, which was previously the role of the pure projection property. In contrast, compositionality and quasicompositionality applied to properties of computations of type $P \ u \ v$, where the “input” type u and the “output” type v may vary independently, making such a notion of round trip inexpressible.

We can formulate a stronger variant of soundness that is idiomatically compositional.

2. The Monadic Profunctor Paradigm of Bidirectional Programming

Definition 2.8.9. A bigenerator $g :: \mathbf{G} \ u \ u$ is *strongly sound* (strongly forward round tripping) if for all $x :: u$, $x \in \text{generate } g$ implies that $\text{check } g \ x = \text{Just } x$.

Whereas soundness relates the generator $\text{generate } g$ to the predicate $\text{toPredicate } g$, strong soundness relates the generator to the *partial function* $\text{check } g$ from which the predicate is derived. The partial function $\text{check } g$ must behave as the identity function (**Just**) on the domain of the generator $\text{generate } g$.

Theorem 2.8.10. Strong soundness implies soundness.

Theorem 2.8.11. Strong soundness is idiomatically compositional.

Idiomatic compositionality means that proving that a bigenerator satisfies strong soundness amounts to checking that all uses of $(\gg=)$ are paired with comap with a semi-inverse relation between their function arguments, and that the primitives are strongly sound.

Lemma 2.8.12. `bool` and `inRange` are strongly sound.

Corollary 2.8.13. `bst` is strongly sound.

Hence, `bst` is a fully round tripping generator, *i.e.*, sound and complete.

Reflective Generators Reflective generators (Chapter 3) build upon our idea of bigenerators, combining them with free generators [Goldstein and Pierce 2022] and getting more out of the backward direction. Free generators consider generators to be parsers of randomness, which can be complemented well by a backward direction that unpacks the choices that led to a value. Thus combining free generators with PMPs gives rise to many appealing applications including shrinking and mutation.

2.9. Monadic Bidirectional Logic Programming

Bigenerators open the door to a whole new world of bidirectional fun. Where reflective generators play with the backward direction, we now explore the forward, as there is no need to limit it to just the **Gen** monad. Why not substitute in lists and play with non-determinism and backtracking, entering the realm of logic programming? For soundness and completeness, any monad that behaves like a set will do, and that is the core idea that helps explore the limits of our new notion of bigenerators.

2.9.1. Construction

We generalise the type **G** (shown below for comparison) from **Gen** in the forward direction to any monad, yielding the **GG** data type:

```
type G      u v = (Fwd Gen :: Bwd Maybe) u v
type GG m u v = Monad m => (Fwd m  :: Bwd Maybe) u v
```

The constructors and destructors also remain the same, just with a more general type:

```
-- constructors:
mkGG :: Monad m => m v -> (u -> Maybe v) -> GG m u v
-- destructors:
generate :: Monad m => GG m u v -> m v
```

```

check      :: Monad m => GG m u v -> u -> Maybe v
toPredicate :: Monad m => GG m u v -> u -> Bool

```

Similar to bigenerators, we provide a smart constructor which accepts a predicate ($v \rightarrow \text{Bool}$) which is then internally turned into a function of type ($v \rightarrow \text{Maybe } v$) for `mkGG`:

```

mkAlignedGG :: Monad m => m v -> (v -> Bool) -> GG m v v
mkAlignedGG gen chk = mkGG gen (\y -> if chk y then Just y else Nothing)

```

We refer to these as *aligned generalised bigenerators* as the view and pre-view types are the same.

Intuitively, the forward direction (`generate`) can still be understood as generating values fulfilling a predicate, and the backward direction (`check`) as a checker for that predicate. The only change is how the generated values are collected, notions of correctness are the same. It is still desirable for generalised generators to be *sound* and *complete*. In proving these, the same concepts as before arise: completeness can be shown through weak completeness and a pure projection, and soundness must be strengthened and proven using idiomatic compositionality.

The bigenerator of Section 2.8 can be replicated trivially by specialising the monad in the above construction to `Gen`. We instead explore the expressive power of programs written in this generalised bigenerator form, capturing logic programming problems involving search. Section 2.10 shows that a generalisation to the `PartialProfunctor` notion allows further power for logic programming.

2.9.2. Choice and Logic Programming

The `MonadPlus` type class provides an interface generalising non-deterministic computation:

```

class Monad m => MonadPlus m where
  mzero :: m a
  mplus :: m a -> m a -> m a

```

In this context, the monad operation `return :: a -> m a` represents the creation of a deterministic computation, with a single choice, `mzero` represents an empty choice (perhaps a kind of failure) and `mplus` captures a non-deterministic choice. From this interface, examples from logic programming can be captured by keeping the monad general, but restricted to `MonadPlus` instances, so that the search strategy can be later selected by specialising the monad in the fashion of “Transforming Functional Logic Programs into Monadic Functional Programs” [Braßel et al. 2010].

By specialising `GG` to monads that are instances of `MonadPlus` we can replicate the examples of Braßel et al. [2010] but generalised to *bidirectional* logic programs. For example, a non-deterministic choice of a boolean can be implemented as an atomic `GG` computation:

```

trueOrFalse :: MonadPlus m => GG m Bool Bool
trueOrFalse = mkGG
  (mplus (pure True) (pure False))
  (const (Just True))

```

The forward ‘generate’ direction takes advantage of the `MonadPlus` interface to lift the two options into the monad via `mplus`. This is the approach taken by Braßel et al. [2010], motivated by expressing the program `True ? False` from the functional logic language Curry [Hanus et al. 1995]. The backwards ‘check’ direction needs to take a `Bool` value and assert that it is either `True` or `False`. Since this is something that is always the case due to the Haskell type system and definition of `Bool`, the check direction can be trivially defined as always returning `Just True`.

2. The Monadic Profunctor Paradigm of Bidirectional Programming

Our monadic bidirectional interface can then be leveraged to compose `trueOrFalse` into more complex examples. For example, to generate/classify lists with a single boolean, we can write:

```
oneBool :: MonadPlus m => GG m [Bool] [Bool]
oneBool = do
  b <- comap singleton trueOrFalse
  return [b]
```

Here, `return` wraps the resulting boolean into a singleton list, and `comap` adjusts its context from `Bool` to `[Bool]` using `singleton :: [a] → Maybe a` to classify singleton list solutions, which returns a `Just` when there is only one element in the list, wrapping that single element.

We build from here to more complex logic programming problems involving search.

N-queens The classic N -queens puzzle involves placing N queen pieces on a chessboard such that they cannot take each other. For example, placing two queens next to each other (horizontally, vertically, or diagonally) is invalid, but placing one a knight's move away from the other is valid. The N -queens problem lends itself well to this framework, showcasing how the monadic style allows bidirectional solutions to be written in idiomatic Haskell, featuring pattern matching and recursion, and without committing to the choice of underlying search monad.

We define the top level as `nQueens` below using the monadic interface, `MonadPlus`, and `MonadFail`:

<pre>nQueens :: (MonadPlus m, MonadFail m) => Int → GG m [Pos] [Pos] nQueens 0 = return [] nQueens n n > 8 = reject n < 0 = reject otherwise = do xs <- comap safeTail (nQueens (n - 1)) x <- comap safeHead (safePlacing xs) return (x:xs)</pre>	<pre>-- Chess positions as integer pairs type Pos = (Int, Int) reject :: MonadFail m => GG m a a reject = mkAlignedGG (fail "No") -- No results (const False) -- Always fail</pre>
--	---

When there are no queens to place (first case), the job is easy: `nQueens` can just return the empty list. We also reject more than eight queens (they won't fit on the board in a civilised manner), or a negative number of queens, handled by the pattern guards in the next case. When it comes to the inductive case (otherwise branch), recursion can calculate the solution to $(n - 1)$ -queens, reducing the problem to calculating where the next queen can be placed by `safePlacing` (below), where non-monadic functions are lifted to the `GG` type using `mkAlignedGG`:

```

safePlacing :: MonadPlus m
  ⇒ [Pos] -- Existing queens positions
  → GG m Pos Pos
safePlacing qs =
  mkAlignedGG
    (choose $ filter noConflicts boardPos)
    (λp → isBoardPos p ∧ noConflicts p)
  where
    -- Checks for conflict with existing queens
    noConflicts p = all (¬ ∘ conflict p) qs

choose :: MonadPlus m ⇒ [a] → m a
choose =
  foldr (λx → mplus (return x)) mzero

```

```

boardPos :: [Pos]
boardPos = [(x, y) | x ← [0..7]
                  , y ← [0..7]]

isBoardPos :: Pos → Bool
isBoardPos (x, y) = 0 ≤ x ∧ x ≤ 7
                  ∧ 0 ≤ y ∧ y ≤ 7

conflict :: Pos → Pos → Bool
conflict (x1, y1) (x2, y2)
  -- horiz. conflict
  | y1 ≡ y2 = True

  -- vert. conflict
  | x1 ≡ x2 = True

  -- diagonal conflict
  | otherwise =
    abs (x1 - x2) ≡ abs (y1 - y2)

```

The `safePlacing` bigenerator, in the check direction, checks whether the position fits on the chessboard and does not conflict with queens already on the board (`noConflicts`). The generate direction filters all possible positions (`boardPos`) for those which are conflict-free and then uses `choose`, to abstract this list to `MonadPlus` (providing operations `mplus` and `mzero`) to choose between these possibilities.

One reasonable specialisation of the monad here is to lists, allowing the solutions to be enumerated in ‘row order’. For example:

```

allSolutions :: [[Pos]]
allSolutions = generate (nQueens 8)

checkOneSolution :: Maybe [Pos]
checkOneSolution = check (nQueens 8 :: GG [] [Pos] [Pos])
  [(2, 7), (3, 2), (6, 1), (7, 4), (4, 6), (5, 3), (1, 5), (0, 0)] -- one solution

```

The N -queens problem can be solved in this way (by induction on the problem size), because N -queens is *antitone* with respect to the check function and a subproblem relation. That is, a potential solution will only be valid (according to `check`) if its subsolution is valid:

$$\frac{x \text{ subproblem of } y}{\text{check } y \implies \text{check } x}$$

This means that the 2-queens solution must be correct to form a correct 3-queens solution, and so on. This makes the solution straightforward to build by monadic sequential composition, because the solution can be formulated inductively by (monadically) composing (via the **do** notation) the subproblem solution with a way to find the current solution, as seen above in `nQueens`.

2. The Monadic Profunctor Paradigm of Bidirectional Programming

However, there are many problems where the relationship between subproblems and predicates is not antitone. To solve this, we introduce the notion of ‘recoverability’ where a configuration which is a non-solution can be ‘recovered’ into a correct solution, providing a bidirectional dual to backtracking. We demonstrate this via the traditional *river crossing problem*.

River Crossing On one side of a river is a fox, a chicken, some grain, and a human who must get everyone across the river by boat. However, there is a catch: the boat is only big enough for two, one of which must be the human to steer, and the fox cannot be left alone with the chicken, nor the chicken be left alone with the grain, lest someone gets eaten.

A little more work is required to solve this problem, because unlike *N*-queens, a bad state can be recovered from by adding extra information. For example, if there is a bank with the chicken and the fox, which does not look good for the chicken, this can be recovered by adding the human to keep the peace. In contrast, if two queens are in conflict, no addition to the board can resolve it.

To implement a solution to this puzzle, the backwards monad needs to support two levels of failure: *recoverable* and *irrecoverable*. Recoverable failures are like the example with the chicken and the fox. In their current state, they will fail, but this can be remedied. Recoverability is represented using the **State** monad, with a boolean indicating whether the current configuration is *unsafe*, meaning that either the fox is left unattended with the chicken, or the chicken with the grain. Both situations may be remediated by adding a human to the configuration. Irrecoverable failures are when the rules of the puzzle are violated, e.g., three characters on the boat. These are represented, as before, with **Maybe**. Adding the new **State** layer for recoverable failure creates *generalised generators with recoverability* (GGRs):

```
newtype GGR m u v = GGR ((:*) (Fwd m) (Bwd (StateT Bool Maybe)) u v)
```

The forward (generate) direction is still a general monad, but the backward (check) direction now has an extra layer of recoverable failure using the **StateT** monad transformer over **Maybe**, giving this monad the ability to keep track of the current validity of a solution as state and short circuit to **Nothing** if the rules of the puzzle are broken (irrecoverable failure).

Setting the monad *m* to lists can be used to solve the river crossing puzzle, but leads to an explosion of solution permutations as the riverbank `[Human, Chicken]` is considered different to `[Chicken, Human]`. In this puzzle, only who is on the bank matters, not their order, suggesting the need for a data type that more closely corresponds to the desired puzzle semantics. The order-agnostic cousin of the list monad, `Set`, seems like the natural solution. However, in Haskell, this leads to an implementational detour because `Set` is not a Haskell monad (its type constraint `Ord` gets in the way of creating a `Monad` instance; essentially `Set` is a monad only on a subcategory of types with an ordering). Luckily, previous work on *relative monads* [Altenkirch et al. 2010] and their implementation in Haskell [Orchard and Mycroft 2012] addresses this problem, allowing for an implementation using `Set`. We elide some of the details here which entails using the `RebindableSyntax` extension to allow the `do`-notation in Haskell to use a relative monad type class `RMonad` instead of the standard `Monad` type class; the programming pattern remains the same.

In terms of scale, solving the river crossing puzzle is a step up from *N*-queens. The style of programming remains the same, there is still a combination of atomic and combined **PMPs**, with the former lifting Haskell functions to perform the puzzle logic, and the latter combining the atomic results into the correct form of solution. We capture the problem with a data type `Actor` of the characters in the problem, a relation compatible of which actors can be safely on the same riverside, and a designation of the riverside by `Side` which can be flipped by rowing:

```
data Actor = Human | Fox | Chicken | Grain
deriving (Show, Eq, Ord)
```

```
compatible :: Actor → Actor → Bool
compatible Fox    Chicken = False
compatible Chicken Grain = False
compatible Chicken Fox    = False
compatible Grain  Chicken = False
compatible _      _        = True
```

```
data Side = L | R
deriving (Show, Eq, Ord)
```

```
row :: Side → Side
row L = R
row R = L
```

```
characters :: Set Actor
characters = S.fromList
           [Human, Chicken, Fox, Grain]
```

Given a set of actors $\mathcal{S}$, we can generate and classify elements of the equivalence class of compatible characters in $\mathcal{S}$ (as defined by `compatible`) as a primitive **GGR** computation constructed via `mkRecoverable :: RMonad m ⇒ m v → (v → Bool) → GGR m v v` (a smart constructor similar to that of bigenerators used before): function:

```
compat :: Set Actor → GGR Set Actor Actor
compat xs = mkRecoverable gen check
where
  check c = (all (compatible c) xs ∨ Human ∈ xs) ∧ ¬ (elem c xs)
  gen | Human ∈ xs = characters S.\\ xs -- add anyone new
    | otherwise = -- add someone non-conflicting
      foldl (λn e → S.filter (compatible e) n) (characters S.\\ xs) xs
```

This is the analogue of `safePlacing` from `nQueens`. We can compose `compat` iteratively to generate and classify sets of actors that are safe together on a single side via `safeSide` which resembles closely the logical structure of the `nQueens` function which similarly generated valid states:

```
safeSide :: Int → GGR Set (Set Actor) (Set Actor)
safeSide 0 = return S.empty
safeSide n -- number of actors on the side
  | n > 4 = reject
  | n < 0 = reject
  | otherwise = do
    xs ← comap tailS (safeSide (n - 1))
    x  ← comap headS (compat xs)
    return (S.insert x xs)
```

Just like `nQueens`, `safeSide` branches on how many actors are required (the first parameter). When there are no actors, it can simply return the empty set. If there are negative actors or more than four, then `safeSide` results in an irrecoverable failure via `reject` (the generate direction of which is just the emptyset `S.empty` but the check direction denotes an irrecoverable failure via `Nothing`). Given a valid size, `safeSide` then inductively creates its solution in the same way as `nQueens`, using `Set` versions of `(:)`, `head`, and `tail`, and using the primitive `compat` for the next compatible element. The inductive step `comap tailS (safeSide (n - 1))` may return an invalid state (e.g., if the human is not present on the side), marked with `False`. However this invalidity is recoverable in the next line which adds a human back in.

2. The Monadic Profunctor Paradigm of Bidirectional Programming

From `safeSide` we can generate states of the entire puzzle (not just a single side of the river), captured by the `PuzzleState` data type:

```
data PuzzleState = PuzzleState
  { leftBank      :: Set Actor
  , boat          :: Set Actor
  , rightBank     :: Set Actor
  , boatLocation  :: Side
  } deriving (Show, Eq, Ord)
```

A transition function `reachableStates :: PuzzleState → Set PuzzleState` (elided for brevity) takes a puzzle state and computes the set of next possible states, these will satisfy the rules of the puzzle (e.g. no more than three on the boat), but may lead to a failing solution (e.g. someone getting eaten). From `reachableStates` we can make a simple bidirectional computation that generates reachable states or classifies those in a set as to whether they follow the rules of the puzzle (where violation is captured by the `Nothing` in the backward direction):

```
reachableState :: PuzzleState → GGR Set PuzzleState PuzzleState
reachableState current
  = mkIrrecoverable
    (reachableStates current)
    (∈ reachableStates current)
```

We can now create the logic bigenerater `safeState` for safe states that obey the rules of the puzzle and won't lead to a failing solution, by using the **do**-notation interface to cross-reference reachable states generated by `reachableStates` with `safeSide`'s notion of what a safe side is:

```
safeState :: PuzzleState → GGR Set PuzzleState PuzzleState
safeState current = do
  (PuzzleState l b r s) ← reachableState current
  l' ← comap (Just ∘ leftBank) (safeSide (S.size l))
  r' ← comap (Just ∘ rightBank) (safeSide (S.size r))
  if l ≡ l' ∧ r ≡ r'
  then return (PuzzleState l b r s)
  else mzeroR
```

For each of these reachable states generated by `reachableState`, `safeState` ensures that the left `l` and right riverbanks `r` are in the set of safe banks given by `safeSide`. Upon a valid and safe state, the state is added, otherwise the program moves on. The execution of the reasoning is achieved by using a `MonadPlus` like interface, adapted for relative monads like `Set`.

With a means to generate the next puzzle state from a current one, all that remains is gathering a sequence of such valid steps into a solution for the river crossing puzzle. Defined as `riverCrossing` below, it first provides the initial state (everyone on the left bank with the boat) and iteratively uses and collects the results of `safeState` until the end state (everyone on the other bank) is reached. Our solution additionally prevents the revisiting of states to avoid solutions that loop:

```
riverCrossing :: GGR Set [PuzzleState] [PuzzleState]
riverCrossing = riverCrossing' S.empty startState
where
  riverCrossing'
```

```

:: Set PuzzleState -- states that have been
→ PuzzleState      -- current state
→ GGR Set [PuzzleState] [PuzzleState]
riverCrossing' ss s = do
  x ← comap headM (safeState s)
  case complete x of
    True  → return [x]
    False → case x 'S.member' (S.insert s ss) of
      True  → mzeroR -- we are not allowing repeated states
      False → do
        xs ← comap tailM (riverCrossing' (S.insert s ss) x)
        return (x:xs)

```

A version of `MonadPlus` for the relative monads interface (with operation `mzeroR`) is used to cull failed solution branches, and the expressive `do`-notation allows for easy composition of the correct solution, using pure Haskell functions to perform puzzle logic, such as establishing whether the solution is complete.

Evaluating `(generate riverCrossing)` produces a set containing both non-looping solutions to the puzzle: one where the fox gets taken before the grain, and one where the grain gets taken before the fox, always with the chicken sticking with the human to avoid it getting eaten/eating. These correct solutions can also be verified in the other direction.

The river crossing puzzle can be characterised as antitone with respect to recoverability.

$$\frac{x \text{ subproblem of } y}{\text{Recoverable } y \implies \text{Recoverable } x}$$

When x is a subproblem of y , if y can be recovered (e.g. a bank containing all the animals), then it must have been the case that x was recoverable (in either case the human can be added). However if y is not recoverable (e.g. having three characters on the boat) we do not know if x was recoverable: additions that created y from x could have invalidated the solution (adding third characters to the boat), or things could already have gone wrong (two animals were on the boat).

The notation of recoverability complicates proof of the round tripping properties for this example. However, we can still achieve a notion of completeness as long as the extra layer of failure is accounted for. A naïve first attempt adapts weak completeness of bigenerators `BiGen` (replayed as (2.6) below) to the new interface for recoverability, labelled as (2.7):

$$(\text{check } g \ x = \text{Just } y) \implies y \in \text{generate } g \quad (2.6)$$

$$(\text{runStateT } (\text{check } g \ x) \ \text{True} = \text{Just } (y, \text{True})) \implies y \in \text{generate } g \quad (2.7)$$

If the forward monad is restricted to be `Foldable` (allowing the use of `∈`), we have a well-typed generalised bigenerator, which may be compositional, but does not properly account for the state, opening the door for a counterexample to slip in. The counterexample arises because the antecedent of completeness above asserts that the state must be `True` after running g . A `GGR` that is not complete can be created that satisfies the round trip law by simply ending with a state of `False` and contradicting the antecedent. To combat this, the check that the state ends as `True`, indicating that no recoverable failures have happened, has been moved to the consequent of the law:

$$(\text{runStateT } (\text{check } g \ x) \ \text{True} = \text{Just } (y, \text{ult})) \implies \text{ult} \wedge y \in \text{generate } g$$

This ensures that the round trip cannot be upheld vacuously, but has the downside that it doesn't provide any insight into what a failing computation should do. Frustratingly, this prevents the proof

2. The Monadic Profunctor Paradigm of Bidirectional Programming

that river crossing upholds this law from being given compositionally, as some of river crossing’s atomic components, such as `compat`, need to be able to fail in a recoverable manner. River crossing can still be proven correct via other methods (and testing suggests that it is), but there is certainly more to explore in this new recoverable frontier.

In summary, while this puzzle is still solvable in our framework, the **PMP** and round tripping laws are complicated by recoverability, which undermines compositionality.

2.10. Generalising partial profunctors to right monadic actions

A useful generalisation to the key structures of our framework presents itself in the application to logic programming. Recall that, so far, our **PMPs** also have the `PartialProfunctor` structure providing `internaliseMaybe :: p u v → p (Maybe u) v`. This yields a contravariant functor map of the form `comap :: PartialProfunctor p ⇒ (u → Maybe u′) → p u′ v → p u v`, capturing the idea that a contextualising function for the backward direction may fail. We generalise this notion such that the contextualising map may be within *any monad* as long as the monad can be ‘internalised’ into the profunctor:

```
class (Profunctor p, Monad m) ⇒ ProfunctorMonadInternalise p m where
  internaliseMonad :: p u v → p (m u) v
```

yielding the contravariant-functor-like map providing extra power to the contextualising function:

```
comap :: (ProfunctorMonadInternalise p m, Profunctor p) ⇒
  (u → m u′) → p u′ v → p u v
comap f = dimap f id ∘ internaliseMonad
```

More generally, this can be thought of as a *right monad action* operating on a contravariant functor, a concept that is encapsulated into the following class:

```
class (Monad m, Contravariant f) ⇒ MonadCoAction m f where
  coAct :: f u → f (m u)
```

Where `coAct` is both unital and associative:

```
unital = contramap return ∘ coAct ≡ id
assoc  = contramap join ∘ coAct  ≡ coAct ∘ coAct
```

where `contramap` is akin to `fmap`, but for contravariant functor.² The `comap` and `contramap` functions resemble each other, with the main difference being their source categories: `comap` maps morphisms of the Kleisli category of the **Maybe** monad, while `contramap` maps morphisms of the base category (i.e., the imagined category `Hask`).

Unpacking the concept of a monad action, the idea of one set ‘acting’ on another, in its simplest form, is represented by the function `act :: X → Y → Y`, where elements of the set X are mapped to endomorphisms on Y . In its uncurried form, left and right actions are distinguished by what side of the tuple the acting set, X , is on:

²See the Haskell standard library in `Data.Functor.Contravariant`.

Definition 2.10.1. A *right action* of a set X on a set Y is defined as follows:

$$\text{actR} :: (Y, X) \rightarrow Y$$

Actions can also respect algebraic structure, for example, a commonly used action is the monoidal action. Here the set X has monoidal structure, and the action can be seen as a homomorphism between the monoid X and the monoid of endomorphisms on Y .

Since monads are monoids in the category of endofunctors, the idea of monoid actions naturally translates to monad actions.

Definition 2.10.2. A *monad action* is when a monad m acts upon an endofunctor f either on the left, or on the right as follows:

$$\begin{aligned} \text{mactL} &:: m (f u) \rightarrow f u \\ \text{mactR} &:: f (m u) \rightarrow f u \end{aligned}$$

Thus coAct corresponds to mactR , but acting contravariantly.

Definition 2.10.3. A *right monad coaction* is a right monad action acting on a contravariant functor:

$$\text{coAct} :: f u \rightarrow f (m u)$$

Thus, $\text{ProfunctorMonadInternalise}$ is a right monad coaction on the first (contravariant) parameter of a profunctor.

We can now restate the property of being compositional when generalising a monadic profunctor to have an arbitrary coaction:

Definition 2.10.4. A property $\mathcal{R}$ is *compositional* with respect to the operations of an M -coacting PMP P if the following conditions hold for all types u, v, w , for all $x :: v$, $p :: P u v$, $k :: v \rightarrow P u w$, $f :: u' \rightarrow M u$,

$$\begin{aligned} &(\text{return } x) \in \mathcal{R}_v^u && \text{(comp-return)} \\ (p \in \mathcal{R}_v^u) \wedge (\forall v. (k v) \in \mathcal{R}_w^u) &\implies (p \gg k) \in \mathcal{R}_w^u && \text{(comp-bind)} \\ p \in \mathcal{R}_v^u &\implies (\text{comap } f p) \in \mathcal{R}_v^{u'} && \text{(comp-comap)} \end{aligned}$$

The only change compared with the previous definition of compositionality (Definition 2.5.5) is that we have generalised the function f applied in comap to an arbitrary monad M .

We will now consider an example that builds on this generalised ability to ‘internalise’ monadic computations inside a profunctor, and its associated generalisation of compositional reasoning.

Example: Grandparents A classic logic programming example asks queries about familial relationships over a database of facts about parentage. Consider the family tree given in Figure 2.1, which can be encoded by a simple binary relationship `parent` where `parent( $x$ ,  $y$ )` means that x is the parent of y , e.g., `parent(Gomez, Wednesday)`. We consider our bidirectional programming view of queries over this knowledge base and the problem of deriving a notion of *grandparent* from the above data. This puzzle is as simple as looking at a family tree and asking who is whose grandparents. For example, for the Addams family tree (Figure 2.1), the question could be, “Who is Wednesday’s grandparent?”. A valid checker would only accept Grandmama as an answer, and a sound and complete enumeration should produce the singleton list containing only Grandmama.

2. The Monadic Profunctor Paradigm of Bidirectional Programming

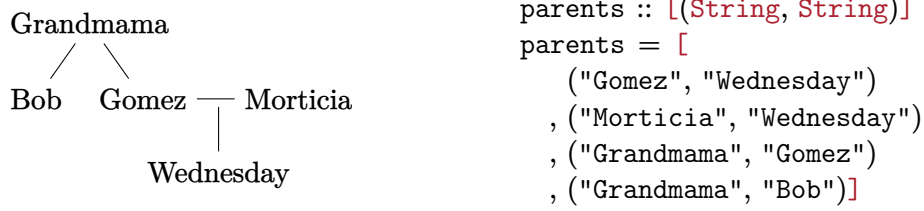


Figure 2.1.: Addams Family Tree

Whilst we previously defined generalised bigenerators **GG** in terms of a forward monad (which could be **Gen**) and backwards **Maybe** monad (to capture checking a predicate), we now generalise **Maybe** to lists, capturing the idea of a list of successes [Wadler 1985]:

```
newtype ND m u v = ND {unND :: (Fwd m :: Bwd []) u v}
```

```
mkND :: Monad m => m v -> (u -> [v]) -> ND m u v
mkND choices check = ND (Fwd choices :: Bwd check)
```

Thus we can see in the smart constructor `mkND` that a primitive bidirectional computation has in the backwards direction a map from a pre-view `u` to zero-or-many views `v`.

The deconstructors for the forward and backward direction are projections, given the names:

```
generateND :: Monad m => ND m u v -> m v
generateND = unFwd ∘ pfst ∘ unND

checkND :: Monad m => ND m u v -> u -> [v]
checkND = unBwd ∘ psnd ∘ unND
```

This type `ND` admits an instance of `ProfunctorMonadInternalise` (alternatively `MonadCoAction`) where we internalise a list argument in the contravariant position simply by the monadic bind of the list monad in the backwards direction:

```
instance Monad m => ProfunctorMonadInternalise (ND m) [] where
  internaliseMonad :: ∀ u v. ND m u v -> ND m [u] v
  internaliseMonad (ND (Fwd py :: Bwd qy)) = ND (Fwd py :: Bwd (λx -> x >>= qy))
```

For this particular problem, we instantiate the forwards monad with the list monad.

The Figure 2.1 relation is captured as a list of string pairs `parents`. From this, `parentOf` provides a non-deterministic computation enumerating the parents of a given person:

```
parentOf :: String -> [String]
parentOf person = do
  (parent, child) ← parents
  guard (person == child)
  pure parent
```

We can define a bidirectional version of `parentOf` to generate parents where the backwards direction checks whether elements are in the parent set:

```

biparentOf :: String → ND [] String String
biparentOf n = mkND
  (parentOf n)
  (λs → if s ∈ (parentOf n) then [s] else [])

```

The obvious (but wrong) approach to get a bidirectional computation of grandparents is to compose `biparentOf` with itself twice, i.e.

```

bigrandparentsOfattempt :: String → ND [] String String
bigrandparentsOfattempt c = biparentOf c >>= biparentOf

```

In the forwards directions (generating), then evaluating:

```

generateND $ bigrandparentsOfattempt "Wednesday"

```

yields the expected result of "Grandmama". However, the backwards direction (checking) does not yield what we expect: `(checkND $ bigrandparentsOfattempt "Wednesday") "Grandmama"` evaluates to `[]`, i.e., it is not calculated that "Grandmama" is the grandparent of "Wednesday", contradicting the forward direction; tripping is violated.

The problem is that the first backwards step (the left-most application `biparentOf "Wednesday"`) checks if "Grandmama" is an element of the set `parentOf "Wednesday"`, which is false because `parentOf "Wednesday"` generates "Gomez" and "Morticia". Thus, the first backwards step yields no results, i.e., `[]`, and so the second backwards step (right-most) does not run.

What is missing is the need to contextualise the first `biParentOf` so that the backwards direction is not given "Grandmama" as its input but is given the children of "Grandmama". Thus we contextualise the first use of `biparentOf` via `comap` and the function `children` which is akin to a generator for children, defined similarly to the original `parentOf` generator:

<pre> bigrandparentsOf :: String → ND [] String String bigrandparentsOf c = do parent ← children 'comap' biparentOf c biparentOf parent </pre>	<pre> children :: String → [String] children parent = do (parent', child) ← parents guard (parent == parent') pure child </pre>
--	---

Evaluating `((checkND $ bigrandparentsOf "Wednesday") "Grandmama")` returns `["Grandmama"]` as expected. The evaluation of the backwards checking direction proceeds by first applying `children "Grandmama"` yielding the `["Gomez", "Bob"]`, each of which is then passed to the backwards direction of `biparentOf "Wednesday"` yielding `["Gomez"]` as the only candidate. Thus the last line of `bigrandparentsOf` is `biparentOf "Gomez"` whose backward direction is not recontextualised and thus considers whether "Grandmama" is the parent of "Gomez" which is correct and thus `["Grandmama"]` is the result as expected.

With the previous `Maybe`-based approach in the river crossing and *N*-queens problems, we could not produce the desired result here as the backwards direction cannot contextualise itself to enough results; a `children :: String → Maybe String` function would not know which child to "pick".

We note that we could also define a variant of the `ND` type here with the logic monad of [Kiselyov, Shan, et al. 2005], in both directions, providing additional backtracking search facilities.

```

newtype BiLogic u v = BiLogic {unBL :: (Fwd Logic :: Bwd Logic) u v}

```

All of the above can be reformulated in this profunctorial monad, producing the same results, without the extra indirection of converting `Logic [a] → [a]` in `children`, providing the full power of the

2. The Monadic Profunctor Paradigm of Bidirectional Programming

Logic monad to both directions. This appears to be the most flexible setting for bidirectional logic programming, combining the profunctorial monad approach of this paper’s main theme along with the `MonadCoAction` above to subsume all the results of this section.

2.10.1. Summary

Generalised generators can be used to solve logic puzzles whose solutions are created inductively. This includes N -queens, and more complex puzzles, such as the river crossing puzzle, that include a notion of recoverability, but excludes puzzles of a transitive nature, for example, resolving who is whose grandparent. By generalising our notion of contravariant functoriality from $a \rightarrow \text{Maybe } b$ arrows to $a \rightarrow m\ b$ arrows with a monad coaction, we can capture a much larger class of bidirectional logic problems.

Generalising the novel notion of bigenerators has further extended the boundary of what bidirectional programs can be applied to, a boundary which we have taken care to pin point and characterise, and pushed back at in a monadic way.

2.11. Discussion and Related Work

Bidirectional transformations are a widely applicable technique used in many domains [Czarnecki et al. 2009]. Among language-based solutions, the lens framework is most influential [Barbosa et al. 2010; Bohannon, J. Nathan Foster, et al. 2008; J. Nathan Foster et al. 2005; N. Foster et al. 2010; Pacheco, Hu, et al. 2014; Rajkumar et al. 2013]. Broadly speaking, combinators are used as programming constructs with which complex lenses are created by combining simpler ones. The combinators preserve round tripping, and therefore the resulting programs are correct by construction. A problem with lens languages is that they tend to be disconnected from more general programming. Lenses can only be constructed by very specialised combinators and are not subject to existing abstraction mechanisms. Our approach allows bidirectional transformations to be built using standard components of functional programming, and gives a reasoning framework for studying compositionality of round tripping.

2.11.1. Applicative bidirectional programming

Existing techniques in the literature and in the wild have leveraged various other common abstractions to compose bidirectional programs, such as categories [Alimarine et al. 2006], and, more closely related to our work, applicative functors. While monadic sequential composition ($\gg=$) allows one computation to depend on the result of a previous one, applicative composition can only sequence independent computations, whose results can nevertheless be combined purely. It is often complemented by a combinator for non-deterministic choice or backtracking in order to allow a limited but often effective form of data dependency.

Rendel and Ostermann [2010] proposed a bidirectional programming interface imitating applicative functors. Although its presentation was centered around parsers and printers, it is also applicable to the settings of lenses and random generators we explored here. To account for the opposite polarities of parsers and printers, which are covariant and contravariant functors, respectively, the interface expects user-defined “partial isomorphisms” to transform and combine parser outputs and deconstruct printer inputs in the same bidirectional program, as opposed to regular functions in “unidirectional” applicative programming.

Our primary contribution is the generalization of that pattern to work with monads, a more expressive abstraction than applicative functors. We also introduce notions of alignment and quasicompositionality, which we believe systematically yield simple criteria to guarantee round-trip properties of bidirectional

programs. Our framework adapts itself particularly well to types of computations that are only *Applicative*. For example, the following combinator combines two bidirectional transformations into one bidirectional transformation on pairs.

```
pair
  :: (Profmonad p
     , PartialProfunctor p
     , Applicative (p (a, b))
     , Functor      (p (a, b)))
     => p a a → p b b → p (a, b) (a, b)
pair a b = (,) < $ > comap (Just o fst) a < * > comap (Just o snd) b
```

The *Alternative* typeclass abstracts common forms of backtracking. We also have examples of “alternative profunctors”, but do not include them in the paper due to the lack of space. Ostermann and Rendel’s developed theory of partial isomorphisms is the basis for their version of a *Functor*. Partial isomorphisms are in fact pairs of pseudo-inverse injections, which our framework allows to be used separately. These injections are also more easily available. In practice, we obtain most of them for free as constructors and field accessors.

As demonstrated above, our framework adapts gracefully to applicative programming, which is in fact a restricted form of monadic programming. By separating the input type from the output type, we can reuse the existing interface of applicative functors without modifying it. The pattern of combining profunctors with applicative functors is actually folklore in the Haskell community, sometimes called *monoidal profunctors*. Besides its generalization to monads, the concepts of alignment and quasicompositionality for verifying round tripping properties are novel to the best of our knowledge.

The codec [Chilton 2018] library we mentioned in Section 2.4 prominently features two applications of this monadic programming style: binary serialization (a form of parsing/printing) and conversion to and from JSON structures (analogous to lenses above).

Opaleye [Purely Agile n.d.], an EDSL of SQL queries for Postgres databases, uses an interface of monoidal profunctors to implement generic operations such as transformations between Haskell datatypes and database queries and responses.

FliPpr [Matsuda and M. Wang 2018b,c] is an invertible language that generates a parser from a definition of a pretty printer. In this paper, our biparser definitions are more similar to those of parsers than printers. This makes sense as it has been established that many parsers are monadic. Similar to the case of HOBiT, there is no discussion of monadic programming in the FliPpr work.

2.11.2. Lenses

More recently, researchers and practitioners have started to realise that some category-theory inspired isomorphisms may turn lenses (which are encapsulated pairs of functions) into representations of single functions. For example, the Haskell lens library [Edward Kmett 2012] uses the so-called *van Laarhoven representation* [Laarhoven 2009] to replace lens composition with normal functional composition. The Purescript language uses a closely related representation: *profunctor optics* [Pickering et al. 2017], a generalization of lenses to which our work bears a striking resemblance.

More precisely, a *Profunctor optic* [Pickering et al. 2017] between a source type s and a view type v is a function of type $p \ v \ v \rightarrow p \ s \ s$, for an abstract profunctor p .

Promonadic definitions are loosely similar to profunctor optics. By abstracting over primitives, *e.g.* `rootL` and `rightL` in Section 2.7, the type of a monadic lens such as `spineL` becomes:

```
L Tree (Maybe Int) (Maybe Int) → L Tree Tree Tree → L Tree [Int] [Int]
```

2. The Monadic Profunctor Paradigm of Bidirectional Programming

This corresponds loosely to a profunctor optic with source type `[Int]`, and two types of views: `Maybe Int` and `Tree`, corresponding to the second and third arguments to the type constructor `L`. This is not exclusive to promonadic lenses. Biparsers and bigenerators are also profunctor-optic-like in the same way, but we chose the example of lenses to better set our work apart from the point of view of profunctor optics. Indeed, the parallel we drew mismatches the notions of “view” and “source” in the two approaches. Among other differences, in the type of promonadic lenses $L\ s\ u\ v$, the source type `s` is internal to the concrete profunctor $L\ s$, whereas profunctor optics operate with an abstract profunctor p and rely on parametricity to preserve the relation between the view and source in $p\ v\ v \rightarrow p\ s\ s$. Profunctor optics and our **PMPs** offer orthogonal composition patterns: profunctor optics can be composed “vertically” using function composition, whereas **PMP** composition is “horizontal” providing sequential composition. In both cases, composition in the other direction can only be obtained by breaking the abstraction.

More dramatically, the framework of *applicative lenses* [Matsuda and M. Wang 2015] uses a different function representation of lens to break out from the point-free restriction, and enable bidirectional programming with explicit recursion and pattern matching. Note that the use of “applicative” in applicative lenses refers to the transitional sense of programming with λ -abstraction and functional application, which is not directly related to *applicative functors* in Haskell [McBride and Paterson 2008]. In this paper, we construct a new composition framework for lenses based on monads, which fits neatly into Haskell’s existing type class hierarchy. This new composition technique not only enables lens programming in the monadic style, but also opens up bidirectional programming to new domains that utilise monadic composition, *e.g.* random generators for testing.

The work on *monadic lenses* [Abou-Saleh et al. 2016] investigates lenses with effects. For instance, a put could require additional input to resolve conflicts when merging a view and a source. Representing effects with monads helps to reformulate the laws that define well-behaved lenses. In contrast, we made the type of lenses itself a monad, and we showed how they can be composed monadically to preserve well-behavedness. Our method is applicable to monadic lenses, yielding what one might call *monadic monadic lenses*: monadically composable lenses with monadic effects. We conjecture that laws for monadic lenses can be adapted to this setting with similar compositionality properties.

2.11.3. Random generators and predicates

Previous approaches to unifying random generators and predicates mostly focused on deriving generators from predicates. One general technique evaluates predicates lazily to drive generation (random or enumerative) [Boyapati et al. 2002; Claessen, Duregård, et al. 2015], but one loses control over the resulting distribution of generated values. *Luck* [Lampropoulos et al. 2017] is a domain-specific language blending narrowing and constraint solving to specify generators as predicates with user-provided annotations to control the probability distribution. In contrast, our programs can be viewed as generators annotated with left inverses with which to derive predicates. This reversed perspective comes with trade-offs: high-level properties would be more naturally expressed in a declarative language of predicates, whereas it is *a priori* more convenient to implement complex generation strategies in a specialised framework for random generators.

And of course, reflective generators [H. Goldstein et al. 2023], which we mentioned in Section 2.8, further demonstrate the usefulness of **PMPs** in this setting.

2.11.4. Future work

This work presents applications of **PMPs** to the design and use of embedded domain-specific languages for bidirectional programming. Partiality is pervasive in our development, and the composition patterns

captured by quasicompositionality require discipline and are still error prone. We hope to develop tools and methods such as type systems to automatically ensure correct usage of these patterns and guarantee compositional properties.

2.12. Conclusions

This paper advances the expressive power of bidirectional programming; we showed that the classic bidirectional patterns of parsers/printers and lenses can be restructured in terms of *Partial Monadic Profunctors* (*PMPs*) to provide sequential composition, with associated reasoning techniques. This opens up a new area in the design of embedded domain-specific languages for BX programming, that does not restrict programmers to stylised interfaces. Our example of bigenerators and their generalisations broadened the scope of BX programming from transformations (converting between two data representations) to non-transformational applications.

To demonstrate the applicability of our approach to real code, we have developed two bidirectional libraries [Liyao Xia 2018], one extending the `attoparsec` monadic parser combinator library to biparsers and one extending `QuickCheck` to bigenerators. One area for further work is studying biparsers with *lookahead*. Currently *lookahead* can be expressed in our extended `attoparsec`, but understanding its interaction with (quasi)compositional round tripping is further work.

Our framework could be applied in library implementations and in the design of bidirectional languages, with either a codec or *PMP* basis for the semantics. We have considered some extensions to our notions of compositionality and congruence to include other programming constructs such as sum types. In fact, our work has inspired the creation of `tomland`³, a bidirectional TOML parser, which uses *PMPs*.

However, this is not the final word on sequentially composable BX programs. In three applications, round tripping properties are similarly split into weak round tripping, which is weaker than the original property but compositional, and purifiable, which is equationally friendly. An open question is whether an underlying structure can be formalised, perhaps based on an adjunction model, that captures bidirectionality even more concretely than *PMPs*.

³<https://kowainik.github.io/posts/2019-01-14-tomland>

3. Reflecting on Random Generation

This chapter continues on with using monadic composition for bidirectional programming, focussing on the bigenerator example introduced in the previous chapter and its potential. It introduces *reflective generators*, an archetype of the goal of this thesis. They combine the functional idioms of monadic composition and property-based testing with bidirectional programming to achieve a powerful embedded domain specific language for specifying random data generators. These generators can reflect upon the choices they make during generation to implement and enhance shrinking and mutation algorithms, generalise an existing algorithm for example-based generation, and much more!

This chapter’s content is taken from *Reflecting on Random Generation* [H. Goldstein et al. 2023], which was published as a Distinguished paper at the 2023 International Conference on Functional Programming. Harrison Goldstein of the University of Pennsylvania and I wrote an original version of this paper in equal collaboration, then Harry led the writing for the revised versions. Our supervisors, Meng Wang and Benjamin C. Pierce advised the project and helped with the editing process.

3.1. Introduction

<pre>bst :: (Int, Int) -> Gen Tree bst (lo, hi) lo > hi = return Leaf bst (lo, hi) = frequency [(1, return Leaf), (5, do x <- choose (lo, hi) l <- bst (lo, x - 1) r <- bst (x + 1, hi) return (Node l x r))]</pre>	<pre>bst :: (Int, Int) -> Reflective Tree Tree bst (lo, hi) lo > hi = exact Leaf bst (lo, hi) = frequency [(1, exact Leaf), (5, do x <- focus (_Node._2) (choose (lo, hi)) l <- focus (_Node._1) (bst (lo, x - 1)) r <- focus (_Node._3) (bst (x + 1, hi)) return (Node l x r))]</pre>
(a) QuickCheck generator.	(b) Reflective generator.

Figure 3.1.: Generators for binary search trees.

Property-based testing, popularized by Haskell’s QuickCheck library [Claessen and Hughes 2000], draws much of its bug-finding power from *generators* for random data. These programs are carefully crafted and encode important information about the system under test. In particular, QuickCheck generators like the one in Figure 3.1a capture what it means for a test input to be *valid*—here, ensuring that a tree satisfies the Binary Search Tree (BST) invariant by keeping track of the minimum and maximum allowable values in each sub-tree. This generator is not just a program for generating BSTs, it *defines* BSTs in the sense that its range is precisely the set of BSTs.

Developers of tools like Hypothesis [David R MacIver, Hatfield-Dodds, et al. 2019]—arguably the most popular PBT framework, with 6,500 stars on GitHub and an estimated 500,000 users [Dodds 2022]—capitalize on this observation and repurpose generators for other tasks, including test-case *shrinking* and *mutation*. These algorithms do not operate directly on data values; rather, shrinking or mutating a value is accomplished by shrinking or mutating the *random choices* that produced that value, and then re-running the generator on the modified choices [David R. MacIver and Donaldson

3. Reflecting on Random Generation

2020]. This amounts to treating generators as *parsers*, taking unstructured randomness and parsing it into structured values—a perspective formalized in terms of *free generators* [Goldstein and Pierce 2022]. Viewing generators as parsers has two advantages: (1) shrinking and mutation can be implemented generically, rather than once per generator, and (2) the modified data values will automatically satisfy any preconditions that the generator was designed to enforce (e.g., the BST invariant), since they are ultimately produced by pushing modified choices through the generator.

Ideally, Hypothesis’s type-agnostic, validity-preserving algorithms should completely subsume more manual ones. Unfortunately, the current Hypothesis approach assumes that the shrinker, for example, is given access to the original random choices that the generator made when producing the value it is shrinking; this doesn’t work when shrinking is separated (in time or space) from generation. In particular, Hypothesis can’t shrink values that it did not generate in the first place—e.g., because they were provided as pathological examples or from crash reports. More subtly, Hypothesis’s shrinking also breaks if the value was modified between generation and shrinking, or saved without also saving a record of the choices.

To use Hypothesis-style shrinking on an arbitrary value, the shrinker needs some way of reconstituting a set of random choices that produce that value. Luckily, inspiration for how to do this can be drawn from the grammar-based testing literature, specifically *Inputs from Hell* [Soremekun et al. 2020], which describes a way to produce test inputs that are similar to an existing one. Starting with a grammar-based generator [Godefroid, Kiezun, et al. 2008], they first use the grammar to parse a value, determining which *productions* must be expanded to produce that value. Then, they bias the generator to expand those productions more often, thus resulting in more values that are similar to the example. In essence, this approach determines which generator choices lead to a desired value by *going backward*, parsing the value with the same grammar that (could have) generated it.

The *Inputs from Hell* approach works well for grammar-based generators, but does not apply to generators that enforce validity. For this, we need a more general solution—one that works with the kinds of *monadic* generators used in QuickCheck, which can enforce arbitrary validity conditions. Such a solution can be found in the bidirectional programming literature. L.-y. Xia et al. [2019a] describe *partial monadic profunctors*, which enrich standard monads with extra operations for describing bidirectional computations. This infrastructure, along with the parsing-as-generation perspective of free generators, enables exactly the kind of bidirectional generation needed to extract random choices from a monadically produced value.

Our main contribution, *reflective generators*, is a language for writing bidirectional generators that can “reflect” on a value to analyze which choices produce that value (see Figure 3.1b). They subsume the grammar-based generators of *Inputs from Hell*, and, critically, they enable Hypothesis-style shrinking and mutation for arbitrary values in the range of a monadic generator. Furthermore, since reflective generators are built on *freer monads*, they can be interpreted numerous ways besides generation and reflection. We discuss three more use cases that demonstrate the versatility of reflective generators as testing utilities.

Following a brief tour through some background (§3.2), we offer the following contributions:

- We present *reflective generators*, a framework that fuses *free generators* and *partial monadic profunctors* into a flexible domain-specific language for PBT generators that can reflect on a value to obtain choices that produce it. (§3.3)
- We develop the theory of reflective generators, defining what it means for reflective generators to be correct along a number of axes and comparing their expressive power to other generation abstractions. (§3.4)
- We demonstrate the core behavior of reflective generators by generalizing prior work on example-

based generation. Our implementation subsumes the *Inputs from Hell* algorithm and extends it to work with monadic generators. (§3.5)

- We apply reflective generators to manipulate user-provided values. Reflective generators enable generator-based shrinking and mutation algorithms to work even when the randomness used to generate test inputs is not available. We show that our shrinkers are at least as effective as other automated shrinking techniques. (§3.6)
- We leverage the reflective generator abstraction to implement other testing tools: checkers for test input validity, “completers” that can randomly complete a partially defined value, and test input producers like enumerators and fuzzers. (§3.7)

We conclude by discussing related work (§3.8) and future directions (§3.9).

3.2. Background

The abstractions we present in this paper rely on a significant amount of prior work. In this section, we review the structures that make reflective generators possible: monadic random generators (§3.2.1), free generators (§3.2.2), and partial monadic profunctors (§3.2.3).

```
class Monad m where
  return :: a -> m a
  (>=>) :: m a -> (a -> m b) -> m b
  do { x <- m; f x } = m >=> f
  type Gen a = Int -> a
```

(a) Definitions for monadic generators.

```
data Freer f a where
  Return :: a -> Freer f a
  Bind :: f a
    -> (a -> Freer f b)
    -> Freer f b
```

```
data Pick a where
  Pick :: [(Weight, Choice, Freer Pick a)]
    -> Pick a
```

(b) Definitions for free generators.

```
class Profunctor p where
  lmap :: (d -> c) -> p c a -> p d a
  rmap :: (a -> b) -> p c a -> p c b
```

```
class Profunctor p => PartialProf p where
  prune :: p b a -> p (Maybe b) a
```

```
class (forall b. Monad (p b), PartialProf p)
  => PMP p
```

(c) Definitions for partial monadic profunctors.

Figure 3.2.: Background definitions.

3.2.1. Monadic Random Generators

The idea of testing executable properties using monadic generators was popularized by QuickCheck and has endured for more than two decades. The core structures used by QuickCheck are shown in Figure 3.2a, where *Gen* represents the type of random generators. It treats the input *Int* as a random seed and uses it to produce a value of the appropriate type. Generators like the one in Figure 3.1a are easy to write because they are *monads* [Moggi 1991]; this structure provides a neat interface for chaining effectful computations.

3. Reflecting on Random Generation

3.2.2. Free Generators

Interfaces like the `Monad` type class can be reified into “free” structures that represent each operation as a data constructor, allowing it to be interpreted in multiple ways. There are several such free structures for the monad interface [Ed Kmett 2023; Trnková et al. 1975, etc.]; we focus on the *freer monad* [Kiselyov and Ishii 2015] structure shown in Figure 3.2b, which reifies the `return` operation as the constructor `Return` and `(>>=)` as `Bind`. The extra type constructor `f` ranges over the operations that are specific to each given monad, for example, the `Get :: State s s` and `Put :: s -> State s ()` operations for `State`.

Our prior work, *free generators* [Goldstein and Pierce 2022], are an instance of this scheme, instantiating `f` as the type constructor `Pick` (Figure 3.2b), which represents a choice between sub-generators. `Pick` is used to implement familiar QuickCheck-style combinators like `choose`, which generates an integer in a given range, and `oneof`, which randomly selects between a list of generators. We use free generators to draw a formal connection between random generation and parsing, interpreting the same free generator as both a generator and a parser.

3.2.3. Partial Monadic Profunctors

Profunctors are a standard construction from category theory, generalizing ordinary functors—structure-preserving maps—to allow for both covariant and contravariant mapping operations. They are realized as the `Profunctor` class (Figure 3.2c), popularized in Haskell by Pickering et al. [2017], where the mapping operations are called `rmap` and `lmap` respectively. The quintessential example of a profunctor is the function type constructor `(->)`. This makes sense, since `b -> a` is contravariant in `b` and covariant in `a`. In that case, `rmap` implements post-composition (of some function `a -> a'`) and `lmap` implements pre-composition (of a function `b' -> b`). Indeed, it is often useful to think of profunctors as function-like: a value of type `p b a` “examines a value of type `b` to produce a value of type `a`” (potentially doing some other effects).

A *monadic profunctor* is a profunctor that is also a monad (i.e., a profunctor `p` such that for any `b`, `p b b` is a monad). L.-y. Xia et al. [2019a] use this extra structure to implement composable bidirectional computations. For example, consider the classic bidirectional programming example of a parser and a pretty printer, which invert one-another. These dual functions can be implemented using the same monadic profunctor:

```
data Biparser b a = { parse :: String -> (a, String); print :: b -> (a, String) }
```

This `Biparser` is a single program that gets “interpreted” in two different ways. As a parser, it ignores the `b` parameter entirely, and simply acts as a parser in the style of `Parsec` [Leijen and Meijer 2001] to produce a value of type `a`. As a pretty printer, it still needs to produce a value of type `a`—after all, the two interpretations share the same code—but now there is no input `String` to parse. Instead, the pretty printer interpretation has a value of type `b` it uses as instructions to produce both the `a` and a `String`. This scheme makes much more sense when the profunctor is *aligned*, that is of type `Biparser a a`; in that case, a properly written pretty printer acts as an identity function, taking a value and reproducing it while also recording its `String` representation.

L.-y. Xia et al. call the first type of interpretation, which ignores its contravariant parameter and just produces an output, a “forward” interpretation, and the second, which acts as an identity function and follows the structure of its contravariant parameter, a “backward” interpretation. We say `parse` goes forward and `print` goes backward. This is an arbitrary choice, but it is helpful as an analogy for the way that certain monadic profunctors are duals of one other.

Writing a `Biparser`, or any other monadic profunctor, is a game of type alignment. In general, aligned programs are desirable, but aligning the types gets tricky around monadic binds. Suppose we

have an aligned $p\ a\ a$ and we want to get an a out and continue with a function of type $a \rightarrow p\ b\ b$ (whose codomain is also aligned). We *cannot* simply use $(\gg=)$ whose type is:

```
(\gg=) :: p b a -> (a -> p b b) -> p b b
```

The problem is the first argument: we have a value of type $p\ a\ a$ but we need one of type $p\ b\ a$ since $p\ b$ is the type of the monad. Luckily, the `lmap` operation (§3.2c) makes it possible to take an aligned profunctor, $p\ a\ a$ and turn it into a $p\ b\ a$ by providing an *annotation* of type $b \rightarrow a$ that says how to focus on some part of a value of type b that has type a . We can thus build up a Biparser by writing a program that looks essentially like a standard monadic parser, but with monadic binds annotated with calls to `lmap` that fix the alignment.

This story is almost complete, but it leaves out cases where annotations need to be partial. Consider a Biparser like this one, which parses either a letter or a number:

```
letter :: Biparser Char Char
number :: Biparser Int Int
data LorN = Letter Char | Number Int

lOrN :: Biparser LorN LorN
lOrN = Letter <$> lmap ??? letter <|> Number <$> lmap ??? number
```

The first annotation should be of type $LorN \rightarrow Char$, but there is no total function of that type: what happens if the `LorN` is a `Number`? The more appropriate annotation type would be $LorN \rightarrow Maybe\ Char$. L.-y. Xia et al. make this possible with *partial monadic profunctors* (PMPs), which add one more operation, `prune`, to capture failure. (We have renamed it `prune` instead of the original “`internaliseMaybe`”.) Unlike monadic profunctors, which can only be annotated with total functions using `lmap`, PMPs can be annotated with partial functions. The combinator `comap` demonstrates this generalized annotation:

```
comap :: PMP p => (c -> Maybe b) -> p b a -> p c a
comap f = lmap f . prune
```

When a PMP like `print` branches (e.g., in `lOrN`) the execution follows both sides (e.g., trying to pretty print both a `Letter` and a `Number`). The partial annotations tell the computation when to *prune* a branch, keeping the search space small and ensuring that PMPs like `print` are efficient.

A concrete example of a partial profunctor is the partial arrow constructor:

```
newtype PartialArr a b = PartialArr {unPA :: a -> Maybe b}
```

This follows on naturally from the intuition that $(\rightarrow)$ is a profunctor. In fact, these are also PMPs, you can find the relevant instances in Appendix 1.

PMPs are complex, and they can be used in a wide variety of ways. We recommend *Composing Bidirectional Programs Monadically* [L.-y. Xia et al. 2019a] for a more thorough explanation.

3.3. The Reflective Generator Language

Reflective generators combine free generators with PMPs, enabling a host of enhanced testing algorithms. In this section, we explain the intuition behind reflective generators (§3.3.1), describe their implementation (§3.3.2), and discuss their various interpretations (§3.3.3).

The basic structure of a reflective generator comes from adding the partial monadic profunctor operations, `lmap` and `prune`, to the `Pick` datatype. We call this extended type `R` (for Reflective) and implement it like this:

```
type Weight = Int
type Choice = Maybe String
```

3. Reflecting on Random Generation

```
data R b a where
  Pick  :: [(Weight, Choice, Freer (R b) a)] -> R b a
  Lmap  :: (c -> d) -> R d a -> R c a
  Prune :: R b a -> R (Maybe b) a
```

The `Pick` constructor here has two small changes from the free generator presentation: we add an extra contravariant type variable `b`, and we modify the choice type to optionally elide choice labels.¹ Then `Lmap` captures contravariant annotations (there is no need to explicitly represent `rmap`, as we will be able to encode it using the monad structure), and `Prune` represents its PMP counterpart with an analogous type. A reflective generator is then a freer monad over `R b`:

```
type Reflective b a = Freer (R b) a
```

3.3.1. Intuition

The type `Reflective b a` of reflective generators should be understood to mean a program that can “reflect on a value `b`, recording choices, while generating an `a`.”

Like PMPs, reflective generators use annotations to fix the types around monadic binds. More intuition of how these should be defined follows from the intuitive interpretation of the types: the goal is to take a generator that reflects on choices in an `a` and turn it into one that reflects on choices in a `b`. Here’s the key: it suffices to show how to *focus on a part of the `b` that contains an `a`*, because that focusing turns a choices into `b` choices. Put another way, the annotation should embed a mapping of type `b -> a` or `b -> Maybe a` that focuses on the `a` part of the `b`. To see this in action, consider the example in Figure 3.1b, paying attention to the first bind in the `Node` branch,

```
do
  x <- focus (_Node._2) (choose (lo, hi))
  ...
```

where `...` continues on to produce the rest of the tree. The call to `choose` results in a `Reflective Int Int`, but the type of the enclosing monad is `Reflective Tree`; as discussed in §3.2.3 we need to add an annotation on the bind that focuses on an `Int` in a `Tree` to get a `Reflective Tree Int`. In the example, we annotate with `focus (_Node._2)` (this syntax is introduced in the next section) but the following is equivalent:

```
comap (\ t -> case t of { Leaf -> Nothing; Node _ x _ -> Just x })
```

As with PMPs like `Biparser`, the process of reflecting on choices is all about the interaction between binds and `comaps`. A value flows through the program, and, at each bind, the `comap` focuses on the part of the value that the left side of the bind should reflect on. If the focusing fails, that branch gets pruned—there is no way to produce the desired value—but if it succeeds, then the left side can reflect on that part of the value, extracting some choices, and reflection can continue on the right side.

With this intuition in mind, we can move onto the technical details of reflective generators.

3.3.2. Implementation

We next describe how reflective generators are implemented and what we’ve done to make them feel familiar and easy to work with. (In §3.9, we discuss plans to validate this belief with a proper user study.)

¹As in *Parsing Randomness* [Goldstein and Pierce 2022], we represent weights in `Pick` as integers for simplicity. Formally they are required to be strictly positive; this will be necessary to prove Theorem 3.4.4.

The Full Story The actual type `R` defined in the Haskell artifact² is a bit more complicated than the one explained above. The actual implementation looks like:

```
data R b a where
  Pick  :: [(Weight, Choice, Freer (R b) a)] -> R b a      -- as before
  Lmap  :: (c -> d) -> R d a -> R c a                      -- as before
  Prune :: R b a -> R (Maybe b) a                          -- as before
  ChooseInteger :: (Integer, Integer) -> R Integer Integer
  GetSize :: R b Int
  Resize  :: Int -> R b a -> R b a
```

First, we add a constructor `ChooseInteger` for picking integers from a range. Technically, this is implementable via `Pick` by simply enumerating all of the integers in the desired range, but doing so is inefficient if the range is large. Adding a separate function for choosing within a range of integers allows us to bootstrap other generators over large ranges, and it makes it easy to implement much more efficient interpretation functions later on.

Second, we add two constructors, `GetSize` and `Resize`, that are analogous to similar operations implemented by `QuickCheck`. Maintaining size control is critical for ensuring generator termination, and, although it is possible to implement sized generators by passing size parameters around manually, internalizing size control cleans up the API of the combinator library and makes generators more readable.

In future sections, we often elide parts of definitions pertaining to these operations, to streamline the presentation, but they do present a couple of theoretical complications that we note in §3.4.

Building a Domain-Specific Language We implement a variety of combinators that make reflective generators easier to read and write, aiming for an interface that captures the full power of reflective generation without straying too far from familiar `QuickCheck` syntax.

The most important reflective generator operation is `Pick`, so we provide a number of choice combinators that are built on top of it:

```
pick      :: [(Int, String, Reflective b a)] -> Reflective b a
labeled   :: [(String      , Reflective b a)] -> Reflective b a
frequency :: [(Int        , Reflective b a)] -> Reflective b a
oneof     :: [Reflective b a] -> Reflective b a
choose    :: (Int, Int) -> Reflective Int Int
```

The most flexible, `pick`, just passes through to the `Pick` constructor, wielding its full power. The rest are simplifications of this operation that represent common use cases. A bit simpler, `labeled` takes only choice labels and no weights—it sets all weights to 1. Finally, `frequency`, `oneof`, and `choose` have the same API as their counterparts in `QuickCheck`, forgoing choice labels.

A brief aside on choice labels: whether or not a user decides to label the choices in a generator depends on two factors. First, it depends on how the generators will be used. In §3.5.1, we discuss a use case that relies heavily on choice labels, and in §3.6 we discuss one that ignores them. Second, whether or not a particular sub-generator is labeled can impact the behavior of use cases that pay attention to labels; in §3.5.1 we discuss intentionally eliding labels as a way of marking parts of the generator whose distributions should not be tuned. As a general rule, we recommend labeling choices in generators, and all generators provided by the reflective generators library are labeled by default, but it is convenient to be able to elide labels when upgrading from a `QuickCheck` generator.

²<https://zenodo.org/record/7988049>

3. Reflecting on Random Generation

Choice operators alone are not enough to build reflective generators, we need infrastructure to glue them together. The bulk of these glue operations follow from the fact that reflective generators are, as expected, PMPs:³

```
instance Profunctor Reflective where
  lmap _ (Return a) = Return a
  lmap f (Bind x h) = Bind (Lmap f x) (lmap f . h)
  rmap = fmap

instance PartialProfunctor Reflective where
  prune (Return a) = Return a
  prune (Bind x f) = Bind (Prune x) (prune . f)
```

Both `lmap` and `prune` commute over `Bind` and do nothing to a `Return` (see the laws in §3.4), so these implementations are straightforward. Behind the scenes, the `Functor`, `Applicative`, and `Monad` operations are implemented for free from the `freer monad`.

Using `lmap` and `prune` on their own is a bit tedious, so we give two combinators that make common use cases much simpler. The `focus` combinator makes it possible to replace pattern matches in `lmap` annotations with *lenses* [John Nathan Foster 2009].

```
focus :: Getting (First b) c b -> Reflective b a -> Reflective c a
focus p = lmap (preview p) . prune
```

The curious reader can dig into the gory details of the types⁴ involved, but it suffices to understand `focus` as a notational convenience that gives a terse syntax for pattern matches:

```
focus (_Node._2) g = (lmap (\ case { Node _ x _ -> Just x; _ -> Nothing }) . prune) g
```

This call to `focus` identifies the part of the value that `g` produces: the second argument to the `Node` constructor. The neat thing about this setup is that `_Node`, a lens that “pattern matches” on a node, can be automatically generated from the `Tree` datatype, and `_2`, which extracts the second element of a k -tuple, is included in the lens library. Together they can match on and extract the part of the value that the reflective generator needs to focus on.

Another convenient helper built from `lmap` and `prune` is `exact`, which operates like `return` but ensures that the returned value is exactly the expected one:

```
exact :: Eq a => a -> Reflective a a
exact a = (lmap (\ a' -> if a == a' then Just a else Nothing)
  . prune) (Pick [(1, Nothing, return a)])
```

Using this function (or manually pruning) at the leaves of a reflective generator is critical: without it, the generator may incorrectly claim to be able to produce an invalid value.

We saw above that combinators like `oneof`, `frequency`, and `choose` align closely with the `QuickCheck` API to make upgrading easier. We provide one more combinator to simplify the upgrade process, `voidAnn`, which can be used in place of an annotation:

```
voidAnn :: Reflective b a -> Reflective Void a
voidAnn = lmap (\ x -> case x of)
```

The `Void` type in Haskell is uninhabited, and thus a reflective generator of type `Reflective Void a` can only be used in limited cases, but `voidAnn` makes it possible to perform the upgrade from Figure 3.1

³Technically, these definitions are not legal Haskell, since both partially apply the `Reflective` type constructor, which is not supported by GHC. In the Haskell artifact we implement the operations as normal functions (rather than type-class methods).

⁴<https://hackage.haskell.org/package/lens-5.2/docs/Control-Lens-Getter.html>

```

bst :: (Int, Int) -> Reflective Void Tree
bst (lo, hi) | lo > hi = return Leaf
bst (lo, hi) =
  frequency
    [ ( 1, return Leaf),
      ( 5, do
        x <- voidAnn (choose (lo, hi))
        l <- voidAnn (bst (lo, x - 1))
        r <- voidAnn (bst (x + 1, hi))
        return (Node l x r) ) ]

```

Figure 3.3.: An intermediate generator with Void as its contravariant type, using voidAnn.

in stages. First, go from Figure 3.1a to Figure 3.3, then go from Figure 3.3 to Figure 3.1b by replacing Void with the correct output type, replacing voidAnn annotations with ones that do appropriate focusing, and replacing return with exact where appropriate. All three generators are shown together in Appendix 2. Experienced reflective generator writers do the upgrade in a single step, but when starting out it may be easier to go through a simpler intermediate generator.

There are a number of other combinators implemented in the artifact, including getSize, resize, standard generators for base types, and higher-order combinators for lists and tuples.

3.3.3. Interpretation

Like free generators, reflective generators do not do anything interesting until they are *interpreted*. An interpretation describes how the inert syntax of the generator program should be executed. As with PMPs, most reflective generator interpretations can be thought of as working either “forward,” simply producing an output of their covariant type, or “backward,” reflecting on a value (and reproduce it *en passant*) while tracking choices. Unlike PMPs, reflective generators do not explicitly pair a forward and a backward interpretation together—in fact, the interpretation in §3.7.2 actually uses both directions at once. Still, directionality of interpretations is often a useful intuition.

The simplest “forward” interpretation turns a reflective generator into a standard QuickCheck generator, shown in Figure 3.4. The free monad part of the syntax is implemented as expected, with Return implemented as return in the Gen monad, and Bind as the monad’s bind. The rest of the syntax is similarly straightforward, with Lmap and Prune doing nothing and Pick interpreted as a weighted random choice.

```

generate :: Reflective b a -> Gen a
generate = interp
  where
    interpR :: R b a -> Gen a
    interpR (Pick gs) = QC.frequency [(w, interpR g) | (w, _, g) <- gs]
    interpR (Lmap _ r) = interpR r
    interpR (Prune r) = interpR r

    interp :: Reflective b a -> Gen a
    interp (Return a) = return a
    interp (Bind r f) = interpR r >>= interp . f

```

Figure 3.4.: The “generate” interpretation.

3. Reflecting on Random Generation

Of course, the value of reflective generators lies in their ability to run “backward,” focusing on sub-parts of a value and reflecting on how they are constructed. This process can be seen using the `reflect` function in Figure 3.5, which interprets a generator to determine which choices could lead to a given value. For example, it would behave as follows when run on `bst` (adapted to record choices):

```
ghci> reflect bst Leaf
[["leaf"]]
ghci> reflect bst (Node Leaf 4 Leaf)
[["node","4","leaf","leaf"]]
```

Here, the constructor choice is indicated with “leaf” or “node”, and the number choice by printing the number. A list of lists is produced so that all possible choice traces are covered. (In these examples, there just happens to be only one trace of choices that could have led to the provided values.)

When interpreting `Pick` the computation splits, each branch representing making one particular choice. In each branch, `Lmap` nodes focus on parts of the value being reflected on; if the focusing fails, a following `Prune` node will filter that branch out of the computation. The monad structure, `Return` and `Bind`, threads the list of recorded choices through the computation, so the final result is a list of the different branches of the computation that were not pruned, along with the choices made in each of those branches. For termination, the `reflect` interpretation requires that any recursion is guarded by focus annotations that focus on smaller parts of the value being reflected on; see §3.4.2 for more details.

```
reflect :: Reflective a a -> a -> [[String]]
reflect g = map snd . interp g
  where
    interpR :: R b a -> (b -> [(a, [String])])
    interpR (Pick gs) = \ b ->                                     -- Record choices made.
      concatMap
        ( \ (_, ms, g') ->
          case ms of
            Nothing -> interp g' b
            Just lbl -> map ( \ (a, lbls) -> (a, lbl : lbls)) (interp g' b)
        ) gs
    interpR (Lmap f r) = \ b -> interpR r (f b) -- Adjust b according to f.
    interpR (Prune r) = \ b -> case b of         -- Filter invalid branches.
      Nothing -> []
      Just a -> interpR r a

    interp :: Reflective b a -> (b -> [(a, [String])])
    interp (Return a) = \ _ -> return (a, [])
    interp (Bind r f) = \ b -> do                -- Thread choices around.
      (a, cs) <- interpR r b
      (a', cs') <- interp (f a) b
      return (a', cs ++ cs')
```

Figure 3.5.: The “reflect” interpretation.

These two interpretations demonstrate the essence of reflective generators, but they are far from the only ones—in total we discuss seven use cases of our different interpretations, all of which can be found in our artifact, and we expect there are use cases for many more. As a user, this gives an amazing amount of flexibility, since a single reflective generator can be interpreted in all of these different ways.

3.4. Theory of Reflective Generators

In this section, we describe more of the theory underlying reflective generators. We discuss various formulations of correctness, including defining what it means to correctly interpret a reflective generator and what it means to correctly write an individual reflective generator (§3.4.1). Next, we explore an interesting property of reflective generators—*overlap*—which has implications for generator efficiency (§3.4.2). Finally, we discuss the expressive power of reflective generators, comparing it to grammar-based generators and to standard monadic ones (§3.4.3).

3.4.1. Correctness

Both interpretations and individual reflective generators can be written incorrectly—the types involved are not strong enough. Here, we describe algebraic properties that the programmer should prove (or test) to ensure good behavior.

Correctness of Interpretations Reflective generators should obey the laws of monads [Moggi 1991], profunctors, and PMPs:

```
(M1)    return a >>= f = f a
(M2)    x >>= return = x
(M3)    (x >>= f) >>= g = x >>= (\ a -> f a >>= g)

(P1)    lmap id = id
(P2)    lmap (f' . f) = lmap f . lmap f'
(PMP1)  lmap Just . prune = id
(PMP2)  lmap (f >=> g) . prune = lmap f . prune . lmap g . prune
(PMP3)  (lmap f . prune) (return y) = return y
(PMP4)  (lmap f . prune) (x >>= g) = (lmap f . prune) x >>= (lmap f . prune) . g
```

Some of these are definitionally true for all reflective generators, thanks to the structure of freer monads:

Lemma 3.4.1. Reflective generators always obey (M1), (M3), (PMP3), and (PMP4).

Proof. By induction on the structure of the generator, using the definitions of `return` and `(>>=)` from Kiselyov and Ishii [2015] and the definitions of `lmap` and `prune` from §3.3.2. See Appendix 3. $\square$

The other equations do not hold in general: they must be established for each interpretation.

We say an interpretation of a reflective generator is *lawful* if it implements a PMP homomorphism to some lawful partial monadic profunctor. Concretely:

Definition 1. An interpretation

$\llbracket \cdot \rrbracket :: \text{Reflective } b \ a \rightarrow p \ b \ a$

is *lawful* iff p obeys the laws of monads, profunctors, and partial monadic profunctors and there exists an *R-interpretation*

$\llbracket \cdot \rrbracket_R :: R \ b \ a \rightarrow p \ b \ a$

such that the following equations hold:

```
\Return a\    = return a
\Bind r f\    = \r\_R >>= \ x -> \f x\
\Lmap f r\_R = lmap f \r\_R
\Prune r\_R  = prune \r\_R
```

3. Reflecting on Random Generation

An alternative approach would be to simply define an interpretation of a reflective generator as a PMP homomorphism along with an interpretation for `Pick`, rather than giving the programmer the freedom to implement lawless interpretations. From a programming perspective, this would behave like a tagless-final embedding [Kiselyov 2012]. We found this tagless-final approach more tedious to program with, but it is available to users if desired (see Appendix 4).

The `generate` instance is indeed lawful, modulo one technical caveat. The classic `Gen` “monad” itself is not actually a lawful monad, but it *is* lawful up to distributional equivalence [Claessen and Hughes 2000]—i.e., generators that produce equivalent probability distributions of values are equivalent, even if they are not equal as Haskell terms. The same caveat applies to the other laws.

Theorem 3.4.2. The *generate* interpretation is lawful up to distributional equivalence.

Proof. Since `Lmap` and `Prune` are both ignored, the other laws are trivial. □

The `reflect` interpretation is also lawful, ignoring the `map_snd` projection that discards the accumulator values.

Theorem 3.4.3. The *reflect* interpretation is lawful.

Proof. We choose

```
p b a = b -> [(a, [String])] = ReaderT b (WriterT [String] []) a
```

which is simply a stack of three monads (reader, writer, and list). Lawfulness follows straightforwardly, by aligning the definition of `reflect` with the lawful implementations of the monad, profunctor, and partial profunctor operations for this combined monad. □

Correctness of a Reflective Generator The proofs of lawfulness for each of the interpretations we want to use can be carried out once and for all, but there is also some work to do for each individual reflective generator, to ensure that its various interpretations will behave the way we expect. We next characterize what it is for a reflective generator to be *correct* and comment on testing for correctness using QuickCheck.

Our correctness criteria is based on similar notions to those of L.-y. Xia et al. [2019a]. We formulate correctness using two interpretations. The `generate` interpretation is the canonical “forward” interpretation, characterizing the set of values that can be produced by a reflective generator when it ignores its contravariant parameter. The canonical “backward” interpretation should characterize the generator’s operation as a generalized identity function, taking a value and reproducing it—`reflect` is almost the right interpretation, but it does extra work to keep track of choices. Thus, we define:

```
reflect' :: Reflective b a -> b -> [a]
```

The `reflect'` interpretation has the same behavior as `reflect`, but it skips the code that tracks choices. It also has a more general, unaligned type. (Alignment is an artificial restriction anyway; `reflect` could also be given an unaligned type, but the alignment better communicates the intended use.) The full code for this and all other interpretation functions can be found in our artifact.

We define soundness and completeness of a reflective generator as follows:

Definition 2. A reflective generator `g` is *sound* iff

$$a \sim \text{generate } g \implies (\text{not } . \text{ null}) (\text{reflect}' g a).^5$$

Where $a \sim \gamma$ means “a can be sampled from QuickCheck generator γ .”

In other words, if the generate interpretation can produce a value, then the reflect' interpretation can reflect on that value without failing.

Definition 3. A reflective generator g is *complete* iff

$$(\text{not } . \text{ null}) (\text{reflect}' g a) \implies a \sim \text{generate } g.$$

In other words, if the reflect' interpretation successfully reflects on a value, then that value should be able to be sampled from the generate interpretation.

As there is no way to check $a \sim \text{generate } g$ directly, completeness is impossible to test. Luckily, L.-y. Xia et al. give an alternative. First they define *weak completeness*:

Definition 4. A reflective generator g is *weak complete* iff

$$a \in \text{reflect}' g b \implies a \sim \text{generate } g.$$

Weak completeness is still impossible to test, but it is *compositional*, meaning it is true of a generator if it is true of its sub-generators. Since the only kind of sub-generator reflective generators can be built from is Pick, we can prove this once and for all:

Lemma 3.4.4. All reflective generators are weak complete.

Proof. By induction on the structure of the generator; see Appendix 5. (Note that this relies on the weights in every Pick being strictly positive.) $\square$

L.-y. Xia et al. also gives a so-called *pure projection property*, which is testable (albeit sometimes intractably⁶):

Definition 5. A reflective generator satisfies *pure projection* iff

$$a' \in \text{reflect}' g a \implies a = a'.$$

To complete the picture, we prove the following:

Theorem 3.4.5. Any weak-complete reflective generator satisfying pure projection is complete.

Proof. Assume $(\text{not } . \text{ null}) (\text{reflect}' g a)$, so there is some a' in $\text{reflect}' g a$. By pure projection, $a = a'$ so a is in $\text{reflect}' g a$. then by weak completeness we have $a \sim \text{generate } g$ as desired. $\square$

The take-away is that testing completeness of a reflective generator directly is impossible, but testing pure projection suffices, where tractable. When determining the correctness of a reflective generator, one should definitely test soundness and, where tractable, test pure projection.

⁵The definition we give for soundness is morally correct, but it will occasionally fail (spuriously) if tested using QuickCheck. The problem is *size*: QuickCheck varies the generator's size parameter while testing, but it does not know to vary the size of the reflect' interpretation to match. Concretely, this means that QuickCheck may test

$$a \sim \text{resize } 100 (\text{generate } g) \implies (\text{not } . \text{ null}) (\text{reflect}' g a).$$

which is effectively evaluating two different generators. To get around this, we should instead test

$$a \sim \text{generate } (\text{resize } n g) \implies (\text{not } . \text{ null}) (\text{reflect}' (\text{resize } n g) a).$$

for all n in a reasonable range.

⁶The precondition of this property is often difficult to satisfy, leading to discards, but since it needs only be tested once for a given generator, the user can likely afford a while trying to falsify it.

3. Reflecting on Random Generation

External Correctness of a Reflective Generator The notions of soundness and completeness above are internal, focused on only the reflective generator itself, but we can also define *external* soundness and completeness with respect to some predicate on the generator's outputs.

We define the following properties:

Definition 6. A reflective generator g is *externally sound* with respect to p iff

$$x \in \text{gen } g \implies p \ x.$$

Definition 7. A reflective generator g is *externally complete* with respect to p iff

$$p \ x \implies (\text{not } . \text{ null}) (\text{reflect } x \ g).$$

Unlike internal soundness and completeness, external soundness and completeness may not be reasonable to check for every reflective generator. Sometimes there is no external predicate to check against; other times there may be a predicate, but the generator may intentionally be incomplete. But it is interesting and useful that both of these are *testable*; normal QuickCheck generators cannot test their own completeness.

3.4.2. Overlap

One last theoretical property of a reflective generator worth noting is its *overlap*.

Definition 8. A reflective generator's *overlap* for a given value is the number of different ways that the value could be produced.

Many reflective generators naturally have an overlap of 1, meaning that there is only one way to generate any given value, but some generators benefit from higher overlap. For example, a generator might pick between two high-level strategies for generating values for the sake of distribution control.

```
g1 = labelled [ ("Z", exact Z), ("S", fmap S (focus _S g1)) ]

gE =
  labelled
    [ ("Z", exact Z),
      ("S", fmap S (focus _S gE)), k
      ("2", fmap (S.S) (focus (_S._S) gE))
    ]

gI =
  labelled
    [ ("Z", exact Z),
      ("S", fmap S (focus _S gI)),
      ("inf", gI)
    ]
```

Figure 3.6.: Reflective generators with unit, exponential, and infinite overlap.

But overlap can cause problems for backward interpretations that care about examining all ways of producing a particular value (e.g., `probabilityOf` which we will define in §3.7). In these cases, overlap may lead to exponential blowup or even non-termination. For example, consider the three generators in Figure 3.6. The first, `g1`, generates natural numbers, each in exactly one way:

```
ghci> reflect g1 (S (S (S (S (S Z)))) -- 5
[["S","S","S","S","S","Z"]]
```

The second, `gE`, can generate numbers in exponentially many ways; specifically, it can generate a number by generating any sum of 1s and 2s that add to the desired total:

```
ghci> length (reflect gE (S (S (S (S (S Z)))))) -- 5
8
ghci> length (reflect gE (S (S (S (S (S (S ...))))))) -- 10
89
```

Computing `reflect gE` of a large number could take a very long time, which may be a problem for some use cases. Finally, we have `gI` which includes the option to make a no-op choice, `"inf"`:

```
ghci> length (reflect gI (S (S (S (S (S Z)))))) -- 5
...
```

Calling `reflect gI` does not terminate—there are infinitely many ways to generate 5. However, we conjecture that any generator with infinite overlap can be made into one that does not, by ensuring that any loop is guarded by some change to the generated structure.

3.4.3. Expressiveness

Reflective generators fall on a spectrum between simple grammar-based generators and complex monadic ones. Here, we show off the different kinds of data constraints that reflective generators can express and discuss a few idioms that they cannot express.

Grammar-Based and Monadic Generators Grammar-based generators [Godefroid, Kiezun, et al. 2008] use a context-free grammar describing the program’s input format as the basis for generating test inputs. For example, the following grammar fragment defines a generator of expression parse trees:

```
term -> factor | term "*" factor | term "/" factor
```

Read as a generator, this says “to generate a term, choose either a factor, a `"*"` node, or a `"/"` node.” Grammar-based generators are useful for generating inputs to a program with a context-free input structure, like expression evaluators, JSON minifiers, or even some compilers. They are often used in fuzzing, which we will discuss more in §3.6.3, for this reason. But grammar-based generators cannot ensure that the values they generate satisfy context-sensitive constraints. One might, for example, want to ensure that the left-hand side of a division does not evaluate to zero:

```
term -> factor | term "*" factor | term "/" nonzero(factor)
```

A complete generator of these expressions would require evaluation to take place during generation, which is not possible as part of a context-free grammar. And this is just the tip of the iceberg: there are a host of context-sensitive constraints that a generator might need to satisfy.

Enter monadic generators. As described in §3.2.1, monadic generators were introduced with QuickCheck and are a domain-specific language for writing generators that produce values satisfying arbitrary computable constraints. Monadic generators can generate binary search trees [Hughes 2019], well-typed terms in a simply-typed lambda calculus, and more. Monadic generators subsume context-free generators, for example, the following generator subsumes the term generator above:

```
term = oneof [fmap Factor factor, liftM2 Mul term factor, liftM2 Div term factor]
```

And with a bit more effort, we can exclude the parse trees with a divide-by-zero error:

```
term = oneof [ fmap Factor factor, liftM2 Mul term factor,
  do f <- factor
    if eval f == 0 then liftM2 Mul term (return f) else liftM2 Div term (return f) ]
```

There is technically one more rung on this ladder: Hypothesis generators. While they are not computationally more powerful than monadic ones, they are implemented in Python and can thus perform arbitrary side-effects while generating. See Appendix 6 for an example of a Hypothesis

3. Reflecting on Random Generation

generator. As we move on to analyzing reflective generators' expressiveness, we continue to focus on pure generators, but incorporating an extra monad argument (and thus, arbitrary effects) to reflective generators is compelling future work.

What Reflective Generators Can Do As a start, reflective generators are certainly at least as powerful as grammar-based generators.

Claim 1. Every grammar-based generator can be turned into a reflective generator via an analogous procedure to the one for monadic generators.

Justification. A grammar can be made into a monadic generator in the following way. For every rule $S \rightarrow \alpha_1 \mid \dots \mid \alpha_n$, we can write a generator

$$s = \text{oneof } [\text{liftM}_i C_1 \mathcal{T}(\alpha_1), \dots, \text{liftM}_j C_n \mathcal{T}(\alpha_n)]$$

where C_1 through C_n are fresh data constructors, and $\mathcal{T}$ translates each production by turning non-terminals into the appropriate sub-generator and turning terminals into Haskell strings. To turn that monadic generator into a reflective generator, simply add focus annotations that extract each argument from each constructor (C). $\square$

For example, this is the term generator that results from translating the grammar-based generator to a reflective generator:

```
term = oneof [
  fmap Factor (focus _Factor factor),
  liftM2 Mul (focus (_Mul._1) term) (focus (_Mul._2) factor),
  liftM2 Div (focus (_Div._1) term) (focus (_Div._2) factor) ]
```

In fact, reflective generators can implement all of the examples we previously listed as the motivation for monadic generators (see the binary search tree generator in Figure 3.1b and the STLC generator fragment below). There are also a variety of examples in §3.5.1 and §3.6 that are expressible by monadic generators and not context-free ones.

To see how reflective generators fare in a complex case, consider a reflective generator for terms in a simply-typed lambda calculus (STLC). The STLC generator is built from two sub-generators:

```
type_ :: Reflective Type Type
expr  :: Type -> Reflective Expr Expr
```

(The underscore prevents `type_` from being interpreted as a keyword.) The STLC generator works by picking a type, then generating a value of that type. The monadic version of the generator would simply write `type_ >>= expr`. But this does not work for a reflective generator; it needs an annotation. Specifically, what's needed is a mapping from `Expr` to `Type` that can focus on the type in the expression. Pleasingly, this focusing is precisely type inference! The type-correct reflective generator is: `comap typeOf type_ >>= expr`.

What Reflective Generators Can't Do Given that reflective generators seem to be able to express so much, it may be easier to characterize what they *can't* express. The biggest limitation of reflective generators is that they cannot represent any approach to generation that fundamentally loses information about previously generated data. For example, in the generator

```
lmap ??? g >>= \ _ -> g'
```

which is missing an annotation, there is no valid annotation to write: the value generated by g cannot be “focused” on as part of the final structure. Why might this come up? One case is when the generator generates a value and then computes some un-invertible function on it; there would be no way to recover the original value to analyze the choices made when producing that value.

We have run across very few generators that fundamentally require an un-invertible function, but one interesting example is some formulations of System F [Girard 1986; Reynolds 1974]. It is possible (though challenging) to write a monadic generator for System F [Palka et al. 2011], but impossible to do so for reflective generators. The problem can be seen when referring back to the reflective generator for STLC terms, which uses type inference to recover a type from an expression. If we tried to translate this generator to one for System F, we would have a problem: type inference for System F is undecidable! Thus, the generator may fail to run backward, even if it works correctly when run forward. Of course, in practice one could write a reflective generator for System F terms which would work modulo some time-outs, but this is a neat example of the dividing line between monadic and reflective generators.

System F is a rare example of a fundamental limitation of reflective generators, but there are a few common generation idioms that reflective generators need to work around. First, reflective generators cannot use the QuickCheck combinator “suchThat”, which samples a value and then, if it does not satisfy some predicate, throws it away, *increases the size parameter*, and samples another. The size manipulation is the problem here: to correctly reflect on a value, the backward direction would need to keep trying generators, recording and throwing away choices, until one succeeds; as far as we know this is not possible with the current structure. The solution is simply to avoid suchThat in favor of generators that satisfy predicates constructively—this would be our recommendation anyway, since suchThat can be extremely slow in complex cases. Second, reflective generators do not support a relatively common idiom where generators pick an integer and then use that integer to bias the generation distribution in some way. This is problematic because there is no way to recover that integer in the backward direction. One could theoretically encode this pattern via pick: instead of `do { i <- choose (0, n); k i }`, write a pick with n equally-weighted branches that each call k on a different value of i . But this is inefficient, so, in practice, a larger rewrite might be required to get the desired distribution of values.

Bottom line: reflective generators have some ergonomic limitations, but they are almost as powerful as monadic generators in practice.

3.5. Example-Based Generation

In this section, we demonstrate the power of reflective generators in the context of a clever generation technique that was an early inspiration for their design: example-based generation (§3.5.1). We replicate and generalize the prior work using reflective generators (§3.5.2).

3.5.1. Inputs from Hell

Inputs from Hell [Soremekun et al. 2020] (IFH) describes an approach to random testing that starts with a set of user-provided example test inputs and randomly produces values that are either quite similar to or quite different from those examples—the idea being that similar values represent “common” inputs and that different ones represent interestingly “uncommon” inputs. By drawing test cases from both of these classes, IFH is able to find bugs in realistic programs.

The IFH approach is based on grammar-based generation. Examples provided by the user are parsed by the grammar, and the resulting parse trees are used to derive weights for a probabilistic context-free grammar (pCFG) that generates the actual test inputs. For example, given a simple grammar for numbers

3. Reflecting on Random Generation

```
num -> "" | digit num          digit -> "1" | "2" | "3"
```

and the example 12, the IFH technique might derive the following pCFGs:

```
num -> 33% "" | 66% digit num    digit -> 50% "1" | 50% "2" | 0% "3"    -- common
num -> 66% "" | 33% digit num    digit -> 0% "1" | 0% "2" | 100% "3"  -- uncommon
```

Each production is given a weight based on the number of times it appears in the parse tree for the provided example (more or less weight, depending on whether the goal is to generate common or uncommon inputs). The first grammar puts more weight on the 1 and 2, since it is trying to generate more inputs like the initial example, whereas the second puts more weight on 3 because it is trying to generate inputs *unlike* 12.

3.5.2. Reflecting on Examples

This process—parse the input, analyze the parse tree, and re-weight the grammar—is straightforward to implement in the setting of grammar-based generation, but the IFH work does not extend to monadic generators. As we discuss in §3.4.3, this is a significant limitation. Reflective generators can bridge this gap by recapitulating the ideas in IFH but using a reflective generator for IFH’s parsing and generation steps.

Implementation We define three functions, corresponding to the parsing, analysis, and re-weighting operations for grammars in the IFH paper:

```
reflect      :: Reflective a a -> a -> [[String]]
analyzeWeights :: [[String]] -> Weights
genWithWeights :: Reflective b a -> Weights -> Gen a
```

We have already seen `reflect`: it reflects on a generated value and produces lists of choices that were made to produce that value. With the choice sequences in hand, `analyzeWeights` aggregates the choices together to produce a set of weights that say how often to expand one rule versus another. This allows a new interpretation, `genWithWeights`, to generate new values by making choices with the weights calculated from the user-provided examples.

When a reflective generator looks like a grammar (as with term in §3.4.3), this process replicates the IFH algorithm exactly. But we have seen that reflective generators are far more powerful than grammar-based generators, so the new algorithm both replicates *and generalizes* IFH, enabling example-based generation for a much larger class of generators.

These interpretations rely heavily on choice labels, which we discuss briefly in §3.3.2. These labels are used by `reflect` and `genWithWeights` to track the choices that should be weighted higher or lower based on the examples. This means that, rather than building reflective generators with one of or frequency, the programmer should use `labeled` or `pick`. Recall that the reflective generators library provides base generators that are labeled as well. However, there is some flexibility here: if the programmer would prefer some choices *not* be re-weighted based on examples, they can simply elide the labels. This is another way that the reflective generators approach generalizes IFH.

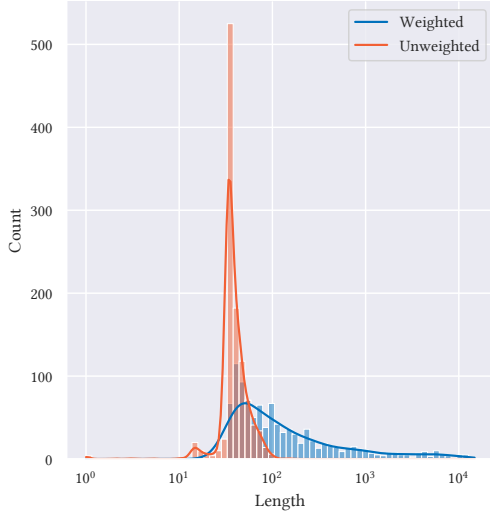
Example-Based Generation in Action Soremekun et al. [2020] use the IFH tuning algorithm as part of a comprehensive testing regime; by contrast, we have found them to be most useful as a quick way to tune a generator to yield a reasonable distribution of sizes and shapes.

To see this in action, consider a generator for JSON documents (the payload) along with a short hash of the document that can be used as a checksum:

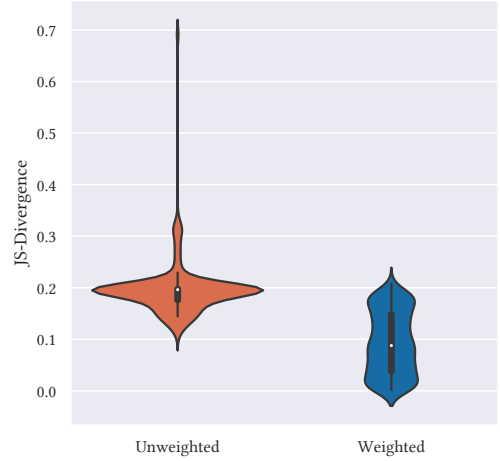
```
withHashcode :: Reflective String String
```

The full generator can be found in Appendix 7. This is inspired by a generator for JSON documents from the IFH paper, the reflective version of which is shown in full in our artifact. Note that, while the JSON part of the generator is equivalent to a context-free one, `withHashCode` is not (since it has to compute the hash during generation).

To demonstrate how these also achieve the goal of IFH-style generators (that the weighted generator is preferable to its unweighted counterpart), we sampled 10 JSON documents that were used in the IFH experiments, ranging from ~ 200 -1,200 bytes long, and used them to weight `withHashCode` in the style of IFH. We generated 1,000 documents from that weighted generator, as well as from the unweighted generator, and compare the results in Figure 3.7.



(a) Length distributions of unweighted and weighted.



(b) Jensen-Shannon divergence of character distributions. Unweighted vs. Examples and Weighted vs. Examples.

Figure 3.7.: Analysis of `withChecksum` tuned by example in the style of *Inputs from Hell*.

Figure 3.7a demonstrates that the weighted generator is far preferable to the unweighted version in terms of its length distribution. The unweighted distribution, shown in orange, is skewed to the left (smaller values) and has a huge spike. Inspecting the data revealed that the payloads of these values are all either `{}` or `[]`, both relatively uninteresting and certainly not worth generating hundreds of times! In contrast, the weighted generator has a varied length distribution. It generates very few trivial values, instead producing a wide distribution that covers more of the input space.

Figure 3.7b focuses on the samples' *character distributions*. We counted the occurrences of each character across all 10 of the example documents, resulting in a probability distribution over characters. Then, for each sample, we computed the Jensen-Shannon divergence⁷ [Lin 1991], between the example distribution and the character distribution of the sample and plotted those divergences in a violin plot. JS divergence measures the difference between two probability distributions, so it is a simple way of getting a sense of how similar or different the characters in the samples are from the ones in the examples. The plots show that the unweighted samples are farther from the the example distribution than the weighted samples.

Without this example-based tuning, the developer of `withHashCode` would need to think carefully about the distribution that they want and even harder about how to alter the generator weights to get

⁷Jensen-Shannon divergence is closely related to the more common Kullback-Leiber (KL) divergence [Kullback and Leibler 1951], but it works better for distributions with differing support because its value is never infinite.

3. Reflecting on Random Generation

there. With example-based tuning, they simply need to assemble 10 or so examples, compute weights from those, and then use those weights for generation instead.

3.6. Validity-Preserving Shrinking and Mutation

The example-based generation in the previous section illustrates some of the benefits of reflecting on choices. In this section, we explore those benefits further, using them to implement test input manipulation algorithms like shrinking and mutation. In this section, we discuss the “internal test-case reduction” algorithms implemented in the Hypothesis framework for PBT (§3.6.1), show that reflective generators make these algorithms much more flexible (§3.6.2), and finally sketch the ways that these ideas also apply to test-case mutation (§3.6.3).

3.6.1. Test-Case Reduction in Hypothesis

Shrinking is the process of turning a large counterexample into a smaller one that still triggers a bug. Shrinking is critical in PBT because bugs are often tickled by very large inputs that are nearly impossible to use for debugging—shrinking makes it much easier to understand which specific bits of the value are actually necessary to trigger the bug.

In QuickCheck, users can either use GHC’s Generics [Magalhães et al. 2010] to derive a shrinker automatically for a given type, or they can write a shrinker by hand. The former is effective in simple cases, as we will see below, but it is not very general—these automatic shrinkers only know about the type structure, so they cannot ensure that the shrunk values satisfy important preconditions nor adequately shrink less structured data like strings. The latter is totally general, but many users find writing shrinkers by hand confusing and error-prone.

This unsatisfying situation led [David R. MacIver and Donaldson \[2020\]](#) to design Hypothesis’s “internal test-case reduction,” which gives the best of both worlds. It solves the generality issue without requiring user effort or understanding. The key insight is that the generator itself already has all of the information needed to produce precondition-satisfying inputs, so the generator should be used as part of the shrinking process. The accompanying clever trick is to *shrink the random choices* used to generate a value, rather than shrinking the value itself.

```
unlabeled = oneof
  [ exact Leaf,
    Branch
      <$> focus (_Branch._1) unlabeled
      <*> focus (_Branch._2) unlabeled ]
```

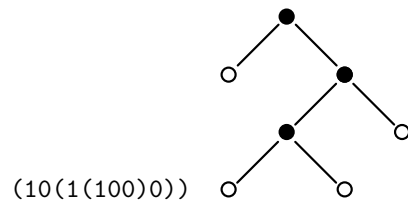


Figure 3.8.: A Hypothesis-inspired reflective generator and a tree that the generator might produce.

Concretely, Hypothesis represents its input randomness as a bracketed string of bits. For example `(10(1(100)0))` produces the tree in Figure 3.8. The first bit says to expand the top-level node, the second says that the left-hand subtree is a leaf, and so on. Each level of bracketing delineates some choices that are logically nested together (in this case, on a particular level of the tree). Hypothesis aims to shrink these bitstrings by finding the *shortlex minimum* string of choices that results in a valid counterexample; shortlex order considers shorter strings to be less than longer strings, and follows lexicographic ordering otherwise (brackets are ignored for the purpose of ordering). In practice, shortlex order turns out to be an effective proxy for complexity of generated test cases: smaller bitstrings tend to produce smaller test cases.

The actual shrinking procedure uses a number of different passes, each of which attempts to shorten the choice string, swap 1s with 0s, or both, resulting in a shortlex-smaller choice string. The passes are described in the Hypothesis paper and available in the open source codebase⁸.

3.6.2. Reflective Shrinking

The downside of the Hypothesis approach is that this style of shrinking only works if the random bitstring that produced the target value is still available—without it, there is nothing to shrink. But there are many reasons one might want to shrink a value for which one does *not* have a corresponding bitstring. In particular, shrinking can be useful for understanding externally provided values that were not produced by the generator at all; for example, if a user submits a bug report containing a printout of some large input that caused a crash, it might be much easier to debug the problem with the help of a shrinker. Similarly, internal shrinking does not work if the value has been modified at all between generation and shrinking, as might be desirable when doing fuzzing-style (see §3.6.3) testing where test inputs are mutated to explore values in a particular region. Luckily, reflective generators can help.

Extracting Bracketed Choice Sequences We implement *reflective shrinking* via another interpretation of reflective generators, with the following type:

```
data Choices = Choice Bool | Draw [Choices]
choices :: Reflective a a -> a -> [Choices]
```

The Choices type describes rose trees with two types of nodes: Choice, which represents a single-bit choice, and Draw which represents some nested sequence of choices;⁹ this type is isomorphic to the bracketed choice sequences that Hypothesis uses. The choices function takes a reflective generator and a value and produces the choice sequences that result in generating that value.

The implementation of choices is similar to that of reflect. It performs a “backward” interpretation of the generator, keeping track of choices as it disassembles a value. This interpretation ignores choice labels, since Hypothesis shrinks at a lower level of abstraction. Instead, the interpretation of a Pick node determines how many bits would be required to choose a branch (by taking the log of the length of the list of sub generators) and then assigns the appropriate choice sequences to each branch. For example:

```
choices (oneof [exact 1, exact 2, exact 3]) 2 = [Draw [Choice False, Choice True]]
```

⁸<https://github.com/HypothesisWorks/hypothesis>

⁹In the Haskell artifact, we use a slightly more complicated type, caching size information to make shortlex comparisons faster.

3. Reflecting on Random Generation

Table 3.1.: Average size of shrunk outputs after reflective shrinking, compared with Hypothesis shrinking, QuickCheck’s genericShrink, and un-shrunk inputs. (Mean and two standard-deviation range.)

*Hypothesis experiments not re-run, data taken from [David R. MacIver and Donaldson 2020].

	Reflective	Hypothesis*	QuickCheck	Baseline
binheap	9.15 (8.00–10.30)	9.02 (9.01–9.03)	9.14 (8.12–10.16)	14.89 (7.01–22.77)
bound5	3.06 (0.60–5.52)	2.08 (2.07–2.10)	17.75 (0.00–62.32)	131.48 (0.38–262.59)
calculator	5.03 (4.54–5.52)	5.00 (5.00–5.00)	5.07 (4.21–5.92)	13.75 (1.60–25.90)
parser	3.70 (2.21–5.20)	3.31 (3.28–3.34)	3.67 (2.69–4.64)	40.04 (0.00–127.51)
reverse	2.00 (2.00–2.00)	2.00 (2.00–2.00)	2.00 (2.00–2.00)	2.67 (0.76–4.57)

(10(1(100)0)) => (1(100)0)
(100)

(a) Shrinks from subTrees.

(10(1(100)0)) => (10(0000))
(10(0000))
(10(000))
(10(00))
(10(0))
(10)

(b) Shrinks from zeroDraws.

(10(1(100)0)) => (0111000)
(1010100)

(c) Shrinks from swapBits.

Shrinking Strategies With an appropriate bracketed choice sequence in hand, shrinking can begin. We implemented a representative subset of the shrinking passes described in the Hypothesis paper: one pass tries shrinking to every available child sequence of the original, a second replaces Draw nodes with zeroes, and a third swaps ones and zeroes to produce lexically smaller choices strings. The results of subTrees, zeroDraws, and swapBits are shown in Figures 3.9a, 3.9b, and 3.9c accordingly.

Replicating Hypothesis Evaluation To check that we replicated Hypothesis shrinking correctly, we replicated one of the experiments from the Hypothesis paper. David R. MacIver and Donaldson borrowed five examples from the SmartCheck repository [Pike 2014] that represent a varied range of shrinking scenarios. Each example comes with a property, a buggy implementation, and a QuickCheck generator; the goal was to run the property to find a counterexample and shrink that counterexample to the smallest possible value.

We upgraded the existing QuickCheck generators to reflective ones, making minor modifications where necessary: we replaced uses of suchThat with generators that satisfied invariants constructively, modified some of the approaches to distribution management, and added appropriate reflective annotations. These modifications are based on the observations from §3.4.3. Then, we ran each experiment 1,000 times and reported the average size of the resulting counterexamples in Table 3.1. Note that the QuickCheck and baseline numbers come from the generate interpretation of the upgraded reflective generator, rather than the original generator.

We find that reflective shrinkers perform just as well as QuickCheck’s genericShrink in all cases, and significantly better in bound5. With a few caveats, reflective shrinkers also match Hypothesis. They exhibit a higher variance in the size of counterexamples that they produce, likely because they only implement a subset of Hypothesis’s shrinking strategies, but nevertheless their counterexamples are on average within 1 unit of Hypothesis (and usually much closer). The worst experiment relative to Hypothesis is bound5; in that example, we suspect the difference is due to differing strategies for generating integers, rather than anything to do with shrinking directly.

A Realistic Example As a final demonstration that reflective shrinkers are useful, we return to a modified version of the JSON example used in §3.5. We define a generator for “`package.json`” files, which are used as a configuration format in Node.js. Programs that process these files may be used by millions of users, so a user may indeed find a bug in the program that a PBT regime did not.

Imagine a scenario where a user finds a bug where a program behaves incorrectly only when the file specifies a specific version of a specific package. In this case, shrinking would be extremely helpful: it would help the developers of the program isolate the exact lines in the JSON file that cause a problem. But shrinking a JSON file like this is impossible for both `genericShrink` and `Hypothesis`. The former does not work because the format is too unstructured: the generator produces JSON strings, rather than a Haskell datatype, so the best the shrinker could do is shorten the string (which would result in invalid JSON). The latter does not even start to shrink, since the JSON file came from a user, and therefore there is no random bitstring to shrink.

A reflective shrinker, however, works perfectly. Appendix 8 shows two JSON documents, the first a full “buggy” version and the second a shrunk version. The shrunk JSON document could point a developer to the precise issue with their program.

3.6.3. Reflective Mutation

HypoFuzz, a tool for coverage-guided fuzzing [Fioraldi et al. 2020] of PBT properties, is a newer and lesser-known part of the Hypothesis ecosystem. Like Crowbar [Dolan and Preston 2017], HypoFuzz uses a PBT generator to aid the fuzzer. Fuzzers try to maximize code coverage by keeping track of a set of interesting inputs and *mutating* them, attempting to explore similar values and hopefully continue to cover new branches of the program. “Mutating well” can be challenging, since naïve mutations will rarely produce valid values; HypoFuzz gets around this concern with the same trick Hypothesis uses for fuzzing: mutate the randomness, not the value.

Internal mutation has all of the same benefits and drawbacks as internal shrinking. On the positive side, it is type agnostic, easy to use, and guarantees validity of the mutated values. On the other side, it assumes that the randomness used to produce a given value is available. It may seem like this drawback is less of an issue for mutation than it is for shrinking, since the fuzzer can just keep track of the random choices associated with each value it wants to mutate, but this is not true of the initial set of *seed inputs*. For optimal fuzzing, the seeds are provided by the user and represent some set of initially interesting values that the fuzzer can play with. but this does not work with Hypothesis-style mutation: the seeds needed for this style are not user-comprehensible values but choice sequences! Once again, reflective generators provide a compelling solution. We can simply extract a choice sequence from each seed using the `choices` interpretation.

3.7. Improving the Testing Workflow

So far we have seen reflective generators in the context of example-based generation, shrinking, and mutation. In this section, we explore several more useful interpretations, demonstrating reflective generators’ power and flexibility.

3.7.1. Reflective Checkers

The “bigenerators” in the original work on PMPs [L.-y. Xia et al. 2019a] can be viewed as a special case of reflective generators. Rather than reflect on a value and produce choices, a bigenerator simply checks whether a value is in the range of the generator, effectively checking if the value satisfies the invariant

3. Reflecting on Random Generation

that the generator enforces. Reflective generators can do this too, by reflecting on the generator’s choices and asking whether or not there exists a set of choices that results in the desired value.

Going further, a reflective generator can calculate the *probability* of generating a particular value with the generate interpretation. We implement this in our artifact as an interpretation, `probabilityOf`, which tracks the different ways of generating a particular value and the likelihood of choosing those different ways. Obviously this works best when the generator’s overlap is low (see §3.4.2)—in cases where overlap is exponential or infinite this may be slow or fail to terminate.

3.7.2. Reflective Completers

A rather different use case for reflective generators is generation based on a *partial value*. For example, imagine a binary search tree with holes:

```
Node (Node _ 1 _) 5 _
```

Reflective generators provide a way to *randomly complete* a value like this, filling the holes with appropriate randomly generated values:

```
Node (Node Leaf 1 Leaf) 5 Leaf
Node (Node Leaf 1 (Node Leaf 3 Leaf)) 5 (Node (Node Leaf 6 Leaf) 7 Leaf)
```

This technique lets the user pick out a sub-space of a generator, defined by some value prefix, and explore that sub-space while maintaining any preconditions that generator enforces. We accomplish this with some carefully targeted hacks, representing a partial value as a value containing undefined: `Node (Node undefined 1 undefined) 5 undefined`.

Suppose, now, that this value were passed into a backward interpretation of `bst` from §3.1—where would things fail? The key insight is that the *only* place a reflective generator manipulates its focused value is when re-focusing. In other words, the only place a backward interpretation can crash on a partial value is while interpreting `Lmap`. Capitalizing on this insight, we wrap the standard `Lmap` interpretation in a call to `catch`, Haskell’s exception handling mechanism:

```
complete :: Reflective a a -> a -> IO (Maybe a)
...
interpR (Lmap f x) b =
  catch
    (evaluate (f b) >>= interpR x)
    (\(_ :: SomeException) -> fmap (: []) (QC.generate (generate (Bind x Return))))
```

As long as no exception occurs, the code works as before. If there is ever an exception, the current value is abandoned and the rest is generated via the generate interpretation. In other words, `complete` mixes both backward and forward styles of interpretation to achieve its goals.

This trick works best for “structural” generators that only do shallow pattern matching in `Lmaps`, things fall apart if the backward direction needs to evaluate the whole term. The clearest example of this is `comap typeOf type_ >>= expr` (recall, it generates a type and then a program of that type); in the backward direction, this generator immediately evaluates the whole term to compute its type. For this generator, `complete` would just generate a totally fresh program.

Users may be able to work around this by making their predicates lazier. For example, one could imagine writing an optimistic type checking algorithm `optimisticTypeOf` that maximizes laziness by blindly trusting user-provided type annotations. The user could then use the reflective generator `comap optimisticTypeOf type_ >>= expr` to complete an incomplete term like `App (Ann (Int -> Int) undefined) (Ann Int undefined)`. The completer would successfully determine that the type of the whole expression is `Int`, and then it would have enough information to complete the undefineds with well-typed expressions.

3.7.3. Reflective Producers

Weighted random generation in the style of QuickCheck is not the only way to get test inputs: both enumeration [Braquehais 2017; Duregård et al. 2012; Runciman et al. 2008] and guided generation [Fioraldi et al. 2020; Zalewski 2022] have been explored as alternatives. Indeed, much of the PBT literature has moved from talking about *generators* to talking about *producers* of test data, where the specific strategy does not matter [Paraskevopoulou et al. 2022; Soremekun et al. 2020]. We say “generators” here because it is familiar and concrete, but reflective generators might better be considered as *reflective producers* because they can be used in these styles.

A reflective generator can be made into an enumerator by interpreting `Pick` as an exhaustive choice rather than a random one. We implement an interpretation

```
enumerate :: Reflective b a -> [[a]]
```

for “roughly size-based” enumeration, leaning heavily on the combinators and techniques found in LeanCheck [Braquehais 2017]. We say “roughly” because, whereas LeanCheck enumerators allow the user to define their own notion of size for each enumerator, reflective generators are limited to a single notion of size based on the number and order of choices needed to produce a given value. A thorough evaluation of this discrepancy would require its own study, but early experiments are promising. For example, `enumerate (bst (1, 10))` reaches size-4 BSTs before its 10th enumerated value and matches the sizes for an idiomatic LeanCheck enumerator (Appendix 9).

While fuzzing is sometimes treated as a separate topic from PBT—focused on finding crash failures by generating inputs external to a system rather than finding more subtle errors in individual functions—a number of recent projects have attempted to bridge the gap, and reflective generators may offer a useful unifying framework for such efforts. We already saw that reflective mutators are helpful in the context of HypoFuzz-style mutation; reflective generators can also be used to interface with an external fuzzer in the style of Crowbar [Dolan and Preston 2017], which is designed to get its inputs from popular fuzzers like AFL or AFL++ [Fioraldi et al. 2020; Zalewski 2022]. Since Crowbar already uses a free-monad-like structure to represent its generators, we can imagine writing an equivalent reflective generator interpretation that works the same way. More generally, reflective generators can be used in any producer algorithm that uses a generator as a parser.

3.8. Related Work

This work builds on the ideas of free generators [Goldstein and Pierce 2022] and partial monadic profunctors [L.-y. Xia et al. 2019a]. Free generators are, in turn, built on top of freer monads [Kiselyov and Ishii 2015], which were initially invented as a better way to represent effectful code in pure languages. While our implementation remains faithful to the basic conception of freer monads, there are many insights from Kiselyov and Ishii that we have not yet explored. Likewise, PMPs are part of the long tradition of bidirectional programming [John Nathan Foster 2009], and it remains to be seen if there are stronger ways to tie reflective generators to work on other bidirectional abstractions.

The concrete realization of reflective generators is also related to Crowbar [Dolan and Preston 2017]. Both libraries are implemented with a syntactic, uninterpreted representation for generators, although the Crowbar version does not incorporate any ideas from monadic profunctors and uses a different type for `bind` that does not normalize as aggressively.

The idea of reflective generators was originally sparked by the tools developed in *Inputs from Hell* [Soremekun et al. 2020], and these tools in turn tie into the broader world of grammar-based generation [Aschermann et al. 2019; Godefroid, Kiezun, et al. 2008; Godefroid, Peleg, et al. 2017; Holler et al. 2012; Srivastava and Payer 2021; Veggiam et al. 2016; J. Wang et al. 2019]. Grammar-based

3. Reflecting on Random Generation

approaches are less expressive than monadic ones, since they can only generate strings from a context-free grammar, and therefore cannot generate complex data structures with internal dependencies.

Replicating example distributions is a classic problem in *probabilistic programming* [Gordon et al. 2014]. While the goals of probabilistic programs are usually quite different from those of PBT generators, there is some overlap in the formalisms used to express these ideas. In particular, one representation of probabilistic programs in the functional programming literature [Ścibior et al. 2018] uses a free monad that is similar to free and reflective generators.

Reflective shrinking and mutation build heavily on ideas in the Hypothesis framework [David R. MacIver, Hatfield-Dodds, et al. 2019; David R. MacIver and Donaldson 2020], but there are other approaches to automated shrinking. As mentioned in §3.6, QuickCheck provides `genericShrink`, which provides a competent shrinker for any Haskell type that implements `Generic`. While `genericShrink` is a decent starting point, it fails to shrink unstructured data (like strings) and values with complex preconditions. Another alternative is provided by Hedgehog [Stanley 2023], another Haskell PBT library. Hedgehog shrinking is similar to Hypothesis shrinking, both using the structure of the generator to automatically shrink values. It has the same limitation: externally provided values cannot be shrunk.

3.9. Conclusion and Future Directions

Reflective generators are a powerful abstraction for producing and manipulating test inputs. We have developed their theory and demonstrated their utility in a variety of testing scenarios, including example-based generation, shrinking, mutation, precondition checking, value completion, enumeration, fuzzing, and more. We plan to build on reflective generators, automating their creation and improving their usability.

Automation and Synthesis of Annotations The Lmap annotations in reflective generators can be arbitrarily complex, but in practice they are usually simple, predictable functions that operate on the input’s structure. We hope that, in a large variety of cases, the annotations can be *synthesized*.

We plan to work with Hoogle+ [James et al. 2020], using its type-based synthesis algorithm to obtain candidate programs for the annotations with no user intervention. This is an especially compelling opportunity because it is easy to tell whether annotations are correct: they must be sound and complete, as described in §3.4. When synthesizing multiple annotations at the same time, the system can even use the number of examples that pass or fail the soundness and completeness properties as a way to infer which annotations are correct and which need to be re-synthesized—if changing an annotation increases the number of passing tests, it is more likely to be correct; if the change causes more tests to fail, it is likely wrong. If this idea works, it could make transitioning from QuickCheck generators to reflective generators almost entirely automatic.

Usability We have taken care to design an API for reflective generators that aligns with existing QuickCheck functions and minimizes programmer effort. Our own experience writing reflective generators studies has been positive, and, except for the aforementioned limitations (§3.4.3), we ran into no issues upgrading existing generators. The automation techniques hypothesized above could make reflective generators even more usable. Still, we certainly do not constitute a representative sample of PBT users: the usability of reflective generators should be studied empirically.

There is a growing push in the PL community to incorporate ideas and techniques from human-computer interaction (HCI) [Chasins et al. 2021], and this is a perfect opportunity to join that movement. We will collaborate with HCI researchers on a usability analysis of reflective generators. Inspired by prior work [Coblentz et al. 2021], this analysis will be useful for refining our design.

Part II.

Binders

4. Embedding by Unembedding

In this part of the thesis, we switch from monadic composition to the compositionality that comes from using higher-order functions. For an embedded language to be higher-order, and enjoy the entailed expressively benefits, it must feature binders. This chapter focusses on the embedding of binders for languages with complex semantics, which previously came at the expense of usability. The presented technique, Embedding by Unembedding, solves this problem, allowing for languages with complex semantics to enjoy an expressive and ergonomic function vernacular.

This chapter’s content is taken from *Embedding by Unembedding* [Matsuda, Frohlich, et al. 2023], which was published at the 2023 International Conference on Functional Programming. This work was done in collaboration with Kazutaka Matsuda of Tohoku University, my supervisor Meng Wang, and Nicolas Wu of Imperial College London.

4.1. Introduction

Programming languages solve problems in an expressive and flexible way. When there is a particular problem to be solved, a *domain-specific language* (DSL) is desirable as it specializes to the problem domain in question. DSLs can be *embedded*, that is created, as a library within another *host* language, to save implementation effort and allow *guest* language users access to host language features. A host language feature that is desirable to reuse is its binding constructs.

Binders are important because they introduce variables, with common binders including **let** and λ -abstractions. A from-scratch implementation of binders is cumbersome and error prone. A popular alternative for embedding a language with binders is Higher-Order Abstract Syntax (HOAS) [Church 1940; G. P. Huet and Lang 1978; Miller and Nadathur 1987; Pfenning and Elliott 1988]. As the name suggests, this technique uses higher-order functions to implement binders, essentially reusing the host’s function binders as guest binders. There has been much development in this area, producing many variants of HOAS including the finally-tagless style [Carette et al. 2009], which supports extensibility in both syntax and semantics; or parametric HOAS (PHOAS) [Chlipala 2008], which adds support for non-compositional semantics.

However, HOAS and its variants encounter issues when trying to embed languages with advanced semantics that do not adhere to the traditional evaluation mode of taking open expressions and mapping free variable assignments to results. For example, reversible/bidirectional computation, exhibited in Fig. 4.1, suffers from this. Here, the open expression `succ x` can be evaluated forward (or “traditionally”) by providing the values of the free variables (in this case the value of `x`) to get the result; or it can

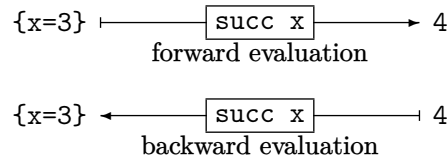


Figure 4.1.: Example of Advanced Semantics: Reversible Computation

4. Embedding by Unembedding

be evaluated backward by providing the result to yield the assignment of x . The problem lies in the attempt to embed guest functions with non-standard semantics as host functions with standard semantics.

So far, this problem has remained undocumented. But we suspect that it is the root cause of some of the struggles faced by language designers. Take the bidirectional programming literature as an example. Much work has gone into this area, with the popular approach of *lenses* and combinators [J. Nathan Foster et al. 2005] being awarded POPL’s most influential paper in 2015, 10 years after its publication. But lenses (and all the follow on work) [Matsuda and M. Wang 2018a,d; Pacheco and Cunha 2010] struggled to include bindings in an embedded setting, resulting in significant difficulties in programming [Miltner et al. 2017]. There is a gap in the literature surrounding this problem, and it is not limited to bidirectional programming. We also investigate the problem in incremental computation, and speculate automatic differentiation, probabilistic programming, and quantum programming as potential targets for future exploration.

This paper pin-points and solves the issue of embedding binders and interpreting open expressions in domains where it was previously not possible. Our solution builds upon a technique known as *unembedding* [Atkey 2009; Atkey et al. 2009], which converts from the HOAS representation to de Bruijn-indexed terms. The crucial insight is that unembedding can be used to bridge the gap between the elegant syntax of HOAS and advanced domain semantics that were previously out of reach. The success of this solution is exhibited by our two substantial case studies, covering state-of-the-art languages in two of the domains afflicted with this problem: incremental computation and reversible computation. The incremental computation example successfully unembeds the Cache-transfer-style (CTS) [Giarrusso et al. 2019] variant of the ILC [Cai et al. 2014]. CTS builds upon the original language, which created incremental functional programs, to create even more incremental programs. In the domain of reversible programming, we create the first embedding of a bidirectional language with binders by embedding HOBiT [Matsuda and M. Wang 2018d], a higher-order lens language.

We name our new embedding framework *Embedding by Unembedding*. It acts as the glue between a familiar HOAS frontend and a system for managing environments, generalizing the original work that converts to de Bruijn-indexed terms, to allow for binders in more interesting semantic domains. We design techniques to tame tedious management of environments and propose generic steps of embedding with the new framework. Specifically, we make the following contributions:¹

- We identify a long-standing problem of being unable to use HOAS for complex application domains of languages with binders, pin-pointing the issue as a mismatch between host and guest language function semantics (§4.2).
- We propose unembedding (§4.3) as a solution to this problem.
- We build a new embedding framework based on unembedding and the finally-tagless style (§4.4).
- We conduct a case study by embedding Giarrusso et al.’s CTS (§4.5). The resulting language is easy to use, with implementation details properly hidden from programmers and scales well to higher-order constructs.
- We conduct another case study by embedding the bidirectional programming language HOBiT [Matsuda and M. Wang 2018d] (§4.6). The resulting language is the first successful embedding of a bidirectional language with binders.

We then discuss related work (§4.7) and future work (§4.8) before concluding (§4.9).

¹A proof-of-concept prototype implementation of Embedding by Unembedding, which also contains the examples used in this paper, is available from <https://doi.org/10.5281/zenodo.7990806>.

4.2. The Problem with **HOAS**

This section identifies and clarifies the problem with **HOAS** that our framework addresses. It begins by covering the necessary background to understand the problem (§4.2.1), then highlights the problem (§4.2.2).

4.2.1. Background: **HOAS**

The motivation for **HOAS** is that the manipulation of variables is cumbersome and sometimes hard to get right [Church 1940; G. P. Huet and Lang 1978; Miller and Nadathur 1987; Pfenning and Elliott 1988]. To do it properly, a number of issues need to be carefully considered: if there were two expressions bound to the same name, which expression is provided when that name is called (*name duplication*); whether this changes if the binders are nested (*nesting*); what happens when a name is called outside of its scope (*out of scope referencing*).

In an embedding setting, **HOAS** completely circumvents these problems by not implementing binders, but reusing the host language’s binding infrastructure. It does this by encoding guest binders as higher-order functions in the host.

For example, **HOAS** would embed a `let` binder as a function with the following type, for some `Exp` (where `Exp` represents some arbitrary semantic domain in this shallow embedding example):

```
letE :: Exp a -> (Exp a -> Exp b) -> Exp b
```

Here, the guest binder is created using a higher-order host function, where the first argument is the expression that gets bound, and the second is where the bound expression gets used. This allows for guest level terms to be written using host binders:

```
term y = letE y (\x -> x)
```

This term expresses a `let` binder that does nothing, taking the variable `y` and doing nothing to it in the body. The point is to highlight how `x` is available to name the bound expression and use it in the body of the `let`, but no implementation effort was needed to control this naming, or its scoping, or the other binder issues. This is the ease and elegance of **HOAS**: an implementer gains the full expressivity of the host’s binders, without any of the effort.

Given its convenience, **HOAS** is the foundation of some prominent embedding techniques including *finally-tagless style* [Carette et al. 2009]. In Haskell, this consists of an embedding whose functions are given by a typeclass [Gibbons and Wu 2014] (or alternatively in ML families by a signature). Finally-tagless style is modular and extensible in both syntax and semantics: constructs can be grouped into different or dependent typeclasses; new constructs can be added as new typeclasses or methods; and new interpretations can be added as new typeclass instances. The `let` expression example can switch to this style, allowing it to generalize over the semantics `Exp`:

```
class Let exp where
  lete :: exp a -> (exp a -> exp b) -> exp b
```

While we have focussed on the finally-tagless [Carette et al. 2009] variant of **HOAS**, this problem also affects parametric **HOAS** (PHOAS) [Chlipala 2008]. It may add support for non-compositional semantics, however, it is still not easy to implement advanced semantics in PHOAS for the same reason that it is hard for **HOAS**.

4.2.2. The Problem

With **HOAS** (and its variants), the semantics of the term under a binder in the guest language is implemented as a host language function. This representation permits only one operation: the

4. Embedding by Unembedding

application of the function. This is sufficient in many cases, but becomes a problem with complex semantic domains, where additional manipulation of variable environments is required outside of the standard instantiation of variables with function arguments. This is not just because these domains often feature non-standard binding constructs, but more crucially, the way the *open expressions* (found inside the binders) are treated differs from standard function bodies.

Take invertible computation as an example. An expressive language in this domain features special backward case constructs and pattern matching (details in §4.6). Moreover, unlike the standard computation model where an expression evaluates to a value under a value environment, the backward semantics is commonly modelled as an interpretation of an open expression (with the binder removed) and a given value to produce a value environment containing the bound variables (see, *e.g.*, [Yokoyama et al. 2011]). For example, the open expression `succ x` from Fig 4.1, when executed backwards with the given value 4 produces a binding environment $\{x = 3\}$. It is not obvious how this behaviour can be encoded by a function as HOAS requires.

A similar problem arises in the exploration of incremental computation, where recomputation is avoided to achieve a performance gain. Cai et al. [2014]’s incremental λ -calculus achieves the goal by interpreting a function as a mapping from input changes to output changes. Open expressions are instrumental here too, as they have clear notions of input and output as opposed to closed ones. The situation is even more complicated with the state-of-the-art Cache-transfer-style (CTS) [Giarrusso et al. 2019] variant, which introduces a notion of “cache” to support non-*self-maintained* computations [Cai et al. 2014]. We defer the details to the case study (§4.5). The key to know here is that this “cache” is a state carried through and updated by the computation, in addition to the already complex semantics of the open expressions. Again, HOAS struggles to encode this behaviour.

These domains share a common characteristic that the complex semantics requires some kind of manipulation of the environment. This is an interesting dilemma. On the one hand, HOAS gives us a simple programming interface as it avoids manipulation of variables. On the other hand, some semantics require access to environments, *i.e.*, assignments to variables, which are hidden by HOAS (for good reason).

In this paper, we advocate Embedding by Unembedding as a solution that works for both HOAS and PHOAS, and allows the embedding of a guest language whose semantics differ from its host, without sacrificing the desirable HOAS (or PHOAS) interface.

4.3. Background: The Technique of Unembedding

Embedding by Unembedding builds upon *Unembedding* [Atkey 2009; Atkey et al. 2009]. This section elaborates on this technique, covering what it is, its original motivation, and why it is suitable for our purposes (§4.3.1); how it leverages de Bruijn-indexed terms as a more amenable representation for their purposes and how this generalizes to solve our embedding problem (§4.3.2); and the details of how the technique works (§4.3.4).

4.3.1. What and Why

Unembedding is a technique that gives a round tripping transformation from a finally-tagless [Carette et al. 2009] language representation to de Bruijn-indexed terms. As the name suggests, the technique is in a sense the opposite of HOAS embedding, converting an embedded implementation to an explicit Abstract Syntax Tree (AST) representation. The original motivation for this work is supporting certain optimizations that require global analysis of the AST [Atkey et al. 2009], to which de Bruijn-indexed terms are more amenable than the finally-tagless representation.

We observe that this technique has much greater applicability. Our iteration on their work distills out the desirable qualities of the representation into a generalization that we will introduce later.

4.3.2. The de Bruijn Representation

We first introduce a representation of environments, as the explicit access to environments is the advantage of de Bruijn representations over HOAS, and this explicit access is also crucial in our generalization presented in this paper. An environment is a collection of values of different types.

The simplest way to achieve this in Haskell is as a tuple, but as this gets messy, a heterogeneous list [Kiselyov, Lämmel, et al. 2004] is more appropriate:

```
data VEnv (as :: [k]) where
  VENil  :: VEnv '[]
  VECons :: a -> VEnv as -> VEnv (a ': as)

exEnv = VECons 1 (VECons 'a' VENil) :: VEnv '[Int,Char]
```

(Here, “`’:`” and “`’[]`” are the type-level versions of the cons operator “`:`” and the nil “`[]`”, respectively.) `VEnv` works the same way as a normal list, but with the addition of a type level list (`as`), allowing it to hold values of different types as desired to implement an environment. For example, `exEnv` stores both an `Int` and a `Char`. This environment of values is a good start, but there are many other types of environment that may be required. For example, the guest types might be distinguished from Haskell types using a wrapper type; or one might wish to store guest expressions, as opposed to values, to represent open terms; or perhaps a value-level representation of a type environment is desired. To accommodate these, a common approach² is to generalize `VEnv` to wrap a container around the stored values:

```
data Env (f :: k -> Type) (as :: [k]) where
  ENil  :: Env f '[]
  ECons :: f a -> Env f as -> Env f (a ': as)
```

Different instantiations of this container `f` create different environments:

```
type VEnv    = Env Identity -- Value environment.
type EEnv exp = Env exp     -- Expression environment.
type TEnv    = Env Proxy    -- Type environment.
```

The `Identity` container³ creates a value environment the same as `VEnv`; `exp` allows an environment to store guest language expressions; `Proxy`⁴ creates a type environment.

This representation allows us to provide a unified interface for manipulation of environments. For example, the environment infrastructure includes a way to index into environments:

```
lookEnv :: Env f as -> Ix as a -> f a
lookEnv (ECons v _)   IxZ      = v
lookEnv (ECons _ env) (IxS n) = lookEnv env n

data Ix (as :: [k]) (a :: k) where
  IxZ :: Ix (a ': as) a      -- At element in question.
  IxS :: Ix as a -> Ix (b ': as) a -- Element lies further into the env.
```

²See, e.g., <http://agda.github.io/agda-stdlib/Data.Star.Environment.html>.

³`Identity` is a container that does nothing, representing the identity functor/monad in Haskell.

It is defined: `newtype Identity a = Identity a`

⁴`Proxy` is a special Haskell type, that attaches a type to a placeholder constructor, holding no data, meaning that no values end up in the environment, but the type level list is preserved.

It is defined: `data Proxy a = Proxy`

4. Embedding by Unembedding

```
class STLCChoas exp where
  lam :: (exp a -> exp b) -> exp (a -> b)
  app :: exp (a -> b) -> exp a -> exp b
```

Figure 4.2.: STLC HOAS Representation

Here, `lookEnv` takes an environment and a witness of type `Ix`, which ensures that a type `a` occurs in a type level list `as`. In the context of `lookEnv`, this corresponds to ensuring that the environment contains the corresponding `f a` value. To extract the value, it walks down the environment to the index at which the value lies using the witness type.

A de Bruijn representation of the **STLC** demonstrates this infrastructure:

```
data DSTLC (env :: [Type]) a where
  DVar :: Ix env a -> DSTLC env a
  DLam :: DSTLC (a ': env) b -> DSTLC env (a -> b)
  DApp :: DSTLC env (a -> b) -> DSTLC env a -> DSTLC env b
```

Intuitively, `DSTLC env a` represents STLC terms that have type `a` and free variables in `env`. Hence, `DLam` says that we can construct a function of type `a -> b` from an expression of type `b` with an extra free variable of type `a`. A variable is represented by its index (`Ix env a`) in the environment. For example, `x` is represented by `DVar IxZ` assuming `x` appears first in the environment, and hence `(\x -> x)` is represented by `DLam (DVar IxZ)`. Using indices, this representation gives a canonical representation to α -equivalent terms. However, indices change when the environment grows *e.g.*, `(\x -> (\y -> x) x)` is represented as this `DSTLC` term: `DLam (DApp (DLam (DVar (IxS IxZ))) (DVar IxZ))`.

4.3.3. The HOAS Representation

In contrast, since in HOAS the variables are represented by using the host's bindings, there is no corresponding method to `DVar`, as presented in Fig. 4.2 with representation of the **STLC** in the finally-tagless style [Carette et al. 2009]. Here, the guest-level bindings are represented by the host-level bindings; *e.g.*, `(\x -> (\y -> x) x)` is represented as `lam (\x -> app (lam (\y -> x)) x)`. A caveat is that we can pass arbitrary functions to `lam`, which, without restrictions, includes those that may not be **STLC** terms, and perhaps perform case analysis on their inputs. It is crucial to exclude such exotic terms [Chlipala 2008], *e.g.*, by using the parametricity of `exp`, which also plays a key role in the unembedding [Atkey 2009].

HOAS is useful for both language EDSL users and implementers, as it uses host-language binders to represent guest-language binders. From the user's perspective, compared with the naïve string representation of variables, it allows us to use the host language's variables for a guest language variables, providing access to the host's name checking and IDE support concerning variables (*e.g.*, displaying their types and automatic completion). From the language implementer's perspective, it frees us from manipulation of names, including name checking, renaming, and substitution. Some advantages, such as removing the need for name-checking and renaming, also apply when comparing HOAS to de Bruijn terms. HOAS also has further advantages over de Bruijn terms, especially from the user's perspective, as variables are more intuitive than indices to program with and enjoy IDE support. It is the goal of this paper is to extend the applicability to HOAS to more semantic domains, while preserving these advantages as much as possible, and this unembedding technique is the key to doing that.

The finally-tagless style of **HOAS** is preferable over plain **HOAS** because it addresses known issues

with plain **HOAS**: exotic terms and defining semantics.⁵ It is one of two state-of-the-art variants that address the issues, with the other being PHOAS [Chlipala 2008]. The use of typeclasses also has the additional advantages over PHOAS of extensibility (a typeclass can have subclasses) and efficiency (no tags are required to distinguish AST nodes—this is why the representation is called “tagless” [Carette et al. 2009]).

4.3.4. Converting between **HOAS** and de Bruijn

Unembedding achieves the best of both worlds: a language with the beloved user-friendly HOAS interface that is also amenable to analysis, by converting between **HOAS** and de Bruijn representations of a language. We expand upon this, using a similar conversion to attach a **HOAS** interface to a first-order representation that supports the semantics of interest.

The conversion from de Bruijn to an open **HOAS** term (OTHoas) falls out quite naturally. In the λ -calculus example, DLam and DApp map directly to lam and app, and DVar looks up what to inline in a value environment:

```
type OTHoas env a = forall exp. STLChoas exp => Env exp env -> exp a
toHOAS :: DSTLC env a -> OTHoas env a
toHOAS (DVar n)   = \g -> lookEnv g n
toHOAS (DLam t)   = \g -> lam (\x -> toHOAS t (ECons x g))
toHOAS (DApp f p) = \g -> app (toHOAS f g) (toHOAS p g)
```

Each line of the toHOAS function constructs a result of type OTHoas, in other words a function from an environment to a guest term. The first line finds its guest term in the provided environment of guest expressions (g) at the location specified by DVar n. The second is translating the λ construct, so creates a lam term. The lam term requires a guest expression of the result type with an environment extended by the variable it introduces. This is created by recursively converting the body of the λ , t, with the required extended environment. Application is converted by providing the app construct with function and argument converted recursively.

Notice that, e.g., OTHoas [a,b] c is isomorphic to forall exp. STLChoas exp => exp a -> exp b -> exp c. Here, the universal quantification in OTHoas ensures that toHOAS cannot produce exotic terms.

The other direction executes the unpacking of the **HOAS** representation into an explicit syntax tree:

```
fromHOAS :: (forall exp. STLChoas exp => exp a) -> DSTLC '[] a
fromHOAS h = runUSTLC h ENil
```

This is more involved because typing environment information needs to be recovered, necessitating an appropriate interpretation of exp: USTLC (the application of runUSTLC to h triggers h to assume this type). Since exp does not contain any information about the typing environment, the basic idea of unembedding is to make this type a function from this information, in the form of a typing environment, to the explicit DSTLC syntax tree:

```
newtype USTLC a = USTLC {runUSTLC :: forall env. TEnv env -> DSTLC env a}
```

To complete USTLC as an interpretation of exp, lam and app must be defined over this type. The interpretation of app is:

```
appSTLC :: USTLC (a -> b) -> USTLC a -> USTLC b
appSTLC (USTLC u1) (USTLC u2) = USTLC (\g -> DApp (u1 g) (u2 g))
```

⁵Defining transformations from a plain **HOAS** tree to its semantics is problematic. This is because the negative occurrence of the **AST** type in constructs such as λ requires a transformation in opposite direction. Even implementing the standard semantics is not straightforward (see, e.g., [Fegaras and Sheard 1996]). Using a typeclass solves this issue because *instantiating* a type parameter of the typeclass with its instance does not share the same issues as *transformation*.

4. Embedding by Unembedding

The environment g is fed to each subexpression u_1/u_2 to allow the construction of a DApp term. The interpretation of lam is trickier, which has the following form (where the type annotations are just for readability):

```
lamSTLC :: (USTLC a -> USTLC b) -> USTLC (a -> b)
lamSTLC k = USTLC $ \ (g :: TEnv env) ->
  let ga = ECons Proxy g :: TEnv (a ': env) -- extend env
      x  = USTLC (\ (g' :: TEnv env') ->      -- prepare arg for k
        weakenManyDSTLC ga g' (DVar IxZ :: DSTLC (a:env) a)) :: USTLC a
  in DLam (runUSTLC (k x) ga)
      -- apply k to x with extended arg to get desired arg for DLam
```

This instantiates lam concretely into the USTLC type that behaves as a λ : introducing a new variable that can be referred to in its body by extending the environment. It extends env to $a:\text{env}$; accordingly, it also prepares its value level representation $ga :: \text{TEnv } (a ': \text{env})$. Recall that $\text{TEnv} = \text{Env Proxy}$. Thanks to the extension, we now can have a variable $\text{DVar IxZ} :: \text{DSTLC } (a:\text{env}) a$, which needs to be converted to $\text{DSTLC env' } a$ to prepare $x :: \text{USTLC } a$ to be fed to the lam 's argument k . This conversion is done by weakening,⁶ which performs appropriate shifting of variables to adjust the environment from ga to the actual incoming one g' .

```
weakenManyDSTLC g1 g2 = weakenBy (compare g1 g2)
```

Here, compare and weakenBy have the following signatures.

```
compare  :: TEnv env1 -> TEnv env2 -> env1 <= env2
weakenBy :: env1 <= env2 -> DSTLC env1 a -> DSTLC env2 a
```

The function compare compares two type environments to have a witness that env2 is an extension of env1 (more precisely, $\text{env2} = \text{env} ++ \text{env1}$ for some env). Then, weakenBy performs weakening by using the witness.

Of course, compare is a partial function, since $\text{env1} <= \text{env2}$ may not hold in general (e.g., $\text{env1} = [a, b]$ and $\text{env2} = [b, a]$). Here, the exclusion of exotic terms matters; the conversion from the HOAS representation is meaningful only from polymorphic term. The universally quantified exp in fromHOAS ensures that the HOAS representation h consists only of lam and app , and hence only the operation on environments are extensions (using lam). As a result, since x appears in a deeper position in k , it will receive an extended environment (g') than the outer one (ga) passed to $k x$. This is why compare always succeeds in fromHOAS . Atkey [2009] proves this fact formally by using the (Kripke) parametricity of h , together with the fact that $\text{toHOAS } (\text{fromHOAS } h) \text{ ENil}$ is equivalent to h .⁷ That is, the two representations are isomorphic.

Thus, unembedding has unpacked the **HOAS** to create an explicit syntax tree. The original work used this to support optimizations and code generation, especially those that require non-local analysis of ASTs [Atkey et al. 2009; Matsuda and M. Wang 2018b; McDonell, Chakravarty, et al. 2013; Stewart 2010; Werk et al. 2012].⁸ Our key observation is that the unembedding technique has so much more potential. The current work is purely syntactic, but by considering semantics, unembedding can be used to support the embedding of languages with advanced computation models.

⁶Note that this assumes that the environment follows a Last In, First Out (**LIFO**) ordering.

⁷The opposite direction $\text{fromHOAS } (\text{toHOAS } t \text{ ENil}) = t$ can be proved by the induction on t without using parametricity.

⁸To be precise, Werk et al. [2012] and Stewart [2010] use the core conversion technique from McDonell, Chakravarty, et al. [2013], described in <https://github.com/mchakravarty/hoas-conv>, which is operationally similar to the unembedding but independently developed.

4.4. Embedding by Unembedding

This section explores using unembedding for embedding languages with “complex semantic domains” and introduces our Embedding by Unembedding framework. The exploration is conducted with the **STLC**. With its standard semantics, it does not require our technique, but our primary focus here is the explanation of our technique and preparation for later sections, where we embed languages with more complex semantics that require Embedding by Unembedding.

The section begins by clarifying what languages we have observed Embedding by Unembedding is applicable to (§4.4.1), before beginning a standard embedding of the **STLC** (§4.4.2), and investigating where unembedding comes in useful (§4.4.3), and finishing in a summary of our technique (§4.4.4).

4.4.1. Applicability

We have observed that Embedding by Unembedding is applicable to languages that: are simply typed; have constructs that are either “first” or “second” order;⁹ have compositional semantics; and support a rule for weakening environments. These qualities constitute sufficient criteria. By “first” or “second” order, we refer to two categories that we split language constructs into: those that introduce binders (which we refer to as *second-order*), and those that do not (*first-order*). The typing rules for each category of construct will adhere to the following general forms and compositional semantics for a certain semantic domain $\mathcal{S}$:¹⁰

$$\frac{\{\Gamma \vdash e_i : A_i\}_{i=1,\dots,n}}{\Gamma \vdash \mathbf{con} \ e_1 \ \dots \ e_n : B} \quad \frac{\{\Gamma, x_{i1} : A_{i1}, \dots, x_{im_i} : A_{im_i} \vdash e_i : B_i\}_{i=1,\dots,n}}{\Gamma \vdash \mathbf{con}' \ (x_{11} \ \dots \ x_{1m_1} \cdot e_1) \ \dots \ (x_{n1} \ \dots \ x_{nm_n} \cdot e_n) : C}$$

Here, $x_{11}, \dots, x_{1m_1}, \dots, x_{n1}, \dots, x_{nm_n}$ are the variables bound by the construct $\mathbf{con}'$. For example in the **STLC**, the function application and the λ abstraction belong to the former and latter sorts of constructs, respectively.

4.4.2. Embedding

Before utilizing unembedding, our embedding of the **STLC** must begin as any other: its syntax and semantics need to be considered and pinned down.

Semantic Domain Firstly, the semantic domain must be identified. This involves considering what the meaning of the embedded language should be. Is it a reversible language where the semantics should run “backwards” by mapping results to free variable assignments? Does it need to support the translation of changes in the inputs to changes in the outputs to facilitate incremental computation? A concrete way of doing this is by reifying it as a datatype in the host language.

As discussed, semantics where unembedding will come in useful are those where open expressions matter. That is, those that must be ready for interpreting the standard typing rule for variables: $\Gamma, x : A, \Gamma' \vdash x : A$. This idea is encapsulated into the `Variables` typeclass below.

```
class Variables (sem :: [k] -> k -> Type) where
  var      :: sem (a ' : as) a
  weaken  :: sem as a -> sem (b ' : as) a
```

⁹We use the term “second-order” specifically in the sense of “second-order abstract syntax”, as used in the line of work by M. P. Fiore, Plotkin, et al. [1999], not in the sense of “second-order λ calculus” (System F).

¹⁰The reader may notice that $\mathbf{con}$ is a special case of $\mathbf{con}'$. We distinguish them for readability ($\mathbf{con}$ can be compared to *app*; $\mathbf{con}'$ can be compared to *lam*), and to highlight that unembedding enables the ability to treat $\mathbf{con}'$.

4. Embedding by Unembedding

If the reified semantic type is an instance of this class, then it is amenable to unembedding. `Variables` represents the class of semantics (`sem`) that depend on some context (`[k]`, a list of types of kind `k`), have one particular variable in focus (`k`), and are able to be unembedded. The `var` method ensures that we can interpret $\Gamma, x : A \vdash x : A$ (where Γ' is empty in the above), and `weaken` ensures that more variables can be added to a context for interpretations. By mixing the two, we can interpret variables that appear in arbitrary positions in a context; *e.g.*, `weaken (weaken var) :: Variables sem => sem (b ': c ': a ': env) a` corresponds to $\Gamma, x : A, y : B, z : C \vdash x : A$.

An astute reader may wonder why we only focus on weakening (`weaken`), when we could have other manipulations (namely, exchange and contraction) that ensure that Γ is essentially a set instead of a sequence. This is because weakening is the only manipulation of environments used in Embedding by Unembedding. That is, `Variables` encapsulates only the minimal set of operations needed, though we expect that the semantic domain must support general extensions and rearrangements on contexts, their actions on the domain, and that semantic functions respect such context manipulations.¹¹

Example 4.4.1. The (standard) semantics of the STLC takes an environment of free variables to a result. This is realized as a concrete Haskell datatype by wrapping up a function from a variable environment to a result as a newtype.

```
newtype STLC env a = Sim { runSim :: VEnv env -> a }
type VEnv = Env Identity
```

Here, `runSim` is exactly the semantics of the STLC: a function from a value environment to a result. The value environment is the `Env` type, where the type container is just `Identity`. The use of the `Env` type provided by our framework helps make this semantic type an instance of the `Variables` typeclass. For `var`, `runSim` can extract the first value from the `VEnv`. For `weaken`, a new `Sim` can be constructed so that it just ignores the first element of the environment:

```
instance Variables STLC where
  var :: STLC (a ': as) a
  var = Sim (\(ECons (Identity x) _) -> x)
  weaken :: STLC as a -> STLC (b ': as) a
  weaken (Sim f) = Sim (\(ECons _ env) -> f env)
```

Language Constructs Semantics in hand, the next step considers how each of the language constructs should behave in that domain. Each construct is represented in the semantic domain as a function over the semantics of terms-in-context.

Example 4.4.2. The STLC only has two constructs: one first-order (`app`), and one second-order (`lam`). The semantic function for a first-order construct such as `app` takes the form of a function over the STLC type. Thus the semantic function for `app` should be a function with two arguments: a function (`f`) and its argument (`x`), where its definition applies `f` to `x`. What differentiates this semantic function from a normal Haskell function representing `app` is that `f` and `x` are wrapped up in the STLC type:

```
appSem :: STLC env (a -> b) -> STLC env a -> STLC env b
appSem fTerm aTerm = Sim (\g ->
  let f = runSim fTerm g -- Unpack function using g.
      x = runSim aTerm g -- Unpack arg using g.
  in f x) -- Apply.
```

¹¹We conjecture that this treatment essentially corresponds to the functor category $[\mathbb{F}, \mathbf{Set}]$ in M. P. Fiore, Plotkin, et al. [1999].

Second-order constructs are slightly more involved, due to the semantics needing to operate on the environment. In the example, `lam` takes a term of the **STLC**, represented by the `STLC` type, that knows how to make the result should it have the argument in its environment, and produces a function term:

```
lamSem :: STLC (a ': env) b -> STLC env (a -> b)
lamSem bTerm = Sim (\g x ->
  let g' = ECons (Identity x) g -- Add x :: a to env.
  in runSim bTerm g')          -- Use to make b.
```

The function term works by storing its argument into the environment and using the original body to produce the result.

Syntax While the semantic functions are an accurate representation of the semantics of the language, they are not very usable. Users want an interface with binders and variable names, not environments and indices, and language designers want to give them this with as little effort as possible. This is why **HOAS** is typically used to represent the syntax of an embedded language.

This step is the same as any other finally-tagless embedding: creating a typeclass to represent the language, where each construct is a method, and binders are achieved as host level functions.

Example 4.4.3. The **STLC** example can use **STLChoas** from Fig. 4.2.

Gap Now the embedded language has both syntax and semantics. All that remains is to connect the two. Normally, this step is as simple as making the semantic type an instance of the finally-tagless syntax. However, this is not straightforward when the semantic type explicitly uses environments.

The **STLC** example has been contrived such that its standard semantics do this needlessly,¹² but domains such as those of our case studies will require it. Thus something is needed to act as a conduit between the **HOAS** syntax and interesting semantics. This something is unembedding.

4.4.3. Unembedding

Unembedding marries syntax and semantics, achieving a complex semantic domain without sacrificing the user-friendly interface. It facilitates making the semantic type an instance of the **HOAS** typeclass.

We create a framework around unembedding so that, on the surface, this connection can be done in two steps: wrap the semantic type in provided wrapper type `EnvI` (for “environment indexed”); apply provided lifting functions (`liftF0n` and `liftS0`) to the semantic functions to instantiate the typeclass methods. Here, the only choice that needs to be made is what lifting function to use. The framework provides a family of lifting functions: a variety for first-order constructs that cater to different numbers of arguments, and one for second-order constructs.

Example 4.4.4. As an example of using lifting functions, consider **STLC**. Here this means that **STLC** is wrapped in `EnvI` so that it can be an instance of **STLChoas**:

```
instance STLChoas (EnvI STLC) where
  app = UE.liftF02 appSem -- not a binder => F0, 2 args => F02
  lam = UE.liftS0n (o11 .. End) lamSem -- binder => S0
```

Here, `app` is defined as the lifting of `appSem`. Because `app` is first-order, one of the `liftF0` functions is used, specifically `liftF02` because `app` has two arguments. As a second-order construct, `lam` is different. Now its arguments can have arguments. To allow for this, `liftS0n` takes a description of what arguments are expected: how many there are and how many arguments (bindings) they themselves have. This

¹²However, the technique would be useful if one interprets **STLC** in an arbitrary cartesian closed category [Elliott 2017].

4. Embedding by Unembedding

description is a list, with each element representing an argument and how many arguments it itself takes. In the case of `lam`, there is one argument that itself has one argument, thus the singleton list of `o11` is needed.

Under the hood, our framework is managing a de Bruijn-index-like first-order representation so that the involvement of the environment is not user-facing. This is a more general application of the unembedding technique, where semantic functions replace de Bruijn terms. Recall that the main challenge that Embedding by Unembedding tackles is the conflict of requirements between the advanced and non-standard guest function semantics and the desire to use HOAS (and thus standard host function semantics) to achieve this. The advanced semantics require an environment; whereas HOAS expects none because the whole point of it is to defer the handling of binders to the host language:

```
semAdv  :: [k] -> k -> Type
semHOAS ::      k -> Type
```

This mismatch is a generalization of the one between de Bruijn indexed terms and HOAS expressions. Thus, a wrapper type can be used to connect the semantic functions to their HOAS representation. This wrapper type, is USTLC from §3.2, but with the first-order semantics (e.g. DSTLC) generalized out as a parameter (*sem*):

```
newtype EnvI sem a = EnvI { runEnvI :: forall env. TEnv env -> sem env a }
```

Regarding implementation of first-order, non-binding constructs, recall that we pass around a value-level representation of the typing environment, `env`, in the implementation of `app` in §3.2. This actually serves as a common pattern. Hence, even without `liftF0` families, we can implement `app` of the **STLC** similarly with `appSem` as below.

```
app_ (EnvI e1) (EnvI e2) = EnvI (\g -> appSem (e1 g) (e2 g))
```

The implementation depends only on the number `n` of arguments of a construct. We provide `liftF0n` to avoid this routine.

Implementation of second-order, binding constructs is analogous to the `lam` case in §3.2, with the implementation of `lam` without `liftS0` following the same story, the only differences being the use of `var` instead of `DVar IxZ`, and the use of `lamSem` instead of a constructor of de Bruijn terms, and a more general implementation of `weakenMany` that uses `weaken` from the `Variables` typeclass.

```
lam_ k = EnvI $ \g :: TEnv env ->
  let ga = ECons Proxy g :: TEnv (a ': env)
      x = EnvI (\g' -> weakenMany ga g' var)
  in lamSem (runEnvI (k x) ga)
```

In general, the lifting of a binding construct is done by: for each argument `k` of type `b` that binds variables of types `a1, ..., an`:

1. Extend `g` to obtain `ga :: TEnv (a1 ': ... ': an ': env)`.
2. Prepare variables `vi :: sem (a1 ': ... ': an ': env) ai` in the semantic domain `sem` by using `var` and `weaken`, and then convert them to `xi :: EnvI sem ai` by weakening:

```
weakenMany :: Variables sem =>
  TEnv env -> TEnv env' -> sem env a -> sem env' a
```

The function `weakenMany` implements our iteration on the original unembedding's `compare` and `weaken`, which repeatedly applies `weaken` from the `Variables` typeclass.

If we ignore manipulation of types, this pattern depends only on the number of bindings for each argument of a construct. So, we provide `liftS0` to perform the routine lifting by knowing the numbers of bindings, as `lam = liftS0n (o11 :. End) lamSem`. Though useful, the implementation of `liftS0` is not in the scope of this paper, and thus we advise curious readers to see Appendix 10.

While the implementation of the lifting functions is quite involved, in reality, this is just the tedious management of environments that is tamed by the Embedding by Unembedding framework. All the language designer actually has to do to connect their interesting semantics to a **HOAS** frontend is use them.

4.4.4. Embedding by Unembedding

Embedding by Unembedding successfully embeds a language with binders and interesting semantics, allowing for the interpretation of an open expressions.

Open Expressions The ability to interpret open expressions is a key feature provided through Embedding by Unembedding that existing embedding techniques have not yet tackled.

The interpretation of open expression is provided generically for any semantics that are members of the `Variables` typeclass:

```
runOpen :: Variables sem => (EnvI sem a -> EnvI sem b) -> sem '[a] b
runOpen f = let gA = ECons Proxy ENil
              x = EnvI $ \g' -> weakenMany gA g' var
              in runEnvI (f x) gA
```

In the specific case where there is only one free variable, `runOpen` takes an open term and transforms it into the semantic type. Since the open term is expressed as a host level function, `runOpen` creates the `EnvI sem a` argument to run it on. It does this by creating a type environment with just the single free variable in it, and then creating a `var` expression where that environment is unified with the incoming one. To ensure that there is indeed only one free variable `runOpen` must be wrapped in `runOpenSTLC` similarly to the original unembedding.

```
runOpenSTLC :: (forall exp . STLChoas exp => exp a -> exp b) -> STLC '[a] b
runOpenSTLC f = runOpen f
```

The framework also provides a more general `runOpenN` for `n` free variables.

Example 4.4.5. Putting the embedded STLC to work builds on top of `runOpenSTLC`:

```
runSTLC :: STLC '[a] b -> a -> b
runSTLC t x = let g = ECons (Identity x) ENil in runSim t g
```

This function operates on the output of `runOpen`, interpreting the STLC term with one free variable as a function from `a` to `b`: the standard semantics of the λ -calculus. Here it is in action, evaluating the application of the identity function to a free variable that has a value of three:

```
term :: STLChoas exp => exp b -> exp b
term x = app (lam (\y -> y)) x

> runSTLC (runOpenSTLC term) 3
3
```

The STLC expression is written as `term`, whose type is what `runOpenSTLC` expects. All that remains is to use `runSTLC` to provide the value of the free variable.

4. Embedding by Unembedding

Correctness The correctness of Embedding by Unembedding—that `runOpenSTLC` implements the standard semantics for this particular case—is understood as a generalization of Atkey [2009]’s original unembedding followed by fusion [Gill et al. 1993]. Continuing to consider our STLC example, unembedding maps the STLChoas representations to the DSTLC in the one-to-one manner, and the semantics of DSTLC is given by replacing constructors with their corresponding semantic functions. Thus, fusing the two processes together by replacing DSTLC constructs with their corresponding semantic functions in the methods of STLChoas for USTLC, we get an instance of STLChoas that coincides with the semantics given for de Bruijn terms. This is what Embedding by Unembedding is doing. In this process, we slightly extend the original discussion to show the correspondence in *open terms*, namely HOAS representations of type `forall exp. STLChoas exp => exp a -> exp b` and de Bruijn terms of type `DSTLC '[a] b`, assuming the Kripke parametricity of the HOAS representations (observe `forall` in the type). The correctness of the fusion can also be proved by using parametricity [Gill et al. 1993]. We can repeat the discussion on a case-by-case basis for languages satisfying the criteria in §4.4.1.

Summary The process of Embedding by Unembedding can be summarized into a recipe, where each different part of the example constitutes a step:

- Step 1. Identify and reify the semantic domain. (Shown for the STLC in Example 4.4.1)
- Step 2. Prepare semantic functions for each construct. (Example 4.4.2)
- Step 3. Provide a HOAS representation of the syntax. (Example 4.4.3)
- Step 4. Implement semantics through appropriate liftings. (Example 4.4.4, recall that all the language designer had to do here was use the provided lifting functions and wrapper type.)
- Step 5. Put the language to work! (Example 4.4.5)

Appendix 11 runs through the application of this recipe to another example: the ILC, the predecessor of the incremental language that is the subject of our incremental computation case study.

An Extensible and Modular Approach A major benefit of embedding is the ease of extending the language, which is indeed the case in Embedding by Unembedding. In fact, our approach features the state-of-the-art modularity and extensibility of the finally-tagless style. To add new constructs, the recipe can just be repeated for the new constructs only, and the language can be created in a modular manner, with different constructs grouped into different and dependant typeclasses. The embedded language presented in §4.5 makes use of this advantage to design the core system simple as possible and enables us to extend the core to incremental primitives for user-defined datatypes. Also, we detail the adding of sequences¹³ to the ILC, which can be found in Appendix 11.

Case Studies We will now demonstrate Embedding by Unembedding on two domains that require the technique to be embedded: incremental and reversible computation. These two domains could not previously be embedded due to the limitations of HOAS: limitations that are now solved by Embedding by Unembedding.

Note that neither of our case studies feature λ binders. This is a consequence of the semantic domains of the languages in question, not the unembedding technique. In the case of the CTS variant of the ILC, the original work [Giarrusso et al. 2019] does not support the *typed* λ abstractions and applications as seen in STLC, as their semantics (which was originally untyped) does not respect types for them.

¹³Sequences are a speedier version of lists in Haskell, and as ILC is about efficiency they are chosen.

We follow suit on choice of binder, avoiding λ , as this is a domain specific issue. In the case of our reversible case study, it is simply difficult to find a semantic domain that appropriately encapsulates the semantics and supports abstractions (see, *e.g.*, [Abramsky 2005; C. Chen and Sabry 2021; Matsuda and M. Wang 2018d, 2020]). As such, we apply the same trick of switching to a let binder, this time deviating slightly from the original language.

4.5. Case Study: Incremental λ -Calculus

The first non-trivial computation model we consider in this paper is incremental computation. This domain is focussed on speeding up recomputation, something that is invaluable in our world of big data [Acar and Y. Chen 2013], and it is a technique that transcends language paradigms, with research into the incrementalization of functional [Y. Chen, Dunfield, Hammer, et al. 2014a], imperative [Hammer et al. 2009], and object orientated languages [Shankar and Bodík 2007].

Unlike normal computations, incremental computation aims to only recompute those outputs that depend on the changes of the input in order to achieve efficiency. Generally, this involves mapping input changes to output changes instead of recomputation. For example, when summing a list, if just one element was incremented, it makes no sense to recompute the sum. It would be far more efficient to just increment the initial sum of the list. Many models of incremental computation exist. Our case study focusses on Cai et al. [2014]’s incremental λ -calculus, specifically an improved variant of it known as the Cache-transfer-style (CTS) [Giarrusso et al. 2019], which propagates changes to the input into changes of the output without using pointers or destructive operations.

This section will cover the unembedding of the CTS [Giarrusso et al. 2019]. (Appendix 11 also walks through the unembedding of the original calculus.) It begins by introducing the CTS variant (§4.5.1) and why it is of interest (§4.5.2), before exploring the interesting parts of the unembedding (§4.5.3), including an evaluation of the performance of the embedded language.

4.5.1. Cache-Transfer-Style Incremental Functions

An incremental function in CTS can be characterized by the following type:

$$\exists C.(A \rightarrow (B, C), (\Delta A, C) \rightarrow (\Delta B, C))$$

The first component (which we call an *initializer*) is a transformation from A to B that also computes a “cache” C . The existential quantification tells us that the cache may differ between computations. The cache is used and updated for the second component (which we call a Δ -*translator*) to translate input changes ΔA into output changes ΔB . Take the list reversal *reverse* as an example. When an insertion to the i th position happens to the original input list xs , an insertion of the same element happens to the $(length\ xs - i)$ th position of the corresponding output. So the cache can be the length of the original input. The cache may also be changed by the input change, which is the reason for the C in the output of the Δ -translator. Here, an insertion also changes the length of the input.

4.5.2. Why Unembedding is Desired and Applicable

As mentioned in §4.2.2, the ability to handle open expressions is crucial to the change-propagation-based incremental computation in general, not limited to Cai et al. [2014]’s original work and the CTS version [Giarrusso et al. 2019]. Unlike closed expressions, open expressions have clear inputs (value environments) and outputs (evaluation results), which makes the notions of input changes and output changes clear. That is, we can view an open expression as a transformation from value environments to resulting values and then consider the change propagation regarding the transformation.

4. Embedding by Unembedding

It is true that we can avoid the direct manipulation of open expressions for particular computation models. For example, for the original **ILC**, where an incremental function has type $(A \rightarrow B, A \rightarrow \Delta A \rightarrow \Delta B)$, we can apply tupling [Chin 1993] to inject the incremental function in $(A, \Delta A) \rightarrow (B, \Delta B)$. Thus, we have a simple HOAS implementation that interprets $\text{exp } a = (a, \text{Delta } a)$ using a type family `Delta` to represent changes on `a`. However, such a simplification is limited to particular models; actually, the tupling approach does not scale to the **CTS** variant [Giarrusso et al. 2019] due to existentially-quantified caches.

The **CTS** example is of interest because it expands the applicability of the original **ILC** to efficiently incrementalize more computations, and the addition of the cache that allows this precludes standard embedding. Giarrusso et al. [2019]’s current system is not ready for a (shallowly) embedded implementation because it targets an untyped calculus, assuming some global program transformations (namely, λ -lifting and a variant of A-normalization [Flanagan et al. 1993]—essentially, the call-by-value CPS transformation), which are not immediately ready for a shallowly embedded implementation in a typed language. Their accompanied implementation provides a deeply-embedded interface, which is designed for a demonstration of their idea, not user consumption: the abstract syntax is λ -lifted and A-normalized, with an interface that uses GADTs and exposes cache types, and the semantics is given by Haskell code generation in strings.

Embedding by Unembedding marks a road to a (shallowly) embedded implementation. First, its ability to access environments enables us to use the straightforward implementation that models a guest term $\Gamma \vdash e : A$ in $\exists C. (\llbracket \Gamma \rrbracket \rightarrow (\llbracket A \rrbracket, C)) \times ((\Delta \llbracket \Gamma \rrbracket, C) \rightarrow (\Delta \llbracket A \rrbracket, C))$. Second, we are able to represent the explicit sharing of computation naturally with a binding construct like `let`. This is crucial; in `let x = e in ... x ... x ...`, interpretation of `e` happens twice in the guest language, while in `letE e (\x -> ... x ... x ...)` the computation happens only once in the evaluation of this expression. Third, we can represent their “higher-order” combinators as binding constructs to enjoy higher-order programming without risking type safety. For example, consider the following, which implements an incremental computation of the cartesian product of given two sequences.

```
cartesianRecreation xs ys = concatMap (\x -> map (\y -> pair x y) ys) xs
```

(Note that we have eluded the type of this example as it contains specifics that we will introduce later.)

It is worth noting that there is a limitation on type safety coming from a gap in types: the original **CTS** [Giarrusso et al. 2019] targets an untyped language, while we here consider a typed EDSL. This gap becomes critical when we consider general λ -abstractions and applications, and we thus do not include λ s in this section. We note, however, that we could have λ s if we compromise type safety to some extent to use dynamic types for caches involved in function values (see Appendix 13.2).

4.5.3. Unembedding Details of Interest

Unembedding **CTS** is fairly standard, following the general recipe in §4.4.4. However, there are two details of the unembedding that are worth exploring: the embedding of its binding construct, and its optimization to the performant implementation necessitated by its domain. These features will be considered in turn.

The Binding Construct The core **CTS** language has first-order constructs: `unit`, `pair`, `fst` and `snd`; and one second-order binding construct: `let`. The semantic domain is created as a direct representation in Haskell of the type of **CTS** incremental functions:

```
data CTS env a = forall c. CTS (VEnv env -> (a, c))           -- Initializer
                ((DEnv env, c) -> (Delta a, c)) -- Translator
```

These semantics encapsulate two evaluation modes: standard evaluation, or the initializer, that takes variable assignments (VEnv) to a result and an initial cache; and incremental evaluation, or the Δ -translator, that takes how variables have changed (DEnv) along with a cache and produces a change in the result (Delta a) along with a new cache. It involves two types of environment: a value environment (VEnv), which slightly differs from the one already introduced in that its container ensures that only types that have a notion of change, a Delta, are contained; and an environment of changes (DEnv), which stores these Delta values.

The semantic domain CTS is an instance of Variables as below.

```
instance Variables CTS where
  var = CTS (\(ECons (PackDiff x) _) -> (x, ()))
        (\(ECons (PackDiffDelta dx) _, _) -> (dx, ()))
  weaken (CTS f tr) = CTS f' tr'
  where
    f' (ECons _ g) = f g
    tr' (ECons _ dg, c) = tr (dg, c)
```

Here, PackDiff and PackDiffDelta wrap the values in the container type for their VEnv and DEnv environments; *i.e.*, VEnv = Env PackDiff and DEnv = Env PackDiffDelta. The method var extracts the head from both value and change environment with the unit cache, weaken applies the initializer and the Δ -translator of the given CTS incremental function to the tails of the value and change environments, respectively.

Following the recipe, the let construct is unembedded as a semantic function over CTS.

```
letSem :: Diff a => CTS env a -> CTS (a ': env) b -> CTS env b
letSem (CTS i1 tr1) (CTS i2 tr2) = CTS i tr
  where
    i g =
      let (x, c1) = i1 g -- Get x::a with i1.
          (y, c2) =
            i2 (ECons (PackDiff x) g) -- Get y::b with i2 on env with x added.
          in (y, (c1, c2)) -- Result, y, is returned with new cache.
    tr (dg, (c1, c2)) = -- Same story but for incremental evaluation.
      let (dx, c1') = tr1 (dg, c1)
          (dy, c2') = tr2 (ECons (PackDiffDelta dx) dg, c2)
      in (dy, (c1', c2'))
```

Here, the constraint Diff a ensures that the type a has Delta a together with appropriate operations; this is actually what PackDiff and PackDiffDelta are wrapping. The function letSem is implemented by defining the initializer (i) and the Δ -translator (tr). They both work the same way: the first argument term is used to produce the a/Delta a, which is then added to the environment, so that the second argument term can produce the desired b/Delta b. The caches from the two argument terms are also combined. This is done in the simplest way: pairing them. Then, the semantic function is linked to the following HOAS syntax

```
let_ :: Diff a => exp a -> (exp a -> exp b) -> exp b
```

with the lifting function.

```
let_ = liftSOn (ol0 :: ol1 :: End) letSem
```

We omit the discussions on the first order constructs unit, pair, fst and snd because their treatment is similar.

An astute reader would notice that we do not use caches in any essential ways in letSem. This is also true for the other constructs above: none of unit, pair, fst and snd uses the cache in any essential

4. Embedding by Unembedding

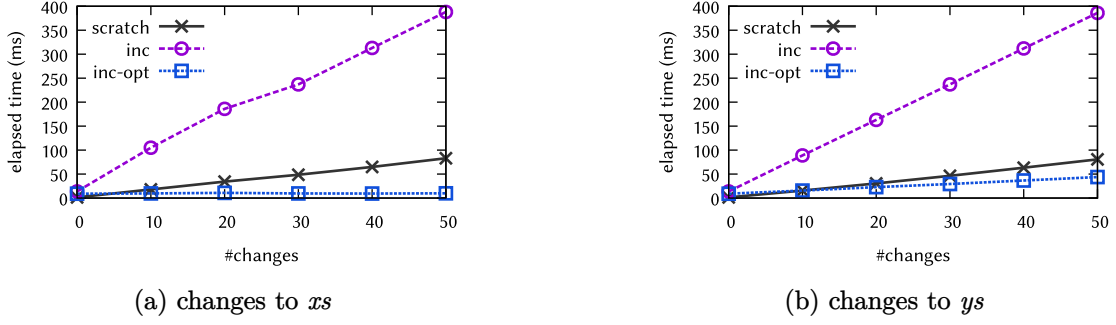


Figure 4.3.: Experimental Results: incremental computation of *cartesian xs ys* when *length xs = length ys = 200*. Key: scratch = recomputation, inc = incremental computation, inc-opt = optimized incremental computation (optimizes away change propagation overhead).

ways either. This is because these constructs are designed to glue together other incremental constructs for particular domains that may use the cache (e.g., sequence manipulating operations discussed below). Here, the extensibility of the finally-tagless representation [Carette et al. 2009] comes in handy: we can keep the core part small and then extend the core afterwards for problem domains of interest.

Being Performant An unembedding of an incremental language would not fulfil its purpose if it was not performant. As such, with this unembedding, we took care to ensure that the incremental evaluation was faster than recomputation, and compared the performance of “eCTS” to the original.

The example we will compare against is (again) the cartesian product. To implement this function, eCTS must first be expanded to include language constructs for manipulating sequences:

$$e ::= \dots \mid \mathbf{emp} \mid \mathbf{concat} \ a \mid \mathbf{single} \ e \mid \mathbf{map} \ (x.e) \ e'$$

(concatMap from §4.5.2 can be implemented as the composition of concat and map). These constructs come with the following typing rules.

$$\frac{}{\Gamma \vdash \mathbf{emp} : \mathbf{Seq} \ A} \quad \frac{\Gamma \vdash e : \mathbf{Seq} \ (\mathbf{Seq} \ A)}{\Gamma \vdash \mathbf{concat} \ e : \mathbf{Seq} \ A} \quad \frac{\Gamma \vdash e : A}{\Gamma \vdash \mathbf{single} \ e : \mathbf{Seq} \ A} \quad \frac{\Gamma, x : A \vdash e : B \quad \Gamma \vdash e' : \mathbf{Seq} \ A}{\Gamma \vdash \mathbf{map} \ (x.e) \ e' : \mathbf{Seq} \ B}$$

Here, $\mathbf{Seq} \ a$ is a type for sequences of a -typed values. Notice that **map** is treated as a second-order construct, instead of a first-order construct involving function types. We do not encode λ s directly in the DSL, but instead rely on the host language (Haskell)’s λ s in programming. The expansion is achieved by repeating the recipe for these new constructs and adding their HOAS as a subclass of the typeclass of the core language. Full details of our implementation can be found in Appendix 13.

The treatment of map, and the modularity and expandability of the language demonstrate a clear advantage of HOAS-based embedding [Carette et al. 2009]. We can extend the system more to implement a realistic example. In Appendix 13.3, we implement an incremental version of HaXML filters [Wallace and Runciman 1999], where transformations are written based on filters of type $\text{Tree} \rightarrow \text{Tree}$ and combinators working on filters are naturally higher-order functions.

Although the straightforward implementation discussed in this section has overheads, with the help of meta-programming techniques, a preliminary evaluation shows that our language has similar performance as the state of the art [Giarrusso et al. 2019]. The details of the implementation can be found in Appendix 13.4. The key point is that it is still based on the same embedding approach, while using Template Haskell [Sheard and S. L. P. Jones 2002] to eliminate some of the overhead

of unembedding, variable accesses and cache composition/decomposition, and it tries to remove the propagation of empty changes.

As a very preliminary experiment, we measure the performance of $f = \lambda z. \text{cartesian } (fst\ z) (snd\ z)$, which is taken from the original CTS paper [Giarrusso et al. 2019] (though we measure multiple changes instead of a single change as in the original paper). Specifically, for an input a_0 and a sequence of changes $\{da_i\}$, we compare the elapsed time of (1) from-scratch computation that computes $\{b_i = f\ a_i\}_{0 \leq i \leq n}$ for each intermediate inputs $a_{i+1} = a_i \oplus da_i$, and (2) incremental computation that first computes the result and a cache $(b_0, c_0) = f_{init}\ a_0$, then propagates each changes $\{(db_i, c_i) = f_{tr}\ (da_i, c_{i-1})\}_{1 \leq i \leq n}$ and finally apply the obtained changes $b_0 \oplus db_1 \oplus \dots \oplus db_n$.

The experimental result is shown in Fig. 4.3. It is conducted on 13-inch MacBook Pro with 2.3 GHz Intel Core i7-1068NG7 CPU and 32 GB memory, using GHC 8.10.7 running on macOS 12.2.1. We use `Data.Sequence.Seq` in the package `containers-0.6.5.1` to represent `Seq`. As we can see, the naive implementation (`inc` in the figure) is expectedly slow; the overhead of change propagation outweighs the recomputation cost. The interesting part is the comparison between `inc-opt` and `scratch`: the incremental version is slower for small number of changes due to the overhead, but as the number of changes grows, the potential of incremental computation becomes clear. It is also interesting to observe that there is a difference between changing xs (Fig. 4.3a) and changing ys (Fig. 4.3b). We think this is due to the effect of removing the unnecessary propagation of empty changes: when xs is changed, only the outer `map` and `concat` are affected, while when ys is changed, both inner and outer `map` and `concat` are affected, making the analysis less effective.

This result is very similar to what is reported in the original CTS paper [Giarrusso et al. 2019]. In summary, the CTS version exhibits an incremental behaviour: taking more time to start-up (the first execution) but a less or equal amount of time for update propagation, which is also the case for us. Moreover, in CTS, the update propagation is comparably faster for changes to xs and seems to take a constant time from their plot. On the other hand, the update propagation is slower for changes to ys and takes a linear time to the input size (the length of xs and ys). Again, this matches our result.

We anticipate that further optimization (as future work) will be able to remove sufficient unembedding overhead to close the gap observed above, which will push the result beyond the state-of-the-art in terms of performance.

4.6. Case Study: Bidirectional Programming Language

A *bidirectional transformation* is a piece of code that can be run in two directions and is expected to satisfy some round tripping properties (i.e., transforming data to another form and back returns the original) [J. Nathan Foster et al. 2005]. This is a technique that allows a programmer to program two related operations at the same time, saving programming time, maintenance effort, and avoids the problem of separate implementations becoming out of sync.

The field of bidirectional programming has found numerous and wide-reaching applications for these including: data synchronization [J. Nathan Foster et al. 2005], gradual development of programs [M. Wang et al. 2010, 2013], deriving parsers from pretty-printers [Matsuda and M. Wang 2013, 2018b], Web/GUI programming [Perez and Nilsson 2015; Rajkumar et al. 2013], model synchronization in software engineering [Stevens 2008a; Xiong et al. 2007; Yu et al. 2012], and view updating in database systems [Bancilhon and Spyrtos 1981; Fegaras 2010; Hegner 2004; Tran et al. 2020].

This domain is a challenge to embed, because to properly develop bidirectional transformations, one requires specialized programming languages that enforce the round tripping properties, preferably by construction.

In this section, we demonstrate that the approach of Embedding by Unembedding is able to address

4. Embedding by Unembedding

the long-standing challenge of embedding bidirectional languages with binders. In particular, we consider a certain class of lenses as a semantic domain, where a class of lens combinators are treated as semantic functions. The unembedding enables us to compose lenses as functions, in order to give extra programming flexibility compared with the standard embedding of lenses as a combinator language. The resulting embedded system is able to closely model the state-of-the-art stand-alone language **HOBiT** [Matsuda and M. Wang 2018d], creating **eHOBiT**.

After introducing lenses (§4.6.1), and highlighting the suitability of unembedding in this case (§4.6.2), this section showcases **eHOBiT** (§4.6.3), investigating its embedding, comparing it to the original, and discussing alternative embedding approaches.

4.6.1. Lenses

Formally, a lens between s and v is defined as a pairs of functions:

```
data Lens s v = L {get :: s -> v, put :: s -> v -> s}
```

Borrowing the terminology of the original application in databases [Bancilhon and Spyratos 1981; J. Nathan Foster et al. 2005; Hegner 1990], s is called *source* and v is called *view*. The view is typically the result of some query on the source. The basic idea of a lens is: `put` merges changes to the view with the source. For example, for a lens `fstL` with `get (x,y) = x`, a corresponding `put` can be defined as `put (_,y) x = (x,y)`. The asymmetric nature of `get` and `put` separates bidirectional transformations (lenses) from inverses, with the former being more general and able to handle non-injective functions.

4.6.2. Why Unembedding is Desired and Applicable

A lens must satisfy round tripping to be considered as *well-behaved* [J. Nathan Foster et al. 2005]. Here we use the system adopted by **HOBiT**, which is a slightly weaker version of the original round tripping [Bancilhon and Spyratos 1981; J. Nathan Foster et al. 2005; Hegner 1990]. The exact definition of the round tripping laws is not important here, which we leave to Appendix 15.1. The point is that our embedding is able to support the compositional reasoning of the property, and thus correctness by construction. This combination of embedding, compositional reasoning, and binding constructs has never been achieved in the bidirectional programming literature.

Open expressions matter in this domain, necessitating unembedding, because a natural approach to a bidirectional language with binders is to interpret an open expression as a lens between value environments and return values. For example, we interpret a guest term `fstL x` as a lens whose `get` takes $\{x = (1,2)\}$ and returns 1, and whose `put` takes the same x and 3 to return $\{x = (3,2)\}$. This is similar to the case of reversible languages where we interpret open expressions as bijections between environments and return values (see, *i.e.*, Yokoyama et al. [2011]). With binding expressions, we are not able to fix the number of free variables in giving its semantic domain. This is why Embedding by Unembedding allows for **eHOBiT**.

HOBiT manipulates algebraic datatypes, and provides a bidirectional case expression, which pattern-matches on a value and performs branching with the matched result in a bidirectional (round tripping) way [Matsuda and M. Wang 2018d]. We could include patterns in a core syntax of a language to embedded, but this makes the system complicated. Instead, we focus on the core behaviour of the case expressions: decomposition of products and branching based on tags of sum-typed values. We mainly focus on the former, which has the guest syntax `let (x,y) = e1 in e2`, and we call `unpair`. The latter construct can be handled similarly but contains subtlety in the problem domain (see three branching lens combinators in J. Nathan Foster et al. [2005]).

```

unpairSem :: LensTerm env (a, b) -> LensTerm (a ': b ': env) r -> LensTerm env r
unpairSem t1 t2 = LT $ \g ->
  let l1 = runLT t1 g
      l2 = runLT t2 (ECons Proxy (ECons Proxy g))
  in letLens l1 (rearrPair l2)

rearrPair :: Lens (VEnv (a ': b ': env)) r -> Lens (VEnv ((a, b) ': env)) r
rearrPair l = L get' put'
  where
    get' (ECons (Just (a, b)) g) = get l (ECons (Just a) (ECons (Just b) g))
    put' (ECons (Just (a, b)) g) r =
      let ECons m (ECons n g') = put l (ECons (Just a) (ECons (Just b) g)) r
      in ECons (Just (fromMaybe a m, fromMaybe b n)) g'

letLens :: Lens (VEnv env) a -> Lens (VEnv (a ': env)) b -> Lens (VEnv env) b
letLens l1 l2 = L get' put'
  where
    get' g = get l2 (ECons (Just (get l1 g)) g)
    put' g b = let a = get l1 g
                ECons m g1' = put l2 (ECons (Just a) g) b
                g2' = put l1 g (fromMaybe a m)
                in merge g1' g2'

```

Figure 4.4.: Semantic Function for Unpairing

4.6.3. Unembedding Details of Interest

The feature of interest is, yet again, the binding construct: `unpair`. Its successful implementation, and more generally that of bidirectional case analysis, is what allows for the embedding of **HOBiT**. As such, its addition to a simple applicative lens language will be detailed, before comparing the new **eHOBiT** to the original.

Unpair The simple bidirectional language that `unpair` will be added to consists of the following first-order constructs: primitive operators (`prim`), `unit` and `pair`. (The full syntax and typing rules for **HOBiT** can be found in Appendix 12.2.)

The language is unembedding to the `LensTerm` semantic type:

```

newtype LensTerm env a = LT {runLT :: TEnv env -> Lens (VEnv env) a}
type VEnv = Env Maybe

```

`LensTerm` realizes the idea of interpreting an open expression. It does this by building upon a lens between value environments and return values with two modifications: it takes `TEnv env` to perform intensional analysis on the structure `env` (to implement `var`), and `Maybe` is used as a container type of the environment to perform runtime linearity checking. We also use the functions: `merge :: VEnv env -> VEnv env -> VEnv env`, which merges component-wise value environments with checking of the linearity represented by `Maybe` and fails when it attempts to merge two `Just`s; and `unitEnv :: TEnv env -> VEnv env`, which returns the value environments filled with `Nothing`, the unit element of `merge`. We shall omit its instance definition of `Variables`.

Fig. 4.4 shows the implementation of `unpair`'s semantic function. The main work is done by lenses manipulating and reformatting the environments: `rearrPair` takes the first two elements of an environment and pairs them; `letLens` is a lens that acts as a `let` for its two argument lenses. Since

4. Embedding by Unembedding

<pre> linesB :: BString → B[String] linesB str = let (f, b) = breakNLB str in case b of '\n' : x : r → f : linesB (x : r) with (> 1) ∘ length by λb_. '\n' : ' ' : b _ → f : [] with (== 1) ∘ length by λb_. lastNL b breakNLB :: BString → B(String, String) breakNLBstr = ... </pre>	<pre> linesB :: HOBiT e => e String -> e [String] linesB str = unpair (breakNLB str) \$ \f b -> case_ "linesB-case" b [consP (constP '\n') (consP varP varP) --> (\x r -> f `consB` linesB (x `consB` r), (> 1) .length, _b _ -> '\n' : ' ' : _b), varP --> (_ -> consB f nilB, (== 1) . length, _b _ -> lastNL _b)] breakNLB :: HOBiT e => e String -> e (String, String) breakNLBstr = ... </pre>
--	---

(a) Original **HOBiT** code (Matsuda and M. Wang [2018d, Fig. 2], with formatting)

(b) Embedded **eHOBiT** code in Haskell

Figure 4.5.: Comparison of *linesB* in **HOBiT** and **eHOBiT** to showcase the faithfulness of the embedding.

these are lenses, this manipulation is implemented as a get and a put function. These lenses both use `fromMaybe` to implement how **HOBiT** handles variables: an updated value of a variable defaults to the original value if the variable is unused. The `fromMaybe` function achieves this defaulting because when its first argument is `Nothing`, it returns its second argument. We are now ready to give `unpair` its HOAS representation:

```
unpair :: exp (a, b) -> (exp a -> exp b -> exp r) -> exp r
```

together with implementation.

```
unpair = liftS0n (o10 :: o12 :: End) unpairSem
```

This `unpair` destructs products, so to perform case analysis on all types, we also implement a function, `branch`, that does the same for sum types. Since algebraic datatypes can be expressed as recursive sum-of-product forms, this is a fairly comprehensive approach.

Comparison of **eHOBiT with **HOBiT**** To demonstrate the programmability and expressiveness of the bidirectional EDSL, we perform a side-by-side comparison between **HOBiT** and our embedded version (Fig. 4.5). This example features further streamlining of the syntax:

```

case_ :: HOBiT exp => String -> exp a -> [Br exp a r] -> exp r
(-->) :: Pat '[] o a
      -> (Func exp o (exp r), r -> Bool, a -> r -> a) -> Br exp a r

```

These methods are derived from `unpair` and `branch`, where `Pat [] [a1, ..., an]` represents patterns of type `a` that bind variables of types `a1 ... an`,¹⁴ and `Func` is a type family defined as `Func exp [a1, ..., an] (exp r) = exp a1 -> ... -> exp an -> exp r`. We leave the details of its definition to Appendix 15.2.

The code comparison implements the running example from the **HOBiT** paper [Matsuda and M. Wang 2018d], which is a bidirectional version of the standard `lines :: String -> [String]` function

¹⁴The first argument of `Pat` is used for the difference list encoding of bindings inspired by Polakow [2015].

in Haskell that splits an input string by newlines as `lines "A\nB\nC" = ["A","B","C"]`. An intricate behaviour of `lines` is that it ignores the last newline as `lines "A\nB\nC\n" = ["A","B","C"]`. We faithfully implement this behaviour and the bidirectional version `linesB` detects and preserves the existence of the last newline in the backward direction.

Unsurprisingly, **HOBiT**'s version comes with less syntactic overhead, but thanks to derived functions including the pattern combinators, our EDSL version has a similar structure akin to the original **HOBiT** version. The syntactic overhead is not negligible, but is a reasonable price to pay for accessing host's functionalities, including typing checking, editor support, advanced compilation, interoperability, and so on.

There are methods existing in the literature for using host-level pattern matching in embedded DSLs, for example, that of [McDonell, Meredith, et al. \[2021\]](#). The basic idea is to use host-level pattern matching to generate branches. For example, a destructor for `Either a b` takes two branches: one of type `f :: exp a -> exp r` and the other of type `g :: exp b -> exp r` in a finally-tagless EDSL. The branches can be unified as `Either (exp a) (exp b) -> exp r`, and host level patterns used to write the unified branch as `\case { Left a -> f a; Right b -> g b }`. However, this approach assumes that the number of branches is fixed for a datatype, and the workarounds they suggest do not scale to recursive types like lists. Also, the approach does not work well with the widely-adopted top-down matching. In our EDSL, `case_` uses the top-down matching both in `get` and `put`, which plays an important role in the `linesB` example (Fig. 4.5b).

Unembedding is also not tied to the specific way that we have expressed bidirectional branching. Our (finally-tagless) embedding supports other approaches: the language can be extended rather easily to include such variants.

4.7. Related Work

This section discusses work related to our framework; including places unembedding has previously been used, similar techniques, further details pertaining to the bidirectional application domain, and a review of languages that have been embedded in a similar manner.

Uses of Unembedding Unembedding [[Atkey 2009](#); [Atkey et al. 2009](#)] and similar techniques have been used in embedding for intensional program manipulation such as code generation. The high-performance array computing EDSL `accelerate` uses the technique to convert a surface language to an internal representation based on de Bruijn terms, with sharing recovery [[McDonell, Chakravarty, et al. 2013](#)]. The technique has been used also for having a better interface for an SMT solver [[Stewart 2010](#)], and a better syntax for probabilistic programs to be converted to OpenCL code [[Werk et al. 2012](#)]. [Matsuda and M. Wang \[2018b\]](#) showed how unembedding can be used to embed a reversible language `FliPpr` [[Matsuda and M. Wang 2013, 2018c](#)], in which users describe pretty-printers with annotations to obtain parsers corresponding to the pretty-printers, in the form of context-free grammars with semantic actions. In contrast, our contribution lies in demonstrating the usefulness of the unembedding for implementing advanced semantics beyond code manipulation. We also provide a general recipe for Embedding by Unembedding that others may build their embedding implementation upon.

Intensional Analysis on Open HOAS Terms `Babybel` [[Ferreira and Pientka 2017](#)] is a meta-programming framework that supports manipulation of open terms in the HOAS representation, based on contextual type theory [[Pientka 2008](#)]. Roughly speaking, a contextual type $[\Gamma \vdash A]$ represents open guest terms that have type A under context Γ . They also provide a first-class substitution so that we can manipulate open terms without worrying about tedious shifting and renaming (both affect Γ). There is

4. Embedding by Unembedding

certain similarity between theirs and ours: both use HOAS and handle open terms. However, there is a critical difference: their HOAS lies in a guest language’s intensional function space, while ours lies in the host language (Haskell)’s extensional one. For example, terms like `lam (\x -> app (lam (\y -> x)) x)` is a guest term instead of a host term in their system. This gives us intensional analysis on such guest terms: for example, they allow patterns like `lam (\x -> m)` to match against the previous term to bind `m` with an open term `app (lam (\y -> x) x)`. Also, unlike the tagless final style [Carette et al. 2009] and PHOAS [Chlipala 2008], the clear separation between the guest and host systems does not require polymorphism to avoid exotic terms. However, having guest terms with `λs` requires a separate embedding technique (though `λs` are only binding constructs in the system), if we use the idea in main-stream languages such as Haskell. For example, in Babybel, they use OCaml’s AST-level pre-processor (PPX) to translate contextual-typed terms into de Bruijn indexed terms. In Haskell, we could use Template Haskell [Sheard and S. L. P. Jones 2002], but Embedding by Unembedding, with future extensions, could serve as another option.

Bidirectional EDSLs Embedding bidirectional languages to benefit the host’s programmability has been a longstanding problem faced by the research community. As discussed in §4.6, the standard approach of representing bidirectional building blocks (such as `lens` [J. Nathan Foster et al. 2007]) as black boxes is unsatisfactory for programmability, limiting programmers to unfamiliar combinators and unconventional program structures. Many efforts have been made to address this problem. Voigtländer [2009]’s semantic bidirectionalization interprets polymorphic functions as bidirectional transformations, which is convenient as programmers write host language’s polymorphic functions, but with quite limited expressiveness—there is no way to specify refined bidirectional behaviours. The applicative lens framework [Matsuda and M. Wang 2015, 2018a] extends semantic bidirectionalization to support `prim`, `unit` and `pair` in §4.6 to use the host language to compose lenses. However, its expressiveness is still limited, as the approach does not scale to the lens combinators. The same authors address the expressiveness issues in HOBiT [Matsuda and M. Wang 2018d], which until now was a stand-alone language.

Applicative Lenses By using the lens variant of `runOpenN`, `runOpenAppLensN` adapted from the `runOpenN` provided by our framework and introduced in §4.4.4, we can construct a lens from a polymorphic function; *e.g.*, we can convert `reverse :: [a] -> [a]` into a lens by `runOpenAppLensN (foldr consB nilB . reverse) :: Lens [a] [a]`. That is, we can mimic Voigtländer [2009]’s semantic bidirectionalization. However, our EDSL is not limited to polymorphic functions; *e.g.*, we can apply concrete lenses by `prim :: HOBiT exp => Lens a b -> exp a -> exp b`, a construct that applies a lens to a result. In fact, this simple DSL is as expressive as the applicative lens framework [Matsuda and M. Wang 2015, 2018a], which uses Yoneda embedding instead of the unembedding, and thus does not scale to second-order constructs.

Similarly Embedded Languages Kiselyov [2019] discussed an approach to implement nondeterministic computation by embedding a core first-order nondeterministic DSL (which can have second-order constructs), without monads. Advantages of this approach is that one can focus on a simple DSL, which is easier to reason about and supports various interpretations (*e.g.*, abstract interpretation and staged interpretation) that are not possible with monads. The HOAS representation of the syntax plays an important role, as it enables us to use host’s `λs` and higher-order functions to define programs in the DSL. Embedding by Unembedding framework encourages this design, as it enables us to embed a core first-order system with HOAS, where previously such an implementation was not obvious, as demonstrated in §4.6 and §4.5. PyRo (<https://pyro.ai/>), a popular probabilistic programming library

in Python, shares a similar design [Bingham et al. 2021; Obermeyer et al. 2020] to Kiselyov [2019], though without using HOAS. This fact suggests a potential application of Embedding by Unembedding.

For a comparison of the unembedded CTS with the original and work related to CTS, see Appendix 14.

4.8. Future Work

Embedding by Unembedding opens the door to embedding a whole new host of languages. We plan to make use of this technique, push it to its limit, and work out where its boundaries lie. From a practical perspective, our future work plans to put Embedding by Unembedding to work. Initially on targets we feel ripe for embedding (*e.g.*, automatic differentiation), but then onto those that may present more of a challenge (perhaps in a linear setting). Then from a theoretical standpoint, we feel that there is much to gain from exploring the theoretical underpinnings of this technique.

Automatic Differentiation Another interesting application of Embedding by Unembedding is automatic differentiation for higher-order languages. Having a correct automatic differentiation for a higher-order language is a challenging topic [Ehrhard and Regnier 2003; Huot et al. 2020; Krawiec et al. 2022; Mazza and Pagani 2021; Sherman et al. 2021; Vytiniotis et al. 2019; F. Wang et al. 2019], especially for the reverse mode, and Embedding by Unembedding sheds a new light on it: we can focus on the core first-order fragment while embedding allows us to use host-level higher-order functions—a system as a whole constitutes a differential higher-order language. Specifically, we can keep a straightforward and easy-to-reason semantics $\llbracket \mathbb{R}^a \rightarrow \mathbb{R}^b \rrbracket = \mathbb{R}^a \rightarrow (\mathbb{R}^b, \mathbb{R}^b \multimap \mathbb{R}^a)$ for the reverse-mode automatic differentiation, where $\multimap$ denotes a linear transformation and thus $\mathbb{R}^b \multimap \mathbb{R}^a$ represents the Jacobian matrix (at the input). By restricting the EDSL type to $\mathbb{R}$ or their products, $\llbracket \Gamma \rrbracket \simeq \mathbb{R}^a$ and $\llbracket A \rrbracket \simeq \mathbb{R}^b$ hold for some a and b to allow the semantic domain as above. Now, Embedding by Unembedding enables higher-order programming in a host language, without risking correctness. As mentioned earlier, we conjectured that the unembedding essentially models a system as a whole in a presheaf category. Thus, in this sense, the embedding-by-unembedding approach would be similar to λ_S [Sherman et al. 2021], which uses a presheaf category to model their higher-order differentiable language, but the unembedding has not been used for their accompanied implementation. Exploring embedded differentiable programming languages is an interesting future direction. One of the challenges in the application is performance—the overhead introduced by the unembedding is unwelcome especially in this problem domain. We believe that staged programming can remove the overhead (see Appendix 13.4 for such a discussion for the incremental calculus).

Probabilistic Programming We are also interested in building upon the work of Nguyen et al. to see how Embedding by Unembedding compares to their use of effect handlers to implement the advanced semantics of probabilistic languages, including using host language variables for random or optimisable variables.

Linear Unembedding A future direction is to develop a linear version of the unembedding. As mentioned in §4.6, linear systems fit better for reversible and bidirectional computations. Since a practical linear programming language is now available as Linear Haskell [J. Bernardy et al. 2018], unembedding that utilizes the host’s linear parametricity attracts more practical interest. Another interesting future direction is to apply Embedding by Unembedding to more practical situations. One candidate is automatic differentiation.

4. Embedding by Unembedding

Criteria for Unembedding In §4.4.1, we mentioned that our approach can be applicable to simply-typed languages where the typing rules and constructs have the designated forms and come with compositional semantics. This description, while intuitive, has points to clarify: for example, why a method $lam :: (e \ a \rightarrow e \ b) \rightarrow e \ (a \rightarrow b)$ has a polymorphic type (abstracting a and b), while we restrict a language to be simply typed; and what are the HOAS representations and semantic functions that corresponds to a construct in general? Specifically, our target languages are the second-order algebras [M. P. Fiore, Plotkin, et al. 1999] as presented in Hamana [2018, 2019], where a symbol (operator) can have a (predicative) polymorphic type. What this means and why this is the case is the subject of future work.

Correctness The correctness of our method, Embedding by Unembedding, intuitively says that the interpretation via `runOpen` coincides with the interpretation of the corresponding de Bruijn term. This is a generalization of the Atkey [2009] proof mentioned in §4.4.4, the formalization of which and expansion to also cover PHOAS is future work. Although the original proof does not cover PHOAS directly, showing that the PHOAS and the tagless-final style are isomorphic by using parametricity [Reynolds 1983; Wadler 1989] is straightforward.

Categorical Exploration We showed in §4.6 that embedding lenses by unembedding leads to an EDSL as expressive as HOBiT [Matsuda and M. Wang 2018d]. This correspondence is not accidental. For terminating computations, we conjecture that the idea of unembedding [Atkey 2009] is to interpret HOAS terms in a presheaf $[C^{op}, \mathbf{Set}]$, *i.e.*, the category of functors from the opposite category of C to the category of the total (hence, terminating) functions, where C denotes the semantic domain of the (non-HOAS) terms. HOBiT adopted a staged semantics [Matsuda and M. Wang 2018d] to evaluate away λ s in the first stage and leave bidirectional constructs for the second stage. It is known that the semantics of two-level languages can be modelled in a presheaf $[C^{op}, \mathbf{Set}]$ (again, assuming termination of the first stage computation), where C denotes the semantic domain for the second stage [Moggi 1998]. This suggests that a language with a presheaf semantics (such as λ_S [Sherman et al. 2021]) might be embedded by unembedding just by embedding the “core part” of the semantic domain (*i.e.*, C). Formalization and further investigation of this is an interesting future direction. What this means and why this is the case is the subject of future work.

Another conjectured relationship to category theory is a connection between the criteria in §4.4.1 and second-order algebras [M. P. Fiore, Plotkin, et al. 1999; Hamana 2018]. The typing rule of the second-order construct `con'` in §4.4.1 has the same form as the one for an operator in a typed second-order algebra used in Hamana [2018]. M. P. Fiore, Plotkin, et al. [1999] discuss the semantics of second-order algebra in a functor category $[\mathbb{F}, \mathbf{Set}]$. Here, $\mathbb{F}$ is a category of manipulations of indices, which can also be seen as manipulation of environments, and in fact include weakening, exchange, and contraction. In §4.4.2, we conjectured a connection between `Variables` and $\mathbb{F}$; they both characterize manipulations of environments, though `Variables` contains the minimal operations needed for Embedding by Unembedding. Thus, it is plausible to conjecture that we can have a unified framework of Embedding and Unembedding based on the second-order algebra and show that the finally-tagless [Carette et al. 2009] HOAS is an initial representation of second-order algebras. Then, we also expect that a language presheaf semantics such ones mentioned above can be modelled by a functor $\mathbb{F} \rightarrow C^{op}$, which can turn a presheaf $[C^{op}, \mathbf{Set}]$ into the functor category $[\mathbb{F}, \mathbf{Set}]$ of interest.

4.9. Conclusion

In this paper, we propose an embedding method called “Embedding by Unembedding”, which is based on the unembedding [Atkey 2009; Atkey et al. 2009] technique that originally gives a bijection between

the HOAS representation and the de Bruijn term representation of a language. We identify that HOAS-based embedding does not directly support compositional semantics defined on terms-in-contexts, and design a general strategy for embedding of DSLs targeting advanced semantic domains. We also give two substantial case studies in the areas of incremental computation and bidirectional transformation, showing the effectiveness and applicability of our general approach.

5. Contextual Embeddings

This chapter continues to extend the variety of modern and complex embedded languages that can feature binders and enjoy the functional vernacular of higher-order programming. In particular, it advances upon Embedding by Unembedding technique in two ways: it avoids type unsafe operations justified by external arguments, and it extends what sort of languages can be embedded with binders to include those with substructural type systems, future proofing the technique for the programming languages of tomorrow.

This chapter’s content is taken from *Contextual Embeddings*, which is currently under submission at the 2026 Conference on Programming Language Design and Implementation (PLDI). This work is a collaboration with Alex Kavvos and Jessica Foster also of the University of Bristol, led by me. As supervisor of Jess and me, Meng Wang took an advisory role in the project.

5.1. Introduction

Programming languages are the tool we reach for to solve computational problems. We take pride in designing languages specialised to different tasks and joy in finding the level of abstraction that’s just right for the task at hand. However, the implementation of a new language—no matter how small—is always a tiresome task: lexing, parsing, and interpreting can be an impediment to rapid prototyping of new languages.

A solution to this problem is to *embed* the new (*guest* or *object*) language into an existing one (the *host* or *meta* language). Hudak [1996] advocates for the technique, citing its versatility and evolvability. It avoids the need for building the infrastructure of a new programming language by reusing that of the old one. This is particularly useful when complex features of the host are taken advantage of, such as its typing system, or when we want our language to be interoperable with other languages embedded in the host. Further, embedded languages let us reuse *cognitive infrastructure*, i.e. if you know how to use the host, you already know a lot about using the guest.

It is particularly important to be able to embed languages with *binding* constructs. Such constructs abstract over variables, for example `let` or a λ -abstraction. *Binders* increase the expressivity of a language. They enable valuable features like functions, local variables, and pattern-matching, making them a must-have feature for any non-trivial language.

Implementing binders from scratch can be cumbersome, due to fiddly issues such as *scope*, *name duplication*, and *capture-avoiding substitution* [Pfenning and Elliott 1988]. The POPLmark challenge [B. E. Aydemir et al. 2005] drew attention to the importance of representing binders and the difficulties involved. As such, embedding languages with binders is a very well explored area [Atkey 2009; Carette et al. 2009; Chlipala 2008; Despeyroux, Felty, et al. 1995; Despeyroux and Hirschowitz 1994; Despeyroux, Pfenning, et al. 1997; G. P. Huet and Lang 1978; Matsuda, Frohlich, et al. 2023; Washburn and Weirich 2003, 2008].

The problem of implementing binders for embedded languages is further complicated as interest extends beyond languages with structural¹ type systems to those where restrictions are placed on the usage of bound variables. In modal type systems, the types can specify when a variable can be used.

¹That is, ordinary systems that support weakening, exchange, and contraction.

5. Contextual Embeddings

This is useful for guiding the correct programming of staged computation [Davies and Pfenning 2001] and ensuring that data does not flow in an undesirable manner [Kavvos 2019]. In substructural type systems, the types can specify how many times a variable can be used. Such systems are already used in influential languages like Rust [Matsakis and Klock 2014], which uses affine typing to control the use of mutable variables, and Haskell, which now has linear functions that ensure their input is used exactly once [J. Bernardy et al. 2018]. Linear type systems in particular have become increasingly relevant as quantum computing develops, with qubits behaving as a non-clonable and non-discardable resource. Hence, modal substructural type systems appear increasingly important to novel programming paradigms, and our embedding techniques must evolve to support them.

The simplest approach to implement binders is a first-order embedding, such as de Bruijn indices [de Bruijn 1972]. Here, variable occurrences are linked to their corresponding binder via indices. For example, the Church encoding of True and False would be expressed with de Bruijn indices as demonstrated with `tt` and `ff`.

```
tt = Lambda (Lambda (Var One)) -- \xy.x, Church encoding of 'true'
ff = Lambda (Lambda (Var Zero)) -- \xy.y, Church encoding of 'false'
```

`Var Zero` refers to the closest bound variable (which we are informally calling ‘y’) and `Var One` refers to the next closest bound variable (which we are informally calling ‘x’). In other words, variables are 0-indexed from the inside out. Unfortunately, de Bruijn indices (and first-order representations in general) are not particularly user-friendly. Using indices instead of names is less intuitive to programmers and error-prone. For example, consider how easy it would be to accidentally define `tt` instead of `ff`.

Thus, following the embedding ethos of reuse, people have turned to Higher-Order Abstract Syntax (HOAS) [G. P. Huet and Lang 1978; Pfenning and Elliott 1988], which uses the host language’s binders to implement guest binders. This technique was developed in the context of logical frameworks [Avron et al. 1987; Harper et al. 1993; Miller and Nadathur 2012], where it was very successful. Using HOAS we may encode the previous examples in a more ergonomic way:

```
data HOAS = Lam (HOAS -> HOAS) | App HOAS HOAS
tt = Lam (\x -> Lam (\y -> x)) :: HOAS -- \xy.x, Church encoding of 'true'
ff = Lam (\x -> Lam (\y -> y)) :: HOAS -- \xy.y, Church encoding of 'false'
```

HOAS is a much more ergonomic embedding technique, as it allows the embedded programmer to reuse any support for binding provided by the host language. However, it creates a host of other problems, e.g. exotic terms and the inability to intensionally analyse code. Hence, the problem of representing binders in embedded languages is **not a solved problem**.

The current state-of-the-art in embedding amounts to using advanced HOAS techniques as a front-end, which we then *unembed* to obtain a more versatile first-order embedding in the back-end. Unembedding was introduced because HOAS does not natively support intensional analysis: it does not allow the host language to pattern-match and analyse terms of the guest language, which is necessary for program transformations, e.g. closure conversion or shrinking reduction optimisations [Atkey et al. 2009; Guillemette and Monnier 2007; McDonell, Chakravarty, et al. 2013]. The importance of unembedding was highlighted by Matsuda, Frohlich, et al. [2023], who used it to expand the types of languages that can be embedded to include those with “advanced semantics” (note this does not encompass modal or substructural examples, which are listed as future work). Previously, such languages were challenging to embed, and had to be implemented from scratch. For example, in *loc. cit.* the authors use unembedding to embed HoBiT [Matsuda and M. Wang 2018d], a bidirectional lens language differentiated by its use of expressive binders instead of rigid combinators.

Implementing unembedding in a type-safe manner is surprisingly hard, even in languages with cutting-edge type systems [Grenrus 2025]. Current Haskell implementations [Atkey et al. 2009; Matsuda,

Frohlich, et al. 2023; McDonell, Chakravarty, et al. 2013] use `unsafeCoerce`, justifying its safety through an external metatheoretic argument. Atkey [2009] dedicates a whole paper to this proof, which is reused / adapted carefully by Matsuda, Frohlich, et al. [2023] and McDonell, Chakravarty, et al. [2013]; they concur that such an argument appears “unavoidable”. Chlipala [2008] encounters a similar problem in the setting of theorem provers with his work on Parametric Higher-Order Abstract Syntax (PHOAS). In Section 3.1, he finds it necessary to assert an additional axiom—an instance of *internal parametricity*—which is not readily available in current theorem provers.

We advance the current state-of-the-art in two ways. First, we tackle the outstanding problem of a *type-safe* unembedding, by avoiding what was previously deemed “unavoidable” [McDonell, Chakravarty, et al. 2013]. Second, we push the boat out by supporting a wide variety of languages with modal and/or substructural type systems. In summary, we offer the following contributions:

- We present *Contextual Embeddings* (§5.3), a technique for embedding languages with binders that combines the precision of first-order representations (§5.2) with an interface as ergonomic as HOAS.
- We demonstrate the core ideas behind Contextual Embeddings with an embedding of the STLC (§5.3.2).
- We justify the embedding with a proof that it is isomorphic to its first-order representation via a type-safe conversion without requiring parametricity (§5.3.3, this is normally type-unsafe, and justified through external arguments). Such a conversion has not been achieved by previous embedding techniques.
- We evidence that this is a flexible and applicable technique with implementations in Haskell, Lean4 and Agda (see artifact). This technique can be used in any modern host language that supports GADTs, type-level data (e.g. dependent types or Haskell’s DataKinds), and a suitable type class system (§5.3.5).
- We showcase the generality of our technique on a plethora of examples spanning three different type systems for embedded languages: structural (§5.3.2), modal (§5.4) and substructural (§5.5), with the latter two not requiring modality or substructural support from the host language. To our knowledge, this is the first technique to encompass all of these categories without support from the host.
- We offer a simplified version of our technique that can uniformly tackle all three categories of type systems (§5.6, §5.7, §5.8). The cost of this simplification is slightly reduced canonicity of the first-order embedding through the addition of some redundant type information, but we will argue that this is a reasonable engineering solution.

5.2. Background: First-Order Embeddings

First-order embeddings will play a pivotal role in the construction of Contextual Embeddings, so it is important to understand their definition (§5.2.1), trade-offs (§5.2.2), and importance as a target for unembedding (§5.2.3).

5. Contextual Embeddings

```

data Ty = Unit | Ty :-> Ty

data Ctx = Empty | Ctx /\: Ty

data Index :: Ctx -> Ty -> Type where
  Top :: Index (ts /\: t) t
  Pop :: Index ts t
      -> Index (ts /\: t') t

data STLC :: Ctx -> Ty -> Type where
  Var :: Index ts t -> STLC ts t
  Lam :: STLC (ts /\: t1) t2
      -> STLC ts (t1 :-> t2)
  App :: STLC ts (t1 :-> t2)
      -> STLC ts t1
      -> STLC ts t2
  Star :: STLC ts Unit

```

Figure 5.1.: The Simply-typed λ -Calculus (**STLC**), as a Generalised Algebraic Data Type (**GADT**)

5.2.1. First-Order Embedding of the STLC

Fig. 5.1 shows a first-order embedding of the **STLC** using a Haskell **GADT**². **Ty** is a data type representing the types of the guest language, in this case, a unit type (**Unit**) and a function type (**:->**). **Ctx** represents the guest’s typing context, which is a “snoc”³ list of guest types. Both of these are lifted to the type level (using Haskell’s **DataKinds** extension) to parametrise our de Bruijn index type (**Index**) and guest terms (**STLC**).

In **Index**, the **Top** and **Pop** constructors can be analogized to the **Zero** and **Successor** constructors of natural numbers, where the natural number here is the de Bruijn index. They are so named in reference to the extra type level work they do: ensuring that the **Index** represents a valid type **t** within the context. **Top**, referencing the most recently bound variable, can only be constructed when the type it is indexing from the context, **t**, was indeed the last one added to the context i.e. the context has shape $(ts \text{ /\: } t)$. Whereas **Pop** constructs an index that can reference a variable bound deeper in the context by “popping” off the most recently bound variable **t**.

STLC is a first-order embedding of the standard **STLC** with a unit type. A term in context **ts** with type **t** can either be: a variable (**Var** holding an **Index**), a λ -abstraction (**Lam**), or an application (**App**). We also add **Star**, which introduces a term of type **Unit**. Following the standard type system of the **STLC**, **Lam** abstracts over an extended context to introduce a function, and **App** eliminates a function by applying it to its argument.

This first-order embedding is appealing for many reasons: it is canonical, *intrinsically* typed, it supports pattern-matching, and it is representable in type-theory.

Canonical and Intrinsically Typed This embedding is one that contains exactly the information needed, no more and no less. Fig. 5.1 is exactly the **STLC** with no redundant information. Part of this arises from the fact it is an *intrinsically-typed embedding* [Reynolds 2003], i.e. one where the host’s typing system is leveraged to only allow typed guest terms. This is desirable due to the benefits of static types and the embedding mantra of reusing host features.

Supports pattern-matching One of the benefits of having an embedded language is we can often perform *metaprogramming* on it: this means we can use the host language to inspect, construct, and analyse programs of the guest language [Lilis and Savidis 2019]. This requires the ability to inductively define functions on its terms by pattern-matching. For example, we can write a function **countVarsDB** that counts how many variables occur in a term.

²While the presentation of this paper will be in Haskell, please see our artifact for examples in Lean and Agda.

³“snoc” indicates that values are added to the end of the list. (“snoc” is “cons” in reverse.)

```

countVarsDB :: STLC ts t -> Int
countVarsDB (Var _)      = 1
countVarsDB (Lam e)      = countVarsDB e
countVarsDB (App e1 e2) = countVarsDB e1 + countVarsDB e2
countVarsDB Star         = 0

```

Representable in Type Theory A significant amount of contemporary work in programming language uses *dependent type theories*, either as a programming language itself, such as Idris [Brady 2011], or as a tool for proving theorems, such as the proof assistants Rocq [Coquand and G. Huet 1988], Lean [Moura et al. 2015], or Agda [Norrell 2007]. A modern embedding should be compatible with dependent type theories. A key requirement is any data types defined as part of the embeddings should be *strictly positive*, so they can be constructed inductively. This first-order embedding is strictly positive.

However, there is one downside of this embedding: it is not user-friendly. The `tt` / `ff` confusion persists.

```

tt = Lam (Lam (Var (Pop Top))) -- \xy.x
ff = Lam (Lam (Var Top))      -- \xy.y

```

5.2.2. Dealbreaking Trade-off

This trade-off between these compelling benefits and usability is not new. In fact, in his original paper de Bruijn [1972] gives three criteria for embeddings:

1. “easy to write and easy to read for the human reader”
2. “easy to handle in metalingual discussion” (nowadays called *metatheory*)
3. “easy for the computer and for the computer programmer”

He argues that this representation satisfies (2) and (3), but not (1). Guillemette and Monnier [2007] also discuss the trade-off of de Bruijn representation. They do this in comparison to a more user-friendly HOAS embedding with the goal of achieving a type-preserving closure conversion. Their experience concurs with the criteria of de Bruijn [1972] highlighting the tension between (1) and (3). The first-order embedding is simply best for performing closure conversion (or other global transformations / optimisations), but it compromises usability for embedded programming.

5.2.3. Rise of the Unembed

Unembedding [Atkey et al. 2009] is the act of translating from some embedding, typically a user-friendly one, into a first-order embedding. Its core is given by a function `unembed`, which performs this translation.

```

unembed :: Embedding -> STLC ts t

```

There is normally also an inverse to this function, i.e. from the first-order representation to the ergonomic one. This creates a setting in which either representation can be used, depending on needs and convenience.

Since the first-order representation is the gold standard in all aspects except usability, we place great importance on support for unembedding, and endeavour to advance the current state-of-the-art to make it type-safe without requirements for external proofs.

5.3. Contextual Embeddings

In this section, we present Contextual Embeddings. We motivate their construction (§5.3.1), and demonstrate their use on the **STLC** (§5.3.2). We then justify the embedding by showing it to be isomorphic to the first-order embedding (§5.3.3). Finally, we explore the practicality of the technique by demonstrating how to run expressions of a Contextual Embedding (§5.3.4), and discussing the scope (§5.3.5) and performance (§5.3.6) of the technique.

5.3.1. Towards an Ergonomic First-Order Embedding

In an effort to create an ergonomic front-end support a type-safe unembedding, we shall start from the first-order embedding itself. Our goal is to make just enough changes to the first-order embedding to make it useable, but not so many that the changes are irreversible, precluding unembedding. The first change, inspired by **HOAS**, is to employ a host function to embed λ -abstraction⁴:

```
data STLC1 :: Ctx -> Ty -> Type where
  Var1 :: Index ts a -> STLC1 ts a
  Lam1 :: (Index (ts :/: a) a -> STLC1 (ts :/: a) b) -> STLC1 ts (a :-> b)
```

This lets us write the identity function using a named variable, piggybacking on the host's binders:

```
ergonomicId = Lam1 (\x -> Var1 x) :: STLC1 Empty (a :-> a)
```

However, there are two significant issues with this representation.

Firstly, it introduces *exotic terms* into the embedding. An exotic term⁵ is one that is representable in the guest language's embedding, but is not actually a term of the guest language [Despeyroux, Felty, et al. 1995]. For example, **exotic** is an exotic term introduced by this change.

```
exotic = Lam1 (\x -> case x of { Top -> ...; Pop _ -> ... })
```

Of course, terms of the **STLC** cannot branch depending on the value of the bound variable's index! The cause of this exotic term is because the host language (in this case Haskell) supports pattern-matching on the **Index** data type. It is important that an embedding excludes such exotic terms to avoid unexpected behaviour. Moreover, admitting exotic terms is counter-productive towards our goal of usability since preventing errors is one of **Molich and Nielsen [1990]**'s heuristics of usability.

We exclude exotic terms by exchanging **Index** for a truncated version **ProxyTop**.

```
data ProxyTop :: Ctx -> Ty -> Type where
  ProxyTop :: ProxyTop (ts :/: t) t
```

ProxyTop is **Index** with only the **Top** constructor. It provides the capability of accessing the most recently bound variable without admitting exotic terms: there is nothing to pattern-match on when there is only one constructor. Adopting this leads to the following version of the embedding.

```
data STLC2 :: Ctx -> Ty -> Type where
  Var2 :: ProxyTop ts a -> STLC2 ts a
  Lam2 :: (ProxyTop (ts :/: a) a -> STLC2 (ts :/: a) b) -> STLC2 ts (a :-> b)
```

⁴We omit the App and Star constructors in this subsection as they're not relevant here.

⁵This is somewhat of a misnomer, as an 'exotic' term is not really a term.

```
notExotic = Lam2 $ \x -> case x of { ProxyTop -> Var2 x }
```

We can further ensure the exclusion of exotic terms by using encapsulation mechanisms of the meta-language, e.g. by making `ProxyTop` *opaque*, so that its constructors are not exported to user-facing modules.

The second problem we have to solve is that we can only access the most recently bound variable. Consider using this embedding to define the constant function $(\lambda xy.x)$.

```
k = Lam2 (\x -> Lam2 (\y -> Var2 ???))
```

To fill in the hole we require a term of type `ProxyTop (('Empty' :/: a) ' :/: b) a`. We would like to use `x`, but it has the type `ProxyTop ('Empty' :/: a) a` instead. The issue is that the context can grow between the binding site of a variable and its use site, but `ProxyTop` only allows us the last variable to be used.

We can solve this if we have evidence that the context at the use site is an extension of the context at the binding site. This evidence is represented by the following type `ExtE`, with `Refl` evidencing that the contexts `ts` are the same, and `Step` supporting an extension (a.k.a. weakening) by one. The function `reify` can combine this evidence with the `ProxyTop` value to recover an appropriate `Index` at use site.

```
data ExtE :: Ctx -> Ctx -> Type where
  Refl  :: ExtE ts ts
  Step  :: ExtE ts ts' -> ExtE ts (ts' :/: t)

reify :: ExtE ts ts' -> ProxyTop ts t -> Index ts' t
reify Refl      ProxyTop = Top
reify (Step e) pt      = Pop (reify e pt)
```

Thus, adding this evidence to the variable constructor allows access to bound variables that were bound deeper in the context.

```
data STLC3 :: Ctx -> Ty -> Type where
  Var3  :: ExtE ts ts' -> ProxyTop ts t -> STLC3 ts' t
  Lam3  :: (ProxyTop (ts :/: a) a -> STLC3 (ts :/: a) b) -> STLC3 ts (a :-> b)
```

```
k' = Lam3 (\x -> Lam3 (\y -> Var3 (Step Refl) x))
```

Alas, that which was cast out through the door has returned by way of the window!⁶ The term `Var3 (Step Refl) x` is just `Var (Pop Top)` with extra steps, and De Bruijn indices have returned. But fear not! Finding the index can be *automated* through the host's *instance resolution* mechanism. Thus we define the type class `ReifyIndex ts ts'` featuring a (modified) `reify` function.

```
class ReifyIndex ts ts' where
  reify :: ProxyTop ts t -> Index ts' t
```

Next, we define instances of this class corresponding to the constructors of `ExtE`.

⁶French proverb: “Chassé par la porte, il revient par la fenêtre.”

5. Contextual Embeddings

```
instance {-# OVERLAPPING #-} ReifyIndex ts ts where
  reify ProxyTop = Top
```

```
instance (ReifyIndex ts ts', ts' ~ (ts' :/: a))
  => ReifyIndex ts ts' where
  reify i = Pop (reify i)
```

The first instance corresponds to the reflexive case, where the context is not extended and our `Index` is guaranteed to be `Top`. The second instance provides the `Step` case. It is defined recursively: assuming we can `reify` an `Index` from `ts` to `ts'`, then we can `reify` from `ts` to `(ts' :/: a)` by `Popping`. In other words, every ‘`snoc`’ corresponds to a `Pop`, until the contexts are (type) equal. You can think of the `ProxyTop` as saving a snapshot of the context at the bind site, so that at the use site we can reify the correct `Index` via instance resolution.

There are some nuances to these instances in Haskell (as well as Agda, which uses a very similar system). We need instance selection to be *unambiguous* and branch based on type equality. Instance selection is based only on the *instance head*, not any of the constraints. Therefore, our two instance heads are `ReifyIndex ts ts` (i.e. the contexts are (type) equal) and `ReifyIndex ts ts'` (i.e. the contexts are not *necessarily* equal, but they could be). These two instances overlap in the case where the contexts are equal, but since `ReifyIndex ts ts` is *strictly more specific* than `ReifyIndex ts ts'`, we can mark it as `OVERLAPPING` to make instance resolution unambiguous: if the contexts are equal, use the reflexive instance, otherwise, use the ‘`snoc`’/`Step` instance.

Notice that even though we are calling it the ‘`snoc`’ instance, there’s no ‘`snoc`’ in the instance head. Instead, we intentionally defer this ‘`snoc`’ check until *after* the instance has already been selected, which we do using the `ts' ~ (ts' :/: a)` type equality constraint. This distinction is subtle, but it gives us more fine-grained control about how instances are *selected* vs. how they are *checked*. This will be especially important for our linear example later and the most general version of our technique. However, this trick also means that Haskell cannot tell that the recursive `ReifyIndex` constraint is structurally smaller, and so we must enable the `UndecidableInstances` extension. It *is* structurally smaller though, meaning this is perfectly safe.

5.3.2. Contextual Embedding of the STLC

Thus we arrive at the Contextual Embedding of the `STLC`.

```
data STLCctx :: Ctx -> Ty -> Type where
  CVar  :: ReifyIndex ts ts' => ProxyTop ts a -> STLCctx ts' a
  CLam  :: (ProxyTop (ts :/: a) a -> STLCctx (ts :/: a) b) -> STLCctx ts (a :-> b)
  CApp  :: STLCctx ts (a :-> b) -> STLCctx ts a -> STLCctx ts b
  CStar :: STLCctx ts Unit
```

`CLam`’s host function lends us Haskell’s usable binder interface. `ProxyTop` protects us from exotic terms by not providing anything to branch on through pattern-matching. And this is all made possible by leveraging the host language’s type classes.

This embedding preserves the benefits of the first-order embedding, while gaining usability.

It is intrinsically typed The embedding is parameterised with contexts and types, and only represents intrinsically-typed terms.

It supports pattern-matching The embedding is a **GADT** (in Haskell) or an indexed inductive type (in type theory), so it is possible to define functions over it by pattern-matching, such as `countVarsCtx`.

```
countVarsCtx :: STLCContext ts t -> Int
countVarsCtx (CVar _) = 1
countVarsCtx CStar     = 0
countVarsCtx (CLam e) = countVarsCtx (e ProxyTop)
countVarsCtx (CApp e1 e2) = (countVarsCtx e1) + (countVarsCtx e2)
```

It is representable in type theory The data type is strictly positive.

It is user-friendly Unlike the first-order embedding, variables have names, helping us define `tt` and `ff` correctly.

```
tt = CLam (\x -> CLam (\y -> CVar x)) :: STLCContext ts (a -> b -> a)
ff = CLam (\x -> CLam (\y -> CVar y)) :: STLCContext ts (a -> b -> b)
```

5.3.3. Isomorphic to First-Order Embedding

One of the greatest advantages of this embedding technique is that it supports type-safe unembedding. While this poses difficulties in HOAS-like embeddings, it is completely prosaic here.

```
unembed :: STLCContext ts t -> STLC ts t
unembed (CVar i)      = Var (reify i)
unembed (CLam e)      = Lam (unembed (e ProxyTop))
unembed (CApp e1 e2) = App (unembed e1) (unembed e2)
unembed CStar         = Star
```

Most of the heavy lifting is hidden in the `CVar` case, where the `ReifyIndex` instance and `ProxyTop` stored inside it are used to reify the de Bruijn index. The only other feature of interest is that every time we come across a binding construct, such as a λ -abstraction, we have pass it the ‘capability’ (`ProxyTop :: ProxyTop (ts :/: t') t'`) that enables it to access the top variable. When this capability reaches a variable, it is automatically reified to the right de Bruijn index.

We complement `unembed` with its inverse, `contextualise`, which translates the first-order embedding back to the Contextual Embedding.

```
contextualise :: STLC ts t -> STLCContext ts t
contextualise (Var i)      = toCVar i
contextualise (Lam e)      = CLam (\_ -> contextualise e)
contextualise (App e1 e2) = CApp (contextualise e1) (contextualise e2)
contextualise Star         = CStar
```

There are two notable cases here: `Lam` and `Var`.

In the `Lam` case, we ignore the variable. That might seem odd at first. To understand it, remember the purpose of this ignored value, the `ProxyTop`, is to enable the recovery of the de Bruijn index. However, in the case of `contextualise`, when we get to the `Var` case we *still have* the de Bruijn index, so the `ProxyTop` is indeed not needed here.

5. Contextual Embeddings

The `Var` case is the most complicated one. To understand how it works, it pays to notice that the constraint in `CVar` is *existentially quantified*. Therefore, starting from an `Index ts' t` we need to produce a `ProxyTop` in a potentially smaller context `ts`, as well as an instance of `ReifyIndex ts ts'`. To achieve this we define an existential type that captures *just* this data.

```
data Var ts' t = forall ts. ReifyIndex ts ts' => MkVar (ProxyTop ts t)
```

You can consider `Var` as a small Contextual Embedding with just variables. This allows us to consider the conversion of variables in isolation. The workhorse that does this isolated conversion is the function, `fromIndex`.

```
fromIndex :: Index ts t -> Var ts t
fromIndex Top = varTop
  where varTop :: forall ts t. Var (ts :/: t) t
        varTop = MkVar (ProxyTop :: ProxyTop (ts :/: t) t)
fromIndex (Pop i) = weakenCVar (fromIndex i)
  where weakenCVar :: Var ts t -> Var (ts :/: t') t
        weakenCVar (MkVar p) = MkVar p
```

For every `Pop`, we weaken the context of the existential wrapper and thereby automatically weaken the `ReifyIndex` instance. For `Top`, we return `ProxyTop` in the existential wrapper. Here, `varTop` uses the `ScopedTypeVariables` language extension to specify the existential type; it would otherwise be ambiguous. Finally, we compose this conversion with a simple function that injects `Var` into the full Contextual Embedding (`STLCctx`) to implement `toCVar`.

```
fromVar :: Var ts t -> STLCctx ts t
fromVar (MkVar i) = CVar i
```

```
toCVar :: Index ts t -> STLCctx ts t
toCVar = fromVar . fromIndex
```

We have proven that `unembed` and `contextualise` form an isomorphism in Lean4. This proof is included in our artifact. `STLC` and `STLCctx` correspond very closely, so the isomorphism proofs are almost entirely taken care of by inducting on the term, simplifying with function definitions and the inductive hypotheses, and reflexivity (with normalisation). The main meat of the proof lies in the isomorphism between `Index` and `ProxyTop` with `ReifyIndex`. Once we have that, the rest follows without much trouble.

```
theorem isoL {ts : Ctx} {t : Ty} {e : STLC ts t}
  : unembed (contextualise e) = e
theorem isoR {ts : Ctx} {t : Ty} {e : STLCctx ts t}
  : contextualise (unembed e) = e
```

indexIsoL: (`reify . fromIndex`) This lemma says that `reify` is the inverse of `fromIndex`. This is the easier direction. The proof itself is a relatively straightforward induction on the index. The intuition is that for every `Pop` in the index, `fromIndex` weakens the `ProxyTop` by one, which `reify` then turns back into a `Pop`.

indexIsoR: (fromIndex . reify) The opposite direction is more difficult because we no longer have a concrete index to induct on. We have only a `ProxyTop` and a `ReifyIndex` instance. In practice, the instance (and therefore the index) is determined by the contexts, so you would hope that we could do some kind of induction on them, but sadly this is not immediately possible. In Haskell, Lean, and Agda, type classes are open world, which means if we are given a `ReifyIndex ts ts` instance, we can't say for certain that it's specifically the base case instance (even though it will be in practice) because another instance for that type could be defined elsewhere. Therefore, we need a 'closed world' axiom which says 'if you give me a `ReifyIndex` instance, it must be one of our two defined instances'. We believe this is a reasonable assertion to make in practice because we can hide the type class from users by not exporting it. Armed with our 'closed world' axiom, we can now complete the proof in this direction by case analysis on the `ReifyIndex` instances.

Bringing this all together completes our proof. The interesting cases are: `isoL`'s `Var` case, which relies on `indexIsoL`; `isoR`'s `CVar` case, which relies on `indexIsoR`; and `isoR`'s `CLam` case, which requires function extensionality.

5.3.4. Assigning Semantics

Of course, for any embedded language, it is important to be able to assign semantics and run embedded code. This is particularly ergonomic with a Contextual Embedding due to the availability of the first-order embedding to expedite the process. We shall demonstrate this by assigning the standard model semantics (interpreting the `STLC` as a mapping from variable assignments to results) to our `STLC` example.

The first step of assigning semantics is defining a mapping from guest language types to host language types so that the host can evaluate the guest's semantics. In Haskell, this can be done with the following type family.

```
type family InterpretType (ty :: Ty) where
  InterpretType Unit      = ()
  InterpretType (a -> b) = (InterpretType a) -> (InterpretType b)
```

This is a type level function that pattern-matches on the guest types and translates them to the corresponding host types, using recursion for function types.

We also extend this mapping for contexts (type-level lists), applying `InterpType` pointwise.

```
type family InterpretCtx (as :: Ctx) where
  InterpretCtx Empty      = '[]
  InterpretCtx (as :/: a) = InterpretType a : InterpretCtx as
```

Now we can define the standard model semantics. Concretely, this (`StdModel`) will be a function from a heterogeneous list (`VEnv`) assigning a value for each of the types in the context to a result. Notice the use of the type families for interpreting the guest types `ts` and `t`. This is essential for the host to execute the `StdModel` function.

```
type StdModel ts t = VEnv (InterpretCtx ts) -> InterpretType t
```

```
data VEnv as where
  ENil  :: VEnv '[]
  ECons :: a -> VEnv as -> VEnv (a ': as)
```

5. Contextual Embeddings

Assigning these semantics to the Contextual Embedding of the **STLC** (**STLCctx**), can be expedited by utilising **unembed**. This simplifies the assignment of semantics to a mapping from the more manageable de Bruijn representation of syntax (**STLC**) to the standard model semantics.

```
interp :: STLCctx ts t -> StdModel ts t
interp = interpDB . unembed

interpDB :: STLC ts t -> StdModel ts t
interpDB (Var i)   env = lookEnv env i
interpDB (Lam f)   env = \x -> interpDB f (consEnv x env)
interpDB (App f x) env = (interpDB f env) (interpDB x env)
interpDB (Star)    env = ()
```

This mapping (**interpDB**) is straightforward, with **lookEnv** looking up values in an environment using an index and **consEnv** adding values to an environment.

5.3.5. Scope of Contextual Embeddings

Host requirements Contextual Embeddings can be embedded into any host that supports GADTs, type-level data (in this case using Haskell’s DataKinds), and has a suitable type class system. The key thing that makes a type class system suitable is its ability to select instances based on type equality, such as in the base case of **ReifyIndex** where both contexts are the same, and structurally recurse on the contexts. Please see our artifact for additional examples in Lean and Agda.

De Bruijn Index Programming It is worth noting that access to the **ProxyTop** constructor allows the user to program as if with de Bruijn indices. Using the **ProxyTop** constructor with a specified type is equivalent to specifying the de Bruijn Index (or more accurately, the de Bruijn level). Consider the following example:

```
id :: forall ts a. STLCctx ts (a -> a)
id = CLam (\x -> CVar (ProxyTop :: ProxyTop (ts/:a) a))
```

This demonstrates how the programmer can avoid the benefits of using the host binder. Instead of providing **x** to **CVar**, the function has been completed by providing **ProxyTop** and its type. Of course, programming this way is as error-prone as programming with de Bruijn indices, so we recommend using a module boundary to hide this constructor.

Undefined and Non-terminating Terms We exclude undefined and non-terminating terms from our analysis. That is, introducing these terms may or may not undermine the benefits of our technique and they should be considered a bug in all cases.

Supported Metaprogramming We support metaprogramming such as global program optimizations. We also consider polymorphic types of guest terms as a form of metaprogramming. For example, the **STLC** does not have polymorphic type variables, but our interface allows for types such as **STLCctx ts (a -> a)**. A theorem prover’s access to the type information and ability to act based on that is another form of supported metaprogramming. All these forms of metaprogramming are guaranteed to produce a well-formed terms thanks to the isomorphism between the Contextual Embedding and the first-order embedding.

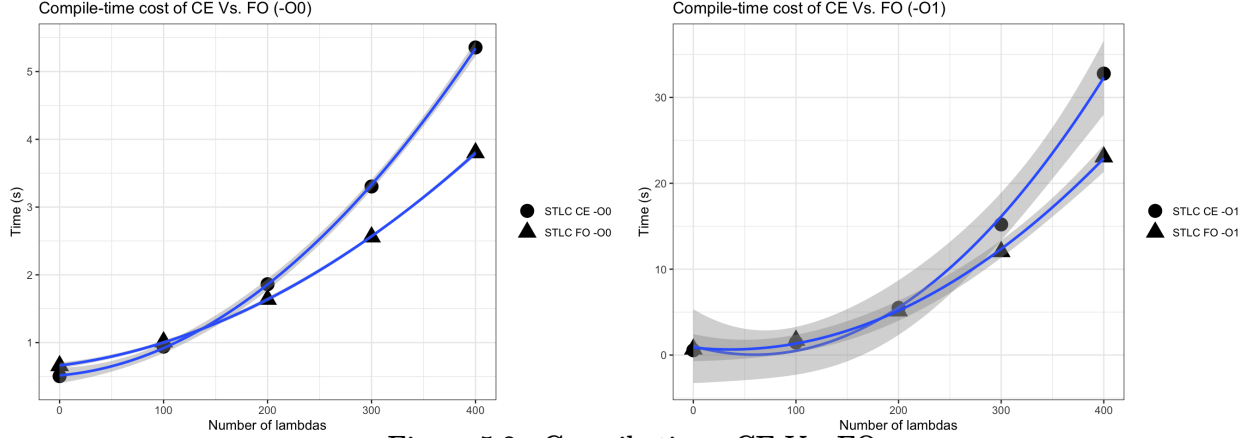


Figure 5.2.: Compile-time: CE Vs. FO

5.3.6. Performance of Contextual Embeddings

To ensure that the overheads of using a Contextual Embedding will not hinder users in terms of performance, we performed some benchmarking. We compared the Contextual Embedding of the **STLC** versus its first-order and **HOAS** counter-parts in terms of compile time, and we investigated the runtime cost of **unembed** and **contextualise**. The benchmarked code was a simple guest language expression of the following format⁷ (just compiled for the compile-time tests, and unembedded / contextualised for the runtime tests):

```
benchmark = CLam $ \y -> CLam $ \x -> ... CLam $ \x -> CVar y
```

We introduce a binder of interest, then separate it from its use-site by varying numbers of binders. These benchmarks were done on a machine with a 4.2GHz Intel i5-3570K CPU, and with GHC 9.10.1 and cabal 3.12.1.0.

Compile-time Overheads Our experiments show that there is indeed a compile-time overhead of the Contextual Embedding versus both the first-order embedding and the **HOAS** embedding. Fig. 5.2 and Fig. 5.3 show how compile time grows as the number of lambdas between the binding site and use site of our variable of interest increases. The tests were run with both the default optimisation flag (-O1) and -O0.

Fig. 5.2 compares our performance with the first-order representation. Fig. 5.3 compares our performance with the **HOAS** representation. The good news is that for a reasonable amount of lambdas (and even the unreasonable amount of 100 lambdas), compile times stay below 2 seconds, even on our very old test machine. Instance resolution costs are relatively negligible, as evidenced by the close compile times of Fig. 5.2. The first-order representation has no instance resolution or host lambdas, yet the compile times are on the same order of magnitude and scale similarly. The difference between our embedding and a **HOAS** embedding is caused by maintenance of the guest language’s typing context (which has brought us benefits elsewhere). We believe that this is a worthwhile trade-off because for a realistic number of binders, our compile times are very similar.

Runtime Overheads Fig. 5.4 shows a very small runtime cost for **unembed** and **contextualise**, on the order of microseconds, which grows linearly with the number of lambdas. This makes sense because

⁷For those unfamiliar with \$, Haskellers hate brackets, so we often replace them with this low-precedence function application operator. **benchmark** could instead be written as `CLam (\y-> CLam (\x-> ... (CVar x)))`.

5. Contextual Embeddings

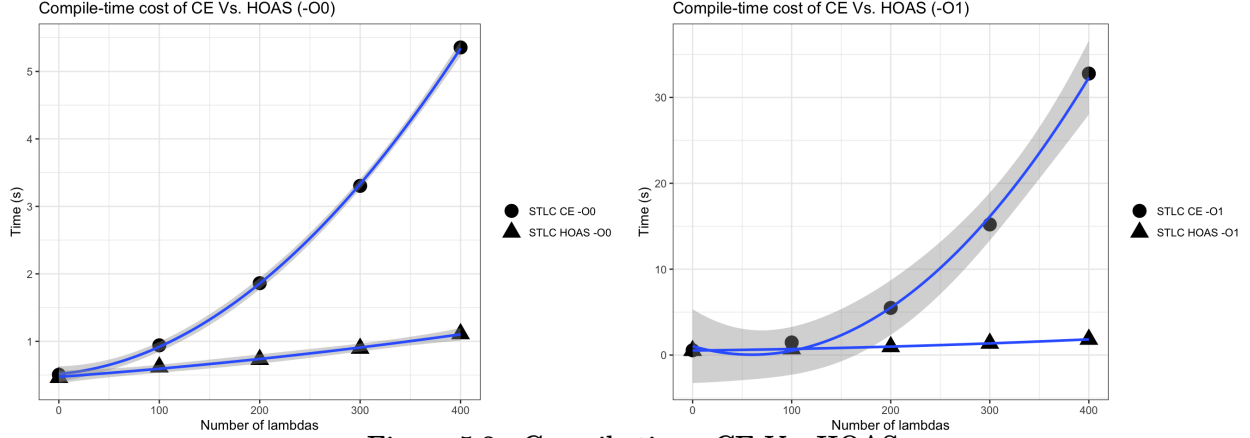


Figure 5.3.: Compile-time: CE Vs. HOAS

in Haskell types are erased, so there is no overhead from them at runtime. Looking at the compiled Haskell core also shows us what happens to the type classes and explains why this representation is so efficient. Consider `const3`:

```
const3 = CLam $ \x -> CLam $ \y -> CLam $ \z -> CVar x
```

When `const3` is compiled, the `CVar x` effectively compiles to `CVar' (\p-> Pop (Pop (case p of ProxyTop -> Top))) x`, where `CVar' :: (ProxyTop ts t -> Index ts' t) -> ProxyTop ts t -> STLCctx ts'` a because once compiled, type classes correspond to carrying around a concrete instance of the contained functions (in this case `reify :: ProxyTop ts t -> Index ts' t`). Thus, at runtime, unembedding amounts to the application of this function to `x`. In the other direction, contextualising involves building up the `reify` function by recursing on the `Index`, which has a very simple structure and can therefore be executed quickly.

5.4. Modal Example: Fitch-Style Modal Lambda Calculus

Moving on from the `STLC`, we present an embedding of a modal lambda calculus, which constitutes our first example of a non-structural language. The defining feature of modal languages is that they control when you can use variables. There are many examples of modal λ -calculi [Kavvos 2016] including Davies and Pfenning [2001] and Clouston [2017]. Both of these examples are amenable to a Contextual Embedding. The former involves two contexts but is otherwise straightforward using the techniques we have already shown. We shall therefore focus on the latter *Fitch-Style* modal λ -calculi, which have a more distinctively modal structure.

In structural languages like the `STLC`, the context grows as we move up a typing derivation, and elements in the context can be used freely at any point. Modal λ -calculi introduce additional controls on the use of variables. In Fitch-style modal λ -calculi, this is achieved by adding a modal type $\Box$, which we represent by the Haskell constructor `Box`.

```
data Ty = Unit | Ty -> Ty | Box Ty
```

It is sometimes helpful to consider terms with box types as “global” terms: they are free to depend on other boxed terms, but not non-boxed terms, as these may depend on “locally” available data. To enforce this, the type of contexts is extended from that of the `STLC` with a lock.

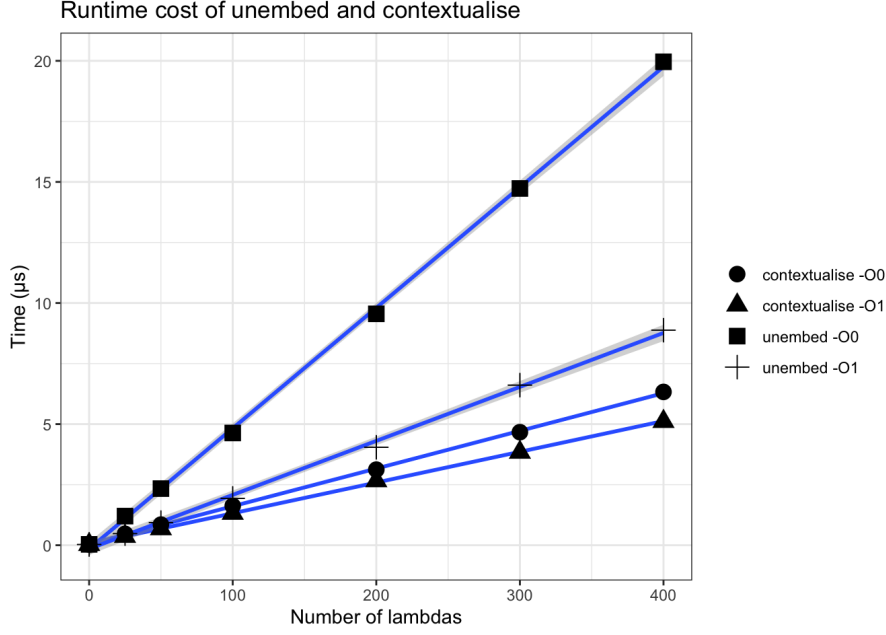


Figure 5.4.: Runtime Cost

```
data Ctx = Empty | Ctx :/: Ty | Lock Ctx
```

Indices are unchanged from the **STLC** and the first-order Fitch-Style Modal Lambda Calculus is embedded as follows:

```
data Fitch :: Ctx -> Ty -> Type where
  -- ...
  Shut :: Fitch (Lock ts) a -> Fitch ts (Box a)
  Open :: UnderLock ts ts' => Fitch ts' (Box a) -> Fitch ts a
```

```
class UnderLock (ts :: Ctx) (ts' :: Ctx) | ts -> ts' where
instance UnderLock (Lock ts) ts
instance UnderLock ts ts' => UnderLock (ts :/: a) ts'
```

`Var`, `Lam`, `App` and `Star` (elided) are unchanged from the **STLC**.⁸ We focus instead on the new constructs `Shut` and `Open`, which introduce and eliminate boxed values respectively. The encapsulation of boxed values is managed by `Shut` who only allows them to arise from a locked context, ensuring that they cannot rely on non-`Box` values in the context. `Open` supervises safe unlocking. The only values that can be opened under a lock are boxed values. It does this by being very specific about its argument. Note that the argument's type is `(Fitch ts' (Box a))`, where `ts'` is specified to be under a lock by the type class constraint⁹. If the argument's type was `(Fitch ts' a)`, `Open` would not be doing its job properly because this would allow any value under a lock to be opened. The specificity of this

⁸But note that `Ctx` has changed, giving `Pop :: Index ts t -> Index (ts :/: t') t` and `Top :: Index (ts :/: t) t` extra meaning, namely that you can't `Pop` past a lock or point to a lock with `Top`.

⁹`UnderLock` takes a context as the first variable (`ts`) and outputs the context 'under' the closest lock as the second variable (`ts'`). The functional dependency `ts -> ts'` ensures that `ts` uniquely determines `ts'`.

5. Contextual Embeddings

argument type allows the host type system to ensure that the modality is properly respected. For example, `bad` would be disallowed by the type system. Here, `x` is added to the context with `Lam` and we shut access to it to introduce the modality. However, this term is bad because it tries to then use `x`, violating the encapsulation of boxed values.

```
bad :: Fitch ts (a :-> Box a)
bad = Lam {- x -} (Shut {- Lock -} (Var (Pop Top {- x -})))
```

Gratifyingly, the Contextual Embedding proceeds very similarly to that of the `STLC` demonstrating the generality of the technique. For completeness, here is the `GADT` of contextual terms:

```
data FitchCtx :: Ctx -> Ty -> Type where
  CVar  :: ReifyIndex ts ts' t => ProxyTop ts t -> FitchCtx ts' t
  CLam  :: (ProxyTop (ts :/: a) a -> FitchCtx (ts :/: a) b) -> FitchCtx ts (a :-> b)
  CApp  :: FitchCtx ts (a :-> b) -> FitchCtx ts a -> FitchCtx ts b
  CStar :: FitchCtx ts Unit
  CShut :: FitchCtx (Lock ts) a -> FitchCtx ts (Box a)
  COpen :: UnderLock ts ts' => FitchCtx ts' (Box a) -> FitchCtx ts a
```

`ProxyTop`, `unembed`, and `contextualise` also proceed as expected.

The only slight change is in the base case instance of `ReifyIndex`.

```
instance {-# OVERLAPPING #-} (ts ~ (ts' :/: t)) => ReifyIndex ts ts t where
  reify ProxyTop = Top
```

This now features a check $(ts \sim (ts' :/: t))$ to ensure that there is a variable to reference, and not a lock. In the unchanged recursive instance, the check that ensured we only extend the context one variable at a time $(ts2 \sim (ts1 :/: a))$ also handily ensures that we don't pop past a lock (because no lock is matched on).

As before, the Contextual Embedding allows users to write terms with ease. As an example, consider the following term that is a *proof* of the K axiom of modal logic¹⁰, i.e. $\Box(A \rightarrow B) \rightarrow \Box A \rightarrow \Box B$.

```
kaxiomCtx :: FitchCtx ts (Box (a :-> b) :-> Box a :-> Box b)
kaxiomCtx = CLam $ \f -> CLam $ \x -> CShut (CApp (COpen $ CVar f) (COpen $ CVar x))
```

5.5. Substructural Example: LLC

For our substructural example, we present the `LLC`. Substructural type systems control how many times you can use variables. In a linear type system, you must use each variable exactly once. This disallows expressions that clone variables such as $\lambda x.(x, x)$, or expressions that drop variables such as $\lambda x.\lambda y.x$.

$$\frac{\text{APP} \quad \Delta_1 \vdash e_1 : A \multimap B \quad \Delta_2 \vdash e_2 : A}{\Delta_1 \Delta_2 \vdash e_1 e_2 : B} \quad \frac{\text{VAR}}{x : A \vdash x : A}$$

Standard presentations of the `LLC` allow the context to be non-deterministically divided between sub-terms, as in the rule `APP` [Polakow 2015], enforcing linearity by the variable rule `VAR` only being

¹⁰This calculus corresponds to a modal logic under the Curry-Howard-Lambek isomorphism [Clouston 2017].

```

data Ty = Unit | Ty :* Ty | Ty :-* Ty

data Ctx = Empty | Ctx :/: CtxElem

data CtxElem = J Ty | N

data Index :: Ctx -> Ctx -> Ty -> Type where
  Top :: Index (ts :/: J t) (ts :/: N) t
  Pop  :: Index ci co t
         -> Index (ci :/: a) (co :/: a) t

data LLC :: Ctx -> Ctx -> Ty -> Type where
  Var  :: Index ci co t -> LLC ci co t
  Lam  :: LLC (ci :/: J t1) (co :/: N) t2
         -> LLC ci co (t1 :-* t2)
  App  :: LLC ci ctx (t1 :-* t2)
         -> LLC ctx co t1 -> LLC ci co t2
  Star :: LLC ci ci Unit
  Pair :: LLC ci ctx t1 -> LLC ctx co t2
         -> LLC ci co (t1 :* t2)
  Letp :: LLC ci ctx (t1 :* t2)
         -> LLC (ctx :/: J t1 :/: J t2)
                (co :/: N :/: N) t
         -> LLC ci co t

```

Figure 5.5.: The First-Order Linear λ -Calculus (*LLC*), as a *GADT*

applicable when the context consists of a single variable. This non-determinism will not work with our type class setup, so instead we opt for a deterministic presentation that features two contexts: an input and an output context [Grenrus 2019; Hodas 1995; Polakow 2015]. Elements of these contexts can either be types or placeholders representing used up values. Linearity is enforced by making sure that in the end, all the values are used up.

For example, consider *varUsage*. This is a linear expression of the *LLC* because the only variable in the input context (of type *J t*) gets used up, as indicated by the *N* replacing this element of the context in the output context. To represent this (see Fig. 5.5, *Ctx* and *CtxElem*), our standard context type has been updated to either wrap context elements in a *J* to indicate they ‘Just’ need to be used up, or replace them with an *N* to indicate that ‘Nothing’ is left.¹¹

```

varUsage :: LLC (Empty :/: J t) (Empty :/: N) t
varUsage = Var Top

```

Fig. 5.5 shows the full Haskell embedding of the first-order *LLC*. It features a base type *Unit*, linear pairs (*:**), and linear functions (*:-**). Our *Index* type is updated to have two contexts and enforce the usage of the represented variable with *Top* matching on a switch from *J* to *N*. When it comes to embedded terms, *Lam* and our two new pair introduction and elimination constructs (*Pair* and *Letp*) are of interest. *Lam* enforces linearity by restricting the body of the lambda to use the newly introduced variable. When pairing two terms together, *Pair* carefully chains together the input and output contexts. These features of *Lam* and *Pair* are also found in *Letp*, as it also carefully chains contexts and enforces the usage of its bound variables.

Creating a Contextual Embedding for this language is more challenging than it seems at first. To appreciate why, consider the essence of the technique. We have been using *ProxyTop* to snapshot the context at the binding site, then at the use site we compare the contexts and recover the de Bruijn index with instance resolution. This works because the context always contains the information needed to recover the de Bruijn index. For example, in the case of the *STLC*, it was the difference in length between bind and use site contexts.

Of importance for considering *LLC* as a candidate for Contextual Embedding is how *ReifyIndex* is defined, in particular, its recursion pattern. Our base case is when the contexts are (type) equal (which works because the use site context is always an extension of the bind site context).

¹¹This naming is inspired by the Haskell’s *Maybe* type, abbreviated to single letters for ease of specifying typing contexts.

5. Contextual Embeddings

Consider now what the stopping point will be in the linear case. Matters here are complicated by two factors: there are now two contexts, and moreover these contexts may change between bind site and use site because variables could have been used up (switching them from J to N in the context). For example, consider `app`:

```
app :: LLC ts ts ((a :-* b) :-* a :-* b)
app = Lam {- \f -} $ Lam {- \x -} $ App (Var {- f -} (Pop Top)) (Var {- x -} Top)

bx :: ProxyTop (ts :: J (a :-* b) :: J a) (ts :: J (a :-* b) :: N) a
ux :: LLC      (ts :: N          :: J a) (ts :: N          :: N) a
```

The use site of `x` is after `f` has been used up, creating a difference between the bind site contexts of `x` (shown in `bx`) and the use site contexts of `x` (shown in `ux`). Notice how the context element corresponding to `f` ($J (a :-* b)$) is replaced by an N . This means that our stopping point can no longer be when the bind and use site contexts are equal, because they won't be.

An alternate solution is implementable, but complex. The key insight is that for any `CVar` sub-expression, exactly one element of the context should switch from J to N . This leaves our type classes with two tasks: inserting this switch in the correct place, and generating the correct index at the use site. The first task is achieved by a type class that uses both bind site contexts and the use site input context to calculate the correct use site output context. The difference in length between either bind site context and the use site context will give the position of where the switch from J to N should be. The rest of the output context is just the same as the input context. With this in place, our recursion pattern for calculating the correct use site index can use this inserted outermost switch from J to N as the stopping point, because any outer elements will exactly match the input context as specified by the first type class. Thus, if the top elements of the context are the same, we add a `Pop`. Then when we hit the switch, this is our base case and we add the `Top`.

The implementation of this is quite involved. It requires one incoherent instance, which is something that must be done with great care to ensure correctness. We have included this implementation in our artifact, but we will not go into the details here because they are very complex and specialised to this particular example. Instead, we'd like to explore a more general solution.

Motivated by how complex and specific this solution was to the linear case, we sought out a simpler and more general solution. We approached this by asking: what if instead of having to extract out the information we need each time, which must be done in a custom and specific way for each new type system, it was already separated out and directly accessible?

5.6. Contextual Embeddings Levelled Up

The issue discussed in the previous section leads us to present a slightly extended kind of Contextual Embedding *with levels*. The key difference is that we make the information that we need to calculate the de Bruijn index more accessible. Previously, this type information was implicitly present in the typing context(s), and had to be extracted. Now, with the information accessible, the type classes can be simplified to work in a uniform manner across different type systems, getting to the essence of our technique.

We will specify exactly what this key “levels” type information is (§5.6.1), demonstrate a Contextual Embedding with Levels on the `STLC` (§5.6.2), discuss the trade-offs of this simplified technique (§5.6.3), before revisiting the `LLC` (§5.7).

```

data Nat = Z | S Nat

data IndexL :: Nat -> Ctx -> Ty -> Type where
  TopL :: IndexL (S l) (ts :/: t) t
  PopL :: IndexL l ts t
  -> IndexL (S l) (ts :/: x) t

data STLCL :: Nat -> Ctx -> Ty -> Type where
  VarL :: IndexL l ts t -> STLCL l ts t
  LamL :: STLCL (S l) (ts :/: a) b
  -> STLCL l ts (a :-> b)
  AppL :: STLCL l ts (a :-> b)
  -> STLCL l ts a -> STLCL l ts b
  StarL :: STLCL l ts Unit

```

Figure 5.6.: A First-Order Embedding of the **STLC** with Levels

5.6.1. Contextual Embedding with Levels

To recover the de Bruijn indices, we have been interested in the differences in length between bind and use site contexts. We will now work with this number directly. We shall call this information “levels” due to its similarity with *de Bruijn Levels*—an alternative in which the indexing goes the other way, i.e. counts the depth of the variable from the top level.

Specifically, what we will refer to as the “level” is a natural number $l + n$, where l is the de Bruijn level, how many binders deep we are, and n is an arbitrary *offset*. This offset will be used to account for context polymorphism. For example, consider the two examples below. `levels` shows how the level increases as we go under binders with an empty outer context. Here, $n = 0$. `levels'` shows the same example again, but in a polymorphic outer context. Here, n is the length of `ts`.

```

levels :: STLCctx Empty (a :-> b :-> b)
levels = CLam (\y -> {- level == 1 -}
  CLam (\x -> {- level == 2 -} CVar x))
levels' :: STLCctx ts (a :-> b :-> b)
levels' = CLam (\y -> {- level == (length ts) + 1 -}
  CLam (\x -> {- level == (length ts) + 2 -} CVar x))

```

For our purposes, it doesn’t matter what n is because what we care about is the relative difference in the level between bind site and use site.

5.6.2. Contextual Embedding of STLC with Levels

To make the levels accessible, they must be added to the first-order embedding as in Fig. 5.6. We represent the levels as a natural number, embedded in the standard way with the `Nat` data type. These levels then parameterise the index and term data types alongside the context and guest type. In `IndexL`: `TopL` ensures that we are at least one level deep, echoing how it checks that the typing context is non-empty, and asserting that the length of a non-empty context is at least one; `PopL` increments the level to keep it in correspondence with the length of the context. In `STLCL`, `LamL` increments the level for its body.

The Contextual Embedding (Fig. 5.7) also gets parameterised by the levels, but the main changes are in the `ProxyTop` and the huge simplification of the `ReifyIndexL` instances. Now, the `ProxyTop` just saves the level as opposed to the whole bind site context. This allows the logic of the type class instances to rely directly on the level instead extracting it from the context. The base case is when the levels are the same, and for every difference between the levels, we add a `PopL`.

The equality constraints on both instances ensure additional type safety. In the first instance, `ts ~ (ts' :/: t)` ensures that the correct type `t` (also saved by the `ProxyTop`) is indeed at the top of the context. In the second instance, `(l' ~ (S l'))` now joins `(ts ~ (ts' :/: t'))` in ensuring we only handle one `PopL` at a time. The elegance of this technique is the triviality of the instance resolution,

5. Contextual Embeddings

```

class ReifyIndexL l l' ts t where
  reify :: ProxyTopL l t -> IndexL l' ts t

instance {-# OVERLAPPING #-}
  ( ts ~ (ts' :/: t))
=> ReifyIndexL l l ts t where
  reify ProxyTopL = TopL

instance ( l' ~ (S l''))
  , ts ~ (ts' :/: t')
  , ReifyIndexL l l' ts' t)
=> ReifyIndexL l l' ts t where
  reify i = PopL (reify i)

data ProxyTopL :: Nat -> Ty -> Type where
  ProxyTopL :: ProxyTopL (S l) t

data STLCContextL :: Nat -> Ctx -> Ty -> Type where
  CVarL  :: ReifyIndexL l l' ts t
    => ProxyTopL l t -> STLCContextL l' ts t
  CLamL  :: ( ProxyTopL (S l) a
    -> STLCContextL (S l) (ts :/: a) b)
    -> STLCContextL l ts (a -> b)
  CAppL  :: STLCContextL l ts (a -> b)
    -> STLCContextL l ts a -> STLCContextL l ts b
  CStarL :: STLCContextL l ts Unit

```

Figure 5.7.: A Contextual Embedding of the **STLC** with Levels

which doesn't rely on these equality constraints (these are checked after), but on the type equality check seeing if the levels are the same (`ReifyIndexL l l ...`) or different (`ReifyIndexL l l' ...`).

5.6.3. Justification of Levels

While this simplification comes at the cost of redundant information in the first-order embedding, we feel that this is reasonable because: this information is not new, it was already part of the context, we have just separated it out for convenience; we can still unembed the Contextual Embedding to the canonical first-order embedding (without the levels); and the levels buy us a simpler, more uniform and adaptable solution.

Unembedding Unembedding from the Contextual Embedding with Levels to the first-order embedding with levels is defined identically to our previous unembedding function. If desired, the extra levels type information can then be forgotten to complete the translation to the canonical first-order embedding.

```

unembed :: STLCContextL l ts t -> STLC ts t
unembed = forgetL . unembedL

unembedL :: STLCContextL l ts t -> STLC l ts t
unembedL (CVarL i) = VarL (reify i)
unembedL (CLamL e) = LamL (unembedL (e ProxyTopL))
unembedL (CAppL e1 e2) = AppL (unembedL e1) (unembedL e2)
unembedL CStarL = StarL

forgetL :: STLC l ts t -> STLC ts t
forgetL (VarL x) = Var (forgetL x)
forgetL (LamL b) = Lam (forgetL b)
forgetL (AppL f x) = App (forgetL f) (forgetL x)
forgetL StarL = Star

```

Uniform Solution The uniformity of the solution is most evident when the Untyped λ -Calculus (**ULC**) is considered. Here the levels not distilled out, the precisely the information that we already have: there are no types in the context, just the level ensuring that nothing out of scope is referenced. The **STLC** builds upon this solution by adding the type information, and instead of bundling these two pieces of information together, we have kept the level separate. Then, as you will see in our Contextual Embedding with Levels of the **LLC**, for a linear language with two contexts, we yet again add typing information separately instead of entangling it with the levels, which made them hard to extract.

```
data Index :: Nat -> Type where
  Top :: Index (S 1)
  Pop :: Index 1 -> Index (S 1)
```

```
data ULC :: Nat -> Type where
  Var  :: Index 1 -> ULC 1
  Lam  :: ULC (S 1) -> ULC 1
  App  :: ULC 1 -> ULC 1 -> ULC 1
  Star :: ULC 1
```

Levelled Up Isomorphism There still exists an isomorphism between the Contextual Embedding with Levels and the first-order embedding with levels. We have extended our Lean proof to show this for the *STLC* and *LLC* (which we shall present next). The proof extension is fairly trivial, with the proof structure remaining the same, just with some straightforward modifications for the different types and/or extra constructors.

5.7. Substructural Example: *LLC* Levelled Up

The Contextual Embedding with Levels of the *LLC* (Fig. 5.8) is constructed from the first-order embedding of the *LLC* following the same steps used for the *STLC*. First, the “level” type information is added. Terms and indices are both parameterised by levels, and levels are incremented by binders. Next, *ProxyTopL* is swapped in for the index type. It is just *ProxyTopL* from the *STLC* but with the top context elements exposed and enforced to go from $(J\ t)$ to N for a *ProxyTopL* for type t . The key gain of levels is clear in *ReifyIndex*, because this works in exactly the same way as the *ULC* and *STLC*, there are just more contexts. The first base case instance still gets triggered by the bind and use site levels matching, there are just now two contexts that are checked to be non-empty. For extra safety, the tops of these contexts are checked to be different because i should get used up and become $o(N)$. The second type class, the recursive case, is still triggered by the levels not matching, adding a *PopL* per difference.

This achieves an embedding of the *LLC* that enforces linearity and allows ergonomic use of host binders:

```
pair :: LLCctxL l ts ts (a :-* b :-* (b :* a))
pair = CLamL $ \x -> CLamL $ \y -> CPairL (CVarL y) (CVarL x)
```

5.8. Further Examples

Contextual Embeddings, especially once levelled up, are quite flexible in supporting different sorts of type systems. In this section, we shall consider a couple of variations including: supporting both linear and unlimited variables (this could be the basis of a mixed quantum and classical language); and numerically limiting how many times a variable can be used. Like the modal example of §5.4, a de Bruijn implementation of these examples must be done carefully, and requires domain specific knowledge: recall our use of the *UnderLock* type class to enforce correct use of the modality. Therefore, the focus of this section will be on sketching how the Contextual Embedding would need adjusted, assuming the existence of a sound de Bruijn implementation. We also plan to explore such implementations as future work.

Unlimited Variables To support a mixture of linear and unlimited variables, the following adjustments to the Contextual Embedding would need to be made. For this setting, we’d expect the guest types to have been extended. For example, *CtxElem* could have been extended with an unlimited case, perhaps: $U\ Ty$. To support this new context element, *Index* and *ProxyTop* will need extra constructors that allow unlimited variables to be used freely:

5. Contextual Embeddings

```

data ProxyTopL :: Nat -> CtxElem -> CtxElem
               -> Ty -> Type where
  ProxyTopL :: ProxyTopL (S l) (J t) N t

class ReifyIndexL l l' i o ci' co' t
  | l l' i o ci' t -> co' where
  reify :: ProxyTopL l i o t
        -> IndexL l' ci' co' t

instance {-# OVERLAPPING #-}
  ( zs ~ (zs' :/: i)
  , out ~ (zs' :/: o) )
=> ReifyIndexL l l i o zs out t where
  reify ProxyTopL = TopL

instance ( l' ~ (S l'') , zs ~ (zs' :/: z)
        , out ~ (out' :/: z)
        , ReifyIndexL l l'' i o zs' out' t )
=> ReifyIndexL l l' i o zs out t where
  reify i = PopL (reify i)

data LLCtxL :: Nat -> Ctx -> Ctx -> Ty -> Type where
  CVarL :: ReifyIndexL l l' i o ci' co' t
        => ProxyTopL l i o t -> LLCtxL l' ci' co' t
  CLamL :: (ProxyTopL (S l) (J a) N a
        -> LLCtxL (S l) (ci :/: J a) (co :/: N) b)
        -> LLCtxL l ci co (a :-* b)
  CAppL :: LLCtxL l ci ctx (a :-* b)
        -> LLCtxL l ctx co a -> LLCtxL l ci co b
  CStarL :: LLCtxL l ci ci Unit
  CPairL :: LLCtxL l ci ctx a
        -> LLCtxL l ctx co b
        -> LLCtxL l ci co (a :* b)
  CLetpL :: LLCtxL l ci ctx (a :* b)
        -> (ProxyTopL (S l) (J a) N a
        -> ProxyTopL (S (S l)) (J b) N b
        -> LLCtxL (S (S l))
        (ctx :/: J a :/: J b)
        (co :/: N :/: N ) t)
        -> LLCtxL l ci co t

```

Figure 5.8.: A Contextual Embedding of the **LLC** with Levels

```

TopU :: Index (S m) (ts :/: U t) (ts :/: U t) t
ProxyTopU :: ProxyTop (S m) (U t) (U t) t

```

And the base case of `ReifyIndex` will need to feature a pattern-match to support this:

```

reify ProxyTopL = TopL
reify ProxyTopU = TopU

```

Now that `ProxyTop` has multiple constructors, they must not be exported from end-user facing modules, otherwise exotic terms could be constructed.

Numerically Limited Variables This time, context elements may be annotated with a natural number indicating how many times they can be used: `L Nat Ty. Index` and `ProxyTop`'s top constructors can be adapted to decrease the natural number (instead of just switching from `J` to `N`) and ensure that no variable whose limit has reached zero is used.

```

TopL :: Index (S m) (ts :/: L (S n) t) (ts :/: L n t) t
ProxyTopL :: ProxyTop (S m) (L (S n) t) (L n t) t

```

5.9. Related Work

5.9.1. Higher-Order Abstract Syntax (**HOAS**) and Friends

Contextual Embeddings builds on a series of research starting with Higher-Order Abstract Syntax (**HOAS**) [G. P. Huet and Lang 1978]. This introduced the trick of reusing the host language's binders, and still stands up in its original setting where pattern-matching on syntax is not a concern.

Follow on research looks into using **HOAS** in other settings. **Despeyroux, Felty, et al.** allow it to be representable in theorem provers with “weakHOAS” (**ProxyTop** is inspired by this work), and **Chlipala** combines this with ideas from **Washburn and Weirich** for the current state-of-the-art.

We directly advance this line of research by addressing the outstanding problem of type-safe unembedding. The key choice that achieves this is the accessibility of the concrete context as a type parameter of guest terms and our truncated index type **ProxyTop**. This differs from **HOAS** where the guest context is inaccessible because it has been handed off to the host language. This is made possible by utilising advances in host languages, namely type-level programming features, which allow us to manage to contexts and perform index reification automatically.

We also bring this line of research into new domains by supporting modal and substructural type systems.

5.9.2. Embedding by Unembedding

Chapter 4 also presents a technique for embedding languages with binders. It presents a framework built upon **Atkey et al.**’s implementation of an isomorphism between de Bruijn and **HOAS** embedded terms. The main contribution is seeing the value of unembedding for embedding languages with “interesting” semantic such as those in bidirectional or incremental domains. Contextual Embeddings are complementary to this work. The interesting case studies can be reskinned to use a Contextual Embedding as a front-end, replacing the type-unsafe translation (justified by **Atkey**’s external proof) to their first-order semantics with a type-safe translation. Regarding performance, the Contextual Embedding’s unembedding also all happens at compile-time, as opposed to Embedding by Unembedding which has to happen at runtime. The biggest advantage of Contextual Embedding over the work of the previous chapter is its support for modal and substructural languages, which is listed as future work there.

5.9.3. Names for Free (**NFF**)

J.-P. Bernardy and Pouillard present a first-order embedding with an ergonomic weakHOAS interface that uses parametricity to ensure that no mistakes are made due to accidentally using the wrong de Bruijn index. This work is similar in that it also uses a type class mechanism to automatically insert the correct index, though the mechanism is different in that they use parameterised variables as ‘free names’ and search through the context until the provided name matches the one in the context, rather than using the difference in levels to derive the index as we do. This is a very neat idea, although it inherently relies on incoherent instances, which can be tricky to get right if you want to do other fancy things with the type class system, as we do with substructural languages. Their paper only presents an untyped language, and while it is trivial to extend this to a typed setting, we found it non-trivial to unembed the language. Since ultimately this is a first-order embedding, the notion of unembedding it might seem strange, but what we mean by this is transforming it into the canonical first order embedding. To do this requires use of **unsafeCoerce** with runtime checks, amounting to a similar type-unsafe technique like that introduced by **Atkey et al.** Thus our advance over this work again amounts to our solving of the problem of a type-safe unembedding and the support of modal and substructural languages.

5.9.4. Embedding a full Linear λ -Calculus (**LLC**) in Haskell

This work from **Polakow** is the only other work we have found that tackles the embedding of a non-structural language into a structural host. They embed the **LLC** into Haskell (without using its linear features). They present a **HOAS** embedding adorned with extra types to intrinsically enforce linearity

5. Contextual Embeddings

of guest terms. This work is similar in that they embed the same presentation of the **LLC** with two contexts [Grenrus 2019; Hodas 1995] and the way they enforce linearity. It also features type-level **Nats** which bear a superficial similarity to our ‘levels’, but are actually more similar to Names For Free, as they only use the **Nats** as a source for fresh names. In our experiments, we found that these **Nats** can be entirely replaced by NFF-style parameterised variables. Again, our contribution lies in the context of unembedding. When it comes to unembedding, this works suffers from the same problems as other **HOAS** embeddings, perhaps more acutely due to the linear setting. Unlike previous uses of unembedding, where **Atkey**’s proof can be appealed to to justify uses of **unsafeCoerce**, there exists no such proof in a linear setting.

5.9.5. Locally Nameless

The locally nameless representation of terms was introduced by Gordon [1994], and has been developed substantially for use in proof assistants by B. Aydemir et al. [2008]. The underlying mathematics of these embeddings has recently been developed by Pitts [2023]. See Charguéraud [2012] for a survey of the use of the technique in formalized metatheory.

The idea of the locally nameless embedding is to mix techniques: while bound variables are represented with de Bruijn Indices, free variables are represented by *name*. This may be a string, but it may also come from any with cofinite quantification (i.e. an infinite type where a fresh name distinct from a finite set can be drawn). As a result, they cannot avoid exotic terms: there is no apparent way for static typing to constrain bound variables to be in-bounds [Charguéraud 2012, §2.4, 3.3]. Moreover, locally nameless representations do not seem amenable to static typing, and require external typing judgements.

While locally nameless representations are an attractive technique for formalizing metatheory in a proof assistant, they do not appear to be a good interface for embedded programming.

5.9.6. Nominal Approaches

Nominal approaches [Pitts 2001] tackle names and binders with a method that is essentially a formalisation of the Barendregt’s variable convention [Barendregt and Barendsen 1984]. The idea is that bound variable names are always fresh, avoiding issues arising from name duplication. The basis of this formalisation is Nominal Logic. While these approaches have achieved great success in systems such as nominal Isabelle [Urban and Tasson 2005], it is unclear how to implement them in settings like Haskell or dependently-typed proof assistants.

5.9.7. First-Order Formalisation

There is a wealth of work on formalising languages with variable binding [G. Allais et al. 2017; M. Fiore and Szamozvancev 2022; Wood and Atkey 2022]. The current state-of-the-art greatly reduces the boilerplate involved with such embeddings, automating an impressive amount of work including correctness properties of generated syntactic operations. This research focusses on generating first-order embeddings, giving it a different and complementary focus to Contextual Embeddings. Since our technique provides a higher-order interface to first-order representations it is appealing to investigate integrating this line of research into our technique. Wood and Atkey’s work is particularly relevant as it focusses on languages with substructural type systems.

5.9.8. Unpublished Related Work

McBride’s Classy Hack and Further Developments A blog post from McBride sparked fruitful progress on the problem of type-safe unembedding. Both on the Agda mailing list [‘effectfully’ and G. ‘g. Allais 2018] and in the Syntactic Haskell package [Axelsson 2025].

The core idea in all these developments is very similar to ours: keep the context information at type level and use instance resolution (or take advantage of an Agda oddity in the case of ‘effectfully’) to allow users to use host binders by automatically injecting them into the correct de Bruijn representation of syntax.

Our work is a full realisation of this core idea, superseding these unpublished works with: implementations in multiple languages (Lean4, Agda, Haskell), unembedding, back-translation (“contextualisation”, which was not covered by these works), proof of correctness, and of course our extension to apply this technique in modal and substructural domains.

Chlipala’s Well-formedness This unpublished work [Chlipala 2011] focusses on avoiding the need to assert an axiom to safely unembed PHOAS [Chlipala 2008]. For unembedding to be safe, the PHOAS term must be well-formed. A well-formed PHOAS term is one that only uses variables introduced by embedded binders. In theory, if a PHOAS term is truly parametric in its variable type, the parametricity should ensure the well-formedness of the term. However, the proof of this is not currently representable in the logic of today’s theorem provers (even though we expect it to be true). So, in this work, instead of asserting this well-formedness for all terms, Chlipala asserts well-formedness individually for terms of interest. Requiring a proof of well-formedness for *dbify* (his version of *unembed*) indeed ensures the safety of the unembedding.

Contextual Embeddings amount to baking this notion of well-formedness (expanded to also support open terms) into our higher-order representation of syntax. Thus direct parallels can be found between this work and ours. For example, in *dbify*, variables are instantiated as de Bruijn levels. This means that it works with the same logic as our *unembed*. The way we have constrained our higher-order syntax with *ReifyIndex* also ensures that contextually embedded terms are well-formed as specified by *wf* (his notion of well-formedness). We think that this agreement validates our approach, and again our extension to modal and substructural domains sets us apart.

5.10. Conclusion and Future Work

In conclusion, we present Contextual Embeddings, a technique for embedding languages with binders that is as ergonomic as HOAS and advances the state-of-the-art in two key ways: support for type-safe unembedding, something that has long evaded prior work; and accommodation of languages with modern, non-structural type systems such as linear or modal languages.

Going forwards, we plan to build on this work as follows.

Benchmarks We’d love to further put our technique through its paces with benchmarks like the POPLmark Challenge [B. E. Aydemir et al. 2005]. We expect the technique to improve the experience of stating theorems and writing test cases, though the core difficulty of the challenge lies in inductive proofs about binding, which would still be carried out on the first-order representation. If this benchmark is fruitful in revealing properties of our technique, we can continue this exploration with the POPLmark Reloaded Challenge [Abel, G. Allais, et al. 2019].

We also think that Abel and J.-P. Bernardy’s unified treatment of modalities will be a good benchmark for exploring how well Contextual Embeddings support non-structural type systems.

5. Contextual Embeddings

More ergonomic Due to the prosaic nature of the `unembed` function, it is natural to think about automating its definition. Perhaps, in Haskell, this can take the form of an `Unembeddable` type class paired with a `DeriveUnembeddable` feature akin to `DeriveFunctor`, or perhaps metaprogramming with `TemplateHaskell`.

Quantum Application Quantum languages are necessarily linear because of the no-cloning theorem. This makes them an attractive target for Contextual Embeddings.

Outside Embedded Languages One could consider our use of the type classes on a more general level: the use of singleton types to guide metaprogramming. We do this with `ProxyTop` to guide reification of the correct de Bruijn index, but this could be applied in areas outside embedded languages, and we are excited to search for potential applications.

6. Conclusion

This thesis focussed on **bringing the benefits of the functional programming vernacular to the complex languages of tomorrow via embedding**. In particular, it brought two key functional idioms to complex languages that are embedded.

Monadic Composition. We brought monadic composition and its expressive `do` notation syntax to the domain of bidirectional programming. This was achieved using Partial Monadic Profunctors (**PMPs**), and proved so successful across the different application domains of bidirectional programming that we propose it as a new paradigm for bidirectional programming. The key contributions presented are the advances in bigenerators including their generalisation for application in logic programming, and their combination with free generators to create reflective generators.

Encapsulating bidirectional transformations as **PMPs** allowed us to flip the script from finding the correct **PMP** for a particular application to exploring what applications could be supported with different **PMPs**. This led to our generalised bigenerators that can solve logic programming puzzles, a new application domain for bidirectional transformations.

The presented work on reflective generators epitomises the goal of this thesis. It introduces a domain specific language for specifying generators that enjoys the expressive syntax of monadic composition and leverages the complex feature of bidirectional programming to increase the power of random data generators. This paper won the distinguished paper award at ICFP23, which highlights the importance of bringing the functional programming vernacular to more domains.

Binders. We tackled the problem of embedding languages with complex semantics and binders by harnessing the power of unembedding. Binders were targeted because their embedding unlocks the functional idiom of higher-order functions. The value of this idiom also lies in compositionality: the ability to take functions as arguments is the important “glue” that facilitates modular code. Higher-order functions, in particular the modularity they grant, are highlighted by **Hughes [1990]** way back in 1990 contributing towards making functional programming matter.

The contributions presented in this thesis increase the domains that can enjoy the benefits of embedded binders. They include a framework for embedding binders into languages in complex domains such as bidirectional and incremental programming, and an expanded technique that increases the type safety of embedding binders by removing reliance on external proofs and supporting more exacting substructural type systems.

This thesis is just the start of bringing the functional vernacular to complex languages via embedding. There is still more to be done: there are more functional idioms to embed, and there are more complex domains to benefit from them. Hopefully, this thesis demonstrates the benefits of this strain of research, and can function as an archetype to support future developments.

Domains that spring to mind as timely targets include quantum computing and cryptography:

- Quantum: with its support for substructural type systems, Contextual Embedding (Chapter 5) is ready to support the embedding of a quantum language with binders. This will be an important step in raising the level of abstraction that quantum programming languages work at from

6. Conclusion

low-level circuit specifications to languages more like those that we use today for classical programming [Cobb et al. 2022].

- Cryptography: discussions have already started about using the increased power of reflective generators (Chapter 3) in the domain of differential privacy [Matsuda, Eramo, et al. 2025]. Differential privacy approaches the problem of gathering statistics over private data without compromising individual privacy. Existing algorithms are monolithic and tend to focus on unstructured data. The idea is that reflective generators, with their monadic compositionality and focus on structured data will aid the creation of modular algorithms. Further, there are ideas to use reflectives to fuzz past cryptography checks, or more broadly apply the bidirectional paradigm to creating parser-formatter pairs for protocol messages.

To conclude, I invite you to consider what domains you think could benefit from the functional vernacular, and whether embedding could help you achieve that.

Bibliography

- ‘effectfully’ and Guillaume ‘gallais’ Allais. 2018. *Typed Jigger in vanilla Agda*. Retrieved July 4, 2025 from <https://lists.chalmers.se/pipermail/agda/2018/010371.html>.
- Andreas Abel, Guillaume Allais, Aliya Hameer, Brigitte Pientka, Alberto Momigliano, Steven Schäfer, and Kathrin Stark. 2019. “POPLMark reloaded: Mechanizing proofs by logical relations.” *Journal of Functional Programming*, 29, e19. DOI: [10.1017/S0956796819000170](https://doi.org/10.1017/S0956796819000170).
- Andreas Abel and Jean-Philippe Bernardy. Aug. 2020. “A unified view of modalities in type systems.” *Proc. ACM Program. Lang.*, 4, ICFP, (Aug. 2020). DOI: [10.1145/3408972](https://doi.org/10.1145/3408972).
- “Reflections on Monadic Lenses.” *A List of Successes That Can Change the World: Essays Dedicated to Philip Wadler on the Occasion of His 60th Birthday*. Springer International Publishing, Cham, 1–31. ISBN: 978-3-319-30936-1. DOI: [10.1007/978-3-319-30936-1_1](https://doi.org/10.1007/978-3-319-30936-1_1).
- Samson Abramsky. 2005. “A structural approach to reversible computation.” *Theor. Comput. Sci.*, 347, 3, 441–464. DOI: [10.1016/j.tcs.2005.07.002](https://doi.org/10.1016/j.tcs.2005.07.002).
- Umut A. Acar, Guy E. Blelloch, and Robert Harper. 2006. “Adaptive functional programming.” *ACM Trans. Program. Lang. Syst.*, 28, 6, 990–1034. DOI: [10.1145/1186632.1186634](https://doi.org/10.1145/1186632.1186634).
- Umut A. Acar and Yan Chen. 2013. “Streaming Big Data with Self-Adjusting Computation.” In: *Proceedings of the 2013 Workshop on Data Driven Functional Programming (DDFP ’13)*. Association for Computing Machinery, Rome, Italy, 15–18. ISBN: 9781450318716. DOI: [10.1145/2429376.2429382](https://doi.org/10.1145/2429376.2429382).
- Artem Alimarine, Sjaak Smetsers, Arjen Weelden, Marko Eekelen, and Rinus Plasmeijer. 2006. “There and back again: arrows for invertible programming.” In: *In Proceedings of the 2005 ACM SIGPLAN workshop on Haskell*. ACM Press, 86–97.
- Guillaume Allais, James Chapman, Conor McBride, and James McKinna. 2017. “Type-and-scope safe programs and their proofs.” In: *Proceedings of the 6th ACM SIGPLAN Conference on Certified Programs and Proofs (CPP 2017)*. Association for Computing Machinery, Paris, France, 195–207. ISBN: 9781450347051. DOI: [10.1145/3018610.3018613](https://doi.org/10.1145/3018610.3018613).
- Thorsten Altenkirch, James Chapman, and Tarmo Uustalu. 2010. “Monads need not be endofunctors.” In: *International Conference on Foundations of Software Science and Computational Structures*. Springer, 297–311.
- Cornelius Aschermann, Tommaso Frassetto, Thorsten Holz, Patrick Jauernig, Ahmad-Reza Sadeghi, and Daniel Teuchert. 2019. “NAUTILUS: Fishing for Deep Bugs with Grammars.” en. In: *Proceedings 2019 Network and Distributed System Security Symposium*. Internet Society, San Diego, CA. ISBN: 978-1-891562-55-6. DOI: [10.14722/ndss.2019.23412](https://doi.org/10.14722/ndss.2019.23412).
- Robert Atkey. 2009. “Syntax for Free: Representing Syntax with Binding Using Parametricity.” In: *Typed Lambda Calculi and Applications, 9th International Conference, TLCA 2009, Brasilia, Brazil, July 1-3, 2009. Proceedings*, 35–49. DOI: [10.1007/978-3-642-02273-9_5](https://doi.org/10.1007/978-3-642-02273-9_5).
- Robert Atkey, Sam Lindley, and Jeremy Yallop. 2009. “Unembedding domain-specific languages.” In: *Proceedings of the 2nd ACM SIGPLAN Symposium on Haskell, Haskell 2009, Edinburgh, Scotland, UK, 3 September 2009*, 37–48. DOI: [10.1145/1596638.1596644](https://doi.org/10.1145/1596638.1596644).
- Arnon Avron, Furio Honsell, and Ian Mason. 1987. *Using Typed Lambda Calculus to Implement Formal Systems on a Machine*. ECS-LFCS-87-31. Laboratory for Foundations of Computer Science, University of Edinburgh. <https://www.lfcs.inf.ed.ac.uk/reports/87/ECS-LFCS-87-31/>.

- Emil Axelsson. 2025. *Syntactic*. Retrieved July 4, 2025 from <https://hackage.haskell.org/package/syntactic-3.8.4/docs/Language-Syntactic-Functional-WellScoped.html>.
- Brian Aydemir, Arthur Charguéraud, Benjamin C. Pierce, Randy Pollack, and Stephanie Weirich. 2008. “Engineering formal metatheory.” In: *Proceedings of the 35th annual ACM SIGPLAN-SIGACT symposium on Principles of programming languages*. Association for Computing Machinery, 3–15. DOI: [10.1145/1328438.1328443](https://doi.org/10.1145/1328438.1328443).
- Brian E. Aydemir et al.. 2005. “Mechanized metatheory for the masses: the PoplMark challenge.” In: *Proceedings of the 18th International Conference on Theorem Proving in Higher Order Logics (TPHOLs’05)*. Springer-Verlag, Oxford, UK, 50–65. ISBN: 3540283722. DOI: [10.1007/11541868_4](https://doi.org/10.1007/11541868_4).
- François Bancilhon and Nicolas Spyrtos. 1981. “Update Semantics of Relational Views.” *ACM Trans. Database Syst.*, 6, 4, 557–575.
- Davi M. J. Barbosa, Julien Cretin, Nate Foster, Michael Greenberg, and Benjamin C. Pierce. 2010. “Matching lenses: alignment and view update.” In: *International Conference on Functional Programming (ICFP)*. ACM, 193–204. DOI: [10.1145/1863543.1863572](https://doi.org/10.1145/1863543.1863572).
- Henk (Hendrik) Barendregt and E. Barendsen. Jan. 1984. “Introduction to lambda calculus.” *Nieuw archief voor wisenkunde*, 4, (Jan. 1984), 337–372.
- Jean-Philippe Bernardy, Mathieu Boespflug, Ryan R. Newton, Simon Peyton Jones, and Arnaud Spiwack. 2018. “Linear Haskell: practical linearity in a higher-order polymorphic language.” *Proc. ACM Program. Lang.*, 2, POPL, 5:1–5:29. DOI: [10.1145/3158093](https://doi.org/10.1145/3158093).
- Jean-Philippe Bernardy and Nicolas Pouillard. 2013. “Names for free: polymorphic views of names and binders.” In: *Proceedings of the 2013 ACM SIGPLAN Symposium on Haskell (Haskell ’13)*. Association for Computing Machinery, Boston, Massachusetts, USA, 13–24. ISBN: 9781450323833. DOI: [10.1145/2503778.2503780](https://doi.org/10.1145/2503778.2503780).
- Eli Bingham, Fritz Obermeyer, Yerdos Ordabayev, and Du Phan. 2021. *Tensor Partial Evaluation*. HOPE 2021: The 8th ACM SIGPLAN Workshop on Higher-Order Programming with Effects. Extend Abstract Available via: <https://icfp21.sigplan.org/details/hope-2021-papers/4/Tensor-Partial-Evaluation>. (2021).
- Aaron Bohannon, J. Nathan Foster, Benjamin C. Pierce, Alexandre Pilkiewicz, and Alan Schmitt. 2008. “Boomerang: resourceful lenses for string data.” In: *Symposium on Principles of Programming Languages (POPL)*. ACM, 407–419.
- Aaron Bohannon, Benjamin C. Pierce, and Jeffrey A. Vaughan. 2006. “Relational lenses: a language for updatable views.” In: *Symposium on Principles of Database Systems (PODS)*. ACM, 338–347.
- Gert-Jan Bottu, Georgios Karachalias, Tom Schrijvers, Bruno C. d. S. Oliveira, and Philip Wadler. 2017. “Quantified Class Constraints.” In: *International Symposium on Haskell (Haskell)*. ACM, Oxford, UK, 148–161. ISBN: 978-1-4503-5182-9. DOI: [10.1145/3122955.3122967](https://doi.org/10.1145/3122955.3122967).
- Chandrasekhar Boyapati, Sarfraz Khurshid, and Darko Marinov. 2002. “Korat: Automated Testing Based on Java Predicates.” In: *International Symposium on Software Testing and Analysis (ISSTA)*. ACM, Roma, Italy, 123–133. ISBN: 1-58113-562-9. DOI: [10.1145/566172.566191](https://doi.org/10.1145/566172.566191).
- Edwin C. Brady. 2011. “IDRIS —: systems programming meets full dependent types.” In: *Proceedings of the 5th ACM Workshop on Programming Languages Meets Program Verification (PLPV ’11)*. Association for Computing Machinery, Austin, Texas, USA, 43–54. ISBN: 9781450304870. DOI: [10.1145/1929529.1929536](https://doi.org/10.1145/1929529.1929536).
- Rudy Matela Braquehais. Oct. 2017. “Tools for Discovery, Refinement and Generalization of Functional Properties by Enumerative Testing.” Publisher: University of York. (Oct. 2017). <http://etheses.whiterose.ac.uk/19178/>.
- Bernd Braßel, Sebastian Fischer, Michael Hanus, and Fabian Reck. 2010. “Transforming functional logic programs into monadic functional programs.” In: *International Workshop on Functional and Constraint Logic Programming*. Springer, 30–47.

- Yufei Cai, Paolo G. Giarrusso, Tillmann Rendel, and Klaus Ostermann. 2014. “A theory of changes for higher-order languages: incrementalizing λ -calculi by static differentiation.” In: *ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI '14, Edinburgh, United Kingdom - June 09 - 11, 2014*, 145–155. DOI: [10.1145/2594291.2594304](https://doi.org/10.1145/2594291.2594304).
- Jacques Carette, Oleg Kiselyov, and Chung-chieh Shan. 2009. “Finally tagless, partially evaluated: Tagless staged interpreters for simpler typed languages.” *J. Funct. Program.*, 19, 5, 509–543. DOI: [10.1017/S0956796809007205](https://doi.org/10.1017/S0956796809007205).
- Arthur Charguéraud. Oct. 2012. “The Locally Nameless Representation.” *Journal of Automated Reasoning - JAR*, 49, (Oct. 2012), 1–46. DOI: [10.1007/s10817-011-9225-2](https://doi.org/10.1007/s10817-011-9225-2).
- Sarah E. Chasins, Elena L. Glassman, and Joshua Sunshine. Aug. 2021. “PL and HCI: better together.” *Communications of the ACM*, 64, 8, (Aug. 2021), 98–106. DOI: [10.1145/3469279](https://doi.org/10.1145/3469279).
- Chao-Hong Chen and Amr Sabry. 2021. “A computational interpretation of compact closed categories: reversible programming with negative and fractional types.” *Proc. ACM Program. Lang.*, 5, POPL, 1–29. DOI: [10.1145/3434290](https://doi.org/10.1145/3434290).
- Yan Chen, Jana Dunfield, and Umut A. Acar. 2012. “Type-directed automatic incrementalization.” In: *ACM SIGPLAN Conference on Programming Language Design and Implementation, PLDI '12, Beijing, China - June 11 - 16, 2012*, 299–310. DOI: [10.1145/2254064.2254100](https://doi.org/10.1145/2254064.2254100).
- Yan Chen, Jana Dunfield, Matthew A. Hammer, and Umut A. Acar. 2014a. “Implicit self-adjusting computation for purely functional programs.” *Journal of Functional Programming*, 24, 1, 56–112. DOI: [10.1017/S0956796814000033](https://doi.org/10.1017/S0956796814000033).
- Yan Chen, Jana Dunfield, Matthew A. Hammer, and Umut A. Acar. 2014b. “Implicit self-adjusting computation for purely functional programs.” *J. Funct. Program.*, 24, 1, 56–112. DOI: [10.1017/S0956796814000033](https://doi.org/10.1017/S0956796814000033).
- Patrick Chilton. 2018. *codec library*. <https://hackage.haskell.org/package/codec>. (2018).
- Wei-Ngan Chin. 1993. “Towards an Automated Tupling Strategy.” In: *PEPM*, 119–132.
- Adam Chlipala. 2008. “Parametric higher-order abstract syntax for mechanized semantics.” In: *ICFP*. Ed. by James Hook and Peter Thiemann. ACM, 143–156. ISBN: 978-1-59593-919-7. DOI: [10.1145/1411204.1411226](https://doi.org/10.1145/1411204.1411226).
- Adam Chlipala. 2011. *Intensional*. Retrieved July 7, 2025 from <http://adam.chlipala.net/cpdt/html/Intensional.html>.
- Alonzo Church. 1940. “A Formulation of the Simple Theory of Types.” *J. Symb. Log.*, 5, 2, 56–68. DOI: [10.2307/2266170](https://doi.org/10.2307/2266170).
- Koen Claessen, Jonas Duregård, and Michal H. Palka. 2015. “Generating constrained random data with uniform distribution.” *Journal of Functional Programming*, 25, e8.
- Koen Claessen and John Hughes. 2000. “QuickCheck: a lightweight tool for random testing of Haskell programs.” In: *Proceedings of the Fifth ACM SIGPLAN International Conference on Functional Programming (ICFP '00), Montreal, Canada, September 18-21, 2000*. Ed. by Martin Odersky and Philip Wadler. ACM, Montreal, Canada, 268–279. DOI: [10.1145/351240.351266](https://doi.org/10.1145/351240.351266).
- Ranald Clouston. 2017. “Fitch-Style Modal Lambda Calculi.” *CoRR*, abs/1710.08326. <http://arxiv.org/abs/1710.08326> arXiv: 1710.08326.
- Adrian Cobb, Jean-Guy Schneider, and Kevin Lee. 2022. “Towards Higher-Level Abstractions for Quantum Computing.” In: *Proceedings of the 2022 Australasian Computer Science Week (ACSW '22)*. Association for Computing Machinery, Brisbane, Australia, 115–124. ISBN: 9781450396066. DOI: [10.1145/3511616.3513106](https://doi.org/10.1145/3511616.3513106).
- Michael Coblenz, Gauri Kambhatla, Paulette Koronkevich, Jenna L. Wise, Celeste Barnaby, Joshua Sunshine, Jonathan Aldrich, and Brad A. Myers. July 2021. “PLIERS: A Process that Integrates User-Centered Methods into Programming Language Design.” *ACM Transactions on Computer-Human Interaction*, 28, 4, (July 2021), 28:1–28:53. DOI: [10.1145/3452379](https://doi.org/10.1145/3452379).

- Thierry Coquand and Gérard Huet. 1988. “The calculus of constructions.” *Information and Computation*, 76, 2, 95–120. DOI: [https://doi.org/10.1016/0890-5401\(88\)90005-3](https://doi.org/10.1016/0890-5401(88)90005-3).
- Krzysztof Czarnecki, J. Nathan Foster, Zhenjiang Hu, Ralf Lämmel, Andy Schürr, and James F. Terwilliger. 2009. “Bidirectional Transformations: A Cross-Discipline Perspective.” In: *International Conference on Theory and Practice of Model Transformations (ICMT)*. Springer Berlin Heidelberg, 260–283. ISBN: 978-3-642-02408-5. DOI: [10.1007/978-3-642-02408-5_19](https://doi.org/10.1007/978-3-642-02408-5_19).
- Rowan Davies and Frank Pfenning. 2001. “A modal analysis of staged computation.” *Journal of the ACM*, 48, 3, 555–604. DOI: [10.1145/382780.382785](https://doi.org/10.1145/382780.382785).
- N.G. de Bruijn. 1972. “Lambda calculus notation with nameless dummies, a tool for automatic formula manipulation, with application to the Church-Rosser theorem.” *Indagationes Mathematicae (Proceedings)*, 75, 5, 381–392. DOI: [10.1016/1385-7258\(72\)90034-0](https://doi.org/10.1016/1385-7258(72)90034-0).
- Joëlle Despeyroux, Amy Felty, and André Hirschowitz. 1995. “Higher-order abstract syntax in Coq.” In: *Typed Lambda Calculi and Applications* (Lecture Notes in Computer Science). Ed. by Mariangiola Dezani-Ciancaglini and Gordon Plotkin. Vol. 902. Springer Berlin Heidelberg, Berlin, Heidelberg, 124–138. DOI: [10.1007/BFb0014049](https://doi.org/10.1007/BFb0014049).
- Joëlle Despeyroux and André Hirschowitz. 1994. “Higher-Order Abstract Syntax with induction in Coq.” In: *Logic Programming and Automated Reasoning* (Lecture Notes in Computer Science). Ed. by Frank Pfenning. Vol. 822. Springer Berlin Heidelberg, Berlin, Heidelberg, 159–173. DOI: [10.1007/3-540-58216-9_36](https://doi.org/10.1007/3-540-58216-9_36).
- Joëlle Despeyroux, Frank Pfenning, and Carsten Schürmann. 1997. “Primitive recursion for higher-order abstract syntax.” In: *Typed Lambda Calculi and Applications* (Lecture Notes in Computer Science). Ed. by Philippe Groote and J. Roger Hindley. Vol. 1210. Springer Berlin Heidelberg, 147–163. DOI: [10.1007/3-540-62688-3_34](https://doi.org/10.1007/3-540-62688-3_34).
- Zac Hatfield Dodds. 2022. current maintainer of Hypothesis (<https://github.com/HypothesisWorks/hypothesis>). Personal communication. (2022).
- Stephen Dolan and Mindy Preston. 2017. “Testing with crowbar.” In: *OCaml Workshop*. https://github.com/ocaml/ocaml.org-media/blob/master/meetings/ocaml/2017/extended-abstract__2017__stephen-dolan_mindy-preston__testing-with-crowbar.pdf.
- Jonas Duregård, Patrik Jansson, and Meng Wang. Sept. 2012. “Feat: functional enumeration of algebraic types.” *ACM SIGPLAN Notices*, 47, 12, (Sept. 2012), 61–72. DOI: [10.1145/2430532.2364515](https://doi.org/10.1145/2430532.2364515).
- Thomas Ehrhard and Laurent Regnier. 2003. “The differential lambda-calculus.” *Theor. Comput. Sci.*, 309, 1-3, 1–41. DOI: [10.1016/S0304-3975\(03\)00392-X](https://doi.org/10.1016/S0304-3975(03)00392-X).
- Conal Elliott. Aug. 2017. “Compiling to Categories.” *Proc. ACM Program. Lang.*, 1, ICFP, (Aug. 2017). DOI: [10.1145/3110271](https://doi.org/10.1145/3110271).
- Leonidas Fegaras. 2010. “Propagating updates through XML views using lineage tracing.” In: *ICDE*. Ed. by Feifei Li et al. IEEE, 309–320. ISBN: 978-1-4244-5444-0.
- Leonidas Fegaras and Tim Sheard. 1996. “Revisiting Catamorphisms over Datatypes with Embedded Functions (or, Programs from Outer Space).” In: *Conference Record of POPL’96: The 23rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, Papers Presented at the Symposium, St. Petersburg Beach, Florida, USA, January 21-24, 1996*. Ed. by Hans-Juergen Boehm and Guy L. Steele Jr. ACM Press, 284–294. ISBN: 0-89791-769-3. DOI: [10.1145/237721.237792](https://doi.org/10.1145/237721.237792).
- Francisco Ferreira and Brigitte Pientka. 2017. “Programs Using Syntax with First-Class Binders.” In: *Programming Languages and Systems - 26th European Symposium on Programming, ESOP 2017, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2017, Uppsala, Sweden, April 22-29, 2017, Proceedings*, 504–529. DOI: [10.1007/978-3-662-54434-1_19](https://doi.org/10.1007/978-3-662-54434-1_19).
- Andrea Fioraldi, Dominik Maier, Heiko Eißfeldt, and Marc Heuse. 2020. “{AFL++} : Combining Incremental Steps of Fuzzing Research.” en. In: Retrieved Dec. 1, 2022 from <https://www.usenix.org/conference/woot20/presentation/fioraldi>.

- Marcelo Fiore and Dmitriy Szamozvancev. Jan. 2022. “Formal metatheory of second-order abstract syntax.” *Proc. ACM Program. Lang.*, 6, POPL, (Jan. 2022). DOI: [10.1145/3498715](https://doi.org/10.1145/3498715).
- Marcelo P. Fiore and Chung-Kil Hur. 2010. “Second-Order Equational Logic (Extended Abstract).” In: *Computer Science Logic, 24th International Workshop, CSL 2010, 19th Annual Conference of the EACSL, Brno, Czech Republic, August 23-27, 2010. Proceedings*, 320–335. DOI: [10.1007/978-3-642-15205-4_26](https://doi.org/10.1007/978-3-642-15205-4_26).
- Marcelo P. Fiore, Gordon D. Plotkin, and Daniele Turi. 1999. “Abstract Syntax and Variable Binding.” In: *14th Annual IEEE Symposium on Logic in Computer Science, Trento, Italy, July 2-5, 1999*, 193–202. DOI: [10.1109/LICS.1999.782615](https://doi.org/10.1109/LICS.1999.782615).
- Cormac Flanagan, Amr Sabry, Bruce F. Duba, and Matthias Felleisen. 1993. “The Essence of Compiling with Continuations.” In: *Proceedings of the ACM SIGPLAN’93 Conference on Programming Language Design and Implementation (PLDI), Albuquerque, New Mexico, USA, June 23-25, 1993*, 237–247. DOI: [10.1145/155090.155113](https://doi.org/10.1145/155090.155113).
- J. Nathan Foster, Michael B. Greenwald, Jonathan T. Moore, Benjamin C. Pierce, and Alan Schmitt. 2005. “Combinators for bi-directional tree transformations: a linguistic approach to the view update problem.” In: *Proceedings of the 32nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2005, Long Beach, California, USA, January 12-14, 2005*, 233–246. DOI: [10.1145/1040305.1040325](https://doi.org/10.1145/1040305.1040325).
- J. Nathan Foster, Michael B. Greenwald, Jonathan T. Moore, Benjamin C. Pierce, and Alan Schmitt. 2007. “Combinators for bidirectional tree transformations: A linguistic approach to the view-update problem.” *ACM Trans. Program. Lang. Syst.*, 29, 3. DOI: [10.1145/1232420.1232424](https://doi.org/10.1145/1232420.1232424).
- John Nathan Foster. 2009. “Bidirectional programming languages.” English. Ph.D. University of Pennsylvania, United States – Pennsylvania. Retrieved Nov. 22, 2022 from <https://www.proquest.com/docview/304986072/abstract/11884B3FBDD4DCFPQ/1>. ISBN: 9781109710137.
- Nate Foster, Kazutaka Matsuda, and Janis Voigtländer. 2010. “Three Complementary Approaches to Bidirectional Programming.” In: *SSGIP*, 1–46.
- Peng Fu, Kohei Kishida, and Peter Selinger. 2020. “Linear Dependent Type Theory for Quantum Programming Languages: Extended Abstract.” In: *Proceedings of the 35th Annual ACM/IEEE Symposium on Logic in Computer Science (LICS ’20)*. Association for Computing Machinery, Saarbrücken, Germany, 440–453. ISBN: 9781450371049. DOI: [10.1145/3373718.3394765](https://doi.org/10.1145/3373718.3394765).
- Paolo G. Giarrusso, Yann Régis-Gianas, and Philipp Schuster. 2019. “Incremental λ -Calculus in Cache-Transfer Style - Static Memoization by Program Transformation.” In: *Programming Languages and Systems - 28th European Symposium on Programming, ESOP 2019, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2019, Prague, Czech Republic, April 6-11, 2019, Proceedings*, 553–580. DOI: [10.1007/978-3-030-17184-1_20](https://doi.org/10.1007/978-3-030-17184-1_20).
- Jeremy Gibbons and Nicolas Wu. Aug. 2014. “Folding Domain-Specific Languages: Deep and Shallow Embeddings (Functional Pearl).” *SIGPLAN Not.*, 49, 9, (Aug. 2014), 339–347. DOI: [10.1145/2692915.2628138](https://doi.org/10.1145/2692915.2628138).
- Andrew Gill, John Launchbury, and Simon L. Peyton Jones. 1993. “A Short Cut to Deforestation.” In: *Proceedings of the conference on Functional programming languages and computer architecture, FPCA 1993, Copenhagen, Denmark, June 9-11, 1993*, 223–232. DOI: [10.1145/165180.165214](https://doi.org/10.1145/165180.165214).
- Jean-Yves Girard. Jan. 1986. “The system F of variable types, fifteen years later.” en. *Theoretical Computer Science*, 45, (Jan. 1986), 159–192. DOI: [10.1016/0304-3975\(86\)90044-7](https://doi.org/10.1016/0304-3975(86)90044-7).
- Patrice Godefroid, Adam Kiezun, and Michael Y. Levin. June 2008. “Grammar-based whitebox fuzzing.” In: *Proceedings of the 29th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI ’08)*. Association for Computing Machinery, New York, NY, USA, (June 2008), 206–215. ISBN: 978-1-59593-860-2. DOI: [10.1145/1375581.1375607](https://doi.org/10.1145/1375581.1375607).

- Patrice Godefroid, Hila Peleg, and Rishabh Singh. 2017. “Learn&fuzz: Machine learning for input fuzzing.” In: *2017 32nd IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 50–59. <https://dl.acm.org/doi/10.5555/3155562.3155573>.
- Harrison Goldstein, Samantha Frohlich, Meng Wang, and Benjamin C. Pierce. Aug. 2023. “Reflecting on Random Generation.” *Proc. ACM Program. Lang.*, 7, ICFP, (Aug. 2023). DOI: [10.1145/3607842](https://doi.org/10.1145/3607842).
- Harrison Goldstein and Benjamin C. Pierce. Oct. 2022. “Parsing Randomness.” *Proceedings of the ACM on Programming Languages*, 6, OOPSLA2, (Oct. 2022), 128:89–128:113. DOI: [10.1145/3563291](https://doi.org/10.1145/3563291).
- Andrew D. Gordon. 1994. “A mechanisation of name-carrying syntax up to alpha-conversion.” In: *Higher Order Logic Theorem Proving and Its Applications* (Lecture Notes in Computer Science). Ed. by Jeffrey J. Joyce and Carl-Johan H. Seger. Vol. 780. Springer Berlin Heidelberg, 413–425. DOI: [10.1007/3-540-57826-9_152](https://doi.org/10.1007/3-540-57826-9_152).
- Andrew D. Gordon, Thomas A. Henzinger, Aditya V. Nori, and Sriram K. Rajamani. May 2014. “Probabilistic programming.” In: *Future of Software Engineering Proceedings* (FOSE 2014). Association for Computing Machinery, New York, NY, USA, (May 2014), 167–181. ISBN: 978-1-4503-2865-4. DOI: [10.1145/2593882.2593900](https://doi.org/10.1145/2593882.2593900).
- Oleg Grenrus. 2019. *Typed tagless-final interpreters for Linear Lambda Calculus de Bruijn indices*. Retrieved July 3, 2025 from <https://okmij.org/ftp/tagless-final/course/LinearLC.hs>.
- Oleg Grenrus. 2025. *PHOAS to de Bruijn conversion*. Retrieved June 27, 2025 from <https://oleg.fi/gists/posts/2025-02-13-phoas-to-db.html>.
- Louis-Julien Guillemette and Stefan Monnier. 2007. “A type-preserving closure conversion in haskell.” In: *Proceedings of the ACM SIGPLAN Workshop on Haskell Workshop* (Haskell ’07). Association for Computing Machinery, Freiburg, Germany, 83–92. ISBN: 9781595936745. DOI: [10.1145/1291201.1291212](https://doi.org/10.1145/1291201.1291212).
- Makoto Hamana. 2018. “Polymorphic Rewrite Rules: Confluence, Type Inference, and Instance Validation.” In: *Functional and Logic Programming - 14th International Symposium, FLOPS 2018, Nagoya, Japan, May 9-11, 2018, Proceedings*, 99–115. DOI: [10.1007/978-3-319-90686-7_7](https://doi.org/10.1007/978-3-319-90686-7_7).
- Makoto Hamana. 2019. “How to prove decidability of equational theories with second-order computation analyser SOL.” *J. Funct. Program.*, 29, e20. DOI: [10.1017/S0956796819000157](https://doi.org/10.1017/S0956796819000157).
- Matthew A. Hammer, Umut A. Acar, and Yan Chen. 2009. “CEAL: A C-Based Language for Self-Adjusting Computation.” In: *Proceedings of the 30th ACM SIGPLAN Conference on Programming Language Design and Implementation* (PLDI ’09). Association for Computing Machinery, Dublin, Ireland, 25–37. ISBN: 9781605583921. DOI: [10.1145/1542476.1542480](https://doi.org/10.1145/1542476.1542480).
- M. Hanus, H. Kuchen, and J.J. Moreno-Navarro. 1995. “Curry: A Truly Functional Logic Language.” In: *Proc. ILPS’95 Workshop on Visions for the Future of Logic Programming*, 95–107.
- Robert Harper, Gordon D. Plotkin, and Furio Honsell. 1993. “A framework for defining logics.” *Journal of the ACM*, 40, 1, 143–184.
- Stephen J. Hegner. 1990. “Foundations of Canonical Update Support for Closed Database Views.” In: *ICDT* (Lecture Notes in Computer Science). Ed. by Serge Abiteboul and Paris C. Kanellakis. Vol. 470. Springer, 422–436. ISBN: 3-540-53507-1.
- Stephen J. Hegner. 2004. “An Order-Based Theory of Updates for Closed Database Views.” *Ann. Math. Artif. Intell.*, 40, 1-2, 63–125.
- Joshua Seth Hodas. 1995. “Logic programming in intuitionistic linear logic: theory, design, and implementation.” Ph.D. Dissertation. USA. UMI Order No. GAX94-27546.
- Christian Holler, Kim Herzig, and Andreas Zeller. Aug. 2012. “Fuzzing with code fragments.” In: *Proceedings of the 21st USENIX conference on Security symposium* (Security’12). USENIX Association, USA, (Aug. 2012), 38. Retrieved Feb. 26, 2023 from.

- Zhenjiang Hu, John Hughes, and Meng Wang. July 2015. “How functional programming mattered.” *National Science Review*, 2, 3, (July 2015), 349–370. eprint: <https://academic.oup.com/nsr/article-pdf/2/3/349/31566307/nwv042.pdf>. DOI: 10.1093/nsr/nwv042.
- Paul Hudak. Dec. 1996. “Building Domain-specific Embedded Languages.” *ACM Comput. Surv.*, 28, 4es, (Dec. 1996). DOI: 10.1145/242224.242477.
- Gérard P. Huet and Bernard Lang. 1978. “Proving and Applying Program Transformations Expressed with Second-Order Patterns.” *Acta Inf.*, 11, 31–55. DOI: 10.1007/BF00264598.
- “Why functional programming matters.” *Research Topics in Functional Programming*. Addison-Wesley Longman Publishing Co., Inc., USA, 17–42. ISBN: 0201172364.
- John Hughes. 2019. “How to Specify It!” In: *20th International Symposium on Trends in Functional Programming*. DOI: 10.1007/978-3-030-47147-7_4.
- Mathieu Huot, Sam Staton, and Matthijs Vákár. 2020. “Correctness of Automatic Differentiation via Diffeologies and Categorical Gluing.” In: *Foundations of Software Science and Computation Structures - 23rd International Conference, FOSSACS 2020, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2020, Dublin, Ireland, April 25-30, 2020, Proceedings*, 319–338. DOI: 10.1007/978-3-030-45231-5_17.
- Nathan Jacobson. 2012. *Basic algebra I*. Courier Corporation.
- Michael B. James, Zheng Guo, Ziteng Wang, Shivani Doshi, Hila Peleg, Ranjit Jhala, and Nadia Polikarpova. Nov. 2020. “Digging for fold: synthesis-aided API discovery for Haskell.” *Proceedings of the ACM on Programming Languages*, 4, OOPSLA, (Nov. 2020), 205:1–205:27. DOI: 10.1145/3428273.
- Mark P. Jones. 1995. “Functional programming with overloading and higher-order polymorphism.” In: *Advanced Functional Programming*. Ed. by Johan Jeuring and Erik Meijer. Springer Berlin Heidelberg, Berlin, Heidelberg, 97–136. ISBN: 978-3-540-49270-2.
- G. A. Kavvos. 2016. “The Many Worlds of Modal λ -calculi: I. Curry-Howard for Necessity, Possibility and Time.” *CoRR*, abs/1605.08106. <http://arxiv.org/abs/1605.08106> arXiv: 1605.08106.
- G. A. Kavvos. 2019. “Modalities, Cohesion, and Information Flow.” *Proceedings of the ACM on Programming Languages*, 3, POPL. eprint: 1809.07897. DOI: 10.1145/3290333.
- Andrew J. Kennedy. 2004. “FUNCTIONAL PEARL Pickler Combinators.” *J. Funct. Program.*
- Oleg Kiselyov. 2012. “Typed Tagless Final Interpreters.” en. In: *Generic and Indexed Programming: International Spring School, SSGIP 2010, Oxford, UK, March 22-26, 2010, Revised Lectures*. Lecture Notes in Computer Science. Ed. by Jeremy Gibbons. Springer, Berlin, Heidelberg, 130–174. ISBN: 978-3-642-32202-0. DOI: 10.1007/978-3-642-32202-0_3.
- Oleg Kiselyov. May 2019. “Effects Without Monads: Non-determinism – Back to the Meta Language.” *Electronic Proceedings in Theoretical Computer Science*, 294, (May 2019), 15–40. DOI: 10.4204/eptcs.294.2.
- Oleg Kiselyov and Hiromi Ishii. 2015. “Freer monads, more extensible effects.” *ACM SIGPLAN Notices*, 50, 12, 94–105. Publisher: ACM New York, NY, USA. <https://dl.acm.org/doi/10.1145/2804302.2804319>.
- Oleg Kiselyov, Ralf Lämmel, and Kean Schupke. 2004. “Strongly Typed Heterogeneous Collections.” In: *Proceedings of the 2004 ACM SIGPLAN Workshop on Haskell (Haskell ’04)*. Association for Computing Machinery, Snowbird, Utah, USA, 96–107. ISBN: 1581138504. DOI: 10.1145/1017472.1017488.
- Oleg Kiselyov, Chung-chieh Shan, Daniel P Friedman, and Amr Sabry. 2005. “Backtracking, interleaving, and terminating monad transformers: (functional pearl).” *ACM SIGPLAN Notices*, 40, 9, 192–203.
- Ed Kmett. 2023. *free: Haskell Package*. (2023). Retrieved July 13, 2023 from [//hackage.haskell.org/package/free](https://hackage.haskell.org/package/free).
- Edward Kmett. 2011. *profunctors library*. <https://hackage.haskell.org/package/profunctors-5.6.2>, accessed January 2021. (2011).

- Edward Kmett. 2012. *lens library*. <https://hackage.haskell.org/package/lens-5.1>, accessed January 2021. (2012).
- Faustyna Krawiec, Simon Peyton Jones, Neel Krishnaswami, Tom Ellis, Richard A. Eisenberg, and Andrew W. Fitzgibbon. 2022. “Provably correct, asymptotically efficient, higher-order reverse-mode automatic differentiation.” *Proc. ACM Program. Lang.*, 6, POPL, 1–30. DOI: [10.1145/3498710](https://doi.org/10.1145/3498710).
- S. Kullback and R. A. Leibler. 1951. “On Information and Sufficiency.” *The Annals of Mathematical Statistics*, 22, 1, 79–86. Publisher: Institute of Mathematical Statistics. Retrieved Feb. 8, 2023 from <https://www.jstor.org/stable/2236703>.
- Twan van Laarhoven. July 2009. *CPS based functional references*. blog post: <http://www.twanvl.nl/blog/haskell/cps-functional-references>. (July 2009).
- Leonidas Lampropoulos, Diane Gallois-Wong, Catalin Hritcu, John Hughes, Benjamin C. Pierce, and Li-yao Xia. 2017. “Beginner’s luck: a language for property-based generators.” In: *Symposium on Principles of Programming Languages (POPL)*. ACM, 114–129.
- Daan Leijen and Erik Meijer. 2001. “Parsec: Direct Style Monadic Parser Combinators For The Real World.” en, 22. <http://www.cs.uu.nl/research/techreps/repo/CS-2001/2001-35.pdf>.
- Yannis Lilis and Anthony Savidis. 2019. “A Survey of Metaprogramming Languages.” *ACM Computing Surveys*, 52, 6. DOI: [10.1145/3354584](https://doi.org/10.1145/3354584).
- J. Lin. Jan. 1991. “Divergence measures based on the Shannon entropy.” *IEEE Transactions on Information Theory*, 37, 1, (Jan. 1991), 145–151. Conference Name: IEEE Transactions on Information Theory. DOI: [10.1109/18.61115](https://doi.org/10.1109/18.61115).
- Ben Lippmeier, (Ed.) . 2015. *Proceedings of the 8th ACM SIGPLAN Symposium on Haskell, Haskell 2015, Vancouver, BC, Canada, September 3-4, 2015*. ACM. ISBN: 978-1-4503-3808-0. DOI: [10.1145/2804302](https://doi.org/10.1145/2804302).
- David R MacIver, Zac Hatfield-Dodds, et al.. 2019. “Hypothesis: A new approach to property-based testing.” *Journal of Open Source Software*, 4, 43, 1891. <https://joss.theoj.org/papers/10.21105/joss.01891.pdf>.
- David R. MacIver and Alastair F. Donaldson. 2020. “Test-Case Reduction via Test-Case Generation: Insights from the Hypothesis Reducer (Tool Insights Paper).” In: *34th European Conference on Object-Oriented Programming (ECOOP 2020)* (Leibniz International Proceedings in Informatics (LIPIcs)). Ed. by Robert Hirschfeld and Tobias Pape. Vol. 166. ISSN: 1868-8969. Schloss Dagstuhl–Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 13:1–13:27. ISBN: 978-3-95977-154-2. DOI: [10.4230/LIPIcs.ECOOP.2020.13](https://doi.org/10.4230/LIPIcs.ECOOP.2020.13).
- José Pedro Magalhães, Atze Dijkstra, Johan Jeuring, and Andres Löb. Sept. 2010. “A generic deriving mechanism for Haskell.” *ACM SIGPLAN Notices*, 45, 11, (Sept. 2010), 37–48. DOI: [10.1145/2088456.1863529](https://doi.org/10.1145/2088456.1863529).
- Nicholas D. Matsakis and Felix S. Klock. Oct. 2014. “The rust language.” *Ada Lett.*, 34, 3, (Oct. 2014), 103–104. DOI: [10.1145/2692956.2663188](https://doi.org/10.1145/2692956.2663188).
- Kazutaka Matsuda, Romina Eramo, Michael Johnson, and Vadim Zaytsev. 2025. “Bidirectional Transformations: Foundations and Applications (NII Shonan Meeting Report No. 231).” *NII Shonan Meeting Reports*, 102, 13–15, 1. DOI: <https://shonan.nii.ac.jp/docs/No.231.pdf>.
- Kazutaka Matsuda, Samantha Frohlich, Meng Wang, and Nicolas Wu. 2023. “Embedding by Unembedding.” *Proceedings of the ACM on Programming Languages*, 7, ICFP, 1–47. DOI: [10.1145/3607830](https://doi.org/10.1145/3607830).
- Kazutaka Matsuda, Zhenjiang Hu, Keisuke Nakano, Makoto Hamana, and Masato Takeichi. 2007. “Bidirectionalization transformation based on automatic derivation of view complement functions.” In: *International Conference on Functional Programming (ICFP)*. ACM, 47–58.
- Kazutaka Matsuda and Meng Wang. 2013. “FliPpr: A Prettier Invertible Printing System.” In: *Programming Languages and Systems - 22nd European Symposium on Programming, ESOP 2013, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2013, Rome, Italy, March 16-24, 2013. Proceedings*, 101–120. DOI: [10.1007/978-3-642-37036-6_6](https://doi.org/10.1007/978-3-642-37036-6_6).

- Kazutaka Matsuda and Meng Wang. 2015. “Applicative bidirectional programming with lenses.” In: *International Conference on Functional Programming (ICFP)*. ACM, 62–74. DOI: [10.1145/2784731.2784750](https://doi.org/10.1145/2784731.2784750).
- Kazutaka Matsuda and Meng Wang. 2018a. “Applicative bidirectional programming: Mixing lenses and semantic bidirectionalization.” *J. Funct. Program.*, 28, e15. DOI: [10.1017/S0956796818000096](https://doi.org/10.1017/S0956796818000096).
- Kazutaka Matsuda and Meng Wang. 2018b. “Embedding invertible languages with binders: a case of the FliPpr language.” In: *Proceedings of the 11th ACM SIGPLAN International Symposium on Haskell, Haskell@ICFP 2018, St. Louis, MO, USA, September 27-17, 2018*, 158–171. DOI: [10.1145/3242744.3242758](https://doi.org/10.1145/3242744.3242758).
- Kazutaka Matsuda and Meng Wang. 2018c. “FliPpr: A System for Deriving Parsers from Pretty-Printers.” English. *New Generation Computing*, 36, 3, 173–202. Publisher: Springer Verlag. DOI: [10.1007/s00354-018-0033-7](https://doi.org/10.1007/s00354-018-0033-7).
- Kazutaka Matsuda and Meng Wang. 2018d. “HOBiT: Programming Lenses Without Using Lens Combinators.” In: *ESOP*, 31–59. DOI: [10.1007/978-3-319-89884-1_2](https://doi.org/10.1007/978-3-319-89884-1_2).
- Kazutaka Matsuda and Meng Wang. 2020. “Sparcl: a language for partially-invertible computation.” *Proc. ACM Program. Lang.*, 4, ICFP, 118:1–118:31. DOI: [10.1145/3409000](https://doi.org/10.1145/3409000).
- Mikaël Mayer, Viktor Kuncak, and Ravi Chugh. Oct. 2018. “Bidirectional Evaluation with Direct Manipulation.” *Proc. ACM Program. Lang.*, 2, OOPSLA, (Oct. 2018), 127:1–127:28. DOI: [10.1145/3276497](https://doi.org/10.1145/3276497).
- Damiano Mazza and Michele Pagani. 2021. “Automatic differentiation in PCF.” *Proc. ACM Program. Lang.*, 5, POPL, 1–27. DOI: [10.1145/3434309](https://doi.org/10.1145/3434309).
- Conor McBride. 2010. *I am not a number, I am a classy hack*. Retrieved Feb. 23, 2025 from <https://mazzo.li/epilogue/index.html%3Fp=773.html>.
- Conor McBride and Ross Paterson. 2008. “Applicative programming with effects.” *J. Funct. Program.*, 18, 1, 1–13.
- Trevor L. McDonell, Manuel M.T. Chakravarty, Gabriele Keller, and Ben Lippmeier. 2013. “Optimising purely functional GPU programs.” In: *Proceedings of the 18th ACM SIGPLAN International Conference on Functional Programming (ICFP ’13)*. Association for Computing Machinery, Boston, Massachusetts, USA, 49–60. ISBN: 9781450323260. DOI: [10.1145/2500365.2500595](https://doi.org/10.1145/2500365.2500595).
- Trevor L. McDonell, Joshua D. Meredith, and Gabriele Keller. 2021. “Embedded Pattern Matching.” *CoRR*, abs/2108.13114. <https://arxiv.org/abs/2108.13114> arXiv: [2108.13114](https://arxiv.org/abs/2108.13114).
- Dale Miller and Gopalan Nadathur. 1987. “A Logic Programming Approach to Manipulating Formulas and Programs.” In: *Proceedings of the 1987 Symposium on Logic Programming, San Francisco, California, USA, August 31 - September 4, 1987*. IEEE-CS, 379–388. ISBN: 0-8186-0799-8.
- Dale Miller and Gopalan Nadathur. 2012. *Programming with higher-order logic*. Cambridge University Press, New York, NY. 306 pp.
- Anders Miltner, Kathleen Fisher, Benjamin C. Pierce, David Walker, and Steve Zdancewic. Dec. 2017. “Synthesizing Bijective Lenses.” *Proc. ACM Program. Lang.*, 2, POPL, (Dec. 2017). DOI: [10.1145/3158089](https://doi.org/10.1145/3158089).
- Eugenio Moggi. July 1991. “Notions of computation and monads.” en. *Information and Computation*. Selections from 1989 IEEE Symposium on Logic in Computer Science 93, 1, (July 1991), 55–92. DOI: [10.1016/0890-5401\(91\)90052-4](https://doi.org/10.1016/0890-5401(91)90052-4).
- Eugenio Moggi. 1998. “Functor Categories and Two-Level Languages.” In: *FoSSaCS*, 211–225. DOI: [10.1007/BFb0053552](https://doi.org/10.1007/BFb0053552).
- Rolf Molich and Jakob Nielsen. Mar. 1990. “Improving a human-computer dialogue.” *Commun. ACM*, 33, 3, (Mar. 1990), 338–348. DOI: [10.1145/77481.77486](https://doi.org/10.1145/77481.77486).
- Akimasa Morihata. 2020. *Short Cut to Incremental Typed Functional Programs*. Informal Proceedings of WPTE 2020: 7th International Workshop on Rewriting Techniques for Program Transformations

- and Evaluation. Available from http://maude.ucm.es/wpte20/papers/WPTE_2020_morihata.pdf. (2020).
- Leonardo de Moura, Soonho Kong, Jeremy Avigad, Floris van Doorn, and Jakob von Raumer. 2015. “The Lean Theorem Prover (System Description).” In: *Automated Deduction - CADE-25 - 25th International Conference on Automated Deduction, Berlin, Germany, August 1-7, 2015, Proceedings* (Lecture Notes in Computer Science). Ed. by Amy P. Felty and Aart Middeldorp. Vol. 9195. Springer, 378–388. DOI: [10.1007/978-3-319-21401-6_26](https://doi.org/10.1007/978-3-319-21401-6_26).
- Shin-Cheng Mu, Zhenjiang Hu, and Masato Takeichi. 2004. “An Algebraic Approach to Bi-directional Updating.” In: *Programming Languages and Systems*. Springer Berlin Heidelberg, 2–20. ISBN: 978-3-540-30477-7.
- Minh Nguyen, Roly Perera, Meng Wang, and Steven Ramsay. 2023. “Effect handlers for programmable inference.” *Haskell 2023*. DOI: [10.1145/3609026.3609729](https://doi.org/10.1145/3609026.3609729).
- Ulf Norrell. 2007. “Towards a practical programming language based on dependent type theory.” PhD thesis. Chalmers University of Technology. <https://research.chalmers.se/en/publication/46311>.
- Fritz Obermeyer, Eli Bingham, Martin Jankowiak, Du Phan, and Jonathan P. Chen. 2020. *Functional Tensors for Probabilistic Programming*. (2020). arXiv: [1910.10775](https://arxiv.org/abs/1910.10775) [stat.ML].
- Martin Odersky and Philip Wadler. 1997. “Pizza into Java: translating theory into practice.” In: *Proceedings of the 24th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL '97)*. Association for Computing Machinery, Paris, France, 146–159. ISBN: 0897918533. DOI: [10.1145/263699.263715](https://doi.org/10.1145/263699.263715).
- Dominic Orchard and Alan Mycroft. 2012. “Categorical programming for data types with restricted parametricity.” *Manuscript, submitted to Post-Proc of Trends in Functional Programming (TFP)*. <https://www.cs.kent.ac.uk/people/staff/dao7/drafts/tfp-structures-orchard12.pdf>.
- Hugo Pacheco and Alcino Cunha. 2010. “Generic Point-free Lenses.” In: *Mathematics of Program Construction*. Ed. by Claude Bolduc, Jules Desharnais, and Béchir Ktari. Springer Berlin Heidelberg, Berlin, Heidelberg, 331–352. ISBN: 978-3-642-13321-3.
- Hugo Pacheco, Zhenjiang Hu, and Sebastian Fischer. 2014. “Monadic combinators for "Putback" style bidirectional programming.” In: *Workshop on Partial Evaluation and Program Manipulation (PEPM)*. ACM, 39–50. DOI: [10.1145/2543728.2543737](https://doi.org/10.1145/2543728.2543737).
- Hugo Pacheco, Tao Zan, and Zhenjiang Hu. 2014. “BiFluX: A Bidirectional Functional Update Language for XML.” In: *International Symposium on Principles and Practice of Declarative Programming (PPDP)*. ACM.
- Michał H. Pałka, Koen Claessen, Alejandro Russo, and John Hughes. 2011. “Testing an Optimising Compiler by Generating Random Lambda Terms.” In: *Proceedings of the 6th International Workshop on Automation of Software Test (AST '11)*. event-place: Waikiki, Honolulu, HI, USA. ACM, New York, NY, USA, 91–97. ISBN: 978-1-4503-0592-1. DOI: [10.1145/1982595.1982615](https://doi.org/10.1145/1982595.1982615).
- Zoe Paraskevopoulou, Aaron Eline, and Leonidas Lampropoulos. June 2022. “Computing correctly with inductive relations.” In: *Proceedings of the 43rd ACM SIGPLAN International Conference on Programming Language Design and Implementation (PLDI 2022)*. Association for Computing Machinery, New York, NY, USA, (June 2022), 966–980. ISBN: 978-1-4503-9265-5. DOI: [10.1145/3519939.3523707](https://doi.org/10.1145/3519939.3523707).
- Ivan Perez and Henrik Nilsson. 2015. “Bridging the GUI gap with reactive values and relations.” In: *Proceedings of the 8th ACM SIGPLAN Symposium on Haskell, Haskell 2015, Vancouver, BC, Canada, September 3-4, 2015*. Ed. by Ben Lippmeier. ACM, 47–58. ISBN: 978-1-4503-3808-0. DOI: [10.1145/2804302.2804316](https://doi.org/10.1145/2804302.2804316).
- Frank Pfenning and Conal Elliott. 1988. “Higher-Order Abstract Syntax.” In: *Proceedings of the ACM SIGPLAN'88 Conference on Programming Language Design and Implementation (PLDI), Atlanta,*

- Georgia, USA, June 22-24, 1988. Ed. by Richard L. Wexelblat. ACM, 199–208. ISBN: 0-89791-269-1. DOI: [10.1145/53990.54010](https://doi.org/10.1145/53990.54010).
- M. Pickering, J. Gibbons, and N. Wu. 2017. “Profunctor optics: Modular data accessors.” en. *Art, Science, and Engineering of Programming*, 1, 2. Publisher: Aspect-Oriented Software Association. Retrieved Jan. 25, 2023 from <https://ora.ox.ac.uk/objects/uuid:9989be57-a045-4504-b9d7-dc93fd508365>.
- Brigitte Pientka. 2008. “A type-theoretic foundation for programming with higher-order abstract syntax and first-class substitutions.” In: *Proceedings of the 35th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2008, San Francisco, California, USA, January 7-12, 2008*. Ed. by George C. Necula and Philip Wadler. ACM, 371–382. ISBN: 978-1-59593-689-9. DOI: [10.1145/1328438.1328483](https://doi.org/10.1145/1328438.1328483).
- Benjamin Pierce. Jan. 2004. “Advanced Topics in Types and Programming Languages,” (Jan. 2004), 3–43.
- Lee Pike. Sept. 2014. “SmartCheck: automatic and efficient counterexample reduction and generalization.” In: *Proceedings of the 2014 ACM SIGPLAN symposium on Haskell (Haskell ’14)*. Association for Computing Machinery, New York, NY, USA, (Sept. 2014), 53–64. ISBN: 978-1-4503-3041-1. DOI: [10.1145/2633357.2633365](https://doi.org/10.1145/2633357.2633365).
- Andrew M. Pitts. 2001. “Nominal Logic: A First Order Theory of Names and Binding.” In: *Theoretical Aspects of Computer Software*. Ed. by Naoki Kobayashi and Benjamin C. Pierce. Springer Berlin Heidelberg, Berlin, Heidelberg, 219–242. ISBN: 978-3-540-45500-4.
- Andrew M. Pitts. 2023. “Locally Nameless Sets.” *Proceedings of the ACM on Programming Languages*, 7, 488–514, POPL. DOI: [10.1145/3571210](https://doi.org/10.1145/3571210).
- Jeff Polakow. 2015. “Embedding a full linear Lambda calculus in Haskell.” In: *Proceedings of the 8th ACM SIGPLAN Symposium on Haskell, Haskell 2015, Vancouver, BC, Canada, September 3-4, 2015*. Ed. by Ben Lippmeier. ACM, 177–188. ISBN: 978-1-4503-3808-0. DOI: [10.1145/2804302.2804309](https://doi.org/10.1145/2804302.2804309).
- Justin Pombrio and Shriram Krishnamurthi. 2014. “Resugaring: Lifting Evaluation Sequences Through Syntactic Sugar.” In: *Programming Language Design and Implementation (PLDI)*. ACM.
- Purely Agile. N.d. *opaleye library*. <https://hackage.haskell.org/package/opaleye>. ().
- Raghu Rajkumar, Sam Lindley, Nate Foster, and James Cheney. 2013. *Lenses for Web Data*. International Workshop on Bidirectional Transformations (BX). (2013).
- Tillmann Rendel and Klaus Ostermann. 2010. “Invertible syntax descriptions: unifying parsing and pretty-printing.” In: *International Symposium on Haskell (Haskell)*, 1–12.
- John C. Reynolds. 1974. “Towards a theory of type structure.” In: *Programming Symposium, Proceedings Colloque sur la Programmation, Paris, France, April 9-11, 1974* (Lecture Notes in Computer Science). Ed. by Bernard Robinet. Vol. 19. Springer, 408–423. DOI: [10.1007/3-540-06859-7_148](https://doi.org/10.1007/3-540-06859-7_148).
- John C. Reynolds. 1983. “Types, Abstraction and Parametric Polymorphism.” In: *Information Processing*. Ed. by R.E.A. Mason. Elsevier Science Publishers B.V. (North-Holland), 513–523.
- John C. Reynolds. 2003. “What do types mean? — From intrinsic to extrinsic semantics.” In: *Programming Methodology* (Monographs in Computer Science). Ed. by Annabelle McIver and Carroll Morgan. Springer New York, 309–327. DOI: [10.1007/978-0-387-21798-7_15](https://doi.org/10.1007/978-0-387-21798-7_15).
- Colin Runciman, Matthew Naylor, and Fredrik Lindblad. Sept. 2008. “Smallcheck and lazy smallcheck: automatic exhaustive testing for small values.” *ACM SIGPLAN Notices*, 44, 2, (Sept. 2008), 37–48. DOI: [10.1145/1543134.1411292](https://doi.org/10.1145/1543134.1411292).
- Christopher Schuster, Tim Disney, and Cormac Flanagan. 2016. “Macrofication: Refactoring by Reverse Macro Expansion.” In: *European Symposium on Programming (ESOP)*. Springer.
- Adam Ścibior, Ohad Kammar, and Zoubin Ghahramani. July 2018. “Functional programming for modular Bayesian inference.” *Proceedings of the ACM on Programming Languages*, 2, ICFP, (July 2018), 83:1–83:29. DOI: [10.1145/3236778](https://doi.org/10.1145/3236778).

- Ajeet Shankar and Rastislav Bodík. 2007. “DITTO: Automatic Incrementalization of Data Structure Invariant Checks (in Java).” In: *Proceedings of the 28th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI ’07)*. Association for Computing Machinery, San Diego, California, USA, 310–319. ISBN: 9781595936332. DOI: [10.1145/1250734.1250770](https://doi.org/10.1145/1250734.1250770).
- Tim Sheard and Simon L. Peyton Jones. 2002. “Template meta-programming for Haskell.” *ACM SIGPLAN Notices*, 37, 12, 60–75. DOI: [10.1145/636517.636528](https://doi.org/10.1145/636517.636528).
- Benjamin Sherman, Jesse Michel, and Michael Carbin. 2021. “ λ_S : computable semantics for differentiable programming with higher-order functions and datatypes.” *Proc. ACM Program. Lang.*, 5, POPL, 1–31. DOI: [10.1145/3434284](https://doi.org/10.1145/3434284).
- Ezekiel Soremekun, Esteban Pavese, Nikolas Havrikov, Lars Grunske, and Andreas Zeller. 2020. “Inputs from Hell: Learning Input Distributions for Grammar-Based Test Generation.” *IEEE Transactions on Software Engineering*. Publisher: IEEE. DOI: [10.1109/TSE.2020.3013716](https://doi.org/10.1109/TSE.2020.3013716).
- Prashast Srivastava and Mathias Payer. July 2021. “Gramatron: effective grammar-aware fuzzing.” In: *Proceedings of the 30th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA 2021)*. Association for Computing Machinery, New York, NY, USA, (July 2021), 244–256. ISBN: 978-1-4503-8459-9. DOI: [10.1145/3460319.3464814](https://doi.org/10.1145/3460319.3464814).
- Jacob Stanley. 2023. *Hedgehog will eat all your bugs*. (2023). Retrieved Jan. 25, 2023 from <https://hedgehog.qa/>.
- Perdita Stevens. 2008a. “A Landscape of Bidirectional Model Transformations.” In: *GTTSE*, 408–424. DOI: [10.1007/978-3-540-88643-3_10](https://doi.org/10.1007/978-3-540-88643-3_10).
- Perdita Stevens. 2008b. “Generative and Transformational Techniques in Software Engineering II.” In: ed. by Ralf Lämmel, Joost Visser, and João Saraiva. Springer-Verlag. Chap. A Landscape of Bidirectional Model Transformations, 408–424. ISBN: 978-3-540-88642-6. DOI: [10.1007/978-3-540-88643-3_10](https://doi.org/10.1007/978-3-540-88643-3_10).
- Don Stewart. 2010. *yices-painless: An embedded language for programming the Yices SMT solver*. In Hackage: <https://hackage.haskell.org/package/yices-painless>. Visited 2022-06-27. (2010).
- Van-Dang Tran, Hiroyuki Kato, and Zhenjiang Hu. 2020. “Programmable View Update Strategies on Relations.” *Proc. VLDB Endow.*, 13, 5, 726–739. DOI: [10.14778/3377369.3377380](https://doi.org/10.14778/3377369.3377380).
- Vera Trnková, Jiří Adámek, Václav Koubek, and Jan Reiterman. 1975. “Free algebras, input processes and free monads.” en. *Commentationes Mathematicae Universitatis Carolinae*, 016, 339–351. <http://dml.mathdoc.fr/item/105628>.
- Christian Urban and Christine Tasson. 2005. “Nominal techniques in Isabelle/HOL.” In: *Proceedings of the 20th International Conference on Automated Deduction (CADE’ 20)*. Springer-Verlag, Tallinn, Estonia, 38–53. ISBN: 3540280057. DOI: [10.1007/11532231_4](https://doi.org/10.1007/11532231_4).
- Spandan Veggalam, Sanjay Rawat, Istvan Haller, and Herbert Bos. 2016. “IFuzzer: An Evolutionary Interpreter Fuzzer Using Genetic Programming.” en. In: *Computer Security – ESORICS 2016* (Lecture Notes in Computer Science). Ed. by Ioannis Askoxylakis, Sotiris Ioannidis, Sokratis Katsikas, and Catherine Meadows. Springer International Publishing, Cham, 581–601. ISBN: 978-3-319-45744-4. DOI: [10.1007/978-3-319-45744-4_29](https://doi.org/10.1007/978-3-319-45744-4_29).
- Janis Voigtländer. 2009. “Bidirectionalization for free! (Pearl).” In: *Symposium on Principles of Programming Languages (POPL)*. ACM, 165–176.
- Dimitrios Vytiniotis, Dan Belov, Richard Wei, Gordon Plotkin, and Martin Abadi. 2019. *The Differentiable Curry*. Program Transformations for Machine Learning@NeurIPS. Available from: <https://openreview.net/pdf?id=ryxuz9SszDB>. (2019).
- Philip Wadler. 1985. “How to replace failure by a list of successes a method for exception handling, backtracking, and pattern matching in lazy functional languages.” In: *Conference on Functional Programming Languages and Computer Architecture*. Springer, 113–128.
- Philip Wadler. 1989. “Theorems for Free!” In: *FPCA*, 347–359.

- Philip Wadler. 1992. “The essence of functional programming.” In: *Proceedings of the 19th ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages (POPL ’92)*. Association for Computing Machinery, Albuquerque, New Mexico, USA, 1–14. ISBN: 0897914538. DOI: [10.1145/143165.143169](https://doi.org/10.1145/143165.143169).
- Philip Wadler. 1995. “Monads for functional programming.” In: *International School on Advanced Functional Programming*. Springer, 24–52.
- Malcolm Wallace and Colin Runciman. 1999. “Haskell and XML: Generic Combinators or Type-Based Translation?” In: *Proceedings of the fourth ACM SIGPLAN International Conference on Functional Programming (ICFP ’99), Paris, France, September 27-29, 1999*. Ed. by Didier Rémi and Peter Lee. ACM, 148–159. ISBN: 1-58113-111-9. DOI: [10.1145/317636.317794](https://doi.org/10.1145/317636.317794).
- Fei Wang, Daniel Zheng, James M. Decker, Xilun Wu, Grégory M. Essertel, and Tiark Rompf. 2019. “Demystifying differentiable programming: shift/reset the penultimate backpropagator.” *Proc. ACM Program. Lang.*, 3, ICFP, 96:1–96:31. DOI: [10.1145/3341700](https://doi.org/10.1145/3341700).
- Junjie Wang, Bihuan Chen, Lei Wei, and Yang Liu. May 2019. “Superion: Grammar-Aware Greybox Fuzzing.” In: *2019 IEEE/ACM 41st International Conference on Software Engineering (ICSE)*. ISSN: 1558-1225. (May 2019), 724–735. DOI: [10.1109/ICSE.2019.00081](https://doi.org/10.1109/ICSE.2019.00081).
- Meng Wang, Jeremy Gibbons, Kazutaka Matsuda, and Zhenjiang Hu. 2010. “Gradual Refinement: Blending Pattern Matching with Data Abstraction.” In: *MPC (Lecture Notes in Computer Science)*. Ed. by Claude Bolduc, Jules Desharnais, and Béchir Ktari. Vol. 6120. Springer, 397–425. ISBN: 978-3-642-13320-6.
- Meng Wang, Jeremy Gibbons, Kazutaka Matsuda, and Zhenjiang Hu. 2013. “Refactoring pattern matching.” *Sci. Comput. Program.*, 78, 11, 2216–2242.
- Geoffrey Washburn and Stephanie Weirich. 2003. “Boxes go bananas: encoding higher-order abstract syntax with parametric polymorphism.” In: *Proceedings of the eighth ACM SIGPLAN international Conference on Functional Programming*. Association for Computing Machinery, 249–262. DOI: [10.1145/944705.944728](https://doi.org/10.1145/944705.944728).
- Geoffrey Washburn and Stephanie Weirich. 2008. “Boxes go bananas: Encoding higher-order abstract syntax with parametric polymorphism.” *Journal of Functional Programming*, 18, 1, 87–140. DOI: [10.1017/S0956796807006557](https://doi.org/10.1017/S0956796807006557).
- Michael Flænø Werk, Joakim Ahnfelt-Rønne, and Ken Friis Larsen. 2012. “An embedded DSL for stochastic processes: research article.” In: *Proceedings of the 1st ACM SIGPLAN workshop on Functional high-performance computing, Copenhagen, Denmark, FHPC@ICFP 2012, September 9-15, 2012*, 93–102. DOI: [10.1145/2364474.2364488](https://doi.org/10.1145/2364474.2364488).
- James Wood and Robert Atkey. 2022. “A Framework for Substructural Type Systems.” In: *Programming Languages and Systems - 31st European Symposium on Programming, ESOP 2022, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2022, Munich, Germany, April 2-7, 2022, Proceedings* (Lecture Notes in Computer Science). Ed. by Ilya Sergey. Vol. 13240. Springer, 376–402. DOI: [10.1007/978-3-030-99336-8_14](https://doi.org/10.1007/978-3-030-99336-8_14).
- Li-yao Xia, Samantha Frohlich, Dominic Orchard, and Meng Wang. 2025. *Source for the paper The Monadic Profunctor Paradigm of Bidirectional Programming*. <https://github.com/Lysxia/monadic-profunctor-bx-paper>. (2025).
- Liyao Xia. 2018. *Further implementations*. <https://github.com/Lysxia/profunctor-monad>. (2018).
- Li-yao Xia, Dominic Orchard, and Meng Wang. 2019a. “Composing Bidirectional Programs Monadically.” In: *Programming Languages and Systems*. Ed. by Luís Caires. Springer International Publishing, Cham, 147–175. ISBN: 978-3-030-17184-1.
- Li-yao Xia, Dominic Orchard, and Meng Wang. 2019b. *Composing bidirectional programs monadically (with appendices)*. Available at <https://arxiv.org/abs/1902.06950>. (2019).

- Yingfei Xiong, Dongxi Liu, Zhenjiang Hu, Haiyan Zhao, Masato Takeichi, and Hong Mei. 2007. “Towards automatic model synchronization from model transformations.” In: *ASE*. Ed. by R. E. Kurt Stirewalt, Alexander Egyed, and Bernd Fischer. ACM, 164–173. ISBN: 978-1-59593-882-4.
- Tetsuo Yokoyama, Holger Bock Axelsen, and Robert Glück. 2011. “Towards a Reversible Functional Language.” In: *RC*, 14–29. DOI: [10.1007/978-3-642-29517-1_2](https://doi.org/10.1007/978-3-642-29517-1_2).
- Yijun Yu, Yu Lin, Zhenjiang Hu, Soichiro Hidaka, Hiroyuki Kato, and Lionel Montrieux. 2012. “Maintaining invariant traceability through bidirectional transformations.” In: *ICSE*. Ed. by Martin Glinz, Gail C. Murphy, and Mauro Pezzè. IEEE, 540–550. ISBN: 978-1-4673-1067-3.
- Michał Zalewski. Nov. 2022. *American Fuzzy Lop (AFL)*. original-date: 2019-07-25T16:50:06Z. (Nov. 2022). Retrieved Nov. 22, 2022 from <https://github.com/google/AFL>.

Appendix

1. Partial Arrows are PMPs

```
instance Profunctor PartialArr where
  dimap f g (PartialArr p) = PartialArr (fmap g . p . f)

instance Monad (PartialArr a) where
  return b = PartialArr (Just . const b)
  (PartialArr p) >>= k = PartialArr (\a -> join (fmap (\x -> unPA (k x) a) (p a)))
    -- Same as the one for (->), but dealing with the maybe
    -- bind for (->): f >>= k = \ r -> k (f r) r

prune :: PartialArr b a -> PartialArr (Maybe b) a
prune (PartialArr p) = PartialArr (join . fmap p)
```

2. Generator Upgrade Progression

Original:

```
bst :: (Int, Int) -> Gen Tree
bst (lo, hi) | lo > hi = return Leaf
bst (lo, hi) = frequency
  [ ( 1, return Leaf ),
    ( 5, do
      x <- choose (lo, hi)
      l <- bst (lo, x - 1)
      r <- bst (x + 1, hi)
      return (Node l x r) ) ]
```

Midpoint:

```
bst :: (Int, Int) -> Reflective Void Tree
bst (lo, hi) | lo > hi = return Leaf
bst (lo, hi) = frequency
  [ ( 1, return Leaf),
    ( 5, do
      x <- voidAnn (choose (lo, hi))
      l <- voidAnn (bst (lo, x - 1))
      r <- voidAnn (bst (x + 1, hi))
      return (Node l x r) ) ]
```

Final:

```
bst :: (Int, Int) -> Reflective Tree Tree
bst (lo, hi) | lo > hi = exact Leaf
bst (lo, hi) = frequency
  [ ( 1, exact Leaf),
```

```
( 5, do
  x <- focus (_Node._2) (choose (lo, hi))
  l <- focus (_Node._1) (bst (lo, x - 1))
  r <- focus (_Node._3) (bst (x + 1, hi))
  return (Node l x r) ) ]
```

3. Proofs of Lemma 3.4.1 (Laws)

This appendix proves the equations from Lemma 3.4.1.

```
(M1)    return a >>= f = f a
(M3)    (x >>= f) >>= g = x >>= (\ a -> f a >>= g)
(PMP3)  (lmap f . prune) (return y) = return y
(PMP4)  (lmap f . prune) (x >>= g) = (lmap f . prune) x >>= lmap f . prune . g
```

Using the following relevant definitions:

```
data Freer f a where
  Return :: a -> Freer f a
  Bind   :: f a -> (a -> Freer f c) -> Freer f c

data R b a where
  Pick :: [(Weight, Choice, Reflective b a)] -> R b a
  Lmap  :: (c -> d) -> R d a -> R c a
  Prune :: R b a -> R (Maybe b) a
  ChooseInteger :: (Integer, Integer) -> R Integer Integer
  GetSize :: R b Int
  Resize  :: Int -> R b a -> R b a

type Reflective b = Freer (R b)

instance Monad (Reflective b) where
  return = Return
  Return x >>= f = f x
  Bind u g >>= f = Bind u (g >=> f)

prune :: Reflective b a -> Reflective (Maybe b) a
prune (Return a) = Return a
prune (Bind x f) = Bind (Prune x) (prune . f)

lmap :: (c -> d) -> Reflective d a -> Reflective c a
lmap f = dimap f id

dimap :: (c -> d) -> (a -> b) -> Reflective d a -> Reflective c b
dimap _ g (Return a) = Return (g a)
dimap f g (Bind x h) = Bind (Lmap f x) (dimap f g . h)
```

Proofs of (M1) and (M3). Immediate, by definition. □

Proof of (PMP3). By rewriting.

```
(lmap f . prune) (return y)
= {- def. return -}
(lmap f . prune) (Return y)
```

```

= {- def. prune (Return case) -}
lmap f (Return y)
= {- def. lmap -}
dimap f id (Return y)
= {- def. dimap (Return case) -}
Return y
= {- def. return -}
return y

```

Thus (PMP3) holds. □

Proof of (PMP4). By induction over the structure of x .

Case $x = \text{Return } a$:

```

(lmap f . prune) (Return a >>= g)
= {- def. >>= (Return case) -}
(lmap f . prune) (g a)
= {- re-bracket -}
(lmap f . prune . g) a
= {- def. >>= (Return case) -}
Return a >>= (lmap f . prune . g)
= {- def. return -}
return a >>= (lmap f . prune . g)
= {- PMP3 -}
(lmap f . prune) (return a) >>= (lmap f . prune . g)
= {- def. return -}
(lmap f . prune) (Return a) >>= lmap f . prune . g

```

Case $x = \text{Bind } r \text{ h}$:

```

(lmap f . prune) (Bind r h >>= g)
= {- def. >>= -}
(lmap f . prune) (Bind r (h >=> g))
= {- def. prune + lmap -}
Bind (Lmap f (Prune r)) (lmap f . prune . (h >=> g))
= {- IH -}
Bind (Lmap f (Prune r)) (lmap f . prune . h >=> lmap f . prune . g)
= {- def. >>= -}
(Bind (Lmap f (Prune r)) (lmap f . prune . h)) >>= (lmap f . prune . g)
= {- def. prune + lmap -}
(lmap f . prune) (Bind r h) >>= lmap f . prune . g

```

Thus (PMP4) holds. □

4. Polymorphic Interpretation Function

```

interpret ::
forall p d c.
(PartialProf p, forall b. Monad (p b)) =>
(forall b a. [(Weight, Choice, Reflective b a)] -> p b a) ->
Reflective d c ->
p d c
interpret p = interp

```

```

where
  interp :: forall b a. Reflective b a -> p b a
  interp (Return a) = return a
  interp (Bind r f) = do
    a <- interpR r
    interpret p (f a)

  interpR :: forall b a. R b a -> p b a
  interpR (Pick xs) = p xs
  interpR (Lmap f r) = lmap f (interpR r)
  interpR (Prune r) = prune (interpR r)

```

5. Proof of Lemma 3.4.4 (Weak Completeness)

Recall that a reflective generator g is weak complete iff

$$a \in \text{reflect}' g b \implies a \sim \text{generate } g.$$

We claim that every reflective generator is weak complete, where the definition of $\text{reflect}'$ is as follows:

```

reflect' :: Reflective b a -> b -> [a]
reflect' = interp
  where
    interp :: Reflective b a -> b -> [a]
    interp (Return x) _ = return x
    interp (Bind r f) b = interpR r b >>= \x -> interp (f x) b

    interpR :: R b a -> b -> [a]
    interpR (Lmap f r') b = interpR r' (f b)
    interpR (Prune r') b = maybeToList b >>= \b' -> interpR r' b'
    interpR (Pick gs) b = gs >>= (\ (_, _, g) -> interp g b)

```

Proof. By mutual induction over the structure of Freer and R.

Given a reflective generator g and a value a :

Case $g = \text{Return } a'$:

Assume $a \in \text{reflect}' (\text{Return } a') b$

$\text{reflect}' (\text{Return } a') b = [a']$, thus $a = a'$.

By definition, $a' \sim \text{return } a'$, so $a \sim \text{return } a' = \text{reflect}' (\text{Return } a')$.

Case $g = \text{Bind } r f$:

Assume $a \in \text{reflect}' (\text{Bind } r f) b$.

Thus, $a \in (\text{interpR}_{\text{reflect}'} r b \gg= \lambda x \rightarrow \text{reflect}' (f x) b)$.

Thus, $\exists a'$ such that $a' \in \text{interpR}_{\text{reflect}'} r b$ and $a \in \text{reflect}' (f a') b$.

By IH_R , $a' \sim \text{interpR}_{\text{generate}} r$.

By IH , $a \sim \text{generate } (f a')$.

Thus, $a \in (\text{interpR}_{\text{generate}} r \gg= \lambda x \rightarrow \text{generate } (f x))$.

Thus, $a \in \text{generate } (\text{Bind } r f)$.

Simultaneously, given an $R r$ and a value a :

Case $r = \text{Lmap } f \ r'$:

Assume $a \in \text{interpR}_{\text{reflect}'} (\text{Lmap } f \ r') \ b$.

Thus, $a \in \text{interpR}_{\text{reflect}'} r' (f \ b)$.

By IH_R , $a \sim \text{interpR}_{\text{generate}} r'$.

Thus, $a \sim \text{interpR}_{\text{generate}} (\text{Lmap } f \ r')$.

Case $r = \text{Prune } r'$:

Assume $a \in \text{interpR}_{\text{reflect}'} (\text{Prune } r') \ b$.

Thus, $a \in \text{maybeToList } b \gg= \backslash b' \rightarrow \text{interpR}_{\text{reflect}'} r' \ b'$.

Thus, $\exists b'$ such that $a \in \text{interpR}_{\text{reflect}'} r' \ b'$

By IH_R , $a \sim \text{interpR}_{\text{generate}} r' \ b'$.

Thus, $a \sim \text{interpR}_{\text{generate}} (\text{Prune } r')$.

Case $r = \text{Pick } \text{gs}$:

Assume $a \in \text{interpR}_{\text{reflect}'} (\text{Pick } \text{gs}) \ b$.

Thus, $a \in (\text{gs} \gg= \backslash (_, _, g) \rightarrow \text{reflect}' \ g \ b)$.

Thus, $\exists g'$ such that $(_, _, g') \in \text{gs}$ and $a \in \text{reflect}' \ g' \ b$.

By IH , $a \sim \text{generate } g'$.

Thus, $a \sim \text{QC.frequency } [(w, \text{interp } g) \mid (w, _, g) \leftarrow \text{gs}]$.

(Recall, we assume weights are positive.)

Thus, $a \sim \text{interpR}_{\text{generate}} (\text{Pick } \text{gs})$.

This completes the proof. □

6. Hypothesis Generator Example

This Hypothesis generator produces valid binary search trees, and can be used for integrated shrinking in the style of [David R. MacIver and Donaldson \[2020\]](#). Since this is just a normal Python function, the generator can use side-effects if desired.

```
@st.composite
def bsts(draw, lo=-10, hi=10):
    if lo > hi:
        return Leaf()
    else:
        if not draw(st.integers(min_value=0, max_value=3)):
            return Leaf()
        x = draw(st.integers(min_value=lo, max_value=hi))
        return Node(x, draw(bsts(lo, x - 1)), draw(bsts(x + 1, hi)))
```

7. JSON with Hash Code Generator

```
withHashCode :: Reflective String String
withHashCode = do
    let a = "{\"payload\":"
    let b = ", \"hashCode\":"
    let c = "}"
    consume a >>- \_ ->
        start >>- \payload -> do
            let hashCode = take 8 (show (abs (hash payload)))
```

```

    consume b >>- \_ ->
    consume hashCode >>- \_ ->
    consume c >>- \_ ->
    return (a ++ payload ++ b ++ hashCode ++ c)
where
    hash = foldl' (\h c -> xor (33 * h) (fromEnum c)) 5381
    consume s = lmap (take (length s)) (exact s)

```

A reflective generator for JSON objects with a hashCode, not expressible with a grammar-based generator. The generator produces a payload, then computes its hash, and then assembles the larger JSON object containing both.

8. Example of Reduced package.json

```

{
  "name": "reflective-generators",
  "description": "What a great project",
  "scripts": {
    "start": "node ./src/server.js",
    "build": "babel ./src -out-dir ./dist",
    "test": "mocha ./test"
  },
  "repository": {
    "type": "git",
    "url": "https://example.com"
  },
  "keywords": [
    "reflective",
    "generators"
  ],
  "author": "test",
  "license": "mit",
  "devDependencies": {
    "babel-cli": "^6.24.1",
    "babel-core": "^6.24.1",
    "babel-preset-es2015": "^6.24.1"
  },
  "dependencies": {
    "express": "^4.15.3",
    "reflective": "^0.0.1"
  }
}

```

```

{
  "name": "a",
  "description": "a",
  "scripts": {
    "start": "a",
    "build": "a",
    "test": "a"
  },
  "repository": {
    "type": "a",
    "url": "a"
  },
  "keywords": [],
  "author": "a",
  "license": "a",
  "devDependencies": {},
  "dependencies": {
    "express": "^4.15.3"
  }
}

```

9. LeanCheck BST Enumerator

```

leanBST :: (Int, Int) -> [[Tree]]
leanBST (lo, hi) | lo > hi = [[Leaf]]
leanBST (lo, hi) =
  cons0 Leaf
    \ / ( choose (lo, hi) >>- \ x ->
          leanBST (lo, x - 1) >>- \ l ->

```

```

leanBST (x + 1, hi) >>- \ r ->
  delay [[Node 1 x r]]
)
where
  (>>-) = flip concatMapT
  choose = concatT [zipWith (\i x -> ofWeight [[x]] i) [0 ..] [lo .. hi]]

```

10. Implementation of liftS0

A complication about the definition of liftS0 is that it must deal with arbitrary number of arguments of a construct, where each argument can bind arbitrary number of variables.

For uniform manipulation of arguments, we first introduce the kind Sig2 k, which describes arguments of second order constructs:

```
data Sig2 k = [k] -> k
```

Here, a type $[a_1, \dots, a_n] \rightarrow b :: \text{Sig2 } k$ represents arguments that binds variables of types $[a_1, \dots, a_n]$ and have type b . This type is intended to be lifted to the type level by Haskell's DataKinds. This enables the definition of a type for unembedded interpretations:

```
data URep (sem :: [k] -> k -> Type) (s :: Sig2 k) where
  UR :: TEnv as -> (Env (EnvI sem) as -> EnvI sem b) -> URep sem (as ':-> b)
```

and one for corresponding semantic objects.

```
data TermRep (sem :: [k] -> k -> Type) (env :: [k]) (s :: Sig2 k) where
  TR :: sem (Append as env) b -> TermRep sem env (as ':-> b)
```

Here, URep takes an value-level representation of as in addition, as otherwise we cannot know how many arguments we pass to a function of type $(\text{Env } (\text{EnvI sem}) \text{ as} \rightarrow \text{EnvI sem } b)$ to obtain a semantic object $(\text{sem } (\text{Append as env}) b)$. Notice that type information is not available in runtime in Haskell in general, unless we explicitly convey them. Together these allow for the definition of liftS0 (Fig. 1). Notice that the $\text{Env } (\text{URep sem}) \text{ ss} \rightarrow \text{EnvI sem } r$ part represents an unembedded interpretation (with value-level representation of binding types), while $(\text{forall env. Env } (\text{TermRep sem env}) \text{ ss} \rightarrow \text{sem env } r)$ the part represents the corresponding semantic function.

However, it is almost certain that users do not want to use such constructors explicitly for lifting, and hence we leverage Haskell's type-level programming to abbreviate this pain. As seen in the main body of the paper, lifting can be done as `liftS0n (o10 :: o11 :: End) letSem`, where users only need to specify the numbers of variables bound by each argument of a construct. The first step is to get rid of Env to represents arguments, and hence we prepare the following type families for “curried” representations.

```

-- Corresponds to (forall env. Env (TermRep sem env) ss -> sem env r)
type family FuncTerm (sem :: [k] -> k -> Type) (env :: [k])
  (ss :: [Sig2 k]) (r :: k) | r -> sem env r where
  FuncTerm sem env '[] r = sem env r
  FuncTerm sem env ((as ':-> a) ': ss) r = sem (Append as env) a
    -> FuncTerm sem env ss r

-- Corresponds to Env (URep sem) ss -> EnvI sem r
type family FuncU (sem :: [k] -> k -> Type) (ss :: [Sig2 k])
  (r :: k) = res | res -> sem r where
  FuncU sem '[] r = EnvI sem r
  FuncU sem ((as ':-> a) ': ss) r = Func (EnvI sem) as (EnvI sem a)
    -> FuncU sem ss r

```

```

liftSO :: forall sem ss r. Variables sem =>
  (forall env. Env (TermRep sem env) ss -> sem env r)
  -> Env (URep sem) ss -> EnvI sem r
liftSO f ks = EnvI $ \g -> f (mapEnv (conv g) ks)
  where conv :: TEnv env -> URep sem s -> TermRep sem env s
        conv g (UR g1 k) = TR $ cnv g g1 k
        cnv :: TEnv env -> TEnv as -> (Env (EnvI sem) as -> EnvI sem a)
              -> sem (Append as env) a
        cnv g g1 k = let {ex_g = appendEnv g1 g; xs = mkXs g g1 ex_g}
                      in runEnvI (k xs) ex_g
mkXs :: TEnv env -> TEnv as' -> TEnv (Append as' env) -> Env (EnvI sem) as'
mkXs _ ENil _ = ENil
mkXs p (ECons _ as) tg@(ECons _ tg')
  = let x = EnvI $ \g' -> weakenMany tg g' var
    in ECons x (mkXs p as tg')

```

Figure 1.: Implementation of liftSO

With these type families, we can define liftSO_n as below

```

liftSOn :: forall sem ss r. Variables sem => Dim ss
  -> (forall env. FuncTerm sem env ss r) -> FuncU sem ss r
liftSOn ns f =
  let h :: forall env. Env (TermRep sem env) ss -> sem env r
      h = fromFuncTerm f
  in toFuncU ns (liftSO @sem h)

```

The function liftSO_n takes Dim ss, which is used to provide a value-level representation of asi required for URep, where ss = [as1:->b1,..., asn:->bn]. Specifying each asi is actually tedious, but an observation suggests that the length of each asi is enough for runtime. The GADT Dim is defined as below, leveraging the fact.

```

data Dim (ss :: [Sig2 k]) where
  End :: Dim '[]
  (..) :: OfLength as -> Dim ss -> Dim ((as ':-> a) ': ss)

data OfLength as where
  LZ :: OfLength '[]
  LS :: OfLength as -> OfLength (a ': as)

```

The function liftSO uses the functions toFuncU and fromFuncTerm, which have the following types.

```

toFuncU :: Dim ss -> (Env (URep sem) ss -> EnvI sem r) -> FuncU sem ss r

fromFuncTerm :: FuncTerm sem env ss r -> Env (TermRep sem env) ss -> sem env r

```

We omit the definitions of them as they are straightforward.

We also provide constants olk defined as ol0 = LZ, ol1 = LS LZ, ol2 = LS ol1 ... for small k for convenience.

11. Unembedding the Incremental λ -Calculus

We clarify here that, unlike the cache-transfer-style variant [Giarrusso et al. 2019] (discussed in Section 4.5), embedding ILC itself is possible even without the Embedding by Unembedding (tupling

can be applied as discussed in §4.5.2). Our primary focus here is to provide another example of the unembedding technique.

11.1. Unembedding ILC

Step 1: Identify the semantic domain The semantics of ILC depends on having infrastructure for representing changes. §4.5 fully implements such an infrastructure, which we will borrow for this example. The change infrastructure consists of a `Diff` typeclass, whose member `a` have an associated `Delta a` type that represents changes (or deltas) that can be made to those types.

With the ability to describe changes, ILC terms should be interpretable in two ways:

1. Using standard evaluation, where variable assignments map to results.
2. Using differential evaluation, where inputs and input deltas are mapped to output deltas.

These semantics can be packaged up into the following concrete datatype:

```
data ILC env a = Inc
  { sEval :: VEnv env -> a, dEval :: VEnv env -> DEnv env -> Delta a }
```

Here, `sEval` performs the standard evaluation, mapping variable assignments (`VEnv`) to a result; `dEval` does the differential evaluation, mapping the inputs (`VEnv`) and changes in them (`DEnv`) to an output change. For readability, we simply (ab)use the `Env` constructors from §4.3 to manipulate these environments, as their actual representations are not important in this section (we will revisit them in §4.5).

Since both parts of this semantic type feature an environment, it is easy to see that it is a member of the `Variables` typeclass. For `var`, `sEval` can extract the first value from the `VEnv`, and `dEval` can ignore the `VEnv` and extract the first delta from `DEnv`. For `weakenOne`, new `sEval` and `dEval` can be created that just ignore the first element of the environments:

```
instance Variables ILC where
  var :: ILC (a : as) a
  var = Inc (\ (ECons (PackDiff a) _) -> a)
          (\_ (ECons (PackDiffDelta da) _) -> da)
  weaken :: ILC as a -> ILC (b : as) a
  weaken (Inc s d) = Inc s' d'
    where
      s' (ECons _ g) = s g
      d' (ECons _ vg) (ECons _ dg) = d vg dg
```

Step 2: Prepare semantic functions for each construct (The core part of) ILC consists of: `unit`, `pair`, `fst`, `snd`, and `let` constructs. Note that `let` is the only second-order construct of this language.

The semantic function for `unit` is a single ILC type where the type in focus is `()`. This means that both `sEval` and `dEval` ignore their arguments and just return `()` and the unit delta respectively:

```
unitSem :: ILC env ()
unitSem = Inc (const ()) (\_ _ -> UnitDelta)
```

Such semantic functions can also have arguments, for example,

`pairSem :: ILC env a -> ILC env b -> ILC env (a, b)` takes one ILC term for the first element of the tuple, and one for the second. It runs both argument expressions, pairing together both the value and delta results. Those for `fst` and `snd` behave similarly: running the pair term and then throwing away

either the first or second element of the result. That for `let_` is more interesting, as it is a second-order construct meaning that it manipulates the environments. It is defined by creating new `sEval` and `dEval` functions:

```
letSem :: Diff a => ILC env a -> ILC (a ': env) b -> ILC env b
letSem (Inc sa da) (Inc sb db)
  = Inc (\g ->
    let x = sa g -- unpack a
        e' = ECons (PackDiff x) g -- add a to env
    in sb e' -- extract b with new env
  )
  (\vg dg ->
    let x = sa vg -- unpack a
        g' = ECons (PackDiff x) vg -- add a to env
        da' = da vg dg -- unpack da
        dg' = ECons (PackDiffDelta da') dg -- add da to env
    in db g' dg' -- extract b with new env
  )
```

Here, the differential evaluation uses both `sa` and `da` to create `a`-typed values and `Delta a`-typed values to be added to the corresponding environments needed for `db`, which mirrors the standard evaluation.

Step 3: Provide the HOAS representation of the syntax For ILC, this means that the language typeclass (`ILChoas`) has five methods. One for each construct:

```
class ILChoas exp where
  unit :: exp ()
  pair :: exp a -> exp b -> exp (a, b)
  fst_ :: exp (a, b) -> exp a
  snd_ :: exp (a, b) -> exp b
  let_ :: Diff a => exp a -> (exp a -> exp b) -> exp b
```

Notice how the types of each method match those of the semantic functions, but with ILC abstracted out into `e`, and the environments omitted as the host language is handling them.

Step 4: Implement semantics through appropriate liftings For ILC, this means that ILC is wrapped in `EnvI` so that it can be an instance of `ILChoas`:

```
instance ILChoas (EnvI ILC) where
  -- constructs without binders => we use liftF0
  unit = UE.liftF00 unitSem -- 0 args => 0
  pair = UE.liftF02 pairSem -- 2 args => 2
  fst_ = UE.liftF01 fstSem -- 1 arg => 1
  snd_ = UE.liftF01 sndSem
  -- our binding construct => we use listS0
  let_ = UE.liftS0n (o10 :: o11 :: End) letSem
```

Here, `unit` is defined as the lifting of `unitSem`. Because `unit` is first-order, one of the `liftF0` functions is used, specifically `liftF0_0` because `unit` has no arguments. The rest of the first-order constructs are defined similarly, differing only in their choice of lifting function, which depends on how many arguments the construct in question has. As a second-order construct, `let_` is different. Now its arguments can have arguments. To allow for this, `liftS0` takes a description of what arguments are expected: how many there are and how many arguments (bindings) they themselves have. This description is a list, with each element representing an argument and how many arguments it itself takes. In the case of `let_`, there are two arguments: the first taking no arguments (`o10`); the second taking one (`o11`).

Step 5: Put the language to work! Putting the embedded ILC to work builds on top of `runOpenILC`:

```
runILC :: Diff a => ILC '[a] b -> a -> (b, Delta a -> Delta b)
runILC t x = let
  vg = ECons (PackDiff x) ENil
  r = sEval t vg
  f da = let dg = ECons (PackDiffDelta da) ENil
         in dEval t vg dg
  in (r, f)
```

This `runILC` operates on the output of `runOpen` and produces two functions: an ordinary transformation and its corresponding change translator. Note that both the input and output types are members of the `Diff` typeclass meaning that they can have changes applied to them by the change infrastructure.

This performs incremental computation! Consider this example term that for some tuple x will take the first element, and pair it with itself:

```
term x = let_ (fst_ x) (\y -> pair y y)
```

It can be executed both normally by extracting the initial result, or incrementally by extracting the change function:

```
> fst (runILC term (7,6))
(7,7)
> (snd (runILC term (7,6))) (PairDelta (DInt 3) (DInt 2))
PairDelta +3 +3
```

In the latter example, the input tuple gets three added to its first element, and two to its second. This gets correctly translated to an output change where three would be added to both elements.

11.2. Extending ILC with Sequences

Embedding by Unembedding shares the modularity and extensibility of tagless-final style. To add new constructs, the recipe can just be repeated, and the language can be created in a modular manner, with different constructs grouped into different and dependant typeclasses. For example, to add sequences¹ to ILC the process unfolds as follows:

Step 1: Identify the semantic domain When it comes to extending the language with constructs, this step is ready done. There is already a semantic type for ILC.

Step 2: Prepare semantic functions for each construct Introducing sequences expands the language's constructors with: `empty`, `singleton`, `concat`, and `map`. Reifying these constructs into the embedding again consists of writing representative functions over the semantic type. For example, `empty` is defined similarly to `unit`:

```
emptySem :: ILC env (Seq a)
emptySem = Inc
  (const S.empty)    -- const the empty sequence
  (\_ _ -> DSeq []) -- const the empty change on sequences
```

Its semantics lift the empty sequence and its `Delta` into the ILC type. The rest of the semantic functions have the following types:

¹Sequences are a speedier version of lists in Haskell, and as ILC is about efficiency they are chosen.

```
singletonSem :: ILC env a -> ILC env (Seq a)
concatSem    :: ILC env (Seq (Seq a)) -> ILC env (Seq a)
mapSem :: (Diff a, Diff b)
      => ILC (a : env) b -> ILC env (Seq a) -> ILC env (Seq b)
```

Exploring their definitions, brings to light different patterns in the way certain constructs behave depending on whether they are constructors, destructors or binders. The constructor `singleton` is similar to `pair` in that it takes an argument to produce the element of the list, then wraps it in the functor. The destructor `concat` can be compared to `fst_` or `snd_`, as it runs its argument and then changes the structure of the result (being sure to propagate this change to the `Delta` type). `map` is the most interesting as it can be defined in terms of `let_` to deal with the environment:

```
mapSem f xs = letSem xs (mapSem' f)
```

Then `mapSem' :: (...) => ILC (a : env) b -> ILC (Seq a : env) (Seq b)` behaves similarly to `concat`, running the first term, then adjusting the result.

Step 3: Provide the HOAS representation of the syntax This step is identical to adding more constructs to a normal tagless-final embedding: adding a new typeclass with those constructs as methods.

```
class ILChoas exp => ILCSeq exp where
  empty    :: exp (Seq a)
  singleton :: exp a -> exp (Seq a)
  concat   :: exp (Seq (Seq a)) -> exp (Seq a)
  map      :: (Diff a, Diff b) => (exp a -> exp b) -> exp (Seq a) -> exp (Seq b)
```

This typeclass has been made dependent on `ILChoas` so that it is clear that it is an extension of the language. The correspondence between the method types and the types of the semantics functions is again evident.

Step 4: Implement semantics through appropriate liftings Instantiating `ILC` as member of this new typeclass follows the same pattern: wrapping it in the `EnvI` type, then defining the methods as the semantic functions using the provided lifting functions. Which lifting function to use is again decided by whether the construct is first or second order and how many arguments it has:

```
instance ILCSeq (EnvI ILC) where
  empty    = UE.liftF00 emptySem
  singleton = UE.liftF01 singletonSem
  concat   = UE.liftF01 concatSem
  map      = UE.liftS0n (o11 :: End) mapSem
```

Step 5: Put the language to work! New constructs allow for more exciting terms to interpret, and with no change in semantics, `runILC` and `runOpen` can still be used together to produce an initial result and a change function. For example, consider the term `cartesian`, which produces the cartesian product of two sequences. It can be run in an expression where the free variables is a pair of two sequences as described by `caInc`:

```
cartesian xs ys = concat (map (\x -> map (\y -> pair x y) ys) xs)
caInc = runILC (runOpen (\zs -> cartesian (fst_ zs) (snd_ zs)))

> let (r, i) = caInc (S.fromList [1..3 :: Int], S.fromList [1..3 :: Int])
> r
```

```

fromList [(1,1),(1,2),(1,3),(2,1),(2,2),(2,3),(3,1),(3,2),(3,3)]
> i (PairDelta (DSeq [Ins 0 0]) mempty)
DSeq [Ins 0 (0,1), Ins 0 (0,2), Ins 0 (0,3)]

```

When run on the sequence that enumerates 1 to 3 paired with itself, the initial result r is indeed the cartesian product of those two sequences, and if the input tuple is updated to add 0 to the first sequence, this change is correctly converted to add the pairs (0,1), (0,2), and (0,3) to the result. Note that `S.` delimitates sequence functions from the Haskell `Data.Sequence`, and `DSeq` describes changes on a sequence as a list of insertions (`Ins`), deletions or replacements.

Thus ILC has been embedded and extended with sequences. It is worth noting that there is a known problem with the ILC. It is only efficient for certain “self-maintainable” functions. The full details of this inefficiency can be found in the sequel ILC paper that introduces the CTS [Giarrusso et al. 2019] variant, but long story short some functions written in ILC can introduce recomputation. In fact, the map semantic function for ILC cannot be implemented without recomputation. Happily, the CTS variant of ILC is able to be embedded, and that is exactly what our first case study in §4.5 does.

12. Syntax and Typing Rules of Languages to Embed

Here, we summarize the syntax and typing rules of the embedded languages discussed in the paper.

12.1. Syntax and Typing Rules of ILC/CTS DSL

The basic syntax is as below.

$$e ::= () \mid (e_1, e_2) \mid \mathbf{fst} \ e \mid \mathbf{snd} \ e$$

For each construct, we have the following type rules.

$$\begin{array}{c}
\frac{}{\Gamma \vdash () : ()} \quad \frac{\Gamma \vdash e_1 : A_1 \quad \Gamma \vdash e_2 : A_2}{\Gamma \vdash (e_1, e_2) : (A_1, A_2)} \quad \frac{\Gamma \vdash e : (A_1, A_2)}{\Gamma \vdash \mathbf{fst} \ e : A_1} \quad \frac{\Gamma \vdash e : (A_1, A_2)}{\Gamma \vdash \mathbf{snd} \ e : A_2} \\
\\
\frac{\Gamma \vdash e_1 : A \quad \Gamma, x : A \vdash e_2 : B}{\Gamma \vdash \mathbf{let} \ x = e_1 \ \mathbf{in} \ e_2 : B}
\end{array}$$

In §4.5, we extend the language as

$$e ::= \dots \mid \mathbf{emp} \mid \mathbf{concat} \ a \mid \mathbf{single} \ e \mid \mathbf{map} \ (x.e) \ e'$$

together with the typing rules below.

$$\frac{}{\Gamma \vdash \mathbf{emp} : \mathbf{Seq} \ A} \quad \frac{\Gamma \vdash e : \mathbf{Seq} \ (\mathbf{Seq} \ A)}{\Gamma \vdash \mathbf{concat} \ e : \mathbf{Seq} \ A} \quad \frac{\Gamma \vdash e : A}{\Gamma \vdash \mathbf{single} \ e : \mathbf{Seq} \ A} \quad \frac{\Gamma, x : A \vdash e : B \quad \Gamma \vdash e' : \mathbf{Seq} \ A}{\Gamma \vdash \mathbf{map} \ (x.e) \ e' : \mathbf{Seq} \ B}$$

Constructs to add depend on problems; for examples, in §13.3, to realized an incremental (but simplified) version of HaXML filters [Wallace and Runciman 1999], we add constructs for trees together with additional operations (such as *filter*) on sequences.

12.2. Syntax and Typing Rules of HOBiT to Embed

The syntax for the embedded HOBiT is as below.

$$e ::= \ell \ e \mid () \mid (e_1, e_2) \mid \mathbf{let} \ (x, y) = e_1 \ \mathbf{in} \ e_2 \mid \mathbf{case} \ e_0 \ \{x_i.e_i \ \mathbf{with} \ \phi_i \ \mathbf{by} \ \rho_i\}_i$$

The constructs come with the following type rules.

$$\begin{array}{c}
\frac{\Gamma \vdash e : A \quad \ell : \text{Lens } A \ B}{\Gamma \vdash \ell e : B} \quad \frac{}{\Gamma \vdash () : ()} \quad \frac{\Gamma \vdash e_1 : A_1 \quad \Gamma \vdash e_2 : A_2}{\Gamma \vdash (e_1, e_2) : (A_1, A_2)} \\
\\
\frac{\Gamma \vdash e_1 : (A, B) \quad \Gamma, x : A, y : B \vdash e_2 : C}{\Gamma \vdash \text{let } (x, y) = e_1 \text{ in } e_2 : C} \\
\\
\frac{\Gamma \vdash e_0 : A_1 + \dots + A_n \quad \{\Gamma, x_i : A_i \vdash e_i : B \quad \phi_i : B \rightarrow \text{Bool} \quad \rho_i : A_1 + \dots + A_n \rightarrow B \rightarrow A_i\}_i}{\Gamma \vdash \text{case } e_0 \text{ of } \{x_i.e_i \text{ with } \phi_i \text{ by } \rho_i\}_i : B}
\end{array}$$

We do not have injections ($\text{InL } e / \text{InR } e$) and unit decompositions ($\text{let } () = e_1 \text{ in } e_2$) in the syntax (and type rules), as the lens application construct ℓe is so strong to express them as derived forms.

13. More Details of Embedded Incremental Language

13.1. Concrete Definition of mapSem'

Fig. 2 shows the definition of mapSem' . We say again that the fact that the changes (specifically, DEnv) form a monoid structure plays an important role in the definition of mapSem' . We can split changes on DEnv ($\text{Seq } a : \text{env}$) into ones on $\text{Seq } a$ and ones on env , and process them separately by transArg and transEnv , respectively. Notice also that the pair (h, htr) is a correct monoid-respecting CTS incremental function, because they are obtained from (f, tr) , which is also a correct monoid-respecting CTS incremental function, by fixing e (and changes to them are also fixed to deEmpty).

13.2. Pseudo Functions

The definition of mapSem' gives us a hint of handling second-order construct whose semantic function treats an input open expression as “functions”. Recall that (h, htr) is a (correct monoid-respecting) CTS incremental function, and dh can be thought as changes to the function (but affects only h). Unlike the original calculi [Cai et al. 2014; Giarrusso et al. 2019] where htr is integrated into dh , we couple it with the original function; such an integration makes the monoid structure dependent on htr , and the resulting function change does not respect the monoid structure.

The idea can be captured by the following datatype.

```

data PFun c a b = PFun (a -> (b, c)) ((Delta a, c) -> (Delta b, c))
instance Diff b => Diff (PFun c a b) where
  newtype Delta (PFun c a b) = DFun (c -> (Delta b, c))
  PFun f tr /+ (DFun df) = PFun f' tr
    where f' a = let (c, b) = f a; (c', db) = df c in (c', b /+ db)

```

(We omit the instance declaration of $\text{Monoid } (\text{Delta } b) \Rightarrow \text{Monoid } (\text{PFun } c \ a \ b)$.) With PFun , the definition of mapSem' becomes as Fig. 3, where the roles of $h/\text{htr}/\text{dh}$ become more evident.

We can provide the semantic function for the application of PFun , but it is not much useful; recall that the goal of PFun is to extract common patterns in the treatment of input open expressions for “higher-order”-like functions such as map . It is worth noting that providing the HOAS representation (and its unembedded implementation) for absPFunSem is impossible, due to the timing of choosing c . In absPFunSem , choice of c can depends on env , and thus the result cannot be expressed as exists c . CTS $\text{env } (\text{PFun } c \ a \ b)$, where c is chosen without knowing env . Handling existential types in the DSL might help, but this means that the DSL is no longer simply-typed and thus requires non trivial extension of the unembedding that targets simply-typed system [Atkey 2009; Atkey et al. 2009].

```

mapSem' :: (Diff a, Diff b) => CTS (a ': env) b -> CTS (Seq a ': env) (Seq b)
mapSem' (CTS f tr) = CTS f' tr'
  where
    f' (ECons (PackDiff xs) g) =
      let h x = f (ECons (PackDiff x) g)
          (ys, cs) = S.unzip (fmap h xs)
      in (ys, (cs, h))

    tr' (ECons (PackDiffDelta dxs) dg, (cs, h)) =
      let dgEmpty = mapEnv (\(PackDiffDelta _) -> PackDiffDelta mempty) dg
          htr (dx, c) = tr (ECons (PackDiffDelta dx) dgEmpty, c)
          dh c        = tr (ECons (PackDiffDelta mempty) dg, c)
          (dys1, cs1) = transArg h htr (dxs, cs)
          (dys2, cs2) = transEnv dh cs1
          h' a = let (b, c) = h a; (dy, c') = dh c in (b /+ dy, c')
      in (dys1 <> dys2, (cs2, h'))

    transArg h htr (DSeq adxs, cs) = iterTr (transArgAtom h htr) (adx, cs)

    transArgAtom h _htr (Ins i x, cs) =
      let (y, c) = h x
      in (DSeq $ pure $ Ins i y, S.insertAt i c cs)
    transArgAtom _h _htr (Del i, cs) =
      (DSeq $ pure $ Del i, S.deleteAt i cs)
    transArgAtom _h htr (Rep i dx, cs) =
      let ci = S.index cs i
          (dy, ci') = htr (dx, ci)
      in (rep i dy, S.update i ci' cs)

    transEnv dh cs =
      let (dys, cs') = S.unzip (fmap dh cs)
      in (mconcat $ zipWith rep [0..] (toList dys), cs')

```

Figure 2.: An implementation of `mapSem'`, where `mapEnv :: (forall x. f x -> g x) -> Env f as -> Env g as` is the mapping function for `Env`.

```

mapSem' g = mapSem (absPFunSem g)

absPFunSem :: Diff a => CTS (a : env) b -> exists c. CTS env (PFun c a b)
absPFunSem (CTS f tr) = CTS f' tr'
  where f' g = (PFun (\a -> f (ECons (PackDiffDelta a) g ))
               (\(da, c) -> tr (ECons (PackDiffDelta da) dgEmpty, c))
               , ( ))
    where dgEmpty = mapEnv (\(PackDiffDelta _) -> PackDiffDelta mempty) g
    tr' (dg, _) = (DFun (\c -> tr (ECons (PackDiffDelta mempty) dg, c)), ( ))

mapSem'' :: Diff b => (forall c. CTS env (PFun c a b)) -> CTS (Seq a : env) b
mapSem'' (CTS f tr) = CTS f' tr'
  where f' (ECons (PackDiffDelta as) g) = let (ph@(PFun h), c) = f g
                                           (as, cs) = Seq.unzip (Seq.map h bs)
                                           in (bs, (cs, c, ph))
    tr' (ECons (PackDiffDelta (DSeq adas)) dg, (cs, c, ph@(PFun h htr))) =
      let (DFun dh, c') = tr (dg, c)
          (dbs1, cs') = iterTr (transArg h htr) (adss, cs)
          (dbs2, cs'') = transEnv dh cs'
      in (dbs1 <> dbs2, (cs'', c', ph /+ DFun dh))

```

Figure 3.: Construction of PFun and an example of its use: we abused the Haskell syntax to use the bare exists.

This issue can be addressed by using dynamic types to encapsulate c , which compromises type safety to some extent.

13.3. Programming Example: Incremental HaXML Filters

To demonstrate the programmability of our embedded incremental language, we implement query Q1 from Use Case “XMP” in XML Query Use Cases² by using an incremental version of HaXML filters [Wallace and Runciman 1999]. This query extracts books that are published by “Addison-Wesley” after 1991, keeping their titles and years, as an input and output example in Fig 4.

We use the following rose tree type to express XML fragments.

```

data Tree = Elem (Fixed String) (Seq Tree)
          | Attr (Fixed String) MyString | Text MyString
type MyString = Seq Char

```

Here, Fixed a represents a values, which are not changed in Δ -propagation.

```

data ADTree = DChild (Delta Tree) | DTag String
            | DAttr String Delta MyString | DText Delta MyString
instance Diff Tree where
  newtype Delta Tree = DTree [ADTree]
  t /+ DTree ads = foldl applyADTree t ads

```

Here, `applyADTree :: Tree -> ADTree -> Tree` applies an atomic change to a given tree: `DChild dt` applies `dt` to the children `ts` of `Elem n ts`, `DTag n'` replaces the element name `n` of `Elem n ts` with `n'`, `DAttr k dv` replaces the attribute name to `k` and changes `v` by `dv` of `Attr k v`, and `DText ds` applies `ds` to `s` of `Text s`; they keep the original tree if it is not in a designated form. One might notice

²<https://www.w3.org/TR/xquery-use-cases/>

```

<bib>
  <book year="1994">
    <title>TCP/IP Illustrated</title>
    <author><last>Stevens</last><first>W.</first></author>
    <publisher>Addison-Wesley</publisher>
    <price>65.95</price>
  </book>
  <book year="1992">
    <title>Advanced Programming in the Unix environment</title>
    <author><last>Stevens</last><first>W.</first></author>
    <publisher>Addison-Wesley</publisher>
    <price>65.95</price>
  </book>
  <book year="2000">
    <title>Data on the Web</title>
    <author><last>Abiteboul</last><first>Serge</first></author>
    <author><last>Buneman</last><first>Peter</first></author>
    <author><last>Suciu</last><first>Dan</first></author>
    <publisher>Morgan Kaufmann Publishers</publisher>
    <price>39.95</price>
  </book>
  <book year="1999">
    <title>The Economics of Technology and Content for Digital TV</title>
    <editor>
      <last>Gerbarg</last><first>Darcy</first>
      <affiliation>CITI</affiliation>
    </editor>
    <publisher>Kluwer Academic Publishers</publisher>
    <price>129.95</price>
  </book>
</bib>

```

(a) Input

```

<bib>
  <book year="1994">
    <title>TCP/IP Illustrated</title>
  </book>
  <book year="1992">
    <title>Advanced Programming in the Unix environment</title>
  </book>
</bib>

```

(b) Corresponding Output

Figure 4.: Input and Output XMLs for Q1 (taken from <https://www.w3.org/TR/xquery-use-cases/>)

that $t \rightarrow t + dt$ cannot change the top-most constructor of t ; to do so, we remove the node and insert a new one at the parent node of t . This simplifies implementations of some key methods such as `children :: exp Tree -> exp (Seq Tree)`, where the change of the top-most construct would trigger the removal of all the elements from the output (and thus it needs to keep the length of the children as a cache if we would consider such changes).

We extend the language to include HaXML filters as Fig. 5, following the general recipe. Then, incremental versions of HaXML filters [Wallace and Runciman 1999] are represented by the following type.

```
type EFilter exp = exp Tree -> exp (Seq Tree)
```

Also, the language comes with many derived filters and filter combinators; some important ones are excerpted in Fig. 6.

Fig. 7 shows an implementation `q1` of the query Q1 in our embedded DSL. Mostly, the code are self-explanatory except some new derived methods: `attr` is a variant of `tag` for attributes, `mkElem a fs` constructs `Elem s ts` where `ts` are constructed by applying the filters `fs`, and `ifAttr s k f` applies `k v` to the input if an input is such an attribute named `s` of value `v`, and otherwise applies `f` to the input. Overall, `q1filter` is implemented quite concisely thanks to incremental filters in our EDSL. However, one can find that the use of `if_` breaks the abstraction using filters, indicated by the explicit argument `a`. Notice that `if_` requires `exp a` for then/else-clauses, and thus we cannot pass expressions of type `EFilter exp = exp Tree -> exp Tree`. This is actually a cost to pay for not having higher-order functions in the DSL. For this particular case, the solution is simple: we prepare families of `if_` such as `if2 :: CTSBool exp => exp Bool -> (exp a -> exp r) -> (exp a -> exp r) -> exp a -> exp r`.³

In the above implementation, we intentionally separated the `liftPred` method into `CTSLiftPred`. The method is unfortunately not compatible with the optimized implementation discussed in §13.4 that uses

³We can provided a unified implementation using type families and GADTs, but cannot choose an appropriate one by using the type of the then/else-clause due to the treatment of the type variable `exp` in Haskell.

```

class CTSTBase exp => CTSTBool exp where
  not :: exp Bool -> exp Bool
  if_ :: exp Bool -> exp a -> exp a -> exp a

class CTSTBool exp => CTSTLiftPred exp where
  liftPred :: (a -> Bool) -> exp a -> exp Bool

class (CTSTSeq exp, CTSTBool exp) => CTSTSeqE exp where
  filter :: (exp a -> exp Bool) -> exp (Seq a) -> exp (Seq a)
  append :: exp (Seq a) -> exp (Seq a) -> exp (Seq a)
  null :: exp (Seq a) -> exp Bool

class (CTSTSeqE exp, CTSTBool e, . . . ) => CTSTFilter exp where
  children :: exp Tree -> exp (Seq Tree)
  makeElem :: String -> exp (Seq Tree) -> exp Tree
  hasTag :: String -> exp Tree -> exp Bool
  ...
  unsingle :: exp (Seq a) -> exp a

```

Figure 5.: Extended Language for HaXML Filters

```

keep :: CTSTFilter exp => EFilter exp
keep = singleton
none :: CTSTFilter exp => EFilter exp
none = \_ -> empty
tag :: CTSTFilter exp => String -> EFilter exp
tag s = filter (hasTag s) . keep
(>>) :: CTSTFilter exp => EFilter exp -> EFilter exp -> EFilter exp
f >> g = concat . (map g) . f
f /> g = f >> children >> g
with :: CTSTFilter exp => EFilter exp -> EFilter exp -> EFilter exp
f `with` g = filter (not . null . g) . f

```

Figure 6.: Derived Filters and Filter Combinators (Excerpt)

```

q1 :: (CTSTFilter exp, CTSTLiftPred exp) => exp Tree -> exp Tree
q1 = unsingle . q1filter

q1filter :: (CTSTFilter exp, CTSTLiftPred exp) => EFilter exp
q1filter = mkElem "bib" [keep /> ((tag "book" `with` byAW `with` after1991)
  >> mkElem "book" [keep /> attr "year", keep /> tag "title"])]

where
  byAW = keep /> tag "publisher" /> text "Addison-Wesley"
  after1991 = keep /> ifAttr "year"
    (\s a -> if_ (liftPred (\str -> (read (toString str) :: Int) > 1991) s)
      (keep a) (none a)) none

```

Figure 7.: Query Q1

staging to eliminate the cost caused unembedding. To avoid this incompatibility, we have to provide methods that replace `toString`, `read` (specialized to `Int`) and `(<)`, which requires substantial effort.

A subtlety in the code is `filter` used inside `with` (see Fig. 6). The associated semantic function with `filter` stores the whole input as a cache because it needs to produce appropriate insertions when a condition becomes true. Thus, the successive applications of `with` in `qfilter` is inefficient in the sense that the associated caches have overlaps. This suggests an optimization by code transformation, but it is not implemented by our current prototype.

13.4. More Efficient Implementation

The implementation based on CTS involves several places to improve in terms of performance: *e.g.*,

1. Manipulation of value-level representations of typing environments that happens in runtime.
2. The empty caches (of the unit type) stored in the whole cache.
3. The function `transEnv` called even when `dh` is the identity change.

We address the first issue by using staging to perform the unembedding-related computation is the first stage and leave the essential computation in the second stage; specifically, we use Template Haskell [Sheard and S. L. P. Jones 2002]. The second issue is addressed by using join lists to manage the structure of caches so that the empty cache works as the identity element for the joining operation. In the combination of staging, we keep only the elements in the join lists in the second stage, but some semantic functions requires us to store the whole cache in a data structure. For this purpose, an optimized semantic domain also keeps the shape of the join list for inter-conversion between join lists in the first stage and ones in the second stage. For the third issue, we add a method `checkEmpty :: Delta a -> Bool` to the typeclass `Diff`, which returns `True` only if the input change is an identical one. Also, we gather uses of variables so that the checking function becomes more accurate for `DFun`, which now has an extra Boolean field for `checkEmpty` that comes true when `checkEmpty` returns true for any element in `de` in the construction (*i.e.*, in `absPFunSem`).

Combining these ideas, the semantic domain now changes to:

```
data CTSQ env a = forall cs us.
  CTSQ (TEnv env)
    (TConn cs)      -- cache shape
    (Env SBool us)  -- uses
    (MCode (Env CodeDiff (Extr env us) -> MCode (Code a, Conn Code cs),
            Env CodeDelta (Extr env us) -> Conn Code cs
            -> MCode (CodeDelta a, Conn Code cs)))
```

Here, we write `Code a` for a type for code of type `a`,⁴ `CodeM a` is the monad satisfying: `CodeM a ≃ (a -> Code r) -> Code a`. The monad is used to share the context in code generation, and avoid materializing whole structure; *e.g.*, `CodeM (a, b)` does not immediately construct pairs in generated code. Also, `CodeD` and `CodeP` are variants of `Code` satisfying `CodeD a ≃ Code (Delta a)` and `CodeP a ≃ Code (PackDiff a)`. The type `Conn f cs` denotes a join list of which contents is `f ci` where `cs = [c1, ..., cn]`. We do not dive more details about this because the implementation is too complex for a paper.⁵

⁴`Code a` is `Q (TExp a)` in GHC before 9, and is `Language.Haskell.TH.Code Q a` in GHC 9.

⁵See the source code (which uses a preliminary formalization of Embedding by Unembedding) available from <https://archive.softwareheritage.org/swh:1:rev:fe7a2d44582d334d83d503d443843351af24d821> for the details.

14. Related Work of Embedded Incremental Language

eCTS vs CTS Though the original CTS incremental λ -calculus [Giarrusso et al. 2019] is untyped, the accompanied implementation contains a simply-typed embedded DSL based on parametric HOAS [Chlipala 2008]. The EDSL is not suitable for users to directly program with; its (P)HOAS representation follows the λ -lifted A'-normal form (a variant of the A-normal form [Flanagan et al. 1993]), and exposes the cache types. Also, the interpretation of the EDSL, code generation by strings, is not backed by the theoretical basis, unlike unembedding [Atkey 2009]. The definition of *mapSem* in Appendix 13 is adopted from their implementation. We note, however, that, to process changes one-by-one, their treatment of functions in the implementation becomes different from the one in the paper, and hence the correctness proof (and that in the accompanied Coq code) is not lifted to the EDSL.

Other Adaptations of CTS Morihata [2020] gives an interface to Cai et al. [2014]'s incremental computation based on short-cut fusion [Gill et al. 1993] with reformulation. In his framework, a function to be incrementalized takes abstract polymorphic methods to be substituted by incremental functions. Thus, his use of the short-cut fusion is similar to the tagless-final style [Carette et al. 2009]. An issue of the original calculus, which the CTS version [Giarrusso et al. 2019] also addresses, is the recomputation of A required by a derivative $A \rightarrow \Delta A \rightarrow \Delta B$. Interestingly, tupling [Chin 1993] does not help here as ΔA is not available for the first run. Their idea can be viewed as Yoneda embedding to have $A \rightarrow \Delta A \rightarrow \Delta B \sim A \rightarrow \forall s.(s \rightarrow \Delta A) \rightarrow (s \rightarrow \Delta B)$. Then, one can tuple it with the original function to have $(A, s \rightarrow \Delta A) \rightarrow (B, s \rightarrow \Delta B)$, which can be viewed as an interpretation domain of $A \rightarrow B$ functions in the DSL. Now the issue of ΔA not being available is resolved by passing *id* to the tupled function yielding $A \rightarrow (B, \Delta A \rightarrow \Delta B)$. This approach scales to propagation of multiple updates: starting from $A \rightarrow \text{Interact } (\Delta A) (\Delta B)$, one obtains $A \rightarrow (B, \text{Interact } (\Delta A) (\Delta B))$. This approach is simple and type preserving, in contrast to Giarrusso et al. [2019] which targets untyped system and performs a global program transformation that requires λ -lifting and A'-normalizing as a preprocess. However, it is unclear how this can apply to higher-order methods such as *map*.

CTS Alternatives We chose the CTS incremental calculus [Giarrusso et al. 2019] as a target because it is simple, easy to reason about, and realistic. There are many other frameworks for incremental computation. One of the most popular ones is self-adjusting computation [Acar, Blelloch, et al. 2006], where programmers create, read and write *modifiabiles* so that a dependency graph among modifiabiles can be extracted to incrementally propagate changes of modifiabiles from input to output. We could extract the core part to make a DSL to embed by unembedding, whose API would be similar to the ML library presented by Acar, Blelloch, et al. [2006]. However, we suspect that the resulting EDSL would have limited benefits as the manipulation of the modifiabiles is the main challenge and not made easy by the embedding. In later work [Y. Chen, Dunfield, and Acar 2012; Y. Chen, Dunfield, Hammer, et al. 2014b], systems are designed to generate modifiable-manipulate code from ordinary functional programs with annotations.

15. More Details of Embedded Bidirectional Language

15.1. Round-tripping Laws

A lens must satisfy round tripping to be considered as *well-behaved* [J. Nathan Foster et al. 2007]. Here we use the system adopted by HOBiT, which is a slightly weaker version of the original round tripping [Bancilhon and Spyrtos 1981; J. Nathan Foster et al. 2007; Hegner 1990].

Definition 9 ($\preceq$ -well-behavedness [Matsuda and M. Wang 2018d]). Let $(S, \preceq)$ and $(V, \preceq)$ be partially-ordered sets. A lens $\ell :: \text{Lens } S \ V$ is called $\preceq$ -well-behaved if it satisfies

$$\begin{aligned} \text{get } \ell \ s = v &\implies v \text{ is maximal} \wedge (\forall v'. v' \preceq v \implies \text{put } \ell \ s \ v' \preceq s) && \text{(Acceptability)} \\ \text{put } \ell \ s \ v = s' &\implies (\forall s''. s' \preceq s'' \implies v \preceq \text{get } \ell \ s'') && \text{(Consistency)} \end{aligned}$$

for any $s, s' \in S$ and $v \in V$, where s is maximal. $\square$

Intuitively, $s_1 \preceq s_2$ means that s_2 is more determined than s_1 . The laws coincide with the conventional well-behavedness when $(\preceq) = (=)$ on S and V . For the sake of this paper, we do not go into many details on round tripping, but emphasize that the property is in fact preserved in our handling by using a type synonym $\text{Lens } S \ V$ for $\preceq$ -well-behaved lenses.

We use the weaker $\preceq$ -well-behavedness so that the embedded language can have a standard simply-typed system that allows both weakening and contraction. Alternatively, we can keep the conventional well-behavedness if we consider a linear-typed language. Though the original unembedding proposal [Atkey 2009; Atkey et al. 2009] relies on the fact that a language to embed can be represented as a second-order algebra [M. P. Fiore and Hur 2010], we conjecture that a similar technique is also possible for linear cases. Recently, Haskell included support for linear types [J. Bernardy et al. 2018], and thus the unembedding of a linear language leveraging the parametricity of a linear-typed host language is an interesting future direction.

15.2. Pattern Combinators

Directly using *unpair* and *branch_n* in programming is cumbersome, and it is desirable if we can write bidirectional branching as ordinary **case** expressions instead. To address this issue, we provide pattern-like combinators on top of the EDSL so far constructed.

We first confirm that patterns p can be viewed as linear-typed expressions. A convenient way to embed linear-typed terms is to use difference lists to represent typing environments (e.g., [Polakow 2015]) as $\Gamma_{\text{in}} \vdash p : A \mid \Gamma_{\text{out}}$, which reads that, by matching a pattern of type p , typing environment Γ_{in} changes to Γ_{out} . Then, a semantics of a pattern can be given as a pair of functions as below.

$$\llbracket \Gamma_{\text{in}} \vdash - : A \mid \Gamma_{\text{out}} \rrbracket = \text{PBij } (\llbracket A \rrbracket, \llbracket \Gamma_{\text{in}} \rrbracket_{\text{L}}) \llbracket \Gamma_{\text{out}} \rrbracket_{\text{L}}$$

The semantics of typing environment $\llbracket \Gamma \rrbracket_{\text{L}}$ is different from the one used for a term-in-context (**VEnv** *env*), as there is no need of weakening and contraction, and using $\llbracket x_1 : A_1, \dots, x_n : A_n \rrbracket = ((\dots ((\), \llbracket A_1 \rrbracket), \dots), \llbracket A_n \rrbracket)$ is fine.

So, the datatype for shallow-embedded patterns can be given as

```
data Pat i o a = Pat (TEnv i -> TEnv o) (PBij (a, LVEEnv i) (LVEEnv o))
```

where $\text{LVEEnv } o$ implements $\llbracket \Gamma \rrbracket_{\text{L}}$; specifically, $\text{LVEEnv} = \text{Env Identity}$. The second parameter of *Pat* computes Γ_{out} from Γ_{in} , which will be used to decompose $\llbracket \Gamma_{\text{out}} \rrbracket_{\text{L}}$ in the embedded DSL (otherwise, it is unclear how *unpair* can be applied). The following are basic combinators for patterns.

```
varP  :: Pat i (i : a) a
unitP :: Pat i i ( )
pairP :: Pat m o a -> Pat i m b -> Pat i o (a, b)
liftP :: PBij a b -> Pat i o b -> Pat i o a
```

They are powerful enough to define many patterns; e.g., the *nil* and *cons* patterns can be defined via partial bijections *unnil* :: $\text{PBij } [a] \ ()$ and *uncons* :: $\text{PBij } [a] \ (a, [a])$ —which are the inverses of the partial bijections *nil* :: $\text{PBij } () \ [a]$ and *cons* :: $\text{PBij } (a, [a]) \ [a]$ corresponding to $[]$ and $(:)$, respectively—as:

Appendix

```

nilP :: Pat i i [a]
nilP = liftP unnil unitP
consP :: Pat m o a -> Pat i m [a] -> Pat i o [a]
consP p1 p2 = liftP uncons (pairP p1 p2 )

```

An important property of `Pat` is that `Pat [] o a` defines a partial bijection between `a` and `LVEEnv o`. Also, we can obtain `TEnv o` as mentioned before, which is used to define the following decomposition function.

```

decompEnv :: HOBiT exp => TEnv o -> (Env exp o -> exp r )
                                     -> (exp (LVEEnv o) -> exp r )
decompEnv ENil      k e = ununit (prim n2uL e) (k Nil)
decompEnv (ECons g ) k e = unpair (prim c2pL e)
                                   (\x xs -> decompEnv g (k . ECons x) xs)

```

Here, `n2uL :: Lens (LVEEnv [ ]) ( )` and `c2pL :: Lens (LVEEnv (a : as)) (a, LVEEnv as)` are the lenses defined by the obvious isomorphism between types.

Thanks to these patterns we can now have the following *derived* method.

```

case_ :: HOBiT exp => String -> exp a -> [Br exp a r ] -> exp r

```

Here, `Br` is a type for branches defined by:

```

data Br exp a r where
  Br :: Pat [ ] o a -> (Env exp o -> exp r )
      -> (r -> Bool) -> (a -> r -> a) -> Br exp a r

```

Note that `o` is existentially quantified in the definition. The implementation of `case_` involves `branch_n` and `decompEnv`, with construction of a lens to convert an input `a` to sums `LVEEnv o1 + ... + LVEEnv on` where `o_i` is the existentially qualified pattern for the *i*th branch. The ease of defining derived combinators demonstrate the clear benefit of embedding, where we can keep the core set of combinators (*i.e.*, semantic functions) small and consequently suitable for complex reasoning. This advantage is decisive in bidirectional languages, as the need to reason about round tripping is no longer at the cost of reduced language features.

We can even add extra features to the language by Haskell's type-level programming. Consider the following type family

```

type family Func exp as r where
  Func exp [ ]      r = r
  Func exp (a : as) r = exp a -> Func exp a r

```

and a converter between the representations (whose straightforward definition is omitted):

```

fromFunc :: Func exp as (exp r ) -> (Env exp as -> exp r )

```

With these, we can define a combinator with the following type.

```

(==>) :: Pat [ ] o a -> (Func exp o (exp r ), r -> Bool, a -> r -> a) -> Br exp a r
p@(Pat _ _) ==> (b,f, recon) = Br p (fromFunc b) f recon

```

As a result, users of the DSL can be oblivious to the implementation and need not touch `Env` at all.